RV32I Minimal SoC – VHDL Implementation

Mateus Telles Nébias

Contents

1 RV32I Minimal SoC – Project Overview **10**
1.1 Introduction . . . 10
1.2 System Architecture . . . 10
1.3 CPU Overview . . . 11
1.4 CPU Datapath Overview . . . 12
1.5 Memory System . . . 14
1.6 Instruction Memory . . . 14
1.7 Bus Interconnect . . . 14
1.8 Peripherals . . . 14
1.9 Trap Handling . . . 15
1.10 Verification Strategy . . . 15
1.11 Documentation Structure . . . 16
1.12 Design Philosophy . . . 16

2 CPU Microarchitecture **16**
2.1 Purpose of This Document . . . 16
2.2 Microarchitecture Overview . . . 17
2.3 Architectural Style . . . 17
2.4 Top-Level CPU Datapath . . . 18
2.5 Program Counter Operation . . . 18
2.6 Instruction Fetch . . . 19
2.7 Decode Stage . . . 20
2.8 Register File Organization . . . 21
2.9 Immediate Generation . . . 22
2.10 ALU Operand Selection . . . 23
2.11 ALU and ALU Control . . . 24
2.12 Branch Comparator and Control Flow Decisions . . . 26
2.13 Load/Store Path . . . 27
2.14 Writeback Path . . . 27
2.15 Single-Cycle Timing Implications . . . 28
2.16 CPU Interaction with the Bus Interconnect . . . 29
2.17 Trap Unit Integration . . . 29
2.18 Instruction Flow by Class . . . 31
2.19 Control and Data Separation . . . 33
2.20 CPU Microarchitecture Summary Diagram . . . 33
2.21 Design Strengths . . . 34

2.22 Verification-Friendly Structure . . . 35
2.23 Limitations and Natural Future Extensions . . . 35
2.24 Final Summary . . . 36

3 ALU Module **36**
3.1 Purpose of the Module . . . 36
3.2 Architectural Role Inside the CPU . . . 37
3.3 Functional Responsibilities . . . 37
3.4 Inputs and Outputs . . . 38
3.5 Data Width and Numeric Interpretation . . . 39
3.6 ALU Control Relationship . . . 39
3.7 Operations Typically Implemented for RV32I . . . 41
3.8 ALU in Different Instruction Classes . . . 42
3.9 Combinational Nature of the Module . . . 43
3.10 Role in the Critical Path . . . 44
3.11 Signed vs Unsigned Behavior . . . 44
3.12 Shift Behavior . . . 45
3.13 Result Usage in the Datapath . . . 46
3.14 Interaction with Other Modules . . . 46
3.15 Verification Expectations . . . 47
3.16 Example Verification Flow . . . 48
3.17 Strengths of This Module in the Project . . . 50
3.18 Design Limitations and Tradeoffs . . . 50
3.19 Summary Diagram . . . 50
3.20 Final Summary . . . 51

4 ALU Control Module **51**
4.1 Purpose of the Module . . . 51
4.2 Architectural Role Inside the CPU . . . 52
4.3 Why This Module Exists . . . 52
4.4 Inputs and Outputs . . . 53
4.5 Relationship with RV32I Instruction Encoding . . . 54
4.6 Typical Decision Structure . . . 54
4.7 Operation Classes Usually Selected . . . 57
4.8 Fixed vs Fully Decoded Cases . . . 58
4.9 Example Decode Reasoning . . . 59
4.10 Combinational Nature of the Module . . . 59
4.11 Position in the Control Path . . . 60
4.12 Interaction with the Main Control Unit . . . 60
4.13 Interaction with the ALU . . . 61
4.14 Handling of Comparison-Related Operations . . . 62
4.15 Handling of Shift Variants . . . 62
4.16 Use in Address Calculation Instructions . . . 64
4.17 Verification Expectations . . . 64
4.18 Example Verification Flow . . . 65
4.19 Design Strengths of the Module . . . 66
4.20 Limitations and Tradeoffs . . . 66
4.21 Summary Diagram . . . 67

4.22 Final Summary . . . 67

5 Branch Compare Module **67**
5.1 Purpose of the Module . . . 67
5.2 Architectural Role Inside the CPU . . . 68
5.3 Why a Dedicated Branch Comparator Exists . . . 68
5.4 Relationship with RV32I Branch Instructions . . . 69
5.5 Inputs and Outputs . . . 69
5.6 Comparison Types Required by RV32I . . . 70
5.7 Signed vs Unsigned Interpretation . . . 71
5.8 Equality vs Ordering Comparisons . . . 72
5.9 Position in the Control-Flow Path . . . 72
5.10 Interaction with the Control Unit . . . 73
5.11 Interaction with the Immediate Generator . . . 74
5.12 Possible Internal Implementation Style . . . 74
5.13 Combinational Nature of the Module . . . 75
5.14 Role in the CPU Critical Path . . . 76
5.15 Branch Decision Semantics . . . 76
5.16 Relationship to Branch Target Computation . . . 76
5.17 Verification Expectations . . . 77
5.18 Example Verification Flow . . . 78
5.19 Interaction with Other Modules . . . 80
5.20 Design Strengths of the Module . . . 80
5.21 Limitations and Tradeoffs . . . 81
5.22 Summary Diagram . . . 81
5.23 Final Summary . . . 81

6 Control Unit Module **82**
6.1 Purpose of the Module . . . 82
6.2 Architectural Role Inside the CPU . . . 82
6.3 Why the Control Unit Exists . . . 83
6.4 Relationship with Instruction Decoding . . . 84
6.5 Inputs and Outputs . . . 85
6.6 Main Responsibilities of the Control Unit . . . 86
6.7 Typical Control Signals . . . 87
6.8 Operation by Instruction Class . . . 89
6.9 Relationship with the ALU Control Module . . . 91
6.10 Relationship with the Register File . . . 93
6.11 Relationship with the Immediate Generator . . . 93
6.12 Relationship with the Load Store Unit . . . 93
6.13 Relationship with the Branch Compare Module . . . 94
6.14 Relationship with the Trap Unit . . . 94
6.15 Control Unit as a Combinational Decoder . . . 94
6.16 Role in the Single-Cycle Critical Path . . . 95
6.17 Possible Internal Decode Style . . . 95
6.18 Verification Expectations . . . 95
6.19 Example Verification Flow . . . 96
6.20 Design Strengths of the Module . . . 98

6.21 Limitations and Tradeoffs . . . 98
6.22 Summary Diagram . . . 98
6.23 Final Summary . . . 99

7 Immediate Generator Module 99
7.1 Purpose of the Module . . . 99
7.2 Architectural Role Inside the CPU . . . 100
7.3 Why Immediate Reconstruction is Required . . . 101
7.4 Immediate Formats in RV32I . . . 101
7.5 Inputs and Outputs . . . 103
7.6 Immediate Selection Process . . . 103
7.7 Sign Extension . . . 104
7.8 Use of Immediate Values in the CPU . . . 104
7.9 ALU Operand . . . 105
7.10 Interaction with Other CPU Modules . . . 105
7.11 Combinational Nature of the Module . . . 106
7.12 Role in the CPU Critical Path . . . 106
7.13 Verification Expectations . . . 107
7.14 Example Verification Flow . . . 107
7.15 Design Strengths of the Module . . . 109
7.16 Limitations and Tradeoffs . . . 109
7.17 Summary Diagram . . . 109
7.18 Final Summary . . . 110

8 Instruction Memory Module 111
8.1 Purpose of the Module . . . 111
8.2 Architectural Role Inside the CPU . . . 111
8.3 Harvard-like Memory Organization . . . 111
8.4 Inputs and Outputs . . . 112
8.5 Instruction Width . . . 112
8.6 Addressing Behavior . . . 113
8.7 Instruction Storage Model . . . 114
8.8 Interaction with the Program Counter . . . 115
8.9 Read-Only Behavior . . . 116
8.10 Implementation Possibilities . . . 117
8.11 Combinational vs Synchronous Behavior . . . 117
8.12 Role in the CPU Critical Path . . . 118
8.13 Interaction with Decode Logic . . . 118
8.14 Verification Expectations . . . 119
8.15 Example Verification Flow . . . 119
8.16 Design Strengths of the Module . . . 121
8.17 Limitations and Tradeoffs . . . 121
8.18 Summary Diagram . . . 121
8.19 Final Summary . . . 122

9 Load Store Unit Module 122
9.1 Purpose of the Module . . . 122
9.2 Architectural Role Inside the CPU . . . 123

9.3 Relationship with RV32I Memory Instructions . . . 123
9.4 Address Generation . . . 124
9.5 Inputs and Outputs . . . 124
9.6 Memory Access Flow . . . 125
9.7 Interaction with the System Bus . . . 127
9.8 Memory Read Operations . . . 127
9.9 Memory Write Operations . . . 128
9.10 Alignment and Addressing Considerations . . . 128
9.11 Interaction with Other CPU Modules . . . 129
9.12 Combinational vs Sequential Behavior . . . 129
9.13 Role in the CPU Critical Path . . . 129
9.14 Verification Expectations . . . 130
9.15 Example Verification Flow . . . 130
9.16 Design Strengths of the Module . . . 132
9.17 Limitations and Tradeoffs . . . 132
9.18 Summary Diagram . . . 132
9.19 Final Summary . . . 133

10 Program Counter Module **133**
10.1 Purpose of the Module . . . 133
10.2 Architectural Role Inside the CPU . . . 134
10.3 Sequential Instruction Execution . . . 134
10.4 Program Counter Update Sources . . . 134
10.5 Inputs and Outputs . . . 135
10.6 Reset Behavior . . . 136
10.7 Interaction with Branch Instructions . . . 136
10.8 Interaction with Jump Instructions . . . 138
10.9 Interaction with Trap Handling . . . 138
10.10 Program Counter as a Register . . . 139
10.11 Role in the CPU Critical Path . . . 140
10.12 Interaction with Other CPU Modules . . . 140
10.13 Verification Expectations . . . 141
10.14 Example Verification Flow . . . 141
10.15 Design Strengths of the Module . . . 143
10.16 Limitations and Tradeoffs . . . 143
10.17 Summary Diagram . . . 143
10.18 Final Summary . . . 144

11 Register File Module **144**
11.1 Purpose of the Module . . . 144
11.2 Architectural Role Inside the CPU . . . 145
11.3 Register Set in RV32I . . . 145
11.4 Special Behavior of Register x0 . . . 146
11.5 Inputs and Outputs . . . 146
11.6 Dual Read Port Architecture . . . 147
11.7 Register Read Behavior . . . 148
11.8 Register Write Behavior . . . 149
11.9 Instruction Interaction . . . 150

11.10 Interaction with Other CPU Modules . . . 151
11.11 Combinational and Sequential Components . . . 151
11.12 Role in the CPU Critical Path . . . 152
11.13 Verification Expectations . . . 152
11.14 Example Verification Flow . . . 153
11.15 Design Strengths of the Module . . . 153
11.16 Limitations and Tradeoffs . . . 154
11.17 Summary Diagram . . . 154
11.18 Final Summary . . . 155

12 Trap Unit Module **155**
12.1 Purpose of the Module . . . 155
12.2 Architectural Role Inside the CPU . . . 156
12.3 What is a Trap . . . 156
12.4 Trap Detection . . . 157
12.5 Illegal Instruction Detection . . . 158
12.6 Trap Handling Flow . . . 159
12.7 Trap Handler Address . . . 160
12.8 Inputs and Outputs . . . 161
12.9 Inputs . . . 161
12.10 Interaction with the Program Counter . . . 161
12.11 Interaction with Other CPU Modules . . . 162
12.12 Combinational vs Sequential Behavior . . . 163
12.13 Role in the CPU Control Path . . . 163
12.14 Verification Expectations . . . 164
12.15 Example Verification Flow . . . 165
12.16 Design Strengths of the Module . . . 167
12.17 Limitations and Tradeoffs . . . 167
12.18 Summary Diagram . . . 167
12.19 Final Summary . . . 167

13 Bus Interconnect Module **168**
13.1 Purpose of the Module . . . 168
13.2 Architectural Role Inside the SoC . . . 168
13.3 Why the Bus Interconnect Exists . . . 169
13.4 Memory-Mapped SoC Model . . . 169
13.5 Inputs and Outputs . . . 170
13.6 Address Decoding . . . 171
13.7 Read Transaction Routing . . . 172
13.8 Write Transaction Routing . . . 173
13.9 One-Master, Multiple-Slave Organization . . . 174
13.10 Relationship with the Load Store Unit . . . 174
13.11 Relationship with Data Memory . . . 175
13.12 Relationship with GPIO . . . 175
13.13 Relationship with Timer . . . 176
13.14 Read Data Multiplexing . . . 176
13.15 Invalid or Unmapped Addresses . . . 176
13.16 Combinational vs Sequential Behavior . . . 177

13.17 Role in the SoC Critical Path . . . 177
13.18 Verification Expectations . . . 178
13.19 Example Verification Flow . . . 178
13.20 Design Strengths of the Module . . . 180
13.21 Limitations and Tradeoffs . . . 180
13.22 Summary Diagram . . . 180
13.23 Final Summary . . . 182

14 Data Memory Module **182**
14.1 Purpose of the Module . . . 182
14.2 Architectural Role Inside the SoC . . . 182
14.3 Memory Organization . . . 183
14.4 Addressing Model . . . 183
14.5 Inputs and Outputs . . . 183
14.6 Write Operation . . . 185
14.7 Read Operation . . . 185
14.8 Interaction with the Load Store Unit . . . 186
14.9 Interaction with the Bus Interconnect . . . 187
14.10 Word Alignment . . . 188
14.11 Sequential Behavior . . . 189
14.12 Role in the CPU Execution Path . . . 190
14.13 Verification Expectations . . . 190
14.14 Example Verification Flow . . . 190
14.15 Design Strengths of the Module . . . 192
14.16 Limitations and Tradeoffs . . . 192
14.17 Summary Diagram . . . 192
14.18 Final Summary . . . 193

15 GPIO Peripheral Module **193**
15.1 Purpose of the Module . . . 193
15.2 Architectural Role Inside the SoC . . . 193
15.3 General Purpose Input/Output Concept . . . 194
15.4 Memory-Mapped Peripheral Model . . . 194
15.5 GPIO Register Interface . . . 195
15.6 Inputs and Outputs . . . 196
15.7 Output Operation . . . 197
15.8 Input Operation . . . 198
15.9 Interaction with the Bus Interconnect . . . 200
15.10 Interaction with the CPU . . . 201
15.11 Sequential Behavior . . . 201
15.12 Example GPIO Use Cases . . . 202
15.13 Verification Expectations . . . 202
15.14 Example Verification Flow . . . 202
15.15 Design Strengths of the Module . . . 204
15.16 Limitations and Tradeoffs . . . 204
15.17 Summary Diagram . . . 204
15.18 Final Summary . . . 205

16 Timer Peripheral Module **205**
16.1 Purpose of the Module . . . 205
16.2 Architectural Role Inside the SoC . . . 205
16.3 Timer Concept . . . 206
16.4 Memory-Mapped Interface . . . 206
16.5 Timer Register Interface . . . 207
16.6 Inputs and Outputs . . . 208
16.7 Counter Operation . . . 209
16.8 Reset Behavior . . . 210
16.9 Timer Read Operation . . . 211
16.10 Timer Write Operation (Optional) . . . 211
16.11 Interaction with the Bus Interconnect . . . 212
16.12 Interaction with the CPU . . . 213
16.13 Example Software Use Case . . . 214
16.14 Sequential Behavior . . . 214
16.15 Verification Expectations . . . 214
16.16 Example Verification Flow . . . 215
16.17 Design Strengths of the Module . . . 217
16.18 Limitations and Tradeoffs . . . 217
16.19 Summary Diagram . . . 217
16.20 Final Summary . . . 218

17 Verification Strategy **218**
17.1 Purpose of the Verification Strategy . . . 218
17.2 Verification Goals . . . 218
17.3 Verification Methodology . . . 219
17.4 Design Under Test (DUT) . . . 220
17.5 Module-Level Verification . . . 221
17.6 Unit Testing Approach . . . 221
17.7 Instruction-Level Verification . . . 224
17.8 Control Flow Verification . . . 224
17.9 Memory Access Verification . . . 226
17.10 Peripheral Verification . . . 227
17.11 Integrated System Verification . . . 228
17.12 Clock and Reset Handling . . . 228
17.13 Expected vs Observed Behavior . . . 229
17.14 Simulation Environment . . . 230
17.15 Debugging Strategy . . . 230
17.16 Coverage Considerations . . . 230
17.17 Benefits of the Verification Strategy . . . 231
17.18 Limitations of the Verification Approach . . . 231
17.19 Summary Diagram . . . 231
17.20 Final Summary . . . 232

18 Testbench Architecture **233**
18.1 Purpose of the Testbench Architecture . . . 233
18.2 Testbench Structure . . . 233
18.3 Design Under Test (DUT) . . . 234

18.4 Clock Generation . . . 234
18.5 Reset Generation . . . 235
18.6 Stimulus Generation . . . 236
18.7 Output Observation . . . 237
18.8 Verification Checks . . . 237
18.9 Sequential Test Execution . . . 238
18.10 Timing Control . . . 240
18.11 Module-Level Testbenches . . . 240
18.12 CPU-Level Testbench . . . 240
18.13 Waveform Analysis . . . 241
18.14 Testbench Advantages . . . 242
18.15 Limitations of Simulation-Based Testbenches . . . 242
18.16 Simulation Workflow . . . 242
18.17 Good Testbench Design Practices . . . 244
18.18 Summary Diagram . . . 244
18.19 Final Summary . . . 245

19 CPU Datapath Diagram **245**
19.1 Purpose of the Datapath Diagram . . . 245
19.2 High-Level CPU Datapath Overview . . . 245
19.3 Instruction Fetch Stage . . . 246
19.4 Instruction Decode Stage . . . 246
19.5 Execution Stage . . . 247
19.6 Memory Access Stage . . . 248
19.7 Writeback Stage . . . 248
19.8 Control Flow Path . . . 249
19.9 Complete CPU Datapath Diagram . . . 249
19.10 Single-Cycle Datapath Characteristics . . . 249
19.11 Critical Path Considerations . . . 250
19.12 Educational Value of the Datapath . . . 250
19.13 Summary . . . 250

20 SoC Architecture Diagram **251**
20.1 Purpose of the SoC Architecture Diagram . . . 251
20.2 System Overview . . . 251
20.3 CPU Core . . . 252
20.4 Instruction Memory . . . 252
20.5 Data Access Path . . . 253
20.6 Bus Interconnect . . . 253
20.7 Data Memory . . . 254
20.8 GPIO Peripheral . . . 255
20.9 Timer Peripheral . . . 255
20.10 Memory Map Concept . . . 255
20.11 Data Flow Through the SoC . . . 256
20.12 Complete SoC Architecture Diagram . . . 257
20.13 Advantages of the SoC Architecture . . . 258
20.14 Educational Value . . . 258
20.15 Summary . . . 259

21 Timer Architecture Diagram **259**
21.1 Purpose of the Timer Architecture Diagram . . . 259
21.2 Overview of the Timer Module . . . 259
21.3 Core Timer Components . . . 260
21.4 Example Timing Measurement . . . 266
21.5 Advantages of the Timer Architecture . . . 266
21.6 Simple Implementation . . . 266
21.7 Educational Value . . . 267
21.8 Summary . . . 267

1 RV32I Minimal SoC – Project Overview

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

1.1 Introduction

This project implements a minimal System-on-Chip (SoC) built around a RV32I RISC-V processor written entirely in VHDL.

The processor is implemented as a single-cycle CPU architecture and integrated with memory-mapped peripherals using a simple bus interconnect.

The goals of the project are:

- study CPU microarchitecture
- design a RISC-V processor in RTL
- integrate a simple SoC architecture
- build a portfolio-level FPGA project

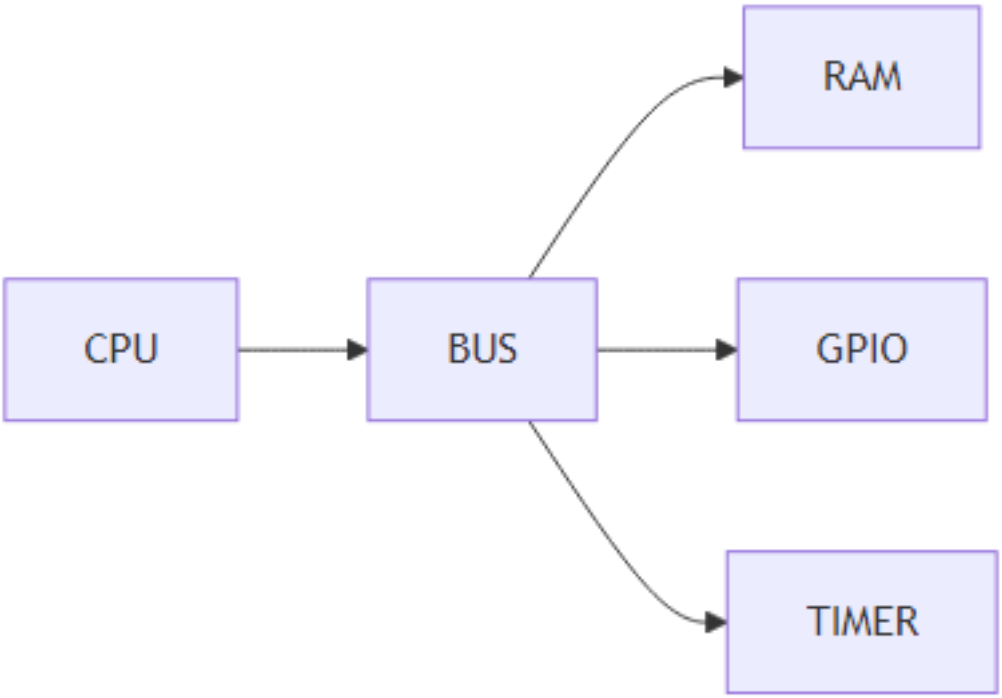
The system was designed to be simple, readable and modular, allowing incremental expansion.

1.2 System Architecture

The SoC consists of the following components:

- RV32I CPU core
- Instruction Memory
- Data RAM
- Bus Interconnect
- GPIO Peripheral
- Timer Peripheral

The CPU communicates with memory and peripherals through a memory-mapped bus.



1.3 CPU Overview

The CPU implements the RV32I base integer instruction set.

Main characteristics:

Feature	Description
ISA	RV32I
Datapath	Single-cycle
Register width	32 bits
Registers	32 general-purpose registers
Instruction width	32 bits
Architecture style	Harvard-like

In a single-cycle architecture, every instruction is executed in exactly one clock cycle.

Execution stages:

1. Instruction Fetch
2. Instruction Decode
3. Execute
4. Memory Access

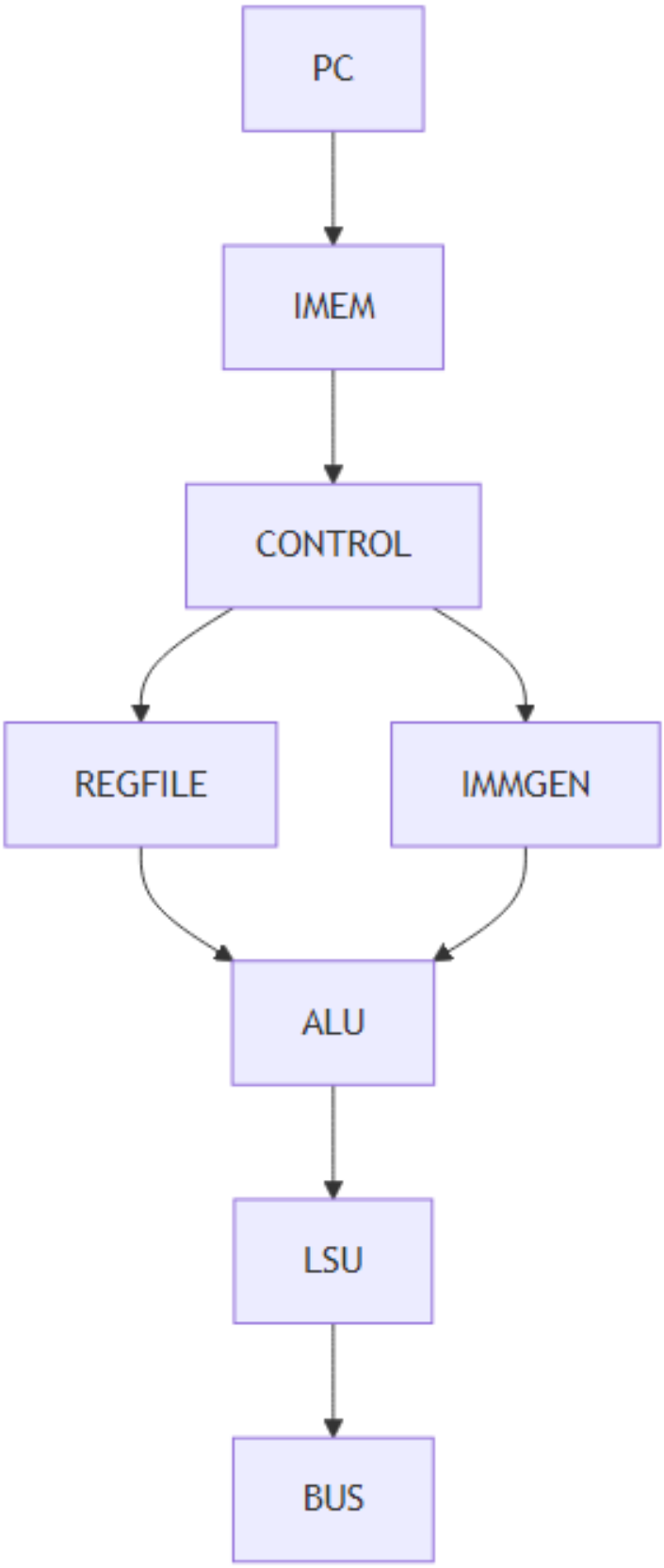
5. Writeback

All stages occur within the same clock cycle.

1.4 CPU Datapath Overview

The CPU datapath is composed of the following modules:

- Program Counter
- Instruction Memory
- Control Unit
- Register File
- Immediate Generator
- ALU
- Branch Comparator
- Load Store Unit
- Trap Unit


PC
IMEM
CONTROL
REGFILE
IMMGEN
ALU
LSU
BUS

1.5 Memory System

Two memory systems exist in the design.

1.6 Instruction Memory

Instruction memory stores the program executed by the CPU.

Characteristics:

- read-only during execution
- addressed by the program counter
- returns 32-bit instructions

1.6.1 Data Memory (RAM)

Data memory is accessed through the Load Store Unit.

Supported operations:

- load word
- store word
- peripheral access through memory mapping

1.7 Bus Interconnect

The bus interconnect routes CPU memory requests to the appropriate device based on address decoding.

Possible targets:

- RAM
- GPIO
- TIMER

This project intentionally uses a simple custom bus instead of more complex standards such as AXI or Wishbone.

Advantages:

- reduced logic complexity
- easier debugging
- better learning visibility

1.8 Peripherals

The SoC contains two peripherals.

1.8.1 GPIO

The GPIO peripheral provides simple memory-mapped I/O.

Typical uses include:

- LEDs
- switches
- FPGA pins
- debugging signals

1.8.2 Timer

The timer peripheral provides:

- cycle counting
- time measurement
- optional interrupt generation

Timers are essential peripherals for embedded software systems.

1.9 Trap Handling

The processor includes a trap unit used to handle exceptional situations such as:

- illegal instructions
- trap requests
- control flow errors

The trap unit also enables future extensions including:

- interrupt handling
- exception handling
- privileged execution modes

1.10 Verification Strategy

Each hardware module is verified using dedicated VHDL testbenches.

Examples include:

- tb_alu.vhd
- tb_register_file.vhd
- tb_program_counter.vhd
- tb_timer.vhd
- tb_cpu.vhd

The verification strategy combines:

- module-level unit tests
- CPU-level integration tests
- peripheral verification

Simulation validates:

- instruction execution
- register writes
- memory transactions
- peripheral interactions

1.11 Documentation Structure

```
docs/

00_project_specification
01_cpu
02_modules
03_peripherals
04_verification
diagrams
```

Each section focuses on a specific aspect of the architecture.

1.12 Design Philosophy

The project prioritizes:

- clarity over complexity
- modular RTL design
- structured documentation
- reproducible simulation

The final system represents a complete educational RISC-V hardware implementation suitable for FPGA experimentation and professional engineering portfolios.

2 CPU Microarchitecture

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

2.1 Purpose of This Document

This document describes the microarchitecture of the CPU used in the **RV32I Minimal SoC** project.

The processor is a **single-cycle RV32I core** written in **VHDL** and integrated into a simple memory-mapped SoC. The goal of this document is to explain how the CPU is organized internally, how data flows through the datapath, how control decisions are made, and how the processor interacts with memory and peripherals.

This document focuses on:

- architectural organization
- datapath structure
- instruction execution flow
- control signal roles
- arithmetic and branch processing
- load/store behavior
- trap integration
- CPU interaction with the SoC bus

2.2 Microarchitecture Overview

The CPU implements the **RV32I base integer instruction set** using a **single-cycle microarchitecture**.

In a single-cycle processor, every instruction completes in one clock cycle. This means that all major phases of instruction execution happen inside a single combinational pass between two active clock edges.

The CPU contains the following main blocks:

- Program Counter
- Instruction Memory Interface
- Control Unit
- Register File
- Immediate Generator
- ALU Control
- ALU
- Branch Comparator
- Load Store Unit
- Trap Unit

At a high level, the processor performs the following operations for each instruction:

1. fetch instruction from instruction memory
2. decode opcode and fields
3. read source registers
4. generate immediate if needed
5. execute arithmetic, logical, branch, or address calculation
6. perform memory access if required
7. select writeback data
8. update program counter

2.3 Architectural Style

The CPU follows a **Harvard-like organization** at the system level:

- instruction fetch uses **instruction memory**

- data accesses go through the **load/store path**
- peripherals are reached through the **bus interconnect**

This separation simplifies the architecture and makes the design easier to understand and verify.

Main properties:

Property	Value
ISA	RV32I
Datapath width	32 bits
Register file size	32 registers
Register width	32 bits
Instruction width	32 bits
Execution model	Single-cycle
Memory access model	Load/store
Peripheral model	Memory-mapped I/O

2.4 Top-Level CPU Datapath

The CPU datapath connects instruction fetch, decode, execute, memory access, and writeback into one direct execution path.

This diagram shows the essential information flow:

- the instruction is fetched using the current PC
- the instruction fields are decoded
- the register file supplies operands
- the immediate generator creates sign-extended immediate values
- the ALU performs arithmetic or address computation
- the branch comparator evaluates branch conditions
- the load/store unit formats data transactions
- writeback returns results to the register file
- PC logic chooses the next instruction address

2.5 Program Counter Operation

The **Program Counter (PC)** stores the address of the current instruction.

Its responsibilities are:

- hold the current instruction address

- provide the fetch address to instruction memory
- update to the next sequential instruction
- redirect control flow on branches, jumps, or traps

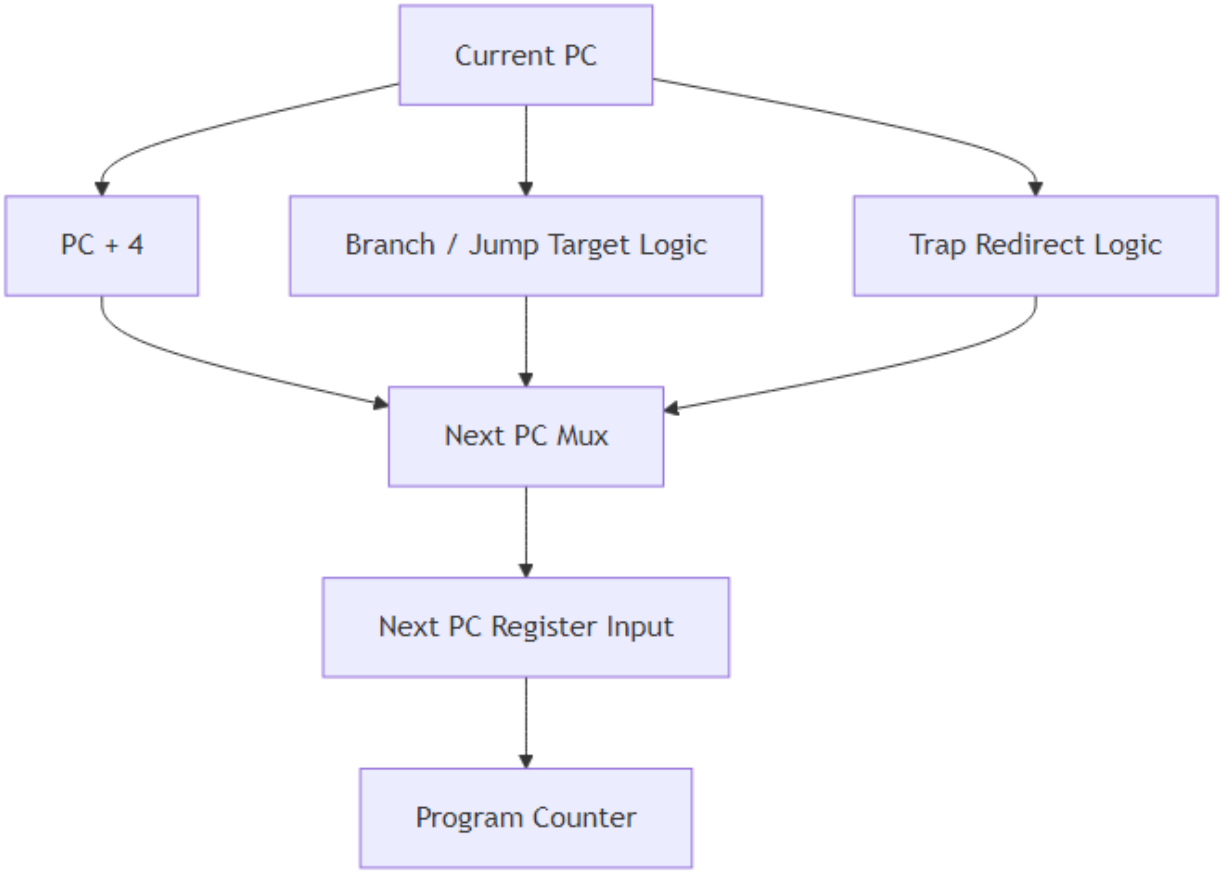
In a standard sequential case:

- next PC = current PC + 4

In control-transfer cases:

- next PC may be replaced by a branch target
- next PC may be replaced by a jump target
- next PC may be replaced by a trap vector or trap-related address

Conceptually:



The PC is therefore one of the central control points in the design.

2.6 Instruction Fetch

Instruction fetch is driven directly by the current PC.

The CPU sends the PC value to instruction memory, which returns a 32-bit instruction word. Because this is a single-cycle design, the fetch result is used immediately by the decode and execution

logic in the same cycle.

The fetched instruction contains the standard RV32I fields:

- opcode
- rd
- funct3
- rs1
- rs2
- funct7
- immediate field fragments

These fields are then routed to the appropriate blocks:

- opcode to the control unit
- rs1 and rs2 to the register file read ports
- rd to writeback destination logic
- immediate fragments to the immediate generator
- funct3 and funct7 to ALU control and branch logic

2.7 Decode Stage

The decode logic interprets the instruction and derives the control behavior required for execution.

The **Control Unit** determines the instruction class and produces control signals such as:

- register write enable
- ALU operand selection
- immediate usage
- memory read enable
- memory write enable
- branch or jump selection
- writeback source selection
- trap-related handling

The **ALU Control** refines ALU behavior based on instruction subtype. For example, it distinguishes between operations that may share the same general instruction format but require different ALU functions.

Examples of instruction categories decoded by the CPU:

- R-type arithmetic and logic
- I-type arithmetic immediate
- load instructions
- store instructions
- conditional branches
- upper immediate instructions
- jump instructions
- system or trap-related instructions

2.8 Register File Organization

The register file implements the standard RV32I integer register set:

- 32 registers
- each register is 32 bits wide
- 2 read ports
- 1 write port

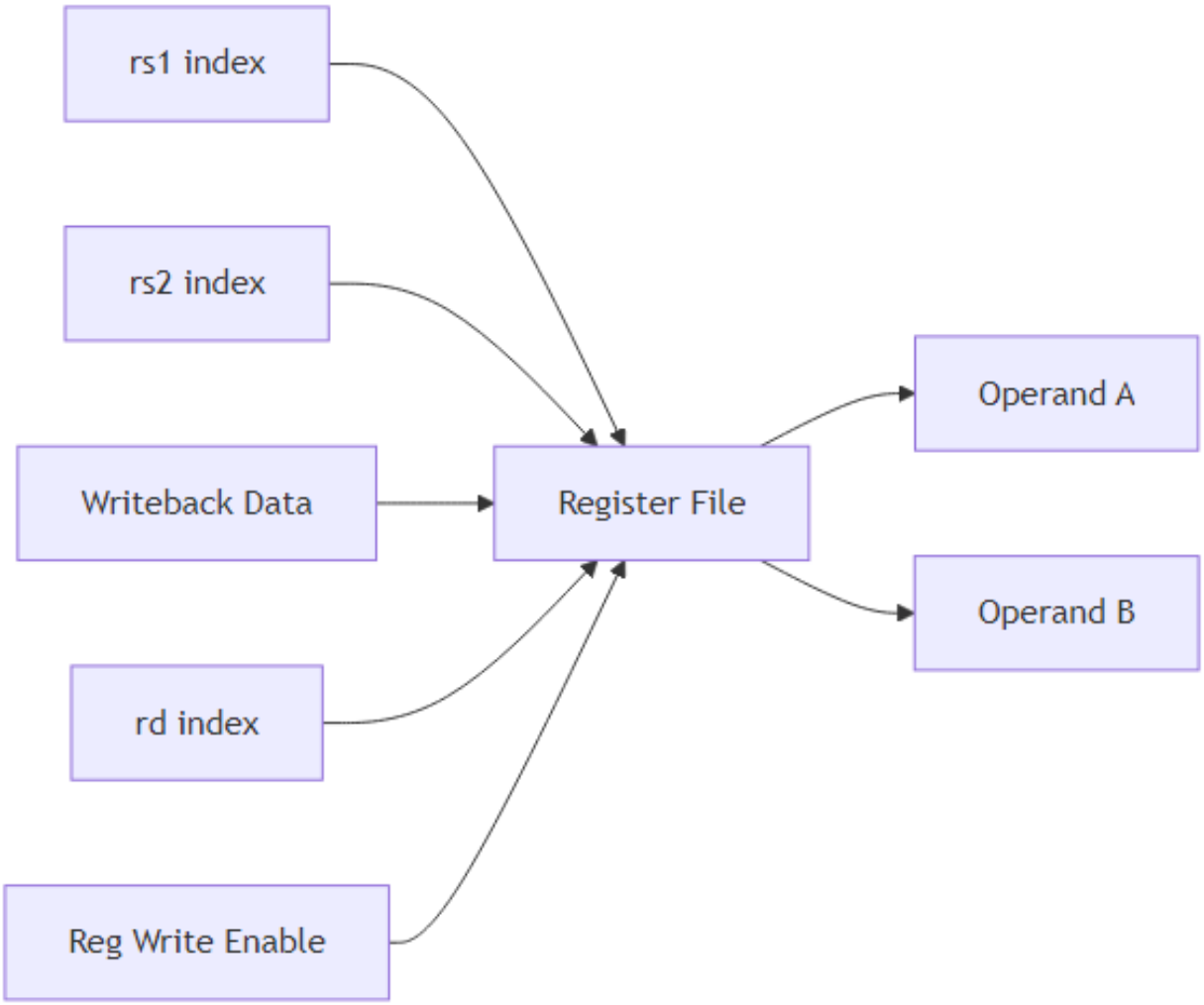
The register file provides:

- source operand A from `rs1`
- source operand B from `rs2`
- destination register selected by `rd`

Typical behavior:

- read ports are combinational
- write occurs when register write is enabled
- register `x0` remains hardwired to zero

Conceptually:



The register file is central to nearly all instructions:

- ALU operations use register operands
- branches compare register operands
- stores use register data as memory write data
- loads write data back into a destination register

2.9 Immediate Generation

The **Immediate Generator** extracts and sign-extends immediate values from the instruction encoding.

RV32I uses multiple immediate formats:

- I-type
- S-type
- B-type
- U-type

- J-type

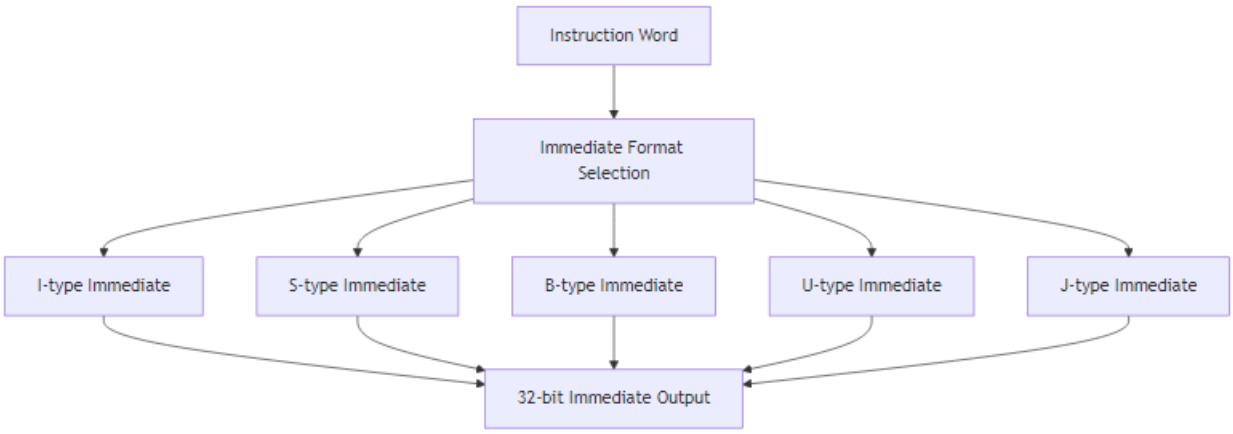
The immediate generator is responsible for:

- selecting the proper bit fields
- reassembling split immediate encodings
- sign-extending when needed
- presenting a uniform 32-bit immediate output

This block is used by:

- immediate ALU operations
- load/store address calculation
- branch target calculation
- upper-immediate instructions
- jump instructions

Conceptually:



Correct immediate decoding is essential because the same 32-bit instruction word places immediate bits in different positions depending on format.

2.10 ALU Operand Selection

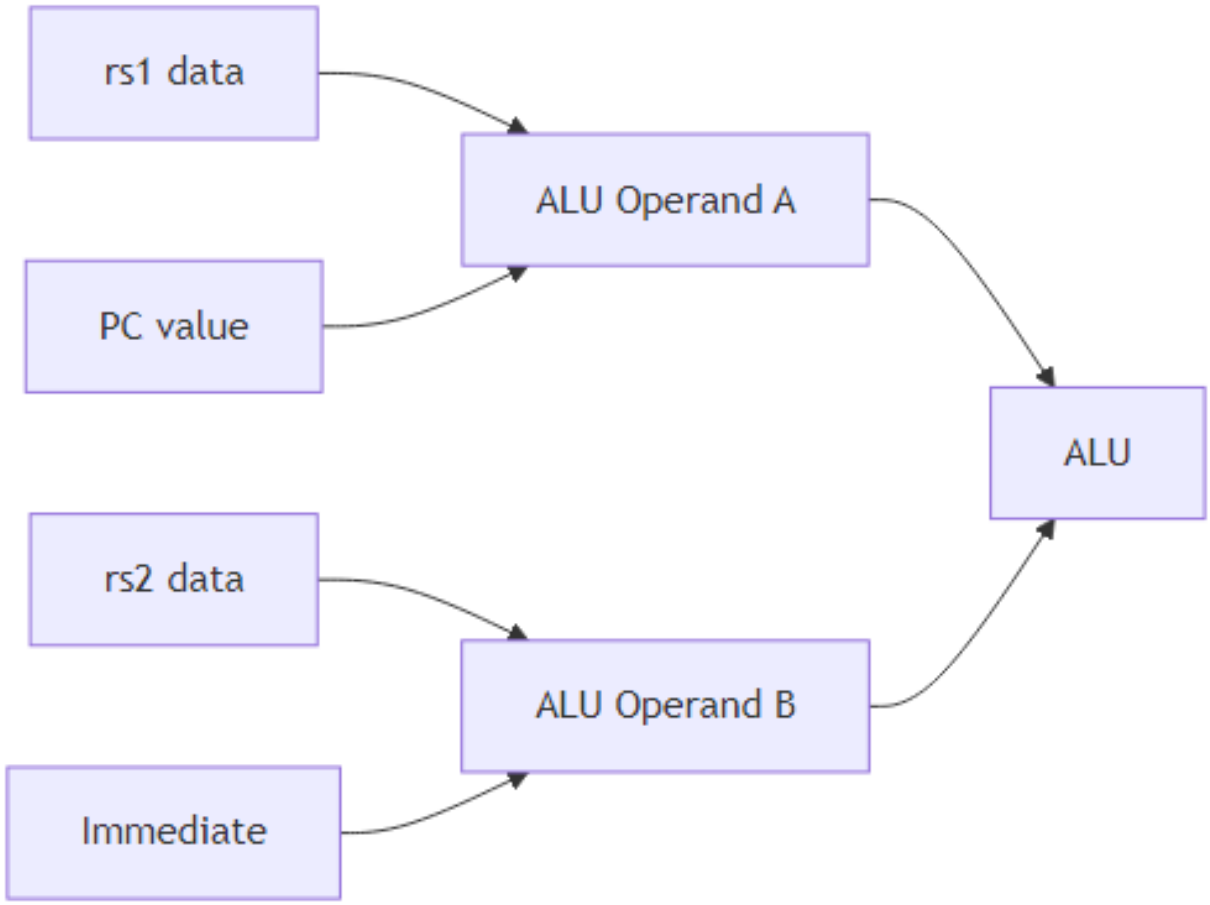
The ALU normally operates on two 32-bit operands. These operands may come from different sources depending on instruction type.

Typical operand selection:

- operand A = register file `rs1`
- operand B = register file `rs2` for R-type instructions
- operand B = immediate value for I-type, load, and store instructions

Some implementations also use specialized operand routing for instructions such as AUIPC, where PC-relative arithmetic may be needed.

Operand selection is controlled by decode signals.



Even in a simple single-cycle design, operand multiplexing is a key part of making the CPU flexible enough to support the full RV32I base integer set.

2.11 ALU and ALU Control

The **ALU** executes arithmetic and logical operations required by RV32I.

Typical operations include:

- ADD
- SUB
- AND
- OR
- XOR
- SLT
- SLTU
- SLL
- SRL
- SRA

The **ALU Control** block translates decoded instruction information into a specific ALU operation

code.

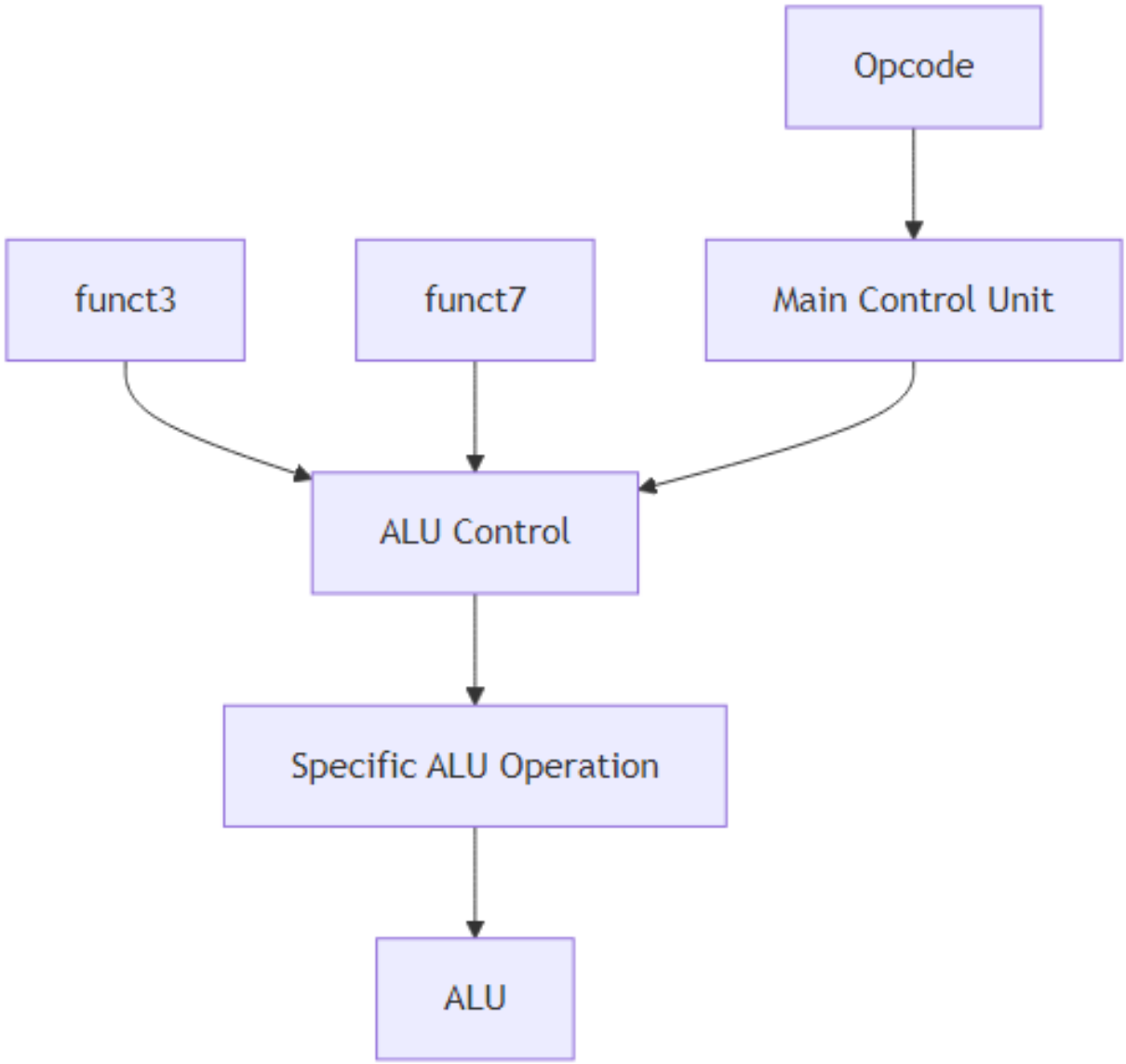
This separation is useful because:

- the main control unit handles broad instruction categories
- ALU control handles operation-level details

For example:

- load and store may both request ALU ADD for address computation
- branch instructions may reuse subtraction or comparison-related logic
- R-type operations select arithmetic or logical functions using funct3 and funct7

Conceptual control structure:



The ALU output is used for multiple purposes:

- arithmetic result

- logical result
- effective address for loads/stores
- base result for writeback
- control-flow target support in some instruction classes

2.12 Branch Comparator and Control Flow Decisions

Conditional branches require comparison of register operands and a decision about whether control flow should be redirected.

The **Branch Comparator** evaluates branch conditions such as:

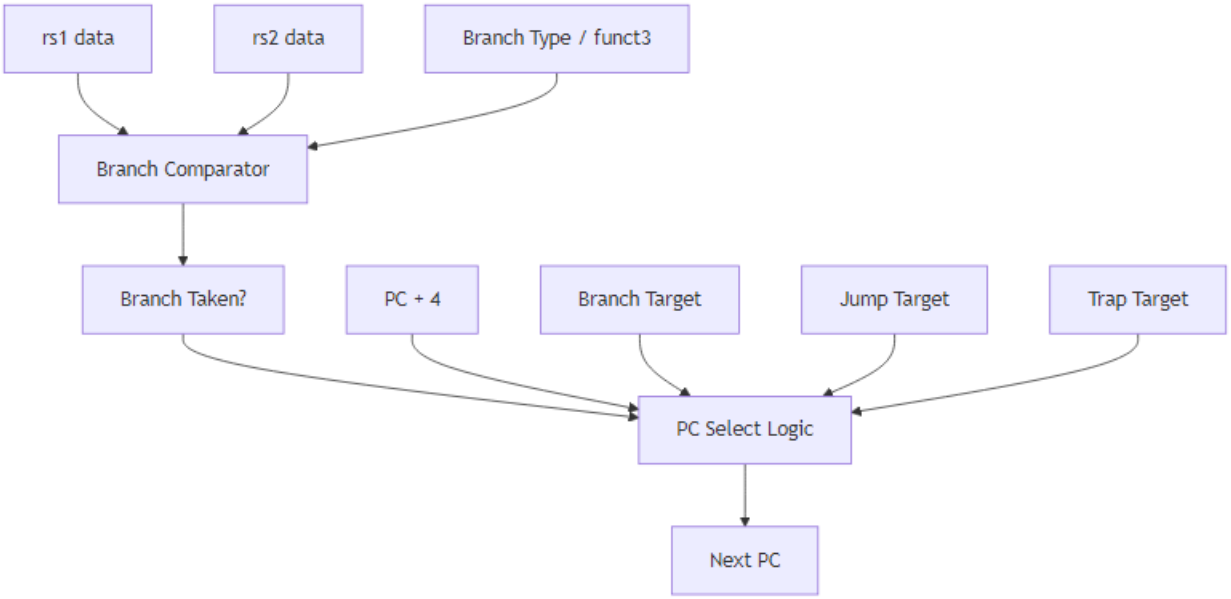
- equal
- not equal
- less than
- greater than or equal
- unsigned comparisons where required

It typically uses data from:

- register operand A
- register operand B
- branch subtype information from the decoded instruction

The result of the branch comparison is then combined with control signals to determine whether the PC will be updated with:

- `PC + 4` for no branch taken
- branch target address for branch taken
- jump target for unconditional control transfer
- trap target when exception/trap logic overrides normal flow



This structure ensures that control flow remains deterministic and that branch outcomes are resolved during the same cycle.

2.13 Load/Store Path

The CPU uses a **load/store architecture**, meaning only dedicated memory instructions access data memory. Arithmetic and logic instructions operate only on registers.

The **Load Store Unit (LSU)** is responsible for memory transaction handling.

Its main functions include:

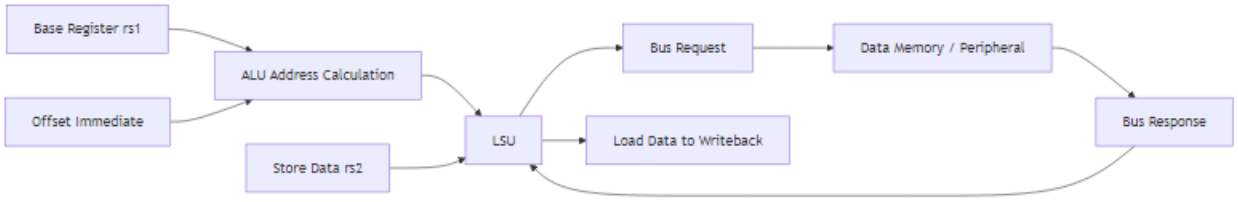
- using the ALU result as an effective address
- selecting memory read or write operation
- formatting store data
- interpreting load return data
- handling access size and data alignment rules as defined by the design
- interfacing with the bus interconnect

Store path:

- base register from `rs1`
- offset immediate from immediate generator
- ALU computes effective address
- register `rs2` supplies store data
- LSU forwards address and write data to the bus

Load path:

- base register from `rs1`
- offset immediate from immediate generator
- ALU computes effective address
- LSU requests a read transaction
- returned memory data is formatted and sent to writeback



The LSU is the bridge between CPU execution and the external memory-mapped SoC environment.

2.14 Writeback Path

At the end of instruction execution, the CPU may need to write a result back to the register file.

The writeback source depends on instruction type. Common writeback sources are:

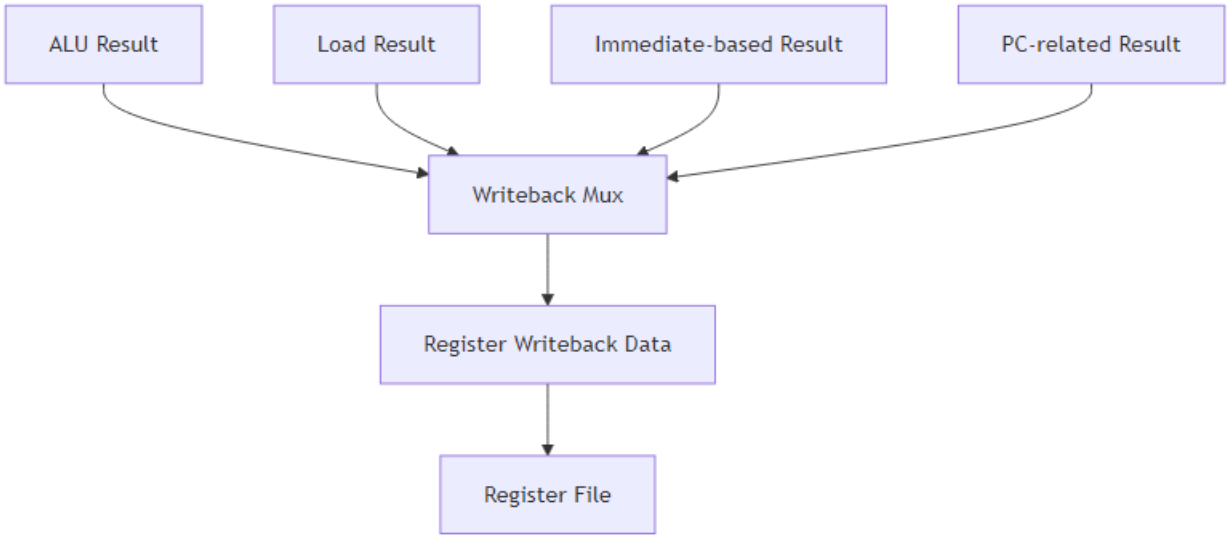
- ALU result

- load data returned by the LSU
- immediate-based value for upper-immediate class instructions
- PC-related result for jump-and-link style instructions

The writeback selection logic chooses the correct result and forwards it to:

- destination register index `rd`
- register file write data input
- register write enable signal

Conceptual writeback structure:



Not every instruction writes back:

- stores generally do not
- purely control-only instructions may not
- trap behavior may suppress or override normal writeback

2.15 Single-Cycle Timing Implications

The single-cycle model greatly simplifies control, but it also imposes a strict timing requirement:

- every instruction must finish all required work in one clock period

This means the clock period must be long enough to cover the worst-case path, which often includes:

- instruction memory access
- decode logic
- register file read
- ALU operation
- data memory or bus access
- writeback path
- PC decision logic

Advantages of single-cycle design:

- simple control logic
- easier debugging
- straightforward understanding of instruction flow
- good educational value

Limitations:

- slower maximum clock frequency
- all instructions take the same cycle time, even simple ones
- memory and peripheral access can become part of the critical path

This design tradeoff is reasonable for a minimal educational RV32I SoC.

2.16 CPU Interaction with the Bus Interconnect

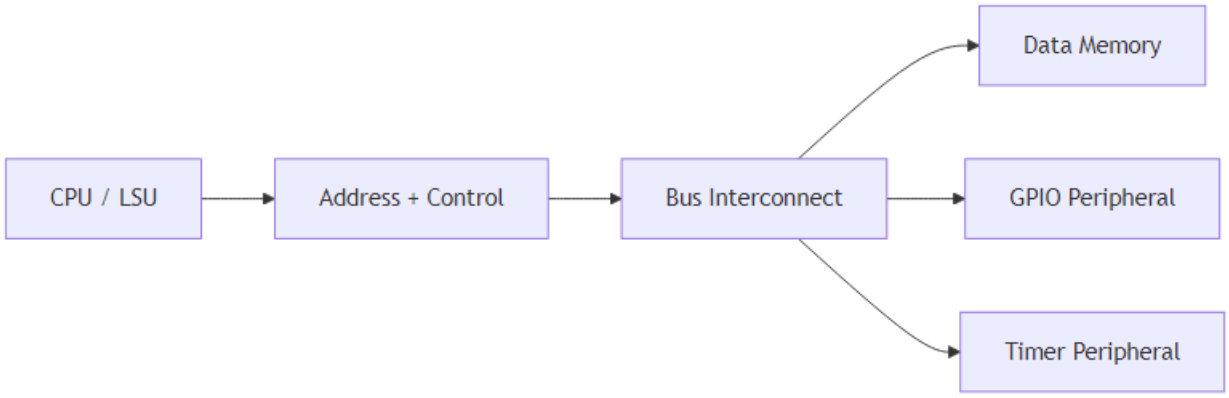
The CPU does not access all external resources directly. Instead, the memory access path passes through a **bus interconnect** that performs address decoding and dispatches the request to the correct target.

Possible destinations include:

- data RAM
- GPIO
- timer

The address produced by the ALU and processed by the LSU is used by the bus interconnect to determine which slave device should respond.

Conceptually:



This arrangement allows the CPU to treat memory and peripherals using the same general load/store mechanism, which is a common embedded SoC approach.

2.17 Trap Unit Integration

The **Trap Unit** adds basic exception or trap-related control capability to the CPU.

Its precise behavior depends on the RTL implementation, but architecturally its role is to:

- detect exceptional execution conditions
- redirect control flow when necessary
- prevent incorrect architectural state updates
- support future expansion toward interrupt/exception handling

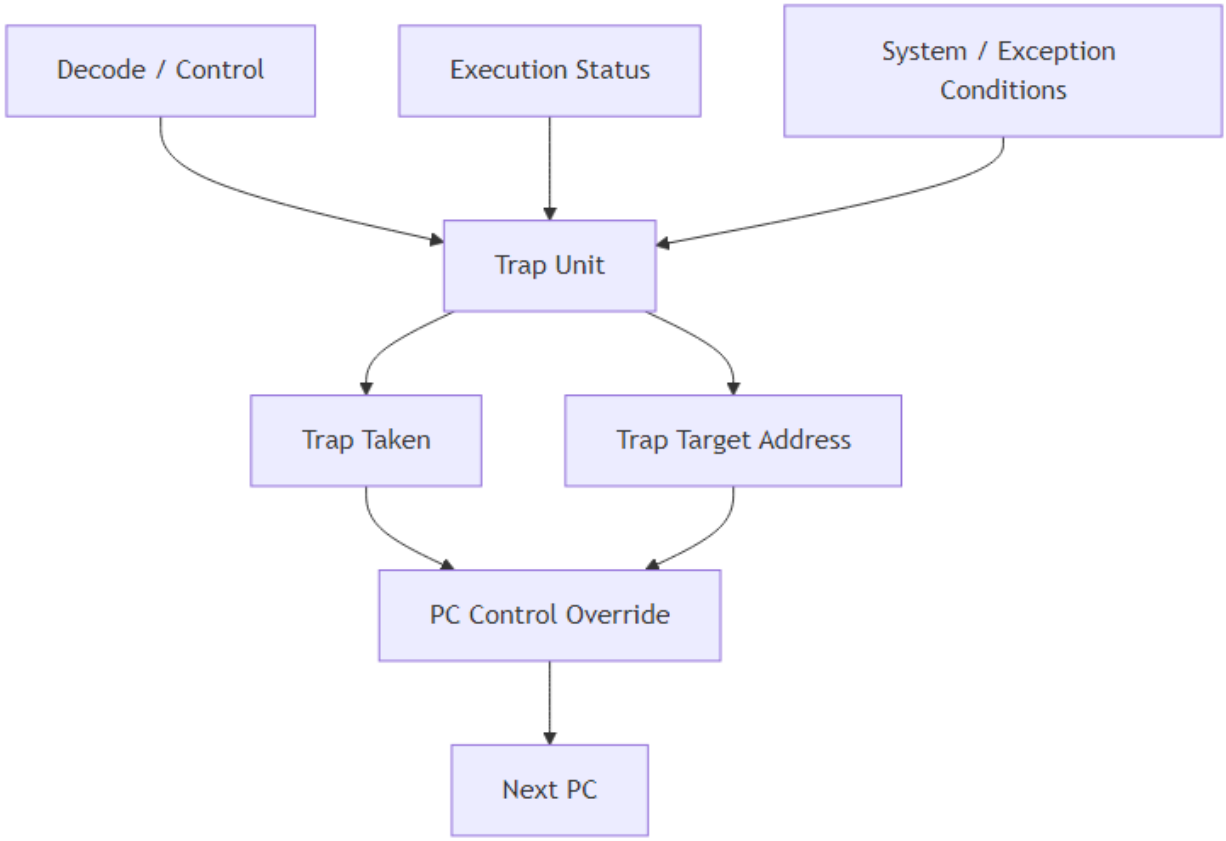
Possible trap-related causes in a minimal CPU may include:

- illegal instruction detection
- explicit trap instruction behavior
- invalid control conditions
- system-level event redirection

The trap unit can influence:

- next PC selection
- writeback enable or suppression
- control-flow override
- status signaling

Conceptually:



The presence of a trap unit is important because it shows the architecture is not limited to only straight-line execution; it already contains a structure for exceptional control flow.

2.18 Instruction Flow by Class

This section summarizes how each major instruction class moves through the CPU.

2.18.1 R-Type Arithmetic / Logic

Examples:

- add
- sub
- and
- or
- xor
- slt
- shift operations

Flow:

1. fetch instruction
2. decode as R-type
3. read `rs1` and `rs2`
4. ALU control selects operation
5. ALU computes result
6. result is written to `rd`
7. PC advances by 4

2.18.2 I-Type Arithmetic

Examples:

- addi
- andi
- ori
- xori
- slti
- shift-immediate operations

Flow:

1. fetch instruction
2. decode as I-type ALU instruction
3. read `rs1`
4. immediate generator creates 32-bit immediate
5. ALU uses register operand and immediate
6. result is written to `rd`
7. PC advances by 4

2.18.3 Load Instructions

Flow:

1. fetch instruction
2. decode as load

3. read base register `rs1`
4. generate offset immediate
5. ALU computes effective address
6. LSU issues read access
7. returned data is formatted
8. load result is written to `rd`
9. PC advances by 4

2.18.4 Store Instructions

Flow:

1. fetch instruction
2. decode as store
3. read base register `rs1`
4. read store data register `rs2`
5. generate offset immediate
6. ALU computes effective address
7. LSU issues write transaction
8. no register writeback occurs
9. PC advances by 4

2.18.5 Branch Instructions

Flow:

1. fetch instruction
2. decode as branch
3. read `rs1` and `rs2`
4. branch comparator evaluates condition
5. immediate generator creates branch offset
6. target address is computed
7. PC selects taken or fall-through path

2.18.6 Jump Instructions

Flow:

1. fetch instruction
2. decode as jump
3. immediate generator creates jump offset
4. target address is computed
5. link result may be prepared for writeback
6. PC is redirected to jump target

2.18.7 Upper Immediate Instructions

Examples:

- lui
- auipc

Flow depends on instruction subtype, but generally includes immediate construction, optional PC-relative arithmetic, and writeback to `rd`.

2.18.8 Trap / System-Related Instructions

Flow:

1. fetch instruction
2. decode trap/system semantics
3. trap unit evaluates action
4. control flow may be redirected
5. normal writeback may be modified or suppressed

2.19 Control and Data Separation

One useful way to understand the CPU is to separate **data paths** from **control paths**.

2.19.1 Data Path Elements

These blocks carry or transform operand data:

- program counter value
- instruction word
- register operands
- immediate values
- ALU results
- load data
- store data
- writeback data

2.19.2 Control Path Elements

These blocks decide what the datapath should do:

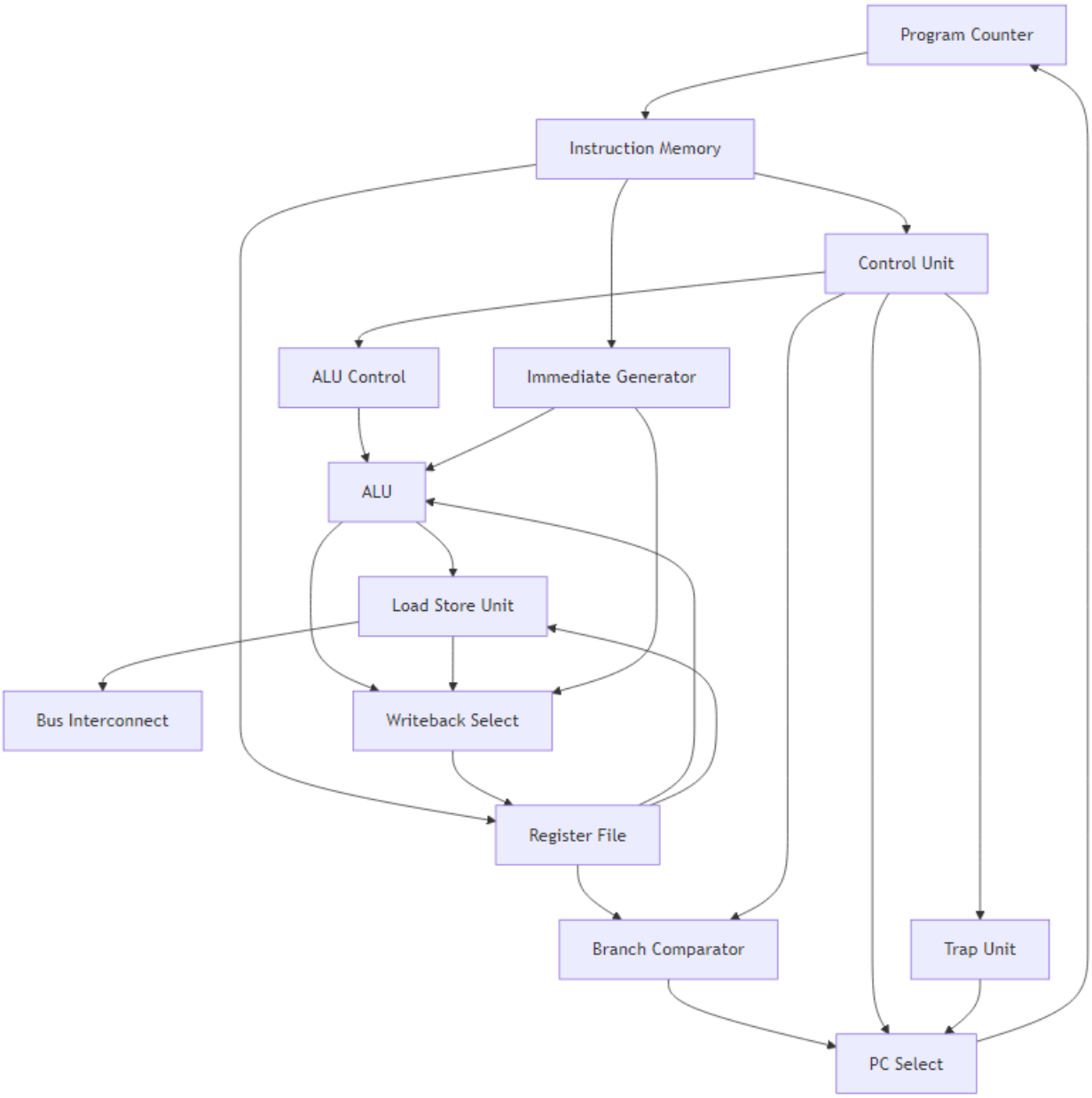
- control unit
- ALU control
- branch decision logic
- PC selection logic
- writeback select logic
- trap unit logic

This separation is important because it explains the architecture clearly:

- datapath moves and transforms values
- control path chooses which operation occurs

2.20 CPU Microarchitecture Summary Diagram

The following summary diagram combines the main CPU blocks and their interconnections.



This diagram represents the CPU as a coordinated set of combinational and state-holding blocks centered around:

- PC state
- register file state
- combinational execution logic
- single-cycle control resolution

2.21 Design Strengths

This CPU microarchitecture has several strengths for an educational and portfolio-oriented project.

2.21.1 Clarity

The single-cycle structure makes the instruction flow easy to follow and document.

2.21.2 Modularity

The CPU is divided into clearly named functional blocks:

- ALU
- control logic
- comparator
- LSU
- trap unit
- register file
- immediate generator

This is excellent for documentation, verification, and future refactoring.

2.21.3 SoC Integration

The CPU is not isolated; it is part of a complete SoC with:

- data memory
- GPIO
- timer
- bus interconnect

That makes the project much more valuable than a standalone ALU-only or toy CPU implementation.

2.22 Verification-Friendly Structure

Because the blocks are separated cleanly, they can be tested individually and then as part of CPU-level integration.

2.23 Limitations and Natural Future Extensions

Although the current microarchitecture is solid for its purpose, the single-cycle model naturally limits performance.

Main limitations:

- long critical path
- fixed one-cycle execution for all instructions
- bus and memory delay directly affect clock period
- no overlap between instruction phases

Natural future extensions could include:

- multi-cycle execution
- pipelining
- hazard handling

- forwarding logic
- interrupt subsystem expansion
- CSR support
- more advanced bus protocol support

These are future possibilities, but even without them the current design is already a complete and coherent RV32I educational core.

2.24 Final Summary

The CPU used in the **RV32I Minimal SoC** is a **32-bit single-cycle RV32I core** designed for clarity, modularity, and complete SoC integration.

Its microarchitecture is built around the cooperation of the following units:

- Program Counter for instruction sequencing
- Instruction Memory for fetch
- Control Unit for decode
- Register File for operand storage
- Immediate Generator for immediate reconstruction
- ALU Control for operation selection
- ALU for arithmetic and logic
- Branch Comparator for conditional control flow
- Load Store Unit for memory-mapped data access
- Trap Unit for exceptional control redirection

Together, these blocks implement a minimal but complete CPU capable of executing RV32I software in a memory-mapped SoC environment with RAM, GPIO, and timer peripherals.

This makes the processor an excellent educational design and a strong foundation for future architectural extensions.

3 ALU Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

3.1 Purpose of the Module

The **ALU (Arithmetic Logic Unit)** is the main combinational execution block of the CPU. Its role is to perform arithmetic, logical, comparison-related, and address-generation operations required by the RV32I instruction set.

Within the CPU microarchitecture, the ALU is used for:

- arithmetic instructions
- logical instructions
- shift operations
- comparison support

- effective address calculation for loads and stores
- branch-related arithmetic support
- writeback result generation

In a single-cycle processor, the ALU is one of the most important blocks because its output is often needed within the same cycle for both control decisions and data writeback.

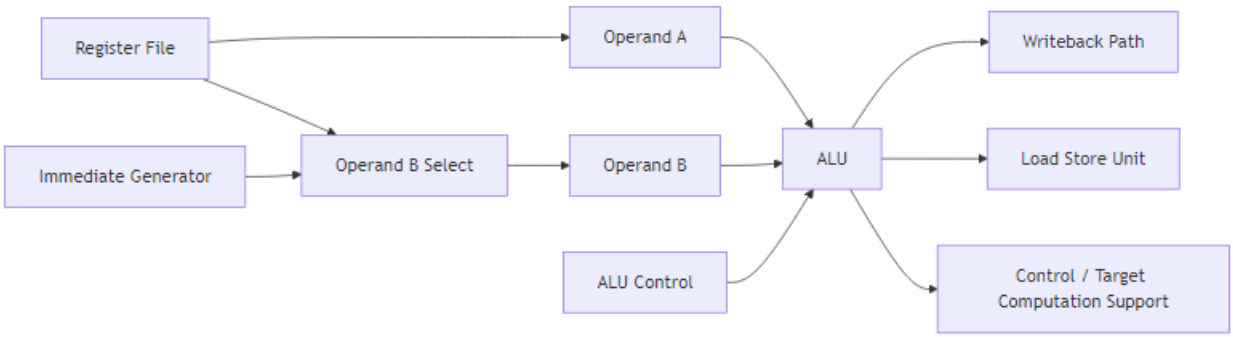
3.2 Architectural Role Inside the CPU

The ALU sits in the execution path between operand generation and result selection.

At a high level, the flow is:

1. the register file provides operands
2. the immediate generator may provide an alternative second operand
3. the control logic selects the ALU operation
4. the ALU computes the result
5. the result is used for writeback, memory addressing, or control flow support

Conceptually:



This makes the ALU the central execution engine of the CPU datapath.

3.3 Functional Responsibilities

The ALU is responsible for computing results for multiple instruction classes.

3.3.1 Arithmetic Operations

Typical arithmetic functions include:

- add
- subtract
- set-less-than
- set-less-than-unsigned

These operations are required by standard RV32I arithmetic instructions and also by address calculations.

3.3.2 Logical Operations

The ALU performs bitwise logical transformations such as:

- AND
- OR
- XOR

These are used by both register-register and register-immediate instructions.

3.3.3 Shift Operations

Shift operations supported in RV32I typically include:

- logical left shift
- logical right shift
- arithmetic right shift

These operations are essential for bit manipulation and low-level software.

3.3.4 Address Calculation

For load and store instructions, the ALU computes:

- effective address = base register + immediate offset

This means memory addressing uses the same execution hardware as arithmetic.

3.3.5 Comparison Support

Although branch decisions may be resolved by a dedicated branch comparator, the ALU still plays a role in signed/unsigned interpretation and in general comparison-related execution support depending on the design partitioning.

3.4 Inputs and Outputs

The exact RTL port names come from the VHDL source, but architecturally the ALU module is expected to expose the following categories of signals.

3.4.1 Inputs

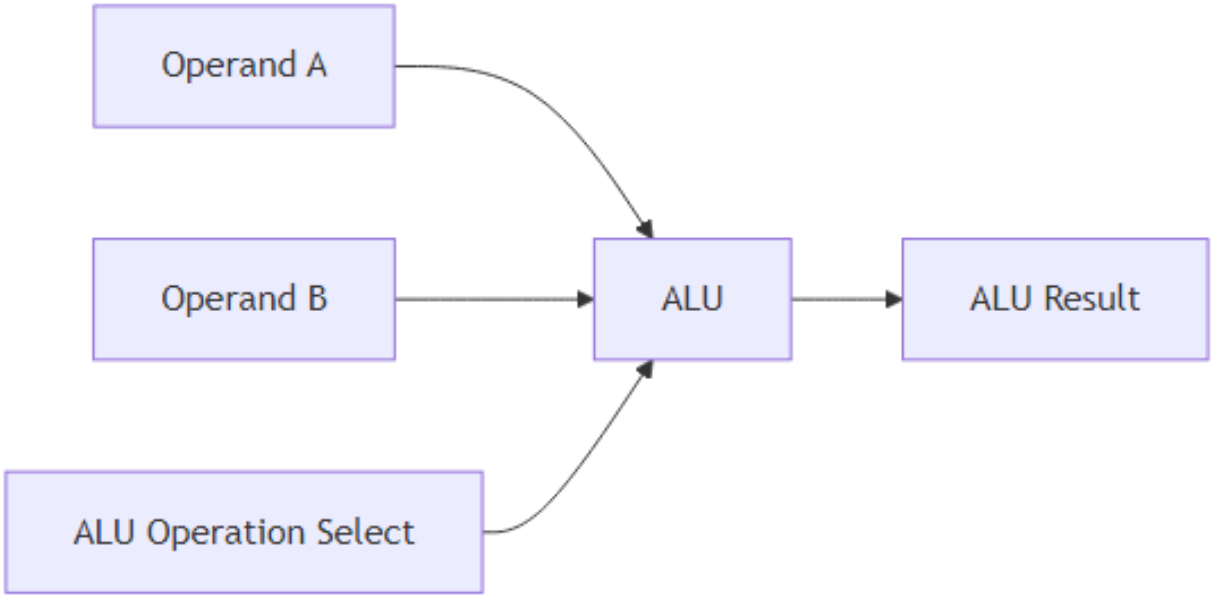
- operand A
- operand B
- ALU operation selector

3.4.2 Outputs

- result
- optional status or comparison-related outputs depending on implementation

The most important output is the 32-bit result value.

Conceptually:



In most RV32I datapaths, the ALU is purely combinational.

3.5 Data Width and Numeric Interpretation

The ALU operates on **32-bit values**, matching the RV32I architecture.

Important numeric interpretations include:

- unsigned arithmetic and logic
- signed comparisons where required
- unsigned comparisons where required
- shift amount handling, usually based on the lower relevant bits

Because RV32I is a 32-bit ISA, all normal ALU outputs are 32 bits wide.

The interpretation of the same bit pattern may vary depending on operation:

- `0xFFFFFFFF` can be treated as a large unsigned number
- the same value can also represent `-1` in signed arithmetic

This distinction matters especially for:

- signed compare
- unsigned compare
- arithmetic right shift
- overflow-insensitive integer behavior

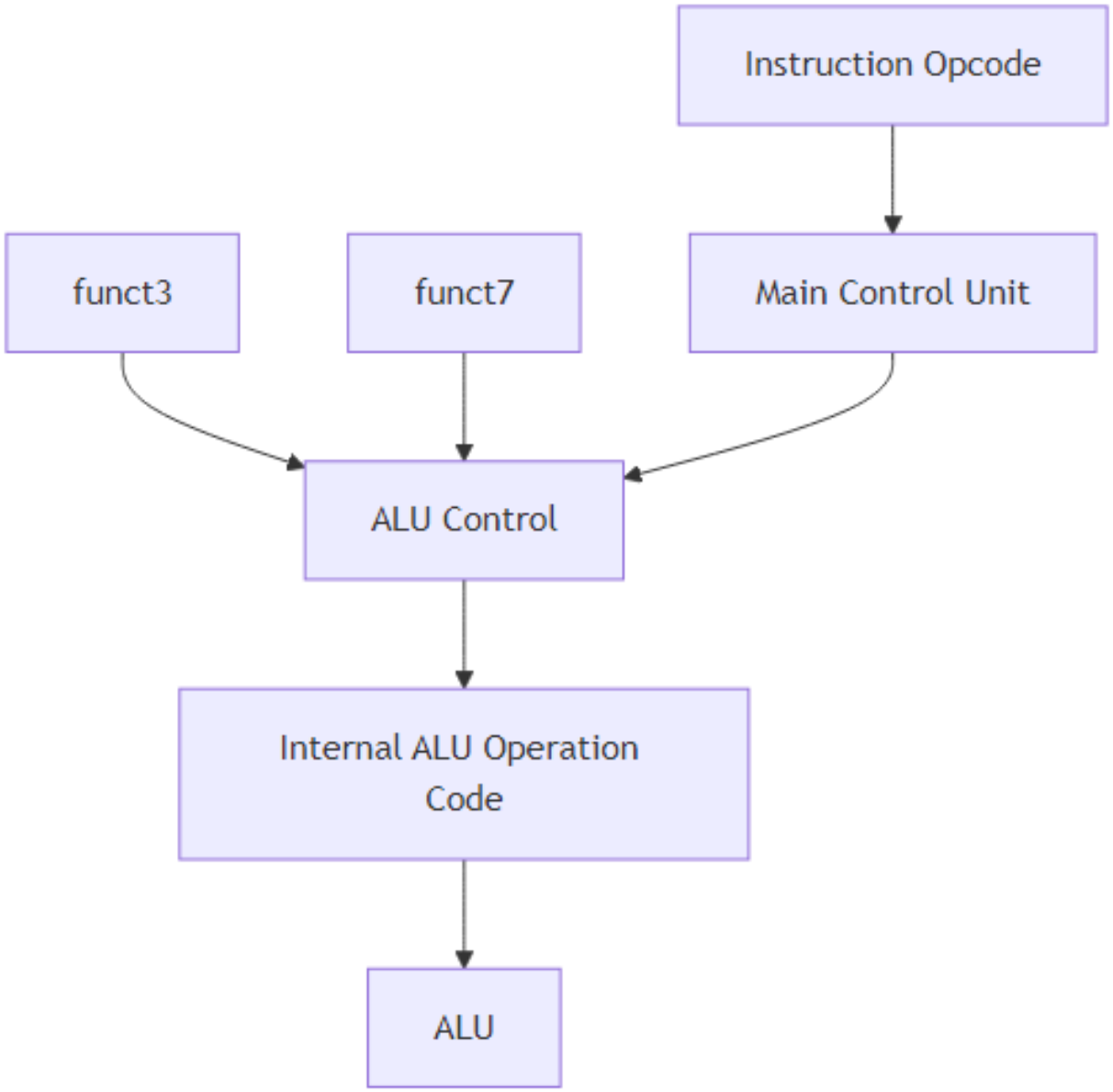
3.6 ALU Control Relationship

The ALU does not decide by itself which operation to execute. That decision comes from the **ALU Control** logic.

The control path typically works as follows:

1. the main control unit identifies the instruction class
2. `funct3` and `funct7` refine the requested operation
3. ALU Control emits a compact internal ALU operation code
4. the ALU decodes that control code and performs the requested function

Conceptually:



This separation improves modularity:

- the control unit remains instruction-oriented
- the ALU remains operation-oriented

3.7 Operations Typically Implemented for RV32I

A minimal RV32I ALU usually supports the following operation families.

3.7.1 Addition

Used by:

- add
- addi
- load address calculation
- store address calculation
- PC-relative arithmetic in some instruction paths

Mathematically:

- result = A + B

3.7.2 Subtraction

Used by:

- sub
- comparison support
- possible branch support depending on design style

Mathematically:

- result = A - B

3.7.3 Bitwise AND

- result = A AND B

3.7.4 Bitwise OR

- result = A OR B

3.7.5 Bitwise XOR

- result = A XOR B

3.7.6 Shift Left Logical

- result = A shifted left by shift amount

3.7.7 Shift Right Logical

- result = A shifted right with zero fill

3.7.8 Shift Right Arithmetic

- result = A shifted right with sign extension

3.7.9 Set Less Than (Signed)

- result = 1 if A < B in signed interpretation
- result = 0 otherwise

3.7.10 Set Less Than Unsigned

- result = 1 if A < B in unsigned interpretation
- result = 0 otherwise

These operations cover the essential RV32I arithmetic/logic requirements.

3.8 ALU in Different Instruction Classes

The ALU is reused across multiple instruction families.

3.8.1 R-Type Instructions

Examples:

- add
- sub
- and
- or
- xor
- sll
- srl
- sra
- slt
- sltu

Flow:

1. read `rs1`
2. read `rs2`
3. decode operation
4. ALU computes result
5. write result to `rd`

3.8.2 I-Type Arithmetic Instructions

Examples:

- addi
- andi
- ori
- xori
- slti
- sltiu
- slli
- srli

- srai

Flow:

1. read `rs1`
2. generate immediate
3. ALU uses register operand and immediate
4. result is written to `rd`

3.8.3 Load Instructions

The ALU is used to compute the effective address:

- address = base register + offset immediate

The load data itself does not come from the ALU, but the address does.

3.8.4 Store Instructions

The ALU computes the target memory address:

- address = base register + offset immediate

The store data comes from the register file, not the ALU result.

3.8.5 Branch and Jump Support

Depending on design partitioning, the ALU may also support:

- branch target arithmetic
- PC-relative address calculation
- control-flow target generation

Even when a dedicated comparator is used for the actual branch decision, the ALU often still participates in forming target addresses.

3.9 Combinational Nature of the Module

The ALU is generally designed as a **pure combinational block**.

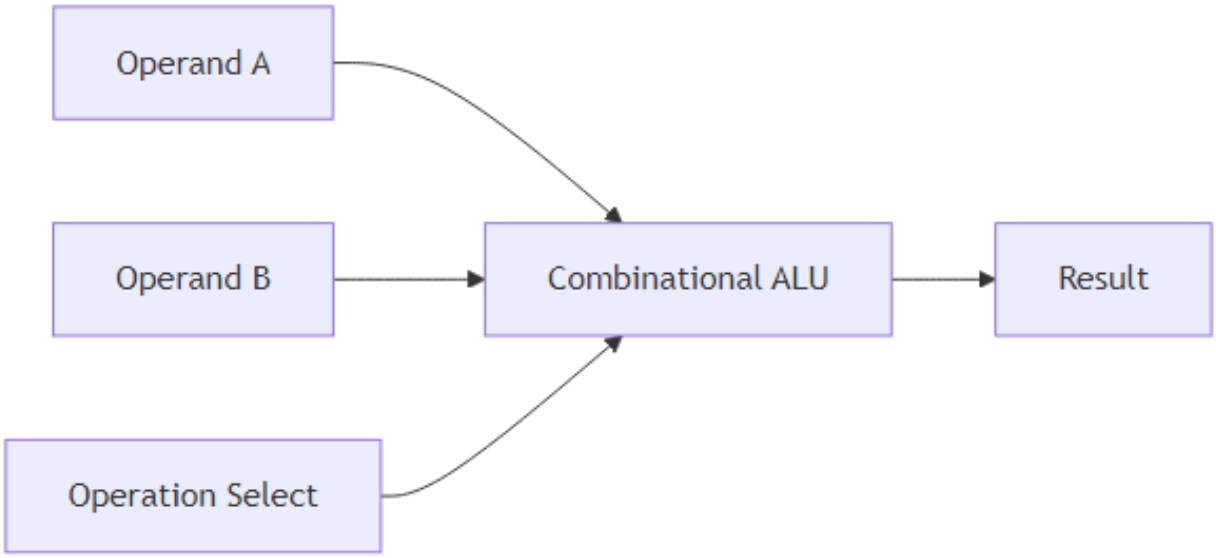
This means:

- no internal state
- no clock input required for result computation
- output depends only on current inputs

Advantages of a combinational ALU:

- immediate response in the same cycle
- simple verification
- natural fit for a single-cycle datapath

Conceptually:



Because the CPU is single-cycle, this combinational behavior is essential.

3.10 Role in the Critical Path

In a single-cycle architecture, the ALU often lies on the critical timing path.

A typical worst-case path may include:

1. PC output
2. instruction memory read
3. decode and control generation
4. register file read
5. operand selection
6. ALU computation
7. memory access or writeback selection
8. register file write or PC update

Because the ALU sits near the center of that path, its implementation directly influences the maximum clock frequency.

This is one reason why ALU design quality is so important even in a simple CPU.

3.11 Signed vs Unsigned Behavior

One subtle but very important part of ALU design is handling signed and unsigned interpretations correctly.

3.11.1 Signed Operations

Examples:

- SLT
- arithmetic right shift
- some subtraction-based comparisons

These operations treat the operands as signed two's complement numbers.

3.11.2 Unsigned Operations

Examples:

- SLTU
- logical shifts
- ordinary bitwise logic

These treat the operands as raw bit patterns or unsigned numbers.

A correct RV32I ALU must preserve this distinction. The hardware may operate on the same 32 physical bits, but the interpretation depends on the selected operation.

3.12 Shift Behavior

Shift operations deserve separate attention because they use one operand as a data value and another as a shift amount.

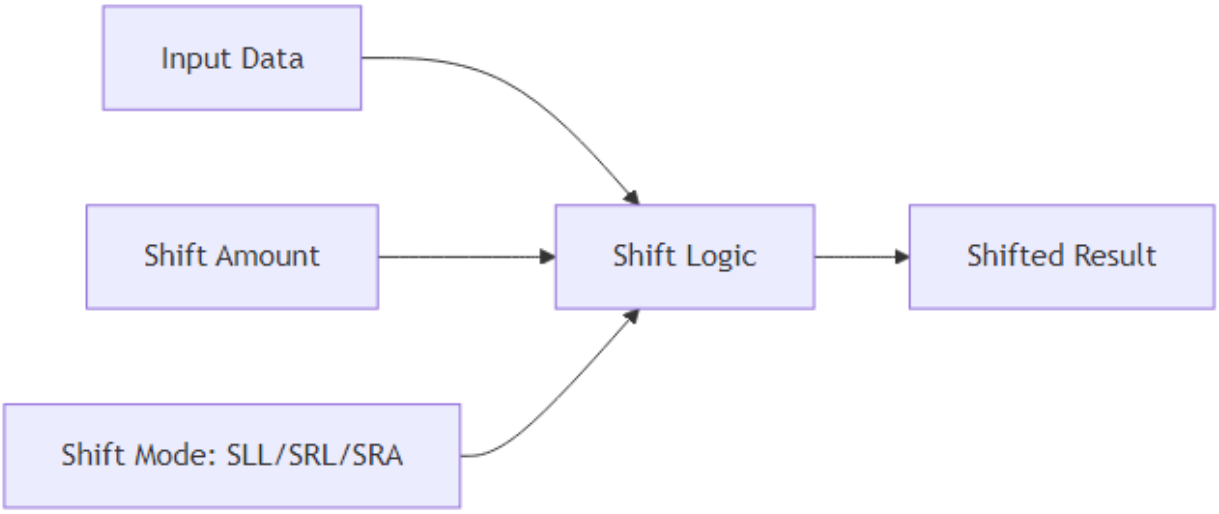
Typical behavior:

- left shift fills with zeros
- logical right shift fills with zeros
- arithmetic right shift replicates the sign bit

In RV32I, the effective shift amount is typically derived from the lower bits of the second operand or immediate.

This ensures the shift range matches a 32-bit architecture.

Conceptually:



Shift correctness is important because many software routines rely on exact bit-level behavior.

3.13 Result Usage in the Datapath

The ALU result can feed different destinations depending on instruction type.

3.13.1 Register Writeback

For arithmetic and logical instructions, the ALU result is written to `rd`.

3.13.2 Memory Address Bus

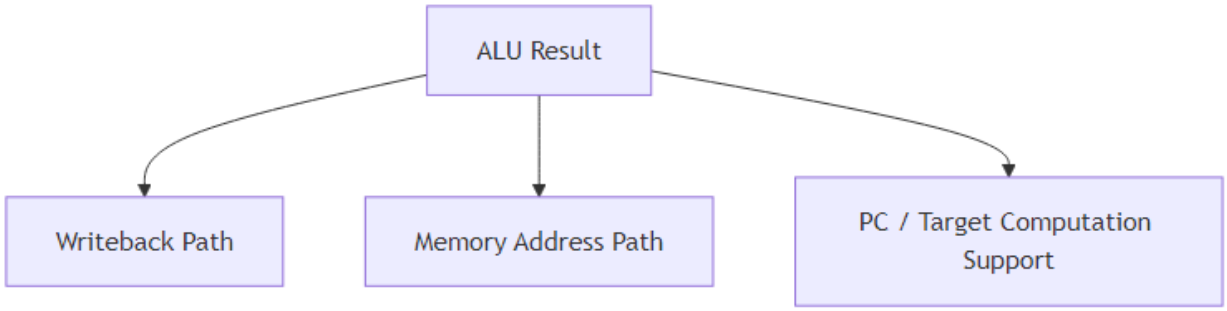
For load/store instructions, the ALU result becomes the effective memory address.

3.13.3 Control-Flow Support

For some instruction paths, the ALU result may contribute to:

- branch target formation
- jump target calculation
- PC-relative computations

Conceptually:



This reuse is one reason the ALU is such a central module in the processor.

3.14 Interaction with Other Modules

The ALU interacts closely with several other CPU modules.

3.14.1 Program Counter

Indirect interaction through control-flow logic and target address computation.

3.14.2 Instruction Memory

No direct arithmetic interaction, but instruction fields determine the ALU operation.

3.14.3 Control Unit

Provides high-level instruction class control information.

3.14.4 ALU Control

Provides the specific execution function code.

3.14.5 Register File

Supplies input operands.

3.14.6 Immediate Generator

Supplies immediate-based second operands.

3.14.7 Branch Comparator

May complement ALU functionality in branch-related decisions.

3.14.8 Load Store Unit

Consumes ALU results as addresses.

3.14.9 Trap Unit

Can override normal execution flow, affecting how ALU results are ultimately used.

3.15 Verification Expectations

A well-designed ALU testbench should verify every supported operation class.

Typical verification targets:

- addition correctness
- subtraction correctness
- signed comparison correctness
- unsigned comparison correctness
- bitwise logic correctness
- shift operation correctness
- corner cases such as zero, all ones, sign boundaries, and large shift values

Representative test scenarios include:

- positive + positive
- positive + negative
- subtraction yielding zero
- signed less-than with negative operands
- unsigned less-than with high-bit differences
- right arithmetic shift of negative values
- left shift by various amounts

- logic operations with complementary bit patterns

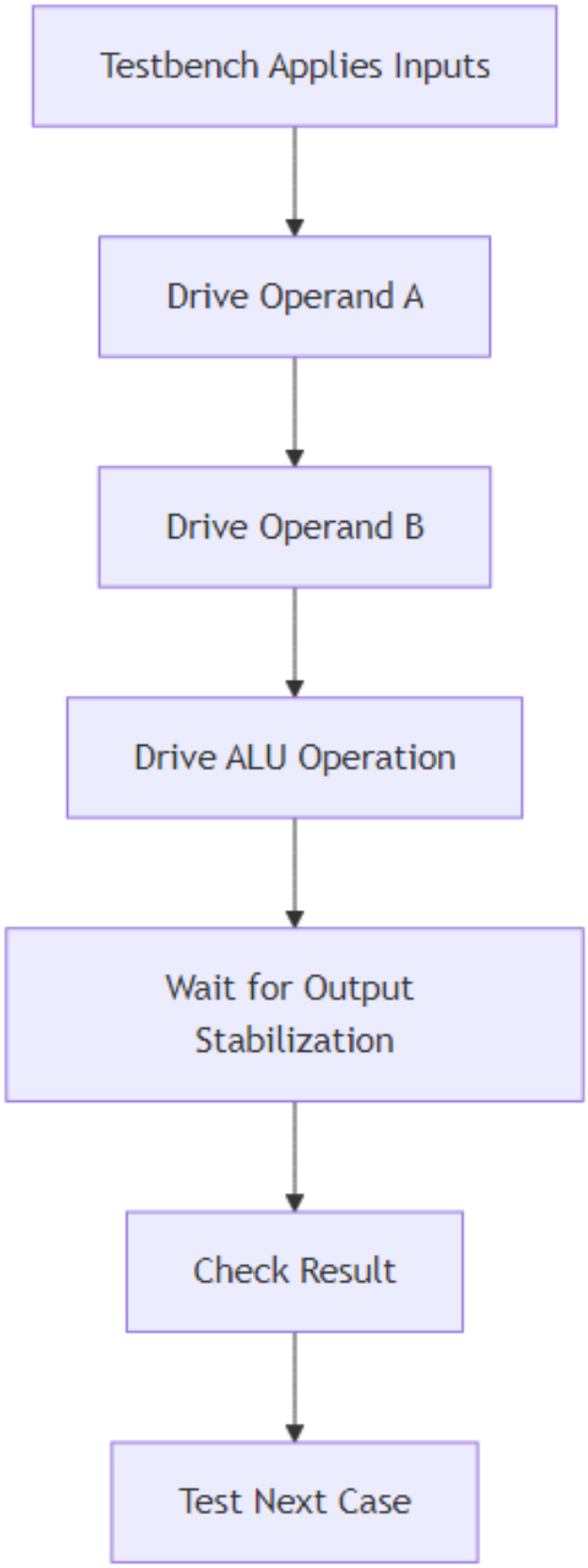
The ALU is usually one of the easiest modules to unit-test because it is combinational and has a clean input-output relationship.

3.16 Example Verification Flow

A simple ALU verification strategy may follow this pattern:

1. drive operand A
2. drive operand B
3. select operation
4. wait for combinational stabilization
5. compare result against expected value
6. repeat for all operation classes

Conceptually:


Testbench Applies Inputs
Drive Operand A
Drive Operand B
Drive ALU Operation
Wait for Output Stabilization
Check Result
Test Next Case

Because the ALU is deterministic and stateless, this style of testing is very effective.

3.17 Strengths of This Module in the Project

The ALU is one of the strongest modules from a documentation and portfolio perspective.

3.17.1 Clear Functionality

Its purpose is easy to understand and maps directly to ISA behavior.

3.17.2 High Reuse

It supports multiple instruction classes with one unified hardware block.

3.17.3 Strong Educational Value

The ALU demonstrates:

- combinational logic design
- arithmetic hardware behavior
- signed vs unsigned reasoning
- operation decoding
- datapath integration

3.17.4 Excellent Verification Candidate

Because it is local, deterministic, and central, it is ideal for detailed testbench documentation.

3.18 Design Limitations and Tradeoffs

Even though the ALU is powerful within this project, some tradeoffs naturally exist in a minimal single-cycle CPU.

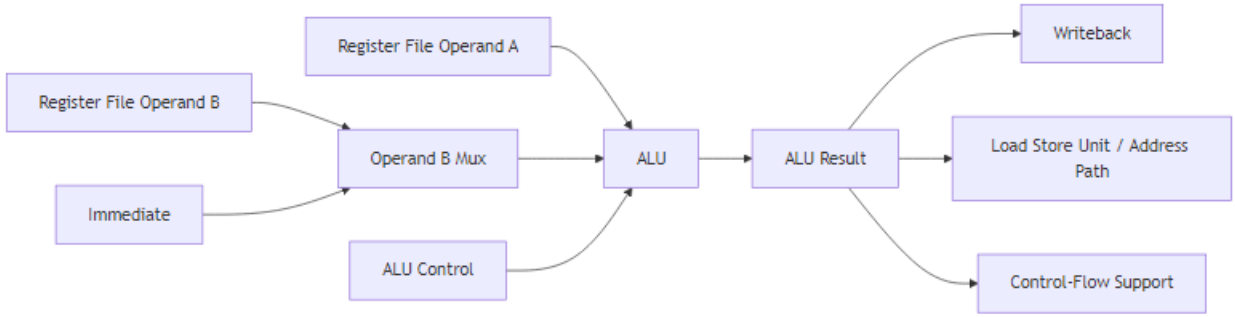
Possible limitations include:

- no pipelined execution inside the ALU
- no multiplication or division support in base RV32I
- timing sensitivity in the single-cycle critical path
- dependency on clean ALU control decoding

These are acceptable tradeoffs for a minimal RV32I educational processor.

3.19 Summary Diagram

The following summary diagram captures the ALU's place in the CPU.



This shows the ALU as the central execution block receiving operands and control, then driving multiple downstream CPU paths.

3.20 Final Summary

The **ALU module** is the central execution engine of the **RV32I Minimal SoC** CPU.

It performs:

- arithmetic operations
- logical operations
- shift operations
- comparison-related operations
- effective address generation

The ALU receives operands from the register file and immediate generator, receives operation selection from ALU control, and produces results used for writeback, memory addressing, and control-flow support.

In a single-cycle RV32I processor, this module is one of the most critical elements both architecturally and in timing terms. Its correctness is essential for proper CPU behavior, and its modularity makes it an excellent building block for documentation, verification, and future architectural extensions.

4 ALU Control Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

4.1 Purpose of the Module

The **ALU Control** module translates decoded instruction information into a specific operation command for the ALU.

Its role is to sit between:

- the high-level instruction decode logic
- the low-level arithmetic and logical execution hardware

In other words:

- the **Control Unit** decides the general class of instruction
- the **ALU Control** decides the exact ALU function to execute

This separation is a classic microarchitectural design choice because it keeps the CPU modular and easier to understand.

The ALU Control is therefore responsible for turning instruction-specific information such as opcode class, `funct3`, and `funct7` into a compact, internal ALU operation code.

4.2 Architectural Role Inside the CPU

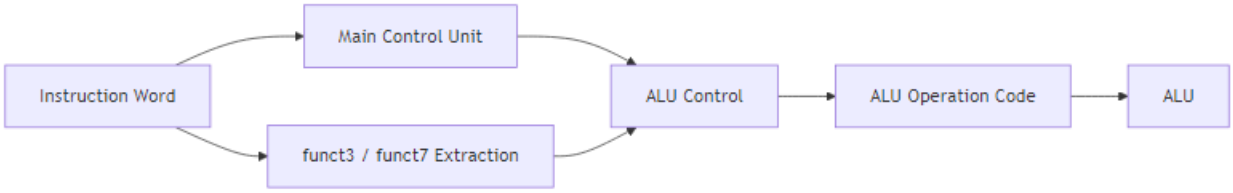
The ALU Control belongs to the CPU control path.

It does not manipulate data values directly. Instead, it determines how the ALU should interpret its operands.

The high-level execution flow is:

1. instruction is fetched
2. control unit classifies the instruction
3. instruction fields such as `funct3` and `funct7` are extracted
4. ALU Control determines the exact ALU operation
5. ALU executes that function on its operands

Conceptually:



This means ALU Control is one of the central modules that converts ISA encoding into real hardware behavior.

4.3 Why This Module Exists

Without ALU Control, the main control unit would need to directly encode every single arithmetic and logical variation of every instruction.

That would make the design:

- more complex
- harder to extend
- harder to document
- harder to verify

By splitting responsibilities:

4.3.1 Main Control Unit

Handles instruction categories such as:

- ALU register-register operation
- ALU immediate operation
- load
- store
- branch
- jump
- upper-immediate
- trap/system behavior

4.3.2 ALU Control

Handles execution details such as:

- add vs sub
- and vs or vs xor
- logical vs arithmetic shift
- signed vs unsigned compare

This separation reduces complexity and improves readability.

4.4 Inputs and Outputs

The exact VHDL port names belong to the RTL implementation, but architecturally the ALU Control is expected to consume control and instruction-detail signals.

4.4.1 Inputs

Typical input categories include:

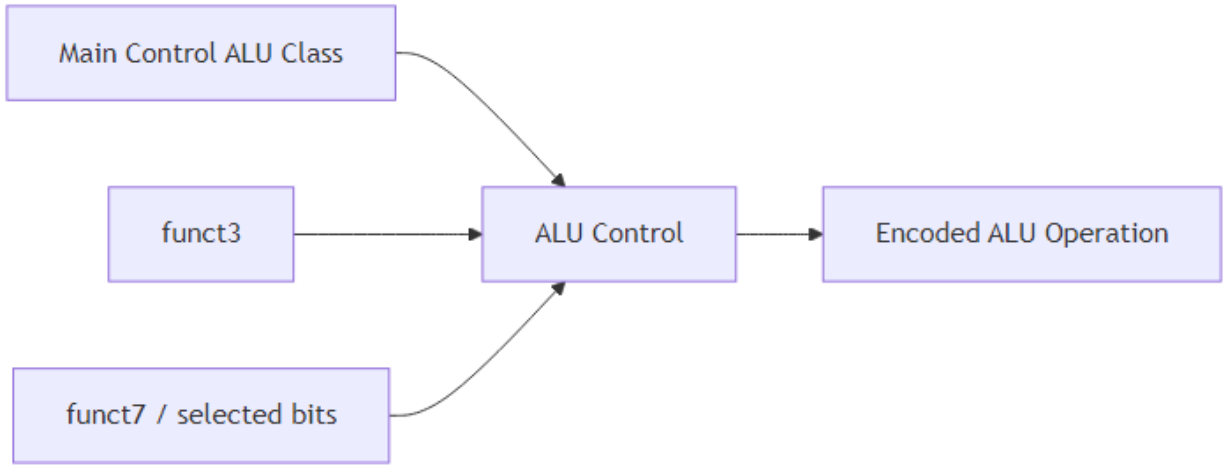
- main ALU operation classification from the Control Unit
- `funct3`
- `funct7` or selected bits from it
- possibly opcode-derived qualifiers

4.4.2 Outputs

Typical output category:

- compact ALU operation code sent to the ALU

Conceptually:



The output is usually much simpler than the original instruction fields, because it only needs to tell the ALU what to do.

4.5 Relationship with RV32I Instruction Encoding

RV32I arithmetic and logical instructions reuse a common structural pattern.

For example:

- many R-type instructions share the same opcode
- several I-type ALU instructions also share another opcode
- their specific behavior is distinguished by `funct3`
- some operations are further distinguished by bits in `funct7`

This is exactly why ALU Control exists.

For example, two instructions can belong to the same general ALU class but still require different execution behaviors:

- ADD and SUB
- SRL and SRA
- SLT and SLTU

The main control logic alone cannot cleanly capture all of those distinctions without becoming overly detailed.

ALU Control resolves this by combining:

- broad instruction category
- function subfields
- internal execution rules

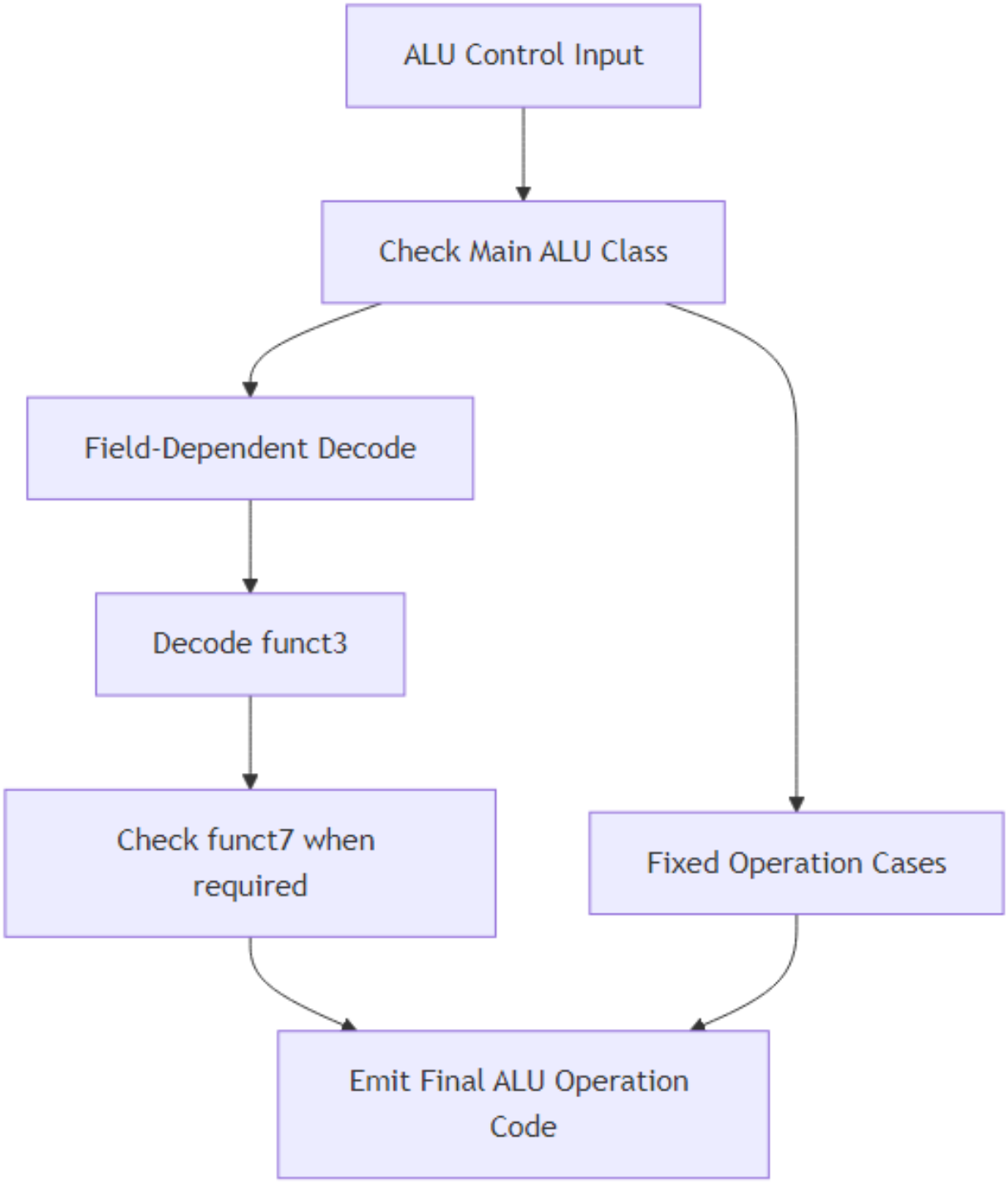
4.6 Typical Decision Structure

The ALU Control logic can be understood as a decision tree.

At a high level:

1. identify whether the instruction is an ALU-style operation or a memory/address calculation
2. determine whether the ALU should do a fixed operation or a field-dependent operation
3. inspect `funct3`
4. inspect the relevant `funct7` bit or bits when needed
5. emit the final ALU operation code

Conceptually:



This kind of logic is usually implemented as a combinational process or combinational selection network in VHDL.

4.7 Operation Classes Usually Selected

A minimal RV32I ALU Control commonly selects the following ALU operations.

4.7.1 ADD

Used by:

- add
- addi
- load address calculation
- store address calculation
- some PC-relative arithmetic paths depending on architecture

4.7.2 SUB

Used by:

- sub
- sometimes branch-related arithmetic support depending on design style

4.7.3 AND

Used by:

- and
- andi

4.7.4 OR

Used by:

- or
- ori

4.7.5 XOR

Used by:

- xor
- xori

4.7.6 SLL

Used by:

- sll
- slli

4.7.7 SRL

Used by:

- srl
- srli

4.7.8 SRA

Used by:

- sra
- srai

4.7.9 SLT

Used by:

- slt
- slti

4.7.10 SLTU

Used by:

- sltu
- sltiu

These operations form the core mapping responsibility of the ALU Control module.

4.8 Fixed vs Fully Decoded Cases

Not all instruction classes need full `funct3` / `funct7` decoding.

4.8.1 Fixed Operation Cases

Some instructions always require the same ALU behavior, regardless of the function fields.

Examples:

- load instructions usually require ADD for address generation
- store instructions usually require ADD for address generation
- certain PC-relative calculations may require ADD

In these cases, ALU Control may simply emit:

- ADD

without deeper decoding.

4.8.2 Fully Decoded Cases

For R-type and I-type ALU operations, deeper decoding is necessary because the same general instruction class contains multiple operations.

This is where `funct3` and `funct7` become essential.

4.9 Example Decode Reasoning

Although the exact encoding depends on the RTL implementation, the reasoning usually follows patterns like these.

4.9.1 R-Type ALU Instructions

General logic:

- main control says: register-register ALU operation
- `funct3` selects the operation family
- `funct7` distinguishes variants where needed

Examples:

- `funct3 = add/sub family`, `funct7 = normal` → ADD
- `funct3 = add/sub family`, `funct7 = subtract variant` → SUB
- `funct3 = shift-right family`, `funct7 = logical` → SRL
- `funct3 = shift-right family`, `funct7 = arithmetic` → SRA

4.9.2 I-Type ALU Instructions

General logic:

- main control says: register-immediate ALU operation
- `funct3` selects most operations directly
- for shift-immediate instructions, selected bits from `funct7` distinguish SRLI from SRAI

This demonstrates why ALU Control is required even in a relatively simple CPU.

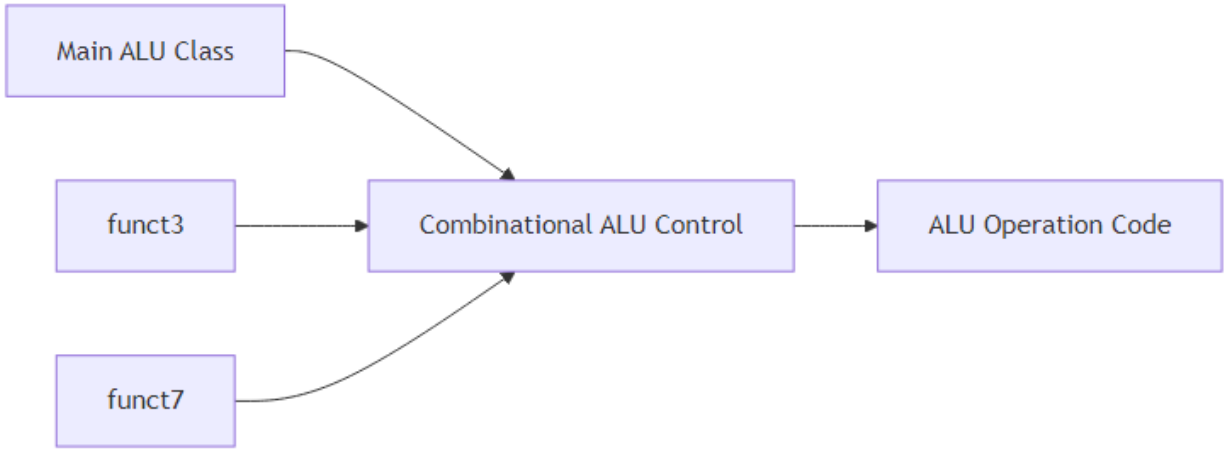
4.10 Combinational Nature of the Module

ALU Control is usually implemented as a **pure combinational block**.

This means:

- no state storage
- no clocked behavior required for its main decision
- output changes immediately in response to its inputs

Conceptually:



This is important in a single-cycle processor because the ALU operation must be available within the same cycle as decode and execution.

4.11 Position in the Control Path

It is useful to understand the ALU Control as part of the broader control path.

4.11.1 Data Path Blocks

These manipulate values:

- register file
- immediate generator
- ALU
- load/store unit

4.11.2 Control Path Blocks

These decide what the datapath should do:

- control unit
- ALU Control
- branch decision logic
- PC selection logic
- trap logic

The ALU Control is therefore a control refinement stage.

It does not choose whether the instruction is a load or a store.
It chooses what exact arithmetic/logical function the ALU must perform.

4.12 Interaction with the Main Control Unit

The ALU Control and main Control Unit work together closely.

4.12.1 Main Control Unit Responsibilities

The main Control Unit decides things such as:

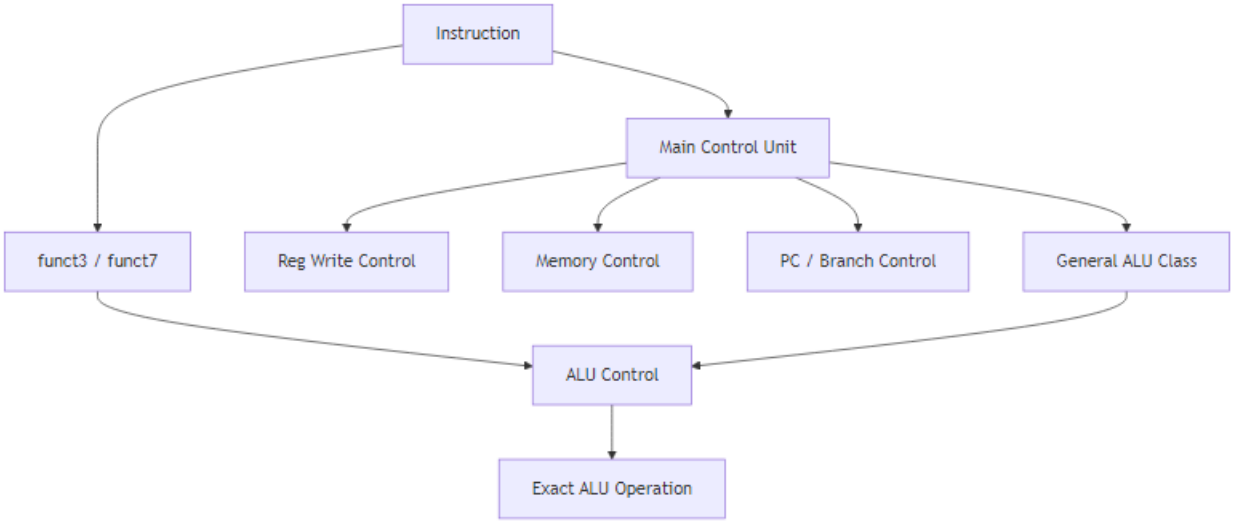
- should the instruction write a register?
- should memory be read?
- should memory be written?
- should an immediate be used?
- is the instruction a branch or jump?
- what is the general ALU behavior class?

4.12.2 ALU Control Responsibilities

The ALU Control takes that general class and resolves:

- exact arithmetic function
- exact logic function
- exact compare-related execution function
- shift mode differences

Conceptually:



This hierarchy keeps the CPU well-structured.

4.13 Interaction with the ALU

The ALU Control output must match the internal expectation of the ALU module.

That means both modules must agree on:

- which encoded control values exist
- what each value means
- which operations are supported

For example, if the ALU Control emits a code meaning `ADD`, the ALU must interpret that exact code as addition.

This makes ALU Control and ALU tightly coupled at the interface level, even though they remain separate modules architecturally.

4.14 Handling of Comparison-Related Operations

RV32I includes signed and unsigned comparison instructions such as:

- SLT
- SLTU
- SLTI
- SLTIU

The ALU Control must distinguish these carefully because they may use similar instruction structures but different numeric interpretation.

This distinction is very important because:

- signed comparison treats the most significant bit as a sign bit
- unsigned comparison treats it as part of magnitude

A correct ALU Control must ensure the ALU performs the intended interpretation.

4.15 Handling of Shift Variants

Shift instructions are another area where ALU Control is especially important.

Shift families include:

- shift left logical
- shift right logical
- shift right arithmetic

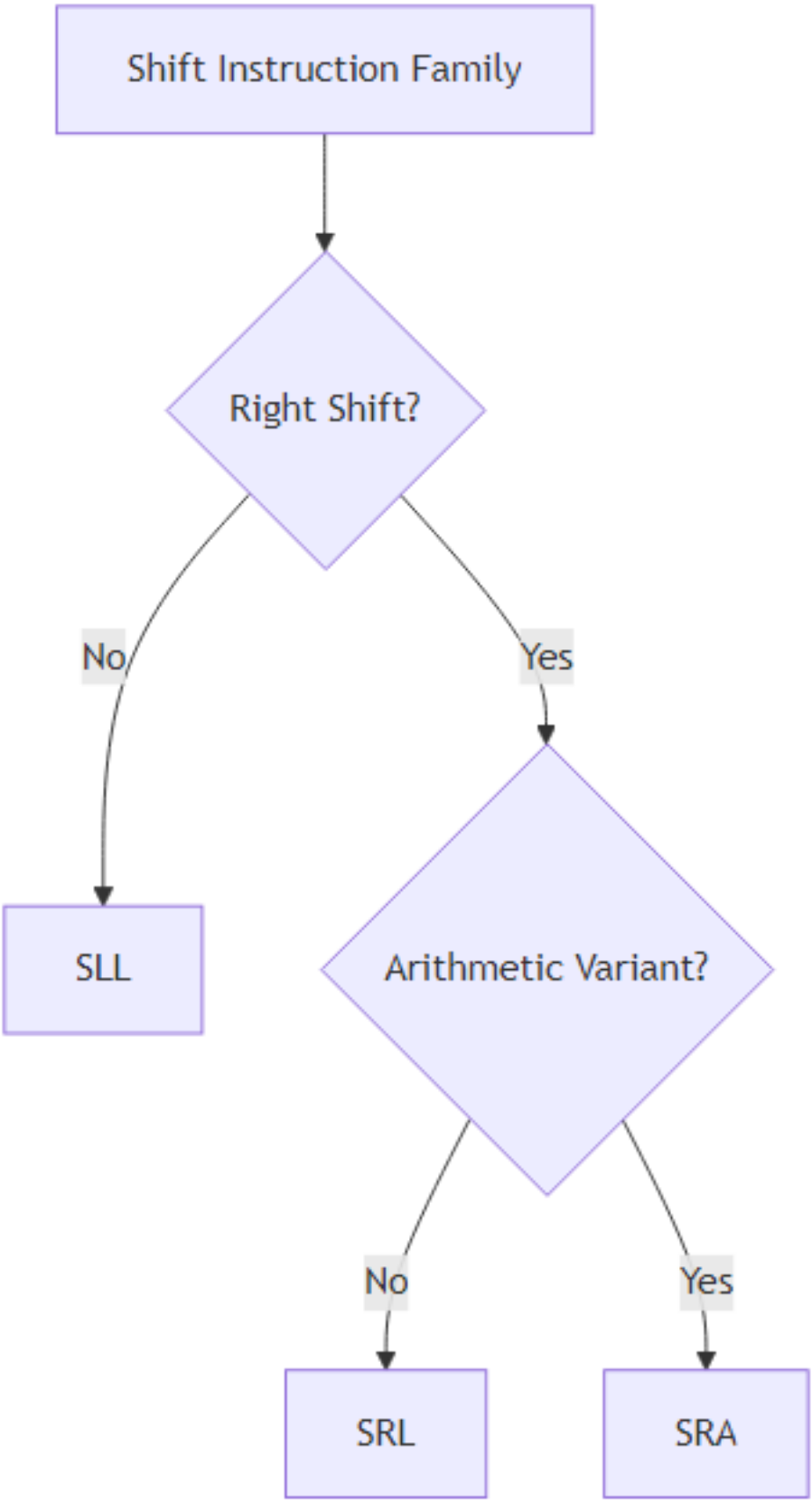
The left-shift family is usually simple to decode, but right shifts often require variant discrimination.

For example:

- one encoding means logical shift right
- another closely related encoding means arithmetic shift right

The ALU Control resolves this ambiguity using the appropriate function bits.

Conceptually:


Shift Instruction Family
Right Shift?
No
Yes
SLL
Arithmetic Variant?
No
Yes
SRL
SRA

This is one of the clearest examples of why the module exists.

4.16 Use in Address Calculation Instructions

Loads and stores generally need only one ALU behavior:

- addition

This is because effective addresses are computed as:

- base register + immediate offset

Therefore, for load/store instructions, ALU Control often receives a main control indication that bypasses detailed arithmetic decode and directly selects the ADD operation.

This is an elegant simplification:

- the ALU is reused for address generation
- ALU Control keeps the selection explicit
- no separate address adder is necessary

4.17 Verification Expectations

A strong ALU Control testbench should verify that instruction field combinations map to the correct ALU operation codes.

This is different from ALU verification:

- ALU verification checks arithmetic outputs
- ALU Control verification checks decode correctness

Typical verification goals include:

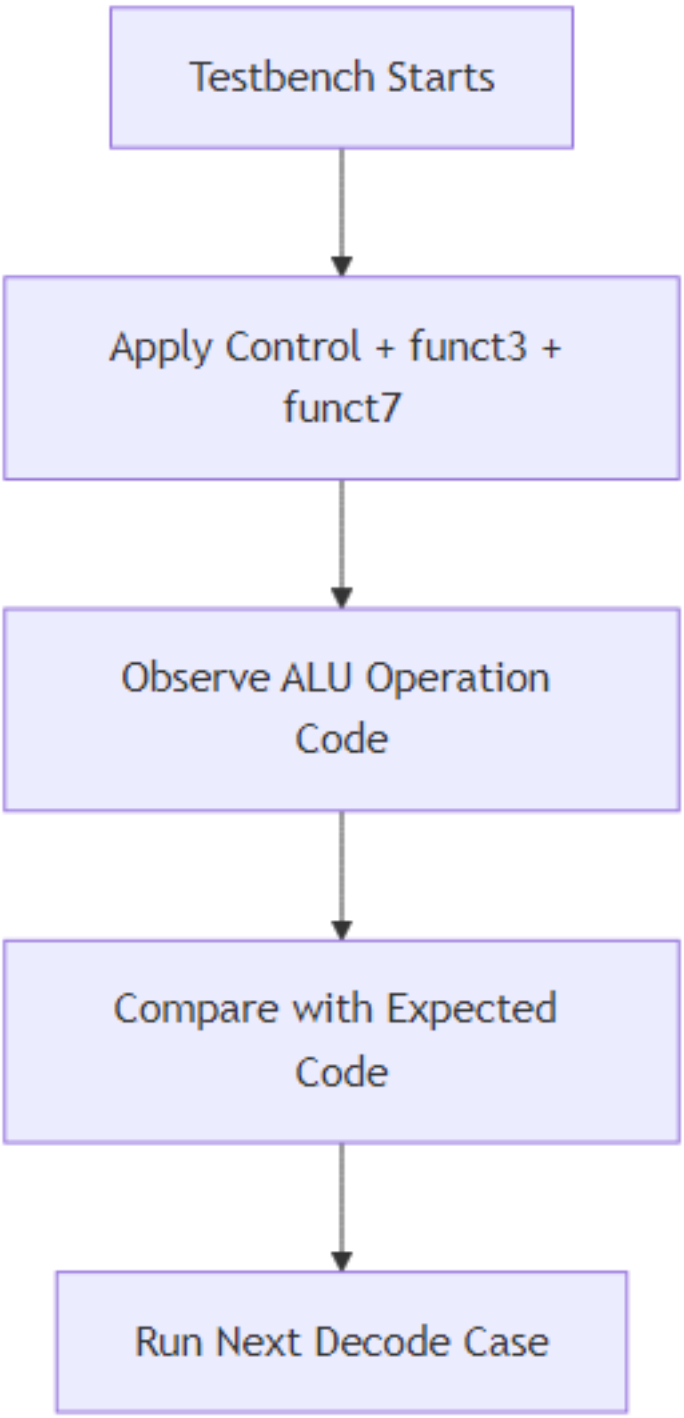
- ADD mapping correctness
- SUB mapping correctness
- AND/OR/XOR mapping correctness
- shift family mapping correctness
- signed vs unsigned compare mapping correctness
- fixed ADD selection for load/store style operations
- invalid or unexpected combinations handled safely according to design intent

Representative test approach:

1. drive a main ALU class
2. drive `funct3`
3. drive `funct7`
4. observe emitted ALU operation code
5. compare against expected result

4.18 Example Verification Flow

Conceptually, a verification flow can look like this:



This is a very good candidate for compact, systematic unit testing because the module is combinational and its behavior is fully determined by encoded inputs.

4.19 Design Strengths of the Module

The ALU Control module brings several benefits to the CPU design.

4.19.1 Better Modularity

It keeps the main control unit from becoming too detailed.

4.19.2 Easier Debugging

If the ALU executes the wrong function, it becomes easier to determine whether the issue is:

- decode/control
- ALU Control mapping
- ALU execution itself

4.19.3 Cleaner Documentation

It is much easier to explain the CPU when instruction-class control and ALU-operation refinement are documented separately.

4.19.4 Reusability

This structure scales well if the CPU is extended in the future.

Possible future extensions might include:

- additional instruction subsets
- more operation encodings
- custom instructions
- multi-cycle or pipelined refinements

4.20 Limitations and Tradeoffs

Although simple and effective, the ALU Control introduces a small extra layer of decode logic in the single-cycle path.

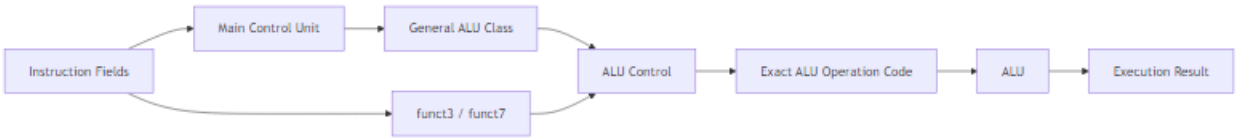
Tradeoffs include:

- additional combinational delay
- need for strict agreement with ALU encoding
- need for careful verification of all encoding combinations

However, these tradeoffs are minor and well justified because the module significantly improves the architecture.

4.21 Summary Diagram

The following summary diagram shows the role of ALU Control in the CPU.



This diagram emphasizes that ALU Control is the bridge between instruction decoding and exact execution behavior.

4.22 Final Summary

The **ALU Control module** is a key part of the control path in the **RV32I Minimal SoC** CPU.

Its responsibility is to convert:

- high-level instruction classification
- function subfields such as `funct3`
- variant information such as `funct7`

into:

- a precise internal ALU operation code

This enables the ALU to perform the correct function for:

- arithmetic instructions
- logical instructions
- shift instructions
- comparison instructions
- address calculations for loads and stores

The module improves modularity, clarity, testability, and scalability of the CPU design. Even though it is relatively small compared to the ALU itself, it is architecturally essential because it translates ISA-level meaning into actual execution commands.

5 Branch Compare Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

5.1 Purpose of the Module

The **Branch Compare** module is responsible for evaluating conditional branch decisions inside the CPU.

In the RV32I architecture, branch instructions do not simply compute a target address; they also need a boolean decision indicating whether the branch condition is true or false. The Branch

Compare module performs that evaluation using source register values and instruction-specific comparison mode information.

Its role is therefore to answer the question:

- should the branch be taken?

This module is essential for correct control flow, because it directly affects the next value loaded into the Program Counter.

5.2 Architectural Role Inside the CPU

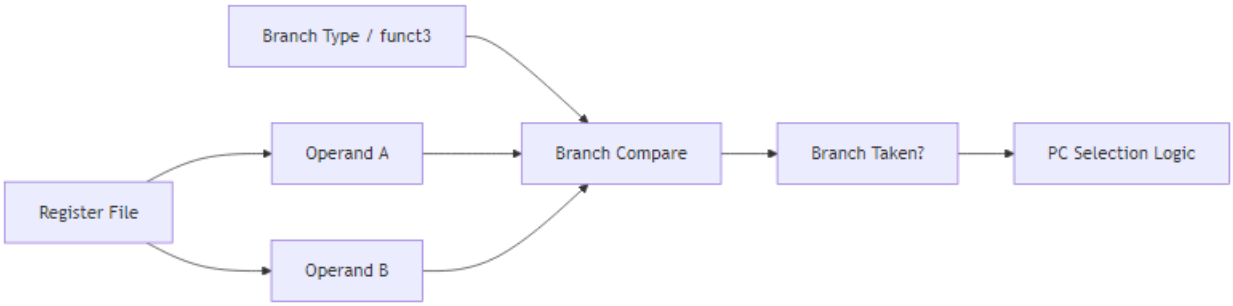
The Branch Compare block belongs to the **control-flow decision path** of the CPU.

Its main inputs conceptually come from:

- the register file operand outputs
- the decoded branch type information
- control logic indicating that the current instruction is a conditional branch

Its main output is a branch decision signal used by the PC selection logic.

Conceptually:



This means the Branch Compare module is not part of the arithmetic datapath in the same way as the ALU, but it is still central to instruction execution because it determines control-flow redirection.

5.3 Why a Dedicated Branch Comparator Exists

A dedicated Branch Compare block is a useful architectural choice because it separates **comparison logic** from **general arithmetic logic**.

Without a separate comparator, the CPU might try to reuse the ALU for branch decisions. While that is possible in some designs, using a dedicated comparator offers important advantages:

- clearer control-flow structure
- simpler documentation
- better modularity
- easier verification
- less coupling between ALU behavior and branch behavior

It also makes the CPU easier to reason about:

- ALU computes arithmetic and logical results
- Branch Compare determines conditional control-flow decisions

This division improves architectural clarity.

5.4 Relationship with RV32I Branch Instructions

The RV32I base ISA includes conditional branch instructions such as:

- BEQ
- BNE
- BLT
- BGE
- BLTU
- BGEU

These instructions compare two source operands and decide whether a branch target should replace the normal sequential next PC.

That means the Branch Compare module must support:

- equality comparison
- inequality comparison
- signed less-than comparison
- signed greater-than-or-equal comparison
- unsigned less-than comparison
- unsigned greater-than-or-equal comparison

These are exactly the semantic checks required by the RV32I branch subset.

5.5 Inputs and Outputs

The exact port names come from the VHDL source, but architecturally the module is expected to use the following signal categories.

5.5.1 Inputs

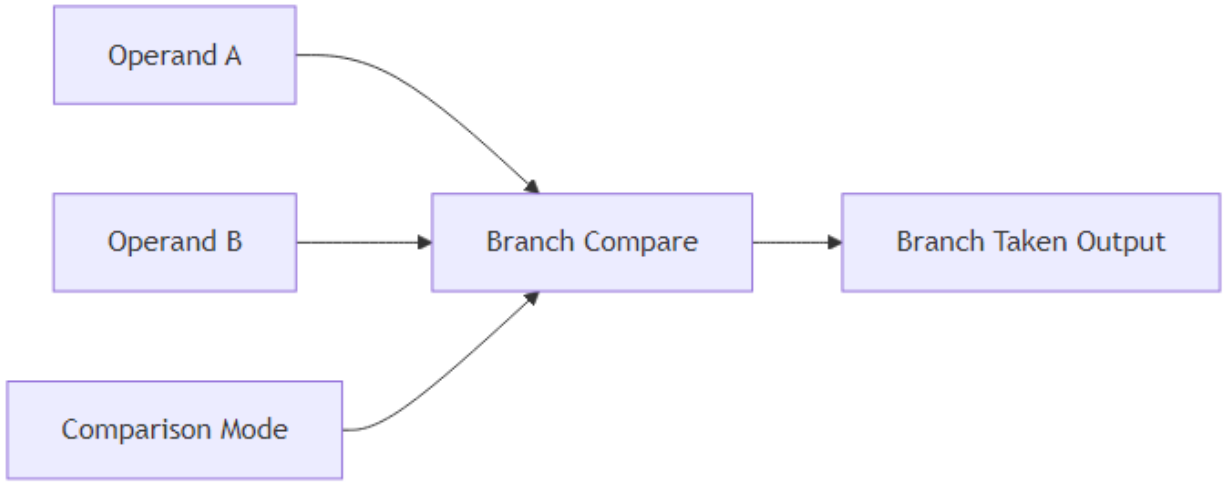
- operand A from the register file
- operand B from the register file
- branch comparison type selection

5.5.2 Outputs

- branch decision result
- possibly helper comparison flags depending on implementation style

The most important architectural output is the final **taken / not taken** decision.

Conceptually:



This output is then used by the PC control logic.

5.6 Comparison Types Required by RV32I

The Branch Compare module must distinguish different semantic comparison modes.

5.6.1 Equal

Used by:

- BEQ

Condition:

- branch taken if A == B

5.6.2 Not Equal

Used by:

- BNE

Condition:

- branch taken if A != B

5.6.3 Signed Less Than

Used by:

- BLT

Condition:

- branch taken if A < B under signed interpretation

5.6.4 Signed Greater Than or Equal

Used by:

- BGE

Condition:

- branch taken if A >= B under signed interpretation

5.6.5 Unsigned Less Than

Used by:

- BLTU

Condition:

- branch taken if A < B under unsigned interpretation

5.6.6 Unsigned Greater Than or Equal

Used by:

- BGEU

Condition:

- branch taken if A >= B under unsigned interpretation

These conditions fully cover the standard RV32I conditional branch class.

5.7 Signed vs Unsigned Interpretation

One of the most important responsibilities of the Branch Compare module is handling the difference between **signed** and **unsigned** interpretation.

This distinction matters because the same 32-bit binary pattern can represent different values depending on whether it is interpreted as:

- a signed two's complement number
- an unsigned binary magnitude

For example:

- `0xFFFFFFFF` is `-1` in signed interpretation
- `0xFFFFFFFF` is a large positive value in unsigned interpretation

This difference directly affects:

- BLT vs BLTU
- BGE vs BGEU

A correct comparator must therefore apply the correct numeric interpretation based on the decoded branch type.

5.8 Equality vs Ordering Comparisons

It is useful to distinguish two major comparison families.

5.8.1 Equality Comparisons

These check only whether bit patterns are identical:

- equal
- not equal

These comparisons do not depend on signedness.

5.8.2 Ordering Comparisons

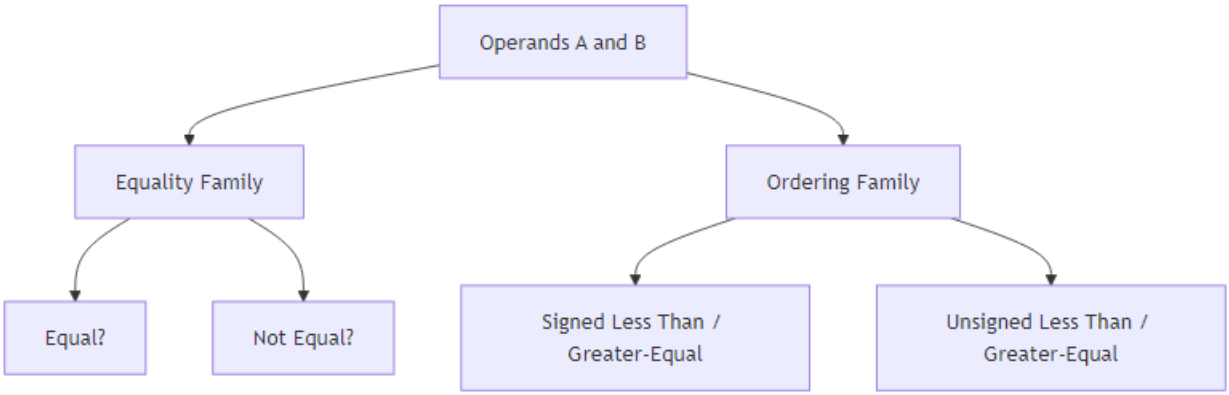
These check relative numeric ordering:

- less than
- greater than or equal

These do depend on signedness.

This distinction often appears naturally in the module's internal logic.

Conceptually:



This is a useful way to document and verify the module behavior.

5.9 Position in the Control-Flow Path

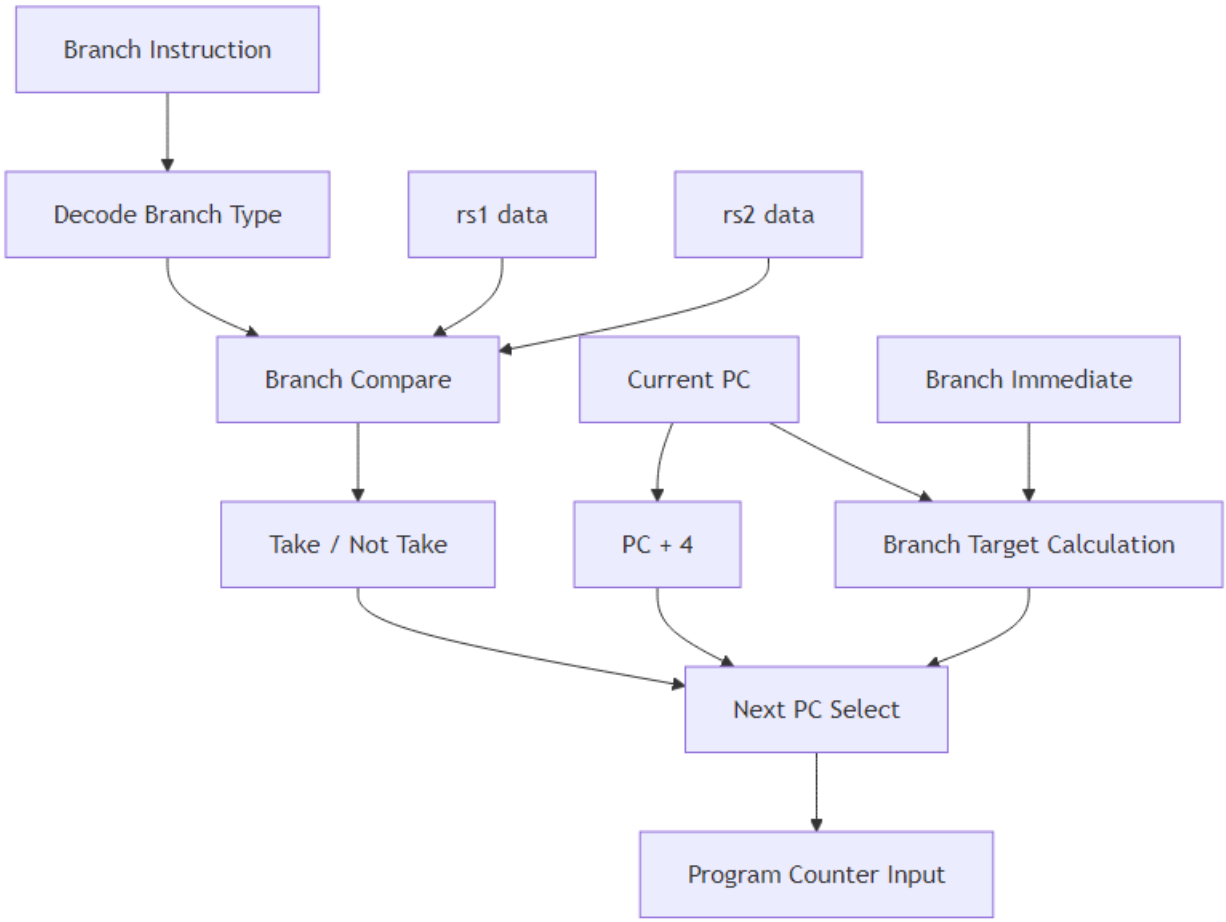
The Branch Compare module does not directly update the Program Counter. Instead, it provides a decision signal to the logic that selects the next PC.

The overall branch flow is typically:

1. fetch branch instruction
2. decode branch type
3. read `rs1` and `rs2`
4. Branch Compare evaluates the condition
5. immediate generator forms the branch offset

6. target address is computed
7. PC logic selects either:
    - `PC + 4`, or
    - the branch target

Conceptually:



The comparator is therefore a decision source for next-PC control.

5.10 Interaction with the Control Unit

The Branch Compare module depends on control information derived from instruction decoding.

The Control Unit determines:

- that the current instruction is a branch
- which branch subtype it is
- what comparison mode should be used

The comparator itself does not decode the full instruction in a broad sense. Instead, it receives a compact comparison mode and applies the corresponding decision rule.

This separation is beneficial because:

- instruction decode stays centralized
- comparison behavior stays specialized
- the CPU remains modular

5.11 Interaction with the Immediate Generator

Although the Branch Compare block does not compute branch target addresses by itself, it works closely with the immediate generation path in the overall branch mechanism.

The branch process conceptually contains two parallel tasks:

5.11.1 Decision Task

Performed by Branch Compare:

- determine whether the branch condition is true

5.11.2 Target Task

Performed by PC/target address logic using the branch immediate:

- determine where to go if the branch is taken

This split is elegant because it separates:

- **should we go?** from
- **where should we go?**

That makes the architecture easier to explain and verify.

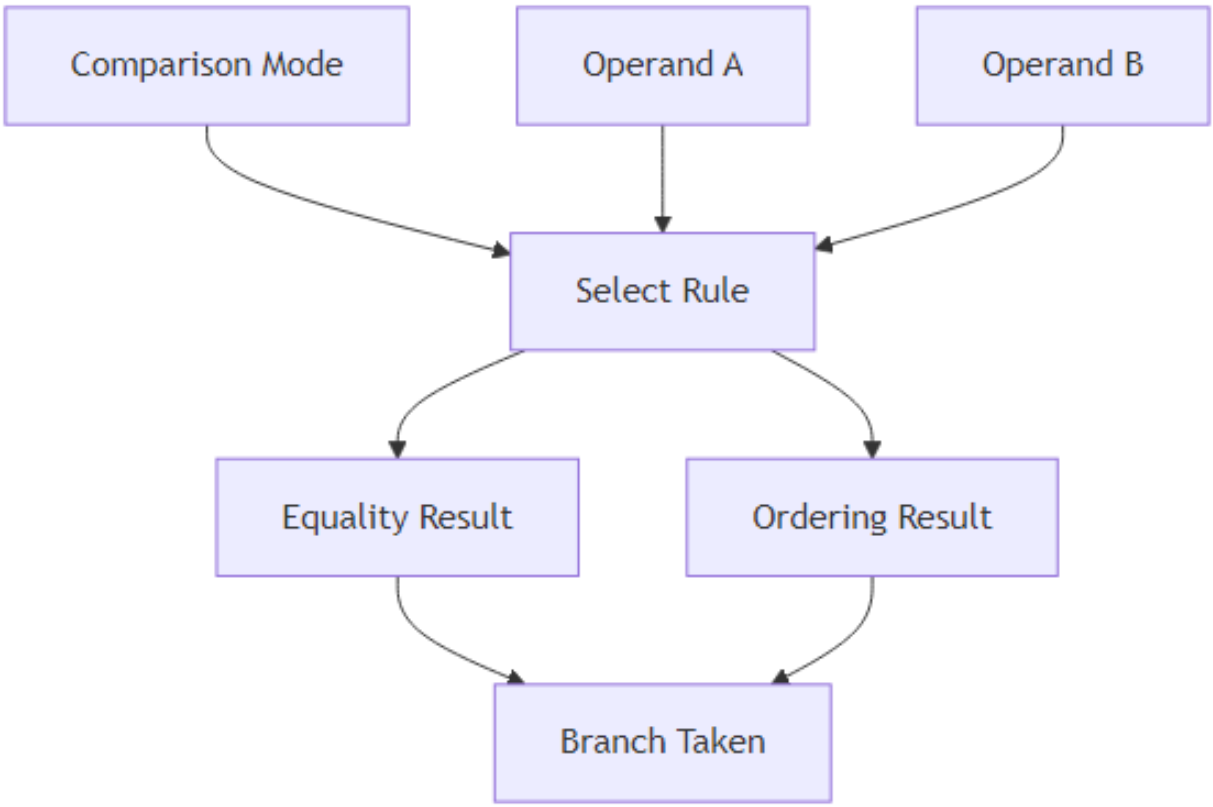
5.12 Possible Internal Implementation Style

The exact RTL implementation can vary, but a Branch Compare module is commonly implemented as a **combinational block**.

Its behavior usually follows a structure like this:

1. inspect comparison mode
2. apply the correct comparison rule
3. produce the final taken/not-taken result

Conceptually:



This style is efficient and well suited for a single-cycle processor.

5.13 Combinational Nature of the Module

The Branch Compare module is generally expected to be **pure combinational logic**.

That means:

- no internal state
- no clock dependency for decision generation
- output depends only on current inputs

This is important because the branch decision must be available in the same cycle as the instruction decode and PC selection process.

Advantages of combinational comparison logic:

- immediate decision generation
- simpler verification
- easy integration with next-PC logic
- natural fit for single-cycle execution

5.14 Role in the CPU Critical Path

Although the comparator may look simpler than the ALU, it can still lie on an important control path in a single-cycle CPU.

A branch instruction may require the following within one cycle:

1. instruction fetch
2. branch decode
3. register file read
4. branch comparison
5. branch target formation
6. PC selection
7. PC update

Because all of this happens in one cycle, the Branch Compare logic directly influences branch timing.

This is especially relevant in a single-cycle architecture, where branch resolution is not deferred to a later stage.

5.15 Branch Decision Semantics

The final comparator output must align exactly with architectural intent.

That means:

- BEQ must take the branch only when operands are equal
- BNE must take the branch only when operands differ
- BLT/BGE must use signed interpretation
- BLTU/BGEU must use unsigned interpretation

Even a small mistake in comparator logic can lead to severe CPU behavior issues, for example:

- wrong loop behavior
- broken conditional execution
- incorrect program flow
- invalid software results

That is why this module, although small, is highly important.

5.16 Relationship to Branch Target Computation

It is important not to confuse two separate concepts:

5.16.1 Branch Comparison

Decides whether the branch condition is true.

5.16.2 Branch Target Computation

Calculates the destination address if the branch is taken.

The Branch Compare module handles only the first of these.

This distinction helps keep the design clean:

- comparator answers yes/no
- target logic produces the address

Together they form complete branch behavior.

5.17 Verification Expectations

The Branch Compare module is an excellent unit-test candidate because it is:

- functionally compact
- stateless
- deterministic
- easy to stimulate

A good testbench should verify all branch modes with representative cases.

5.17.1 Equality Cases

- equal values
- different values
- zero vs zero
- max value vs max value

5.17.2 Signed Ordering Cases

- negative vs positive
- positive vs negative
- equal signed values
- boundary cases near sign transitions

5.17.3 Unsigned Ordering Cases

- high-bit differences
- small vs large values
- equal values
- max unsigned vs small unsigned

Representative verification goals:

- BEQ behavior correctness
- BNE behavior correctness
- BLT behavior correctness
- BGE behavior correctness
- BLTU behavior correctness

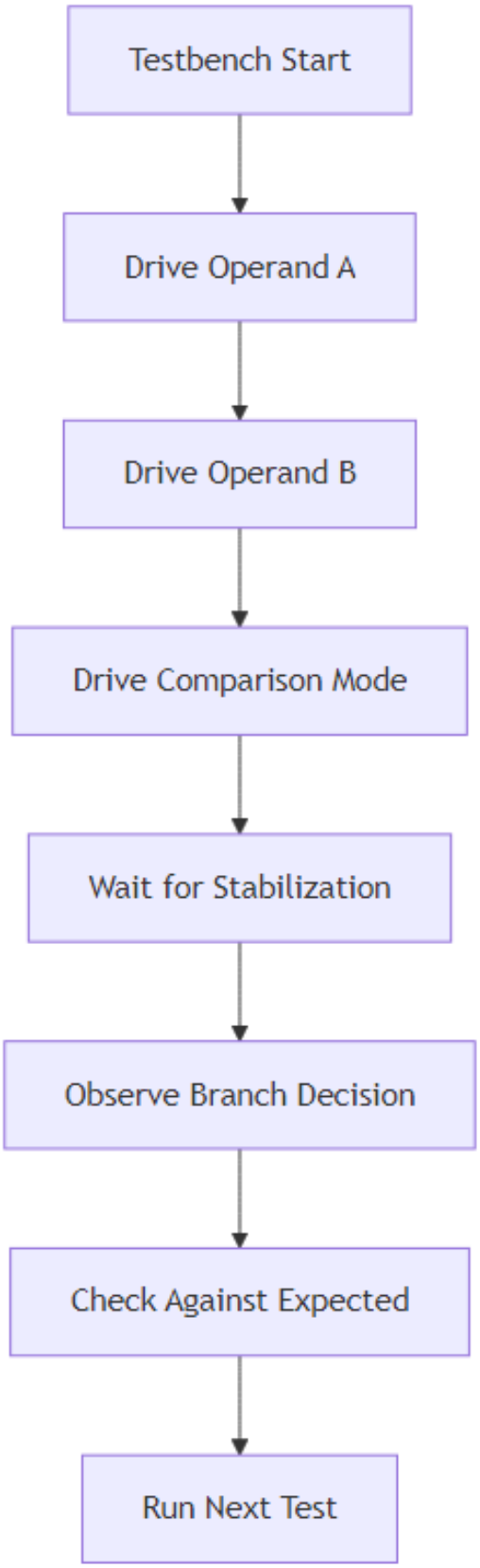
- BGEU behavior correctness

5.18 Example Verification Flow

A simple verification strategy may follow this structure:

1. drive operand A
2. drive operand B
3. drive branch comparison mode
4. wait for combinational stabilization
5. observe branch decision output
6. compare against expected result
7. repeat for all branch types and edge cases

Conceptually:


Testbench Start
Drive Operand A
Drive Operand B
Drive Comparison Mode
Wait for Stabilization
Observe Branch Decision
Check Against Expected
Run Next Test

Because the module is combinational, verification can be systematic and compact.

5.19 Interaction with Other Modules

The Branch Compare module interacts with several other CPU blocks.

5.19.1 Register File

Provides the two source operands to compare.

5.19.2 Control Unit

Provides the branch subtype or comparison mode.

5.19.3 Program Counter Logic

Consumes the taken/not-taken decision.

5.19.4 Immediate Generator

Provides the branch offset used elsewhere for target calculation.

5.19.5 Trap Unit

May override the normal PC decision path, even if the branch comparator produces a valid branch result.

This means the comparator is one input to control-flow logic, not necessarily the final authority in all exceptional cases.

5.20 Design Strengths of the Module

This module contributes several architectural benefits.

5.20.1 Clear Separation of Concerns

It isolates branch condition evaluation from arithmetic execution.

5.20.2 Easier Documentation

It is much easier to document control flow when branch comparison is its own block.

5.20.3 Easier Verification

Its limited, well-defined behavior makes testbench creation straightforward.

5.20.4 Better CPU Readability

The CPU microarchitecture becomes more understandable when branch decisions are handled by a dedicated comparator.

5.21 Limitations and Tradeoffs

A dedicated Branch Compare module introduces a small extra block in the control path, which means:

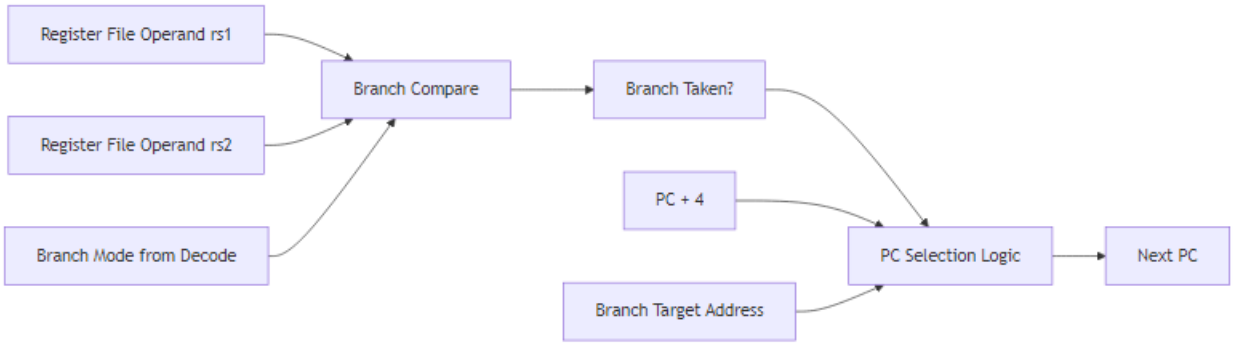
- additional decode-to-decision wiring
- separate verification effort
- slight extra combinational logic

However, these tradeoffs are minor and usually well justified because the module improves modularity and design clarity.

In a minimal educational CPU, this is usually the preferred approach.

5.22 Summary Diagram

The following summary diagram shows the Branch Compare block in its CPU context.



This emphasizes the comparator's role as a decision block in the control-flow path.

5.23 Final Summary

The **Branch Compare module** is the dedicated control-flow comparison block of the **RV32I Minimal SoC** CPU.

Its purpose is to evaluate the conditions required by RV32I branch instructions, including:

- equality
- inequality
- signed ordering
- unsigned ordering

It receives operands from the register file, receives comparison mode information from instruction decode, and produces a final taken/not-taken decision used by the next-PC selection logic.

Although smaller than the ALU, this module is architecturally critical because it determines whether conditional control-flow redirection occurs correctly. Its modularity improves clarity, verification, and overall CPU organization.

6 Control Unit Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

6.1 Purpose of the Module

The **Control Unit** is the central instruction-decode block of the CPU.

Its purpose is to interpret the fetched instruction and generate the control signals that coordinate the behavior of the datapath. In a single-cycle RV32I processor, this module is one of the most important architectural elements, because every instruction must be fully decoded and executed within the same clock cycle.

The Control Unit answers questions such as:

- should the register file write back a result?
- should the ALU use a register operand or an immediate?
- should memory be read?
- should memory be written?
- is the instruction a branch or jump?
- what broad ALU behavior class is required?
- should exceptional or trap-related behavior be triggered?

Without the Control Unit, the datapath blocks would exist, but they would not know how to cooperate for any given instruction.

6.2 Architectural Role Inside the CPU

The Control Unit belongs to the **control path** of the processor.

It receives instruction information from the fetch/decode path and generates the set of steering signals that configure the datapath for the current instruction.

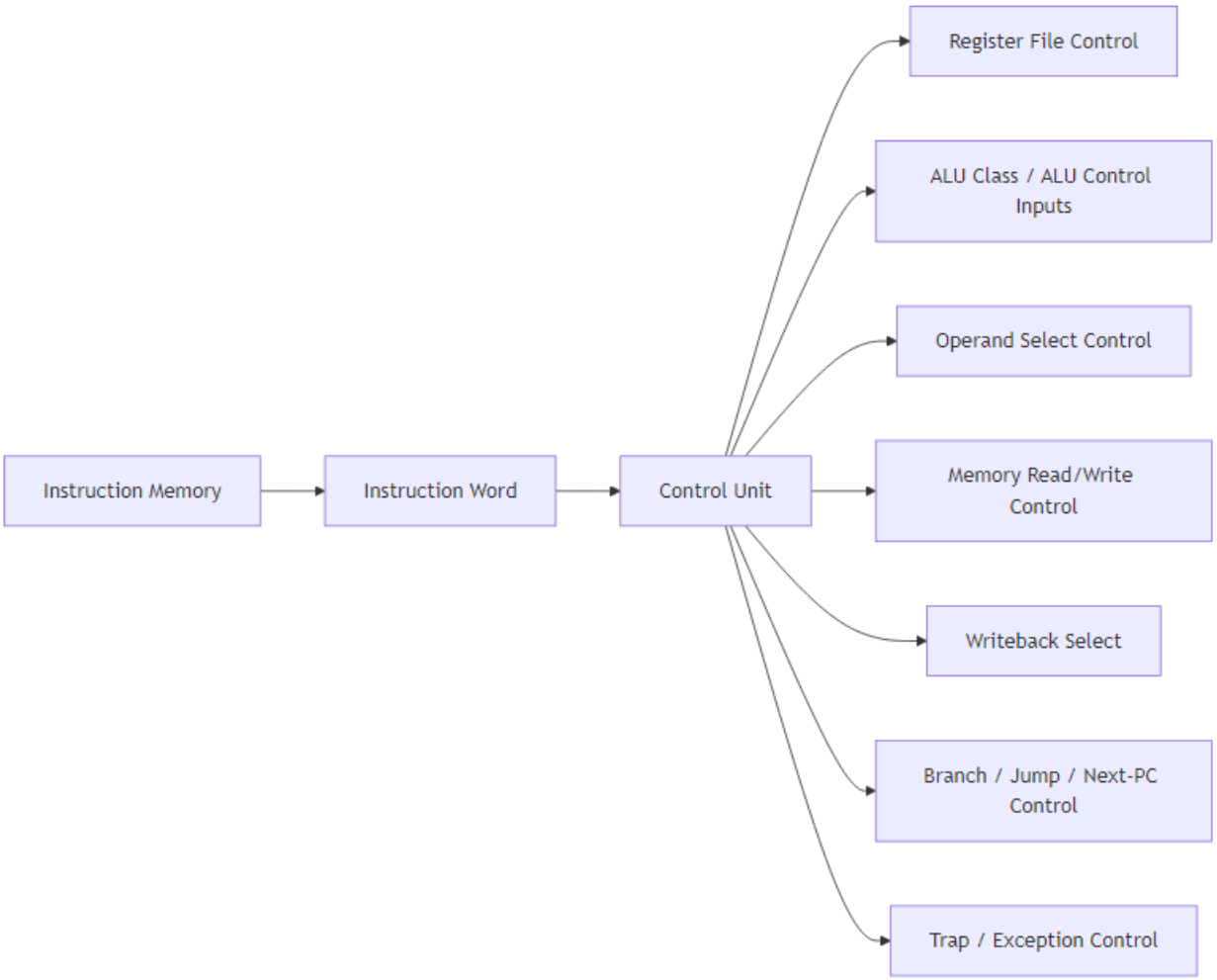
Conceptually, the Control Unit sits between:

- the fetched instruction
- the execution datapath
- the control-flow selection logic
- the memory access path
- the writeback path

At a high level:

1. instruction memory returns the instruction word
2. the Control Unit decodes the instruction
3. control signals are generated
4. the rest of the CPU behaves according to those signals

Conceptually:



This makes the Control Unit the module that transforms ISA encoding into CPU behavior.

6.3 Why the Control Unit Exists

The CPU datapath contains multiple reusable hardware blocks:

- register file
- immediate generator
- ALU
- branch comparator
- load/store unit
- bus interconnect interface

- trap logic

Each instruction uses only a specific subset of these blocks and uses them in a specific way.

For example:

- an ADD instruction needs register operands, ALU execution, and writeback
- a LOAD instruction needs register plus immediate address calculation, memory read, and writeback
- a STORE instruction needs address calculation and memory write, but no register writeback
- a BRANCH instruction needs comparison and PC selection, but usually no writeback

The Control Unit determines which of these behaviors must be enabled for the current instruction.

6.4 Relationship with Instruction Decoding

The Control Unit interprets the instruction at the architectural level.

It typically uses fields such as:

- opcode
- and, depending on implementation style, instruction classification results

In many CPU organizations, the main Control Unit focuses primarily on the **opcode** or top-level instruction class, while more detailed decode is delegated to specialized blocks such as:

- ALU Control
- Branch Compare
- Immediate Generator
- Trap Unit

This is a good design choice because it keeps the main decoder manageable.

A common split is:

6.4.1 Main Control Unit

Determines the broad behavior class:

- ALU instruction
- load
- store
- branch
- jump
- upper immediate
- trap/system

6.4.2 Specialized Blocks

Refine the exact behavior:

- ALU Control decides the exact ALU function
- Branch Compare decides branch conditions

- Immediate Generator reconstructs the correct immediate format
- Trap Unit handles exceptional control redirection

6.5 Inputs and Outputs

The exact VHDL port names come from the RTL implementation, but architecturally the Control Unit is expected to consume decoded instruction identity information and emit datapath steering signals.

6.5.1 Inputs

Typical input categories:

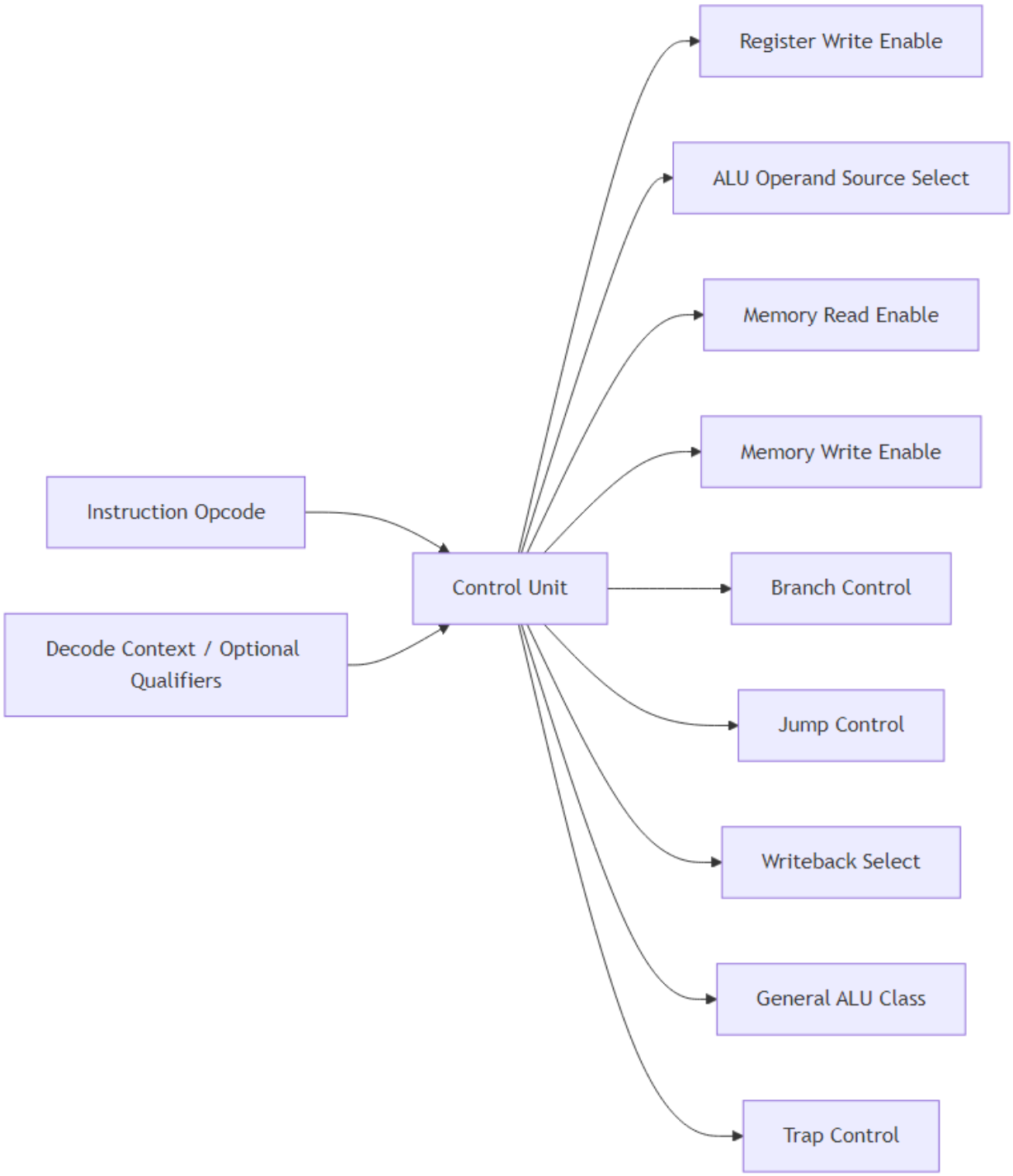
- instruction opcode
- possibly instruction validity qualifiers
- possibly trap-related qualifiers or decode context

6.5.2 Outputs

Typical control outputs may include:

- register write enable
- ALU source selection
- memory read enable
- memory write enable
- branch enable
- jump enable
- writeback source selection
- broad ALU operation class
- trap-related control outputs

Conceptually:



These outputs collectively define the instruction's behavior in the datapath.

6.6 Main Responsibilities of the Control Unit

The Control Unit has several core responsibilities.

6.6.1 Instruction Classification

It determines what broad type of instruction is currently being executed.

Examples:

- register-register ALU instruction
- register-immediate ALU instruction
- load
- store
- conditional branch
- jump
- upper immediate
- system or trap-related instruction

6.6.2 Datapath Steering

It decides which datapath resources should be used and how.

Examples:

- whether operand B comes from `rs2` or from the immediate generator
- whether memory access is active
- whether writeback is enabled

6.6.3 Writeback Policy

It determines whether a destination register should receive a result and from which source.

6.6.4 Control-Flow Direction

It indicates whether the current instruction affects the next-PC path through:

- sequential execution
- conditional branch
- unconditional jump
- trap/exception redirection

6.6.5 Coordination with Specialized Control Blocks

It provides the top-level control context used by:

- ALU Control
- Branch Compare
- Load Store Unit
- Trap Unit

6.7 Typical Control Signals

A single-cycle RV32I Control Unit often generates signals conceptually similar to the following.

6.7.1 Register Write Enable

Indicates whether the current instruction writes a result to `rd`.

Used by:

- ALU operations
- loads
- jump-and-link type instructions
- upper-immediate instructions

Usually not used by:

- stores
- pure branches
- many trap-control cases

6.7.2 ALU Source Select

Determines whether the ALU's second operand comes from:

- register file operand `rs2` or
- immediate generator output

Used to distinguish:

- R-type ALU instructions
- I-type and memory-addressing instructions

6.7.3 Memory Read Enable

Indicates that a load operation is in progress.

6.7.4 Memory Write Enable

Indicates that a store operation is in progress.

6.7.5 Branch / Jump Control

Indicates that next-PC logic should consider branch or jump redirection.

6.7.6 Writeback Select

Determines which source is written back to the register file.

Possible categories include:

- ALU result
- load data
- immediate-based result
- PC-related result

6.7.7 General ALU Class

Provides a coarse execution category to ALU Control.

6.8 Operation by Instruction Class

The easiest way to understand the Control Unit is to see how it behaves for each major RV32I instruction class.

6.8.1 R-Type Arithmetic / Logic Instructions

Examples:

- add
- sub
- and
- or
- xor
- sll
- srl
- sra
- slt
- sltu

Typical control behavior:

- read operands from register file
- use `rs2` as ALU operand B
- enable register writeback
- disable memory read/write
- select ALU result for writeback
- request detailed ALU decode

6.8.2 I-Type Arithmetic Instructions

Examples:

- addi
- andi
- ori
- xori
- slti
- sltiu
- slli
- srli
- srai

Typical control behavior:

- read `rs1`
- use immediate as ALU operand B

- enable register writeback
- disable memory access
- select ALU result for writeback

6.8.3 Load Instructions

Typical control behavior:

- read base register
- use immediate offset
- enable ALU address calculation
- enable memory read
- disable memory write
- enable register writeback
- select load data for writeback

6.8.4 Store Instructions

Typical control behavior:

- read base register and store data register
- use immediate offset
- enable ALU address calculation
- disable memory read
- enable memory write
- disable register writeback

6.8.5 Branch Instructions

Typical control behavior:

- read both source operands
- enable branch comparison path
- generate branch offset path
- disable memory access
- disable normal register writeback
- enable PC decision logic for conditional redirection

6.8.6 Jump Instructions

Typical control behavior:

- enable jump target logic
- enable PC redirection
- enable register writeback when link semantics apply
- choose the correct writeback source for the link result

6.8.7 Upper Immediate Instructions

Examples:

- lui

- auipc

Typical control behavior depends on subtype, but usually includes:

- immediate-based result generation
- register writeback enable
- no memory access

6.8.8 Trap or System-Related Instructions

Typical control behavior:

- signal trap logic
- possibly override normal next-PC selection
- possibly suppress or redirect normal writeback
- coordinate with trap handling path

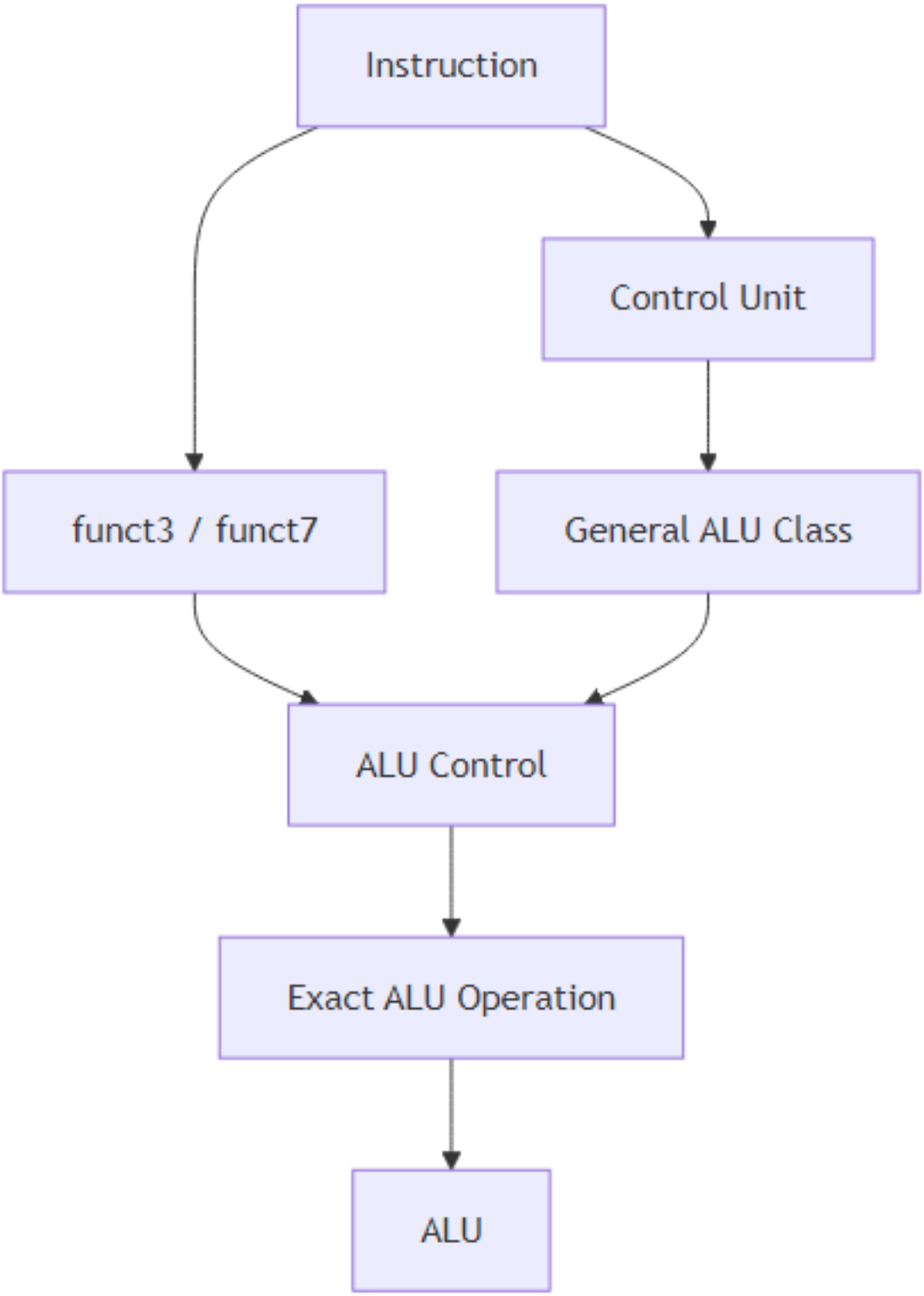
6.9 Relationship with the ALU Control Module

The Control Unit does not usually specify the exact ALU function directly for all instructions. Instead, it sends a **general ALU behavior class** to the ALU Control module.

The relationship is:

- Control Unit says what kind of ALU usage is needed
- ALU Control says what exact ALU operation is needed

Conceptually:



This makes the CPU hierarchy cleaner and easier to scale.

6.10 Relationship with the Register File

The Control Unit determines how the register file participates in execution.

It influences:

- whether a result is written back
- whether the instruction consumes one or two source operands
- which result source is written to `rd`

For example:

- ALU instructions write to `rd`
- loads write to `rd`
- stores do not write to `rd`
- branches usually do not write to `rd`

The register file itself does not know instruction semantics; it simply acts according to control signals.

6.11 Relationship with the Immediate Generator

The Immediate Generator reconstructs a 32-bit immediate from the instruction, but the Control Unit determines whether that immediate is needed and how it should be used at a high level.

Examples:

- I-type ALU instructions use an immediate operand
- loads and stores use an offset immediate
- branches use a branch immediate
- jumps use a jump immediate
- upper-immediate instructions use U-type immediate behavior

The Control Unit therefore helps select the execution path in which the generated immediate participates.

6.12 Relationship with the Load Store Unit

The Load Store Unit handles memory access formatting and data movement, but it requires the Control Unit to determine when a memory operation is active.

The Control Unit provides high-level memory behavior such as:

- load vs store
- memory read enable
- memory write enable
- whether writeback should come from memory data

This relationship is essential because memory transactions are not meaningful without decode context.

6.13 Relationship with the Branch Compare Module

The Branch Compare module evaluates conditions, but it relies on the Control Unit to indicate the instruction class and, in many designs, the branch subtype or enable context.

The Control Unit therefore determines:

- whether the current instruction is actually a branch
- whether the next-PC logic should consider comparison outcomes
- how branch behavior should fit into overall control flow

This keeps branch semantics tied to instruction decode rather than scattered throughout the datapath.

6.14 Relationship with the Trap Unit

The Trap Unit handles exceptional control redirection or system-style events, but the Control Unit often identifies the relevant instruction class or control context first.

Examples of possible interactions include:

- illegal instruction classification
- trap/system instruction recognition
- exceptional decode conditions
- suppression of normal writeback or normal PC progression

In this way, the Control Unit and Trap Unit cooperate to maintain correct architectural behavior when execution leaves the ordinary sequential path.

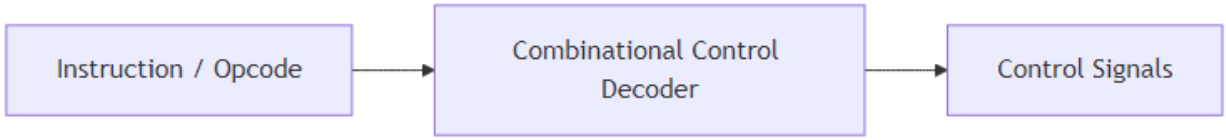
6.15 Control Unit as a Combinational Decoder

In a single-cycle CPU, the Control Unit is generally implemented as a **combinational decode block**.

This means:

- it does not usually hold execution state
- its outputs depend directly on the current instruction inputs
- the correct control outputs must appear in the same cycle as execution

Conceptually:



This is a natural fit for single-cycle execution, but it also means the Control Unit contributes directly to the decode-to-execute timing path.

6.16 Role in the Single-Cycle Critical Path

Because the CPU is single-cycle, the Control Unit lies early in the critical path.

A typical instruction may require within one cycle:

1. PC provides instruction address
2. instruction memory returns instruction
3. Control Unit decodes instruction
4. control signals configure datapath
5. ALU / compare / memory path executes
6. writeback or PC decision completes

This means the Control Unit must be:

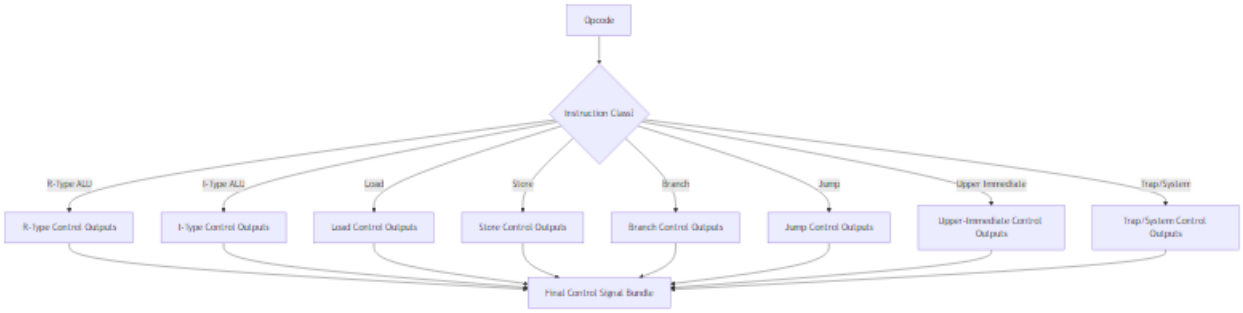
- logically correct
- fast enough for the intended clock target
- clearly structured to avoid decode ambiguity

Even though it may look "simple" compared to data-processing blocks, it is fundamental to timing closure in a single-cycle CPU.

6.17 Possible Internal Decode Style

The exact RTL implementation may vary, but Control Units are often structured as decision logic based on opcode classes.

A conceptual decode flow may look like this:



This kind of structure is easy to document and verify.

6.18 Verification Expectations

A strong Control Unit testbench should verify decode behavior across all major instruction classes.

The goal is not to test arithmetic correctness directly, but to verify that the correct control signals are emitted for each instruction type.

Typical verification targets include:

- register write enable correctness

- memory read/write decode correctness
- ALU source select correctness
- branch/jump control correctness
- writeback select correctness
- trap/system classification correctness

Representative instruction groups to test:

- R-type ALU instructions
- I-type ALU instructions
- loads
- stores
- branches
- jumps
- upper-immediate instructions
- trap/system instructions

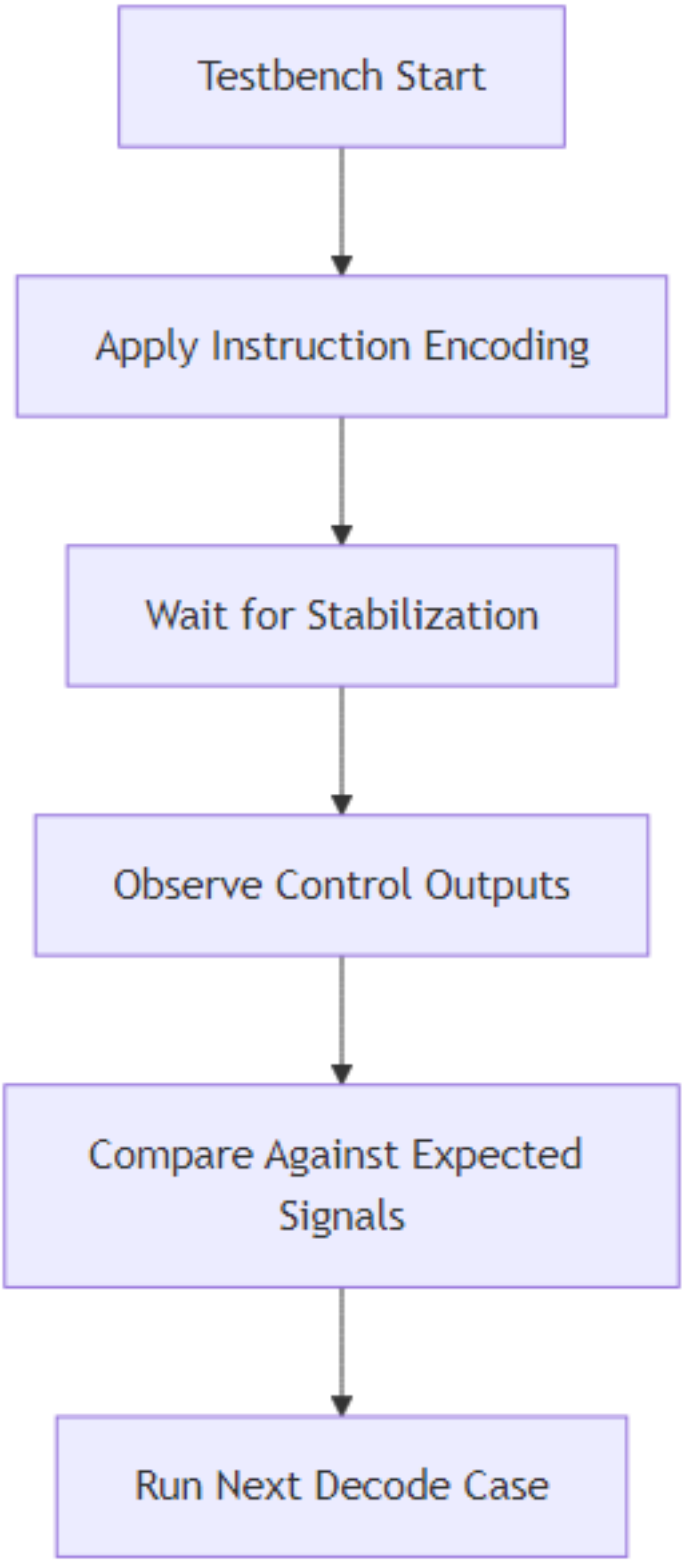
A decode bug in the Control Unit can cause many downstream failures even if all other datapath modules are individually correct.

6.19 Example Verification Flow

A simple Control Unit verification pattern may look like this:

1. apply an instruction opcode or instruction word pattern
2. wait for combinational stabilization
3. observe generated control outputs
4. compare each relevant signal against expected values
5. repeat for all instruction classes and representative variants

Conceptually:



Because the Control Unit is typically combinational, systematic table-driven testing works very well.

6.20 Design Strengths of the Module

The Control Unit provides several major benefits to the CPU architecture.

6.20.1 Centralized Decode Logic

Instruction meaning is interpreted in one main place instead of being scattered throughout the CPU.

6.20.2 Improved Modularity

Different execution blocks can stay specialized because they rely on the Control Unit for coordination.

6.20.3 Easier Documentation

It becomes much easier to explain the CPU when top-level instruction behavior is documented through a dedicated decode block.

6.20.4 Easier Debugging

If a whole instruction class behaves incorrectly, the Control Unit is a natural place to inspect first.

6.20.5 Scalable Organization

This style supports future extensions more naturally than a monolithic, unstructured decode scheme.

6.21 Limitations and Tradeoffs

A centralized Control Unit also introduces some tradeoffs.

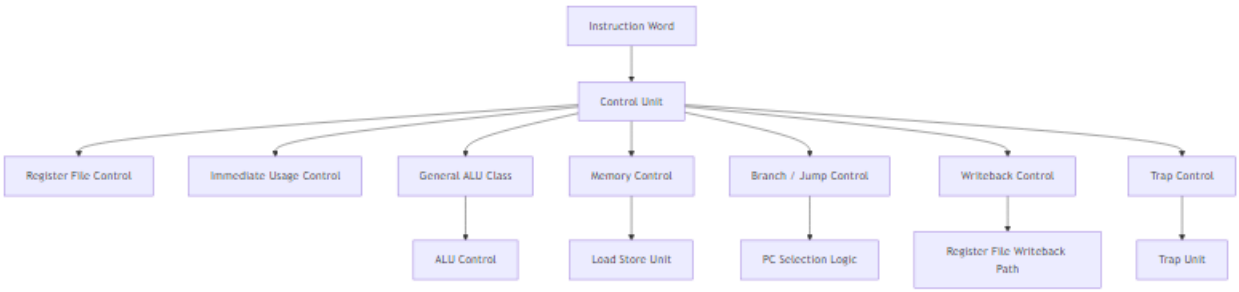
Possible tradeoffs include:

- added combinational delay in decode
- need for careful coordination with all downstream modules
- need for consistent signal naming and semantics
- possibility that one decode bug affects many parts of the CPU

However, these tradeoffs are normal and acceptable. In practice, a well-designed Control Unit is essential for a clean CPU architecture.

6.22 Summary Diagram

The following summary diagram shows the Control Unit in context.



This highlights the Control Unit as the source of top-level execution behavior.

6.23 Final Summary

The **Control Unit module** is the central decode and coordination block of the **RV32I Minimal SoC** CPU.

Its job is to interpret the current instruction and generate the control signals that configure:

- the register file
- operand selection
- ALU usage
- memory access behavior
- branch/jump behavior
- writeback selection
- trap-related control flow

It acts as the main bridge between ISA-level instruction encoding and actual datapath operation. In a single-cycle RV32I processor, this makes the Control Unit one of the most architecturally important modules in the design.

A correct and well-structured Control Unit is essential for clarity, modularity, verification quality, and overall CPU correctness.

7 Immediate Generator Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

7.1 Purpose of the Module

The **Immediate Generator** is responsible for extracting and reconstructing immediate values from the instruction word.

In the RV32I instruction set architecture, immediate values are encoded in different bit positions depending on the instruction format. The Immediate Generator collects those bits, assembles them into the correct order, performs sign extension when required, and outputs a uniform 32-bit immediate value that can be used by the rest of the CPU.

This module is essential because many instructions rely on immediate operands, including:

- arithmetic immediate instructions
- load instructions
- store instructions
- branch instructions
- jump instructions
- upper-immediate instructions

Without this module, the CPU datapath would have to handle multiple immediate formats separately, making the architecture much more complicated.

7.2 Architectural Role Inside the CPU

The Immediate Generator belongs to the **decode stage** of the CPU datapath.

It operates directly on the instruction word and provides an immediate value that is used by several execution paths.

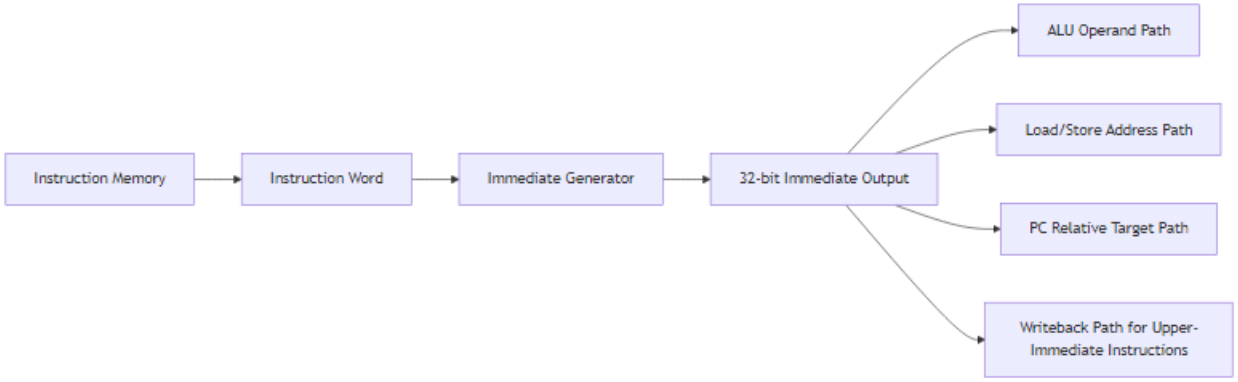
Conceptually, the Immediate Generator sits between:

- the instruction fetch/decode path
- the execution datapath

High-level flow:

1. instruction is fetched from instruction memory
2. the instruction is decoded
3. the Immediate Generator extracts the immediate bits
4. the reconstructed immediate is forwarded to the datapath

Conceptually:



This makes the Immediate Generator a shared decode component used by many different instruction classes.

7.3 Why Immediate Reconstruction is Required

Unlike register indices or opcodes, immediate values in RISC-V are not always stored as contiguous bit fields.

Instead, they are arranged in different ways depending on the instruction type.

This design improves encoding efficiency but requires hardware to reconstruct the final immediate value.

The Immediate Generator performs the following tasks:

1. select the correct instruction format
2. extract immediate bits from the instruction
3. rearrange them into the correct order
4. perform sign extension when required
5. output a 32-bit immediate value

This keeps the rest of the CPU datapath simple and uniform.

7.4 Immediate Formats in RV32I

The RV32I architecture defines several instruction formats that contain immediate values.

The Immediate Generator must support all of them.

7.4.1 I-Type Immediate

Used by:

- arithmetic immediate instructions
- load instructions
- some system instructions

Properties:

- contiguous immediate field
- sign-extended

Example usage:

```
addi rd, rs1, imm
lw   rd, imm(rs1)
```

7.4.2 S-Type Immediate

Used by:

- store instructions

Properties:

- immediate split across two instruction regions
- must be reassembled
- sign-extended

Example usage:

```
sw rs2, imm(rs1)
```

7.4.3 B-Type Immediate

Used by:

- conditional branch instructions

Properties:

- immediate split across multiple instruction fields
- reassembled in a special order
- represents a **branch offset**
- sign-extended

Example usage:

```
beq rs1, rs2, offset
```

7.4.4 U-Type Immediate

Used by:

- LUI
- AUIPC

Properties:

- immediate placed in upper bits
- lower bits typically zeroed
- does not represent a simple offset

Example usage:

```
lui rd, imm
auipc rd, imm
```

7.4.5 J-Type Immediate

Used by:

- jump instructions

Properties:

- immediate split across several fields
- reassembled into a jump target offset
- sign-extended

Example usage:

```
jal rd, offset
```

7.5 Inputs and Outputs

Although the exact signal names depend on the RTL implementation, the architectural interface is straightforward.

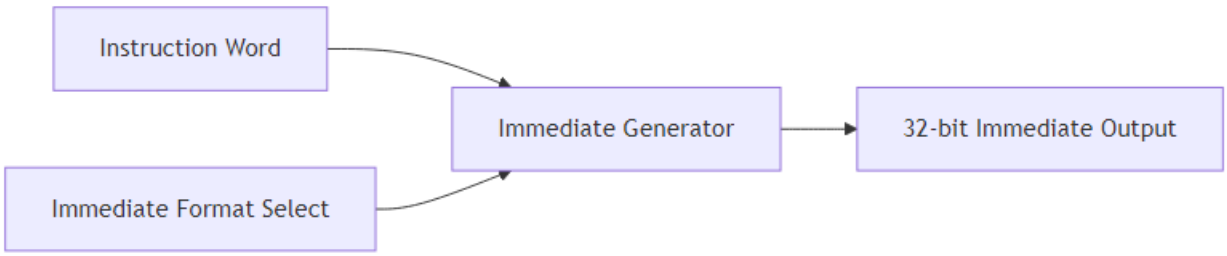
7.5.1 Inputs

- instruction word
- instruction format selection or decode information

7.5.2 Outputs

- 32-bit immediate value

Conceptually:



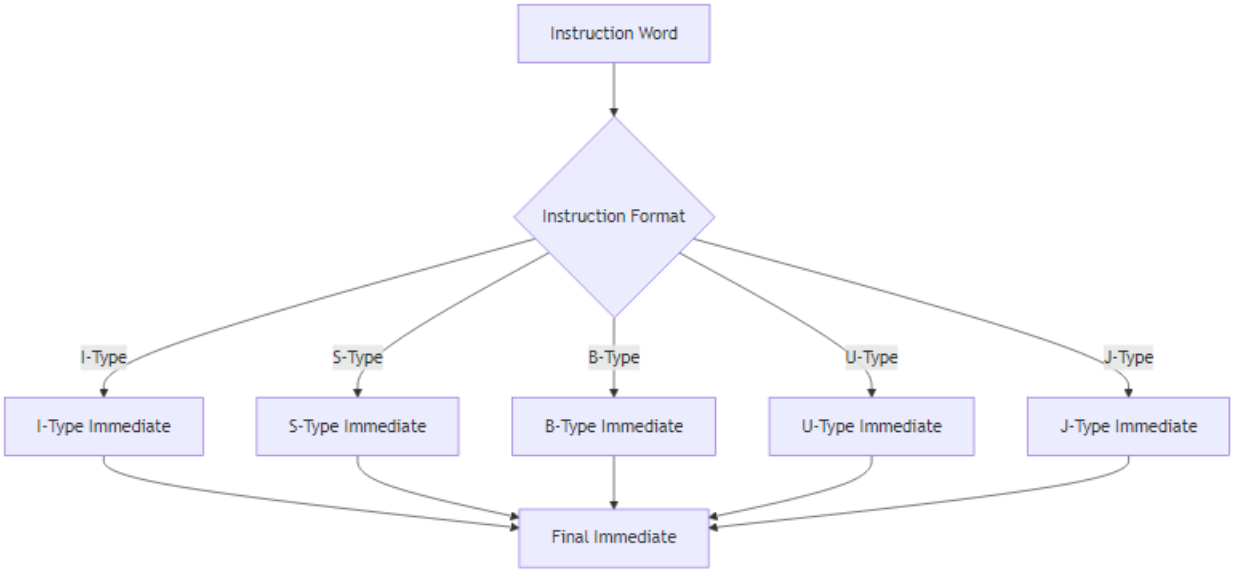
This interface makes the Immediate Generator easy to reuse and easy to test.

7.6 Immediate Selection Process

The Immediate Generator must determine which immediate format to use for the current instruction.

This is usually controlled by decode logic.

Conceptually:



Each path reconstructs the immediate value differently.

7.7 Sign Extension

Most immediate values in RV32I are **sign-extended**.

This means the most significant bit of the encoded immediate becomes the sign bit of the 32-bit result.

Sign extension is required because:

- the instruction format may encode fewer than 32 bits
- the CPU still needs a full 32-bit operand

Example concept:

Correct sign extension is critical for proper behavior of:

- negative offsets
- backward branches
- negative arithmetic immediates

7.8 Use of Immediate Values in the CPU

The immediate output from the Immediate Generator feeds multiple datapath components.

7.9 ALU Operand

Used by arithmetic immediate instructions.

Example:

```
addi x5, x1, 10
```

The ALU receives:

- operand A = register file value
- operand B = immediate

7.9.1 Address Generation

Used by loads and stores.

Example:

```
lw x5, 16(x1)
```

The ALU computes:

```
address = rs1 + immediate
```

7.9.2 Branch Targets

Used to compute conditional branch targets.

Example:

```
beq x1, x2, label
```

Branch offset comes from the immediate generator.

7.9.3 Jump Targets

Used for unconditional control transfer.

Example:

```
jal x1, label
```

7.9.4 Upper-Immediate Instructions

Example:

```
lui x5, imm
```

The immediate itself becomes the basis of the writeback result.

7.10 Interaction with Other CPU Modules

The Immediate Generator interacts with several parts of the CPU.

7.10.1 Control Unit

Determines which instruction format is active.

7.10.2 ALU

Consumes the immediate as an operand for arithmetic or address calculation.

7.10.3 Program Counter Logic

Uses branch and jump immediates to form control-flow targets.

7.10.4 Load Store Unit

Uses immediate values for memory addressing offsets.

7.10.5 Writeback Logic

Some instructions use immediate values directly for register writeback.

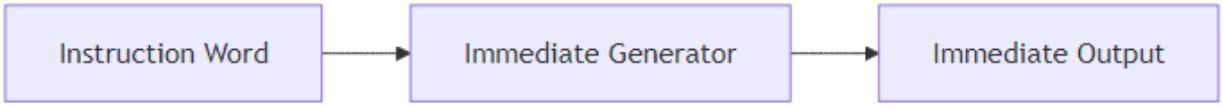
7.11 Combinational Nature of the Module

The Immediate Generator is typically implemented as **pure combinational logic**.

Characteristics:

- no internal state
- no clock dependency
- output determined only by instruction input

Conceptually:



This makes the module simple, predictable, and efficient.

7.12 Role in the CPU Critical Path

In a single-cycle CPU, the Immediate Generator lies early in the decode stage.

A typical instruction execution path may include:

1. instruction fetch
2. immediate reconstruction
3. operand selection
4. ALU computation
5. memory or writeback path
6. PC update

Because immediate values are often required by the ALU in the same cycle, the Immediate Generator must be fast enough to avoid slowing down execution.

7.13 Verification Expectations

The Immediate Generator is well suited to unit testing because its behavior is deterministic.

A testbench should verify:

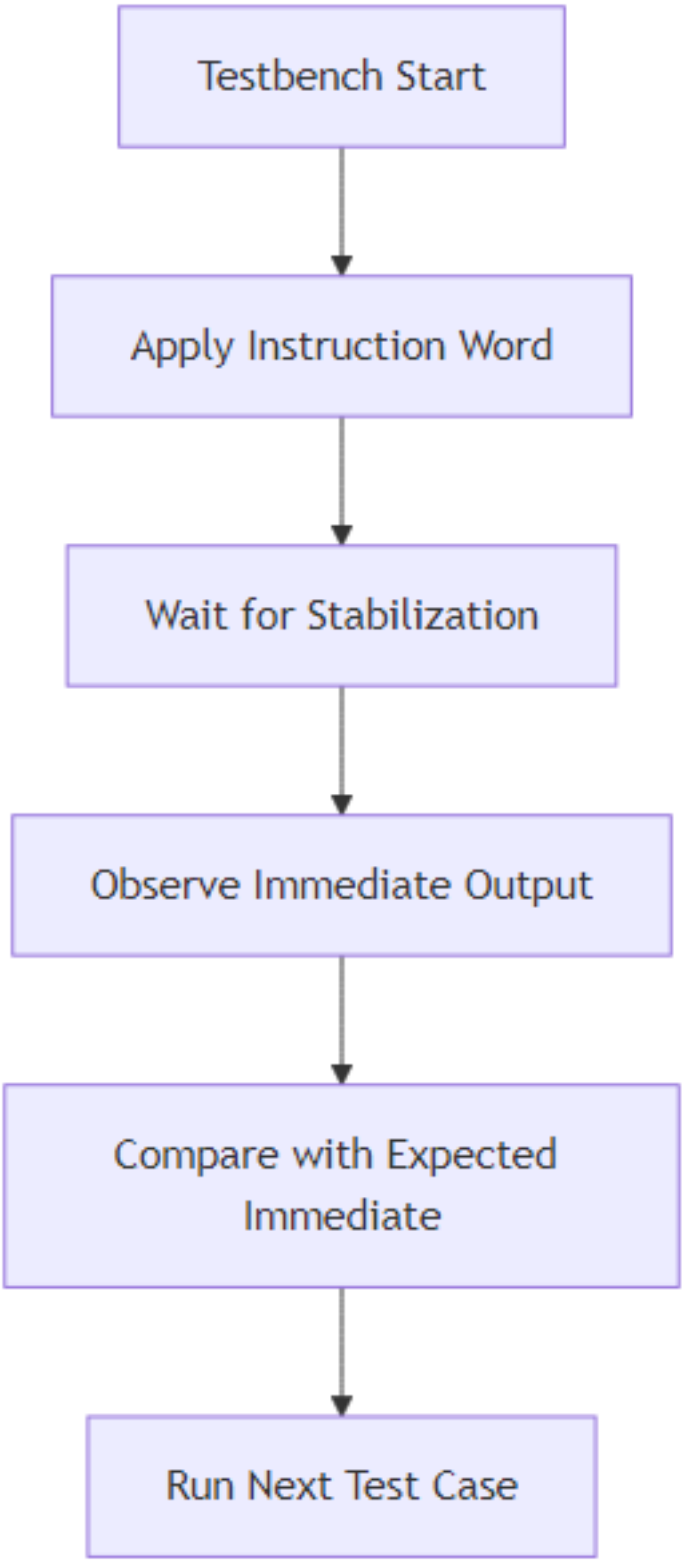
- I-type immediate extraction
- S-type immediate reconstruction
- B-type branch offset reconstruction
- U-type immediate placement
- J-type jump offset reconstruction
- sign extension correctness

Test cases should include:

- positive values
- negative values
- maximum positive immediates
- maximum negative immediates
- boundary conditions

7.14 Example Verification Flow

A typical test sequence may follow this structure:



Because the module is combinational, verification can be performed efficiently using systematic instruction patterns.

7.15 Design Strengths of the Module

The Immediate Generator contributes several architectural advantages.

7.15.1 Clean Datapath

All immediate formats are handled in one location.

7.15.2 Simpler ALU Logic

The ALU always receives a consistent 32-bit operand.

7.15.3 Clear Decode Structure

Instruction reconstruction logic is isolated from execution logic.

7.15.4 Easy Verification

The module has a small number of well-defined behaviors.

7.16 Limitations and Tradeoffs

The Immediate Generator introduces a small amount of combinational logic into the decode stage.

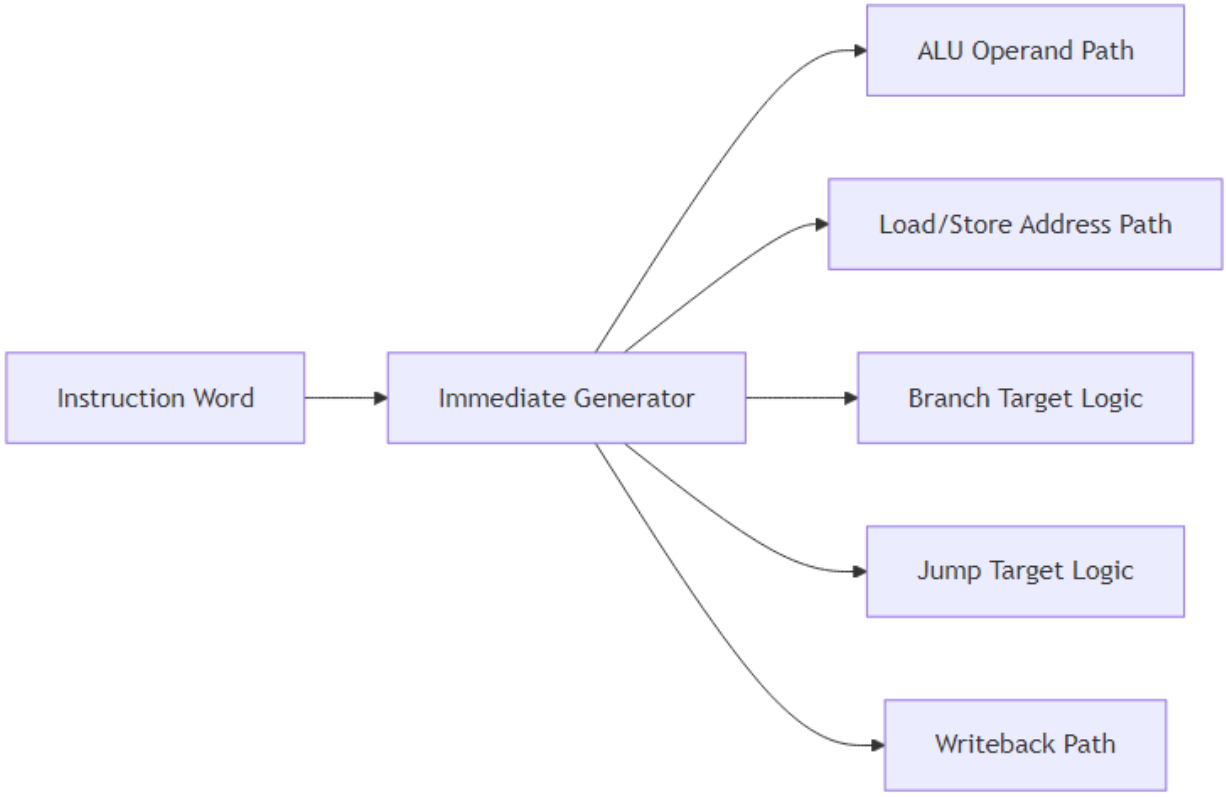
Tradeoffs include:

- additional wiring complexity
- dependence on correct format selection
- possible timing contribution to the decode stage

However, these costs are small compared to the benefits of modularity and clarity.

7.17 Summary Diagram

The following diagram summarizes the role of the Immediate Generator in the CPU.



This highlights the Immediate Generator as a central decode component used by many parts of the datapath.

7.18 Final Summary

The **Immediate Generator** module reconstructs immediate values from the instruction encoding and provides a consistent 32-bit operand for the CPU datapath.

It supports multiple instruction formats defined by RV32I, including:

- I-type
- S-type
- B-type
- U-type
- J-type

The module extracts immediate bits, rearranges them into the correct order, applies sign extension where necessary, and forwards the resulting value to execution units such as the ALU, load/store path, and control-flow logic.

Although conceptually simple, the Immediate Generator plays a fundamental role in enabling immediate-based arithmetic, address generation, and control-flow operations within the RV32I CPU.

8 Instruction Memory Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

8.1 Purpose of the Module

The **Instruction Memory** module stores the program executed by the CPU.

In the RV32I Minimal SoC architecture, the Instruction Memory provides the instruction word corresponding to the current value of the Program Counter (PC). The CPU fetches one instruction per clock cycle in the single-cycle execution model.

This module is therefore responsible for:

- storing the program instructions
- returning the instruction located at the address requested by the PC
- supporting deterministic instruction fetch during execution

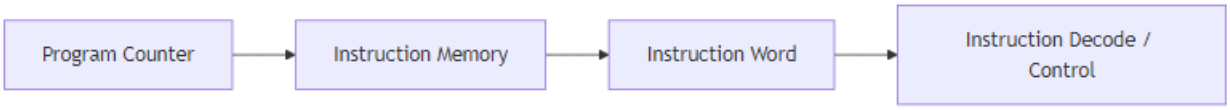
Instruction memory is **read-only during program execution**, which simplifies the architecture and ensures predictable instruction behavior.

8.2 Architectural Role Inside the CPU

Instruction Memory belongs to the **instruction fetch stage** of the CPU.

Its main role is to translate the current Program Counter value into the instruction that must be executed.

Conceptually:



The instruction word is then forwarded to multiple decode and execution components of the CPU.

8.3 Harvard-like Memory Organization

The RV32I Minimal SoC follows a **Harvard-like architecture**, meaning that instruction and data memory paths are separated.

This means:

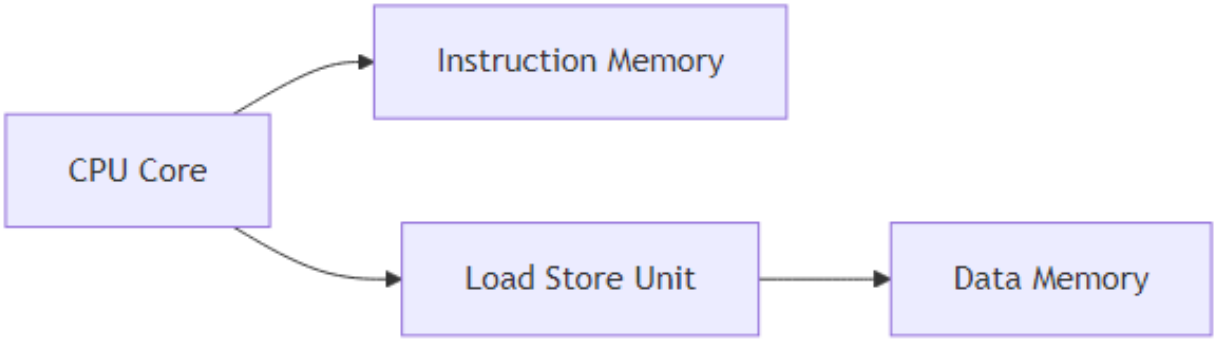
- **Instruction Memory** is used only for fetching instructions
- **Data Memory** is used for load/store operations

Advantages of this separation include:

- simpler memory access logic

- reduced contention between instruction and data accesses
- clearer datapath structure
- easier debugging and verification

Conceptually:



This organization allows the CPU to fetch instructions while also performing data operations without architectural ambiguity.

8.4 Inputs and Outputs

The exact port names depend on the VHDL implementation, but the architectural interface is straightforward.

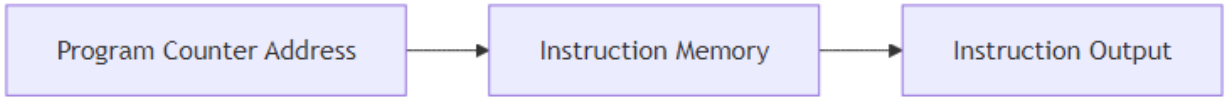
8.4.1 Inputs

- instruction address (from Program Counter)

8.4.2 Outputs

- instruction word

Conceptually:



The returned instruction is typically **32 bits wide**, matching the RV32I instruction size.

8.5 Instruction Width

The RV32I instruction set uses **fixed-length 32-bit instructions**.

Therefore:

- every instruction occupies one 32-bit word
- instruction addresses increment by 4 bytes

- instruction alignment is naturally enforced

Example instruction sequence:

```
Address     Instruction
0x00000000  instruction 0
0x00000004  instruction 1
0x00000008  instruction 2
0x0000000C  instruction 3
```

This regular structure simplifies instruction fetch and decode.

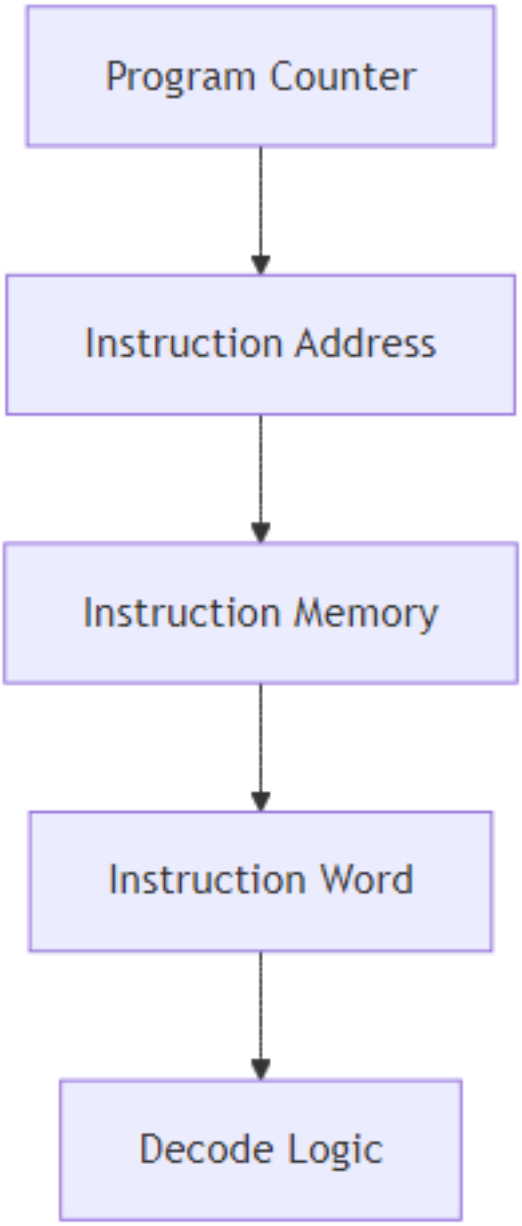
8.6 Addressing Behavior

Instruction Memory receives the current Program Counter value and returns the instruction stored at that address.

Typical fetch flow:

1. PC provides instruction address
2. instruction memory reads the corresponding word
3. instruction is forwarded to decode logic

Conceptually:



In a single-cycle CPU, this entire process happens within the same clock cycle.

8.7 Instruction Storage Model

Instruction Memory typically contains a preloaded program image.

Common approaches include:

- initialized memory arrays in VHDL
- memory initialization files
- ROM-style instruction storage

- simulation-only program loading

Example conceptual memory structure:

Each memory entry stores one instruction word.

8.8 Interaction with the Program Counter

The Program Counter determines which instruction will be fetched.

Normal sequential execution follows the pattern:

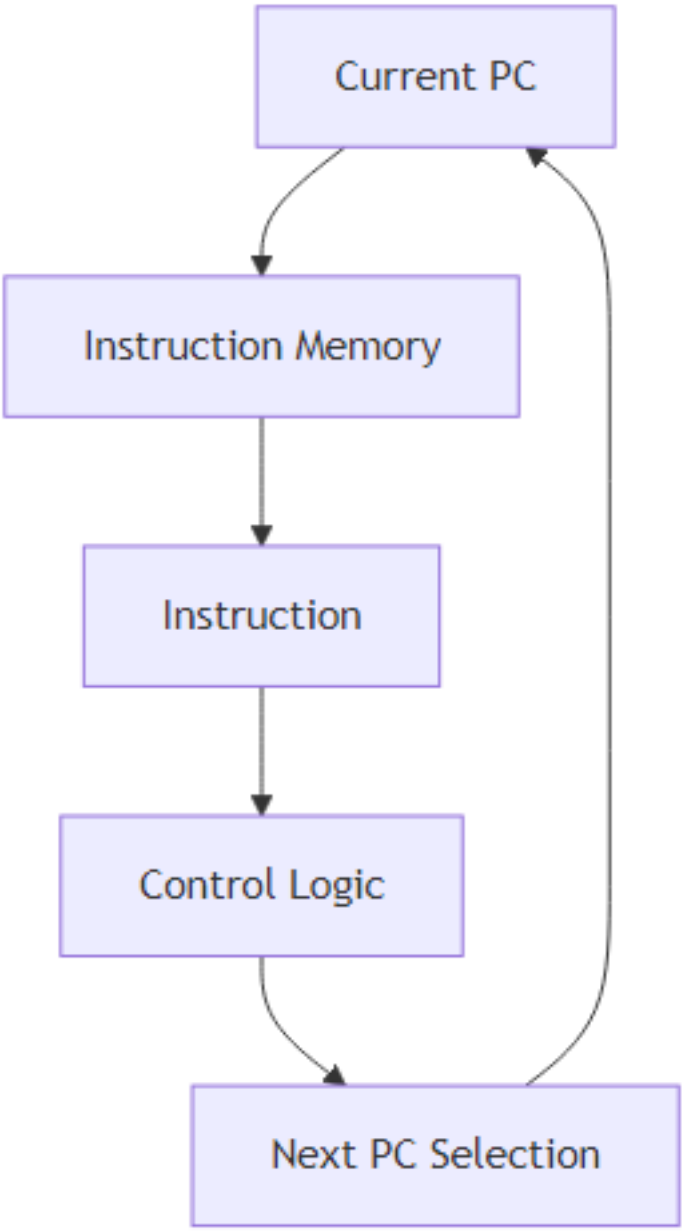
```
next PC = current PC + 4
```

However, control-flow instructions may change the PC.

Examples:

- branches
- jumps
- trap redirection

Conceptually:



This loop continues for every instruction cycle.

8.9 Read-Only Behavior

In the RV32I Minimal SoC design, instruction memory is treated as **read-only during execution**.

This means:

- the CPU cannot modify instructions
- no write interface is required
- the memory behaves like ROM during runtime

Advantages include:

- simpler design
- fewer control signals
- improved reliability
- easier simulation

Program loading typically occurs before execution begins.

8.10 Implementation Possibilities

The exact hardware implementation can vary depending on the target platform.

Common approaches include:

8.10.1 ROM-based Implementation

Instructions are hard-coded into a ROM structure.

Advantages:

- simple
- stable
- deterministic

8.10.2 Block RAM Initialization (FPGA)

Instructions are stored in FPGA block RAM initialized at configuration time.

Advantages:

- realistic hardware behavior
- scalable memory capacity

8.10.3 Simulation Memory Array

For simulation, instruction memory may be implemented using a VHDL array initialized from a file.

Advantages:

- flexible testing
- easy program replacement

8.11 Combinational vs Synchronous Behavior

Instruction memory can be implemented as either:

8.11.1 Combinational Read Memory

The instruction output changes immediately when the address changes.

Advantages:

- simple behavior
- ideal for educational single-cycle CPUs

8.11.2 Synchronous Memory

Instruction output changes on a clock edge.

Advantages:

- realistic hardware timing
- matches FPGA memory behavior

For a minimal single-cycle design, either approach can be acceptable depending on the implementation.

8.12 Role in the CPU Critical Path

Instruction memory lies at the **very beginning of the CPU critical path**.

Typical execution path:

1. Program Counter provides address
2. instruction memory returns instruction
3. Control Unit decodes instruction
4. operands are selected
5. ALU or memory operations occur

Because of this, instruction memory latency contributes directly to the maximum achievable clock frequency.

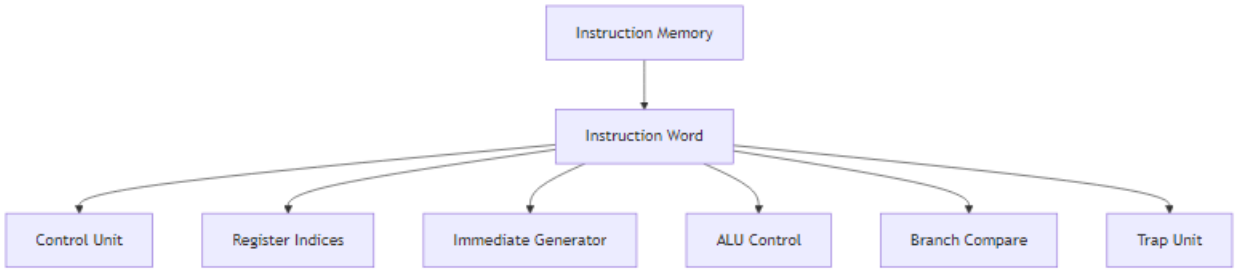
8.13 Interaction with Decode Logic

The instruction output feeds the decode stage.

Instruction fields are distributed to several modules:

- Control Unit
- Register File
- Immediate Generator
- ALU Control
- Branch Compare
- Trap Unit

Conceptually:



This makes instruction memory the entry point for all instruction-driven behavior.

8.14 Verification Expectations

Instruction memory is relatively simple to verify compared to execution units.

Typical verification goals include:

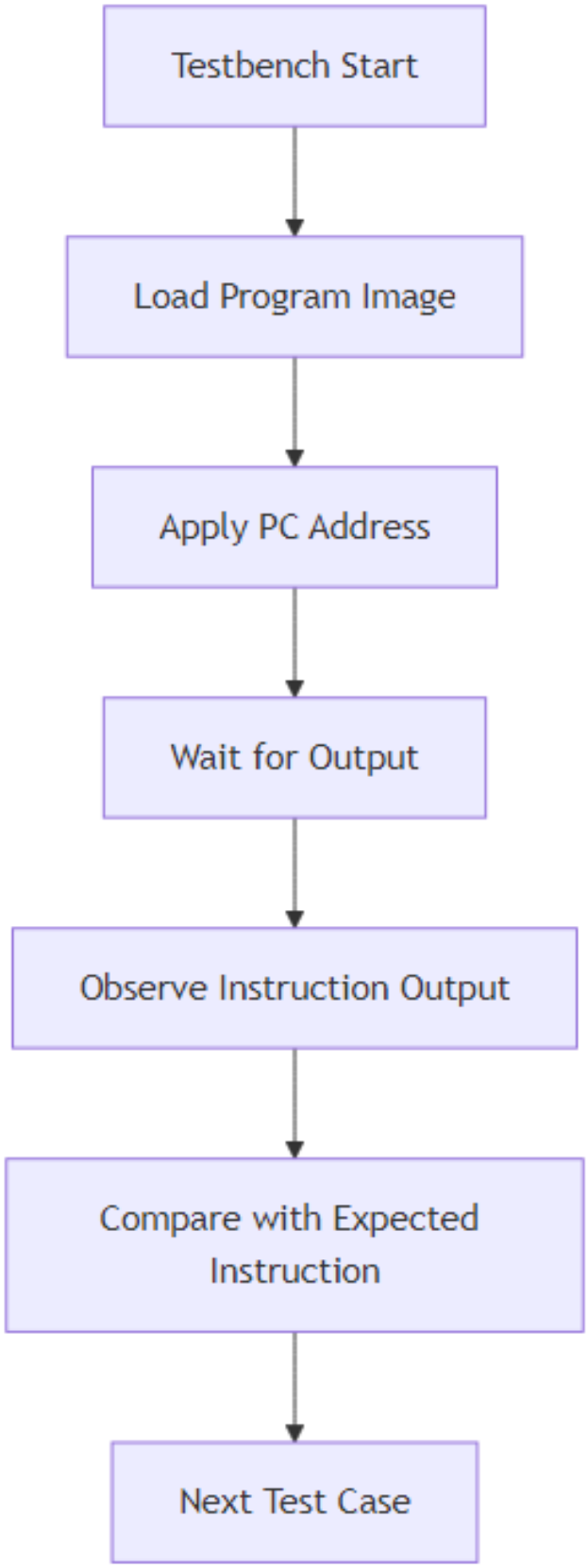
- correct instruction retrieval
- correct addressing behavior
- correct instruction alignment
- correct program initialization

Example test scenarios:

- sequential instruction fetch
- branch target fetch
- jump target fetch
- boundary addresses

8.15 Example Verification Flow

A basic testbench procedure may follow this structure:


Testbench Start
Load Program Image
Apply PC Address
Wait for Output
Observe Instruction Output
Compare with Expected Instruction
Next Test Case

Because the module is deterministic and stateless during execution, verification is straightforward.

8.16 Design Strengths of the Module

The Instruction Memory module contributes several advantages to the CPU design.

8.16.1 Simplicity

Its functionality is limited to instruction storage and retrieval.

8.16.2 Clear Architecture

It cleanly separates instruction fetch from data memory operations.

8.16.3 Deterministic Behavior

Instruction output depends only on the address input.

8.16.4 Easy Verification

The module has minimal logic and predictable behavior.

8.17 Limitations and Tradeoffs

A simple instruction memory implementation introduces some limitations.

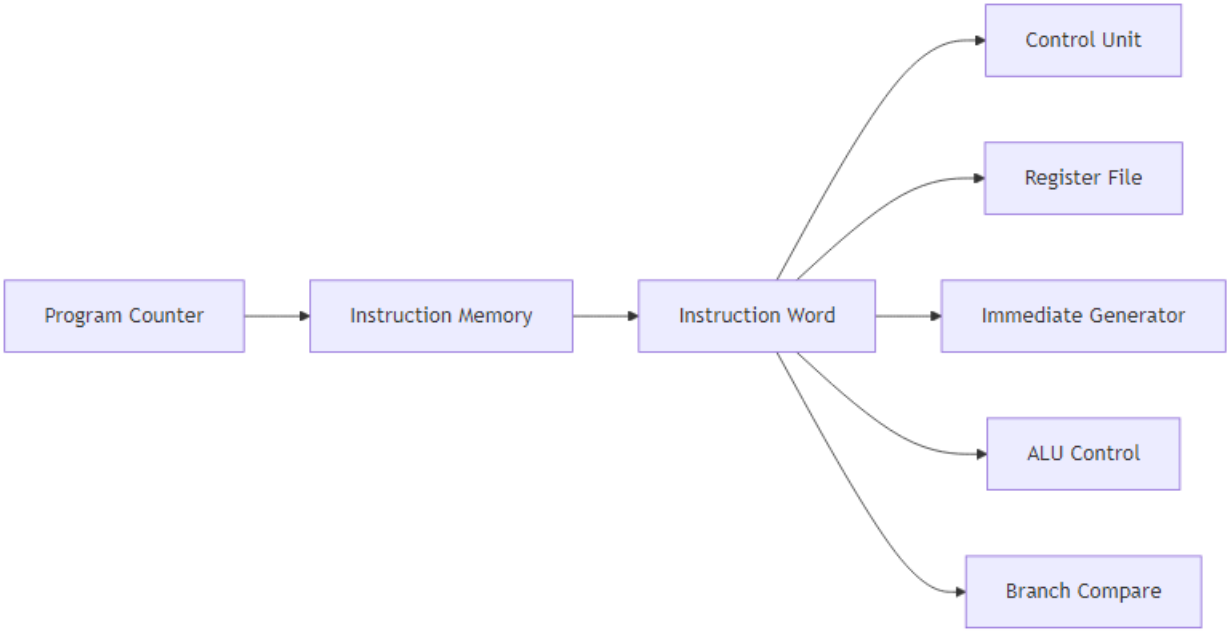
Possible tradeoffs include:

- fixed program image during execution
- limited flexibility for dynamic code modification
- dependence on correct initialization

However, these tradeoffs are acceptable for a minimal educational CPU design.

8.18 Summary Diagram

The following diagram summarizes the role of Instruction Memory in the CPU.



Instruction Memory therefore acts as the **entry point of the execution pipeline**.

8.19 Final Summary

The **Instruction Memory module** stores the program executed by the RV32I Minimal SoC CPU.

It receives the current Program Counter address and returns the corresponding 32-bit instruction word.

The module plays a fundamental role in the instruction fetch stage and provides the starting point for instruction decoding and execution.

Although conceptually simple, Instruction Memory is critical because every instruction executed by the CPU originates from it. Its design emphasizes clarity, deterministic behavior, and straightforward integration with the Program Counter and decode logic.

9 Load Store Unit Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

9.1 Purpose of the Module

The **Load Store Unit (LSU)** is responsible for handling all data memory access operations in the CPU.

In the RV32I architecture, memory operations occur through explicit load and store instructions. The Load Store Unit provides the logic that connects the CPU datapath with the system memory

and memory-mapped peripherals.

The LSU is therefore responsible for:

- generating memory addresses
- performing memory reads
- performing memory writes
- routing memory data between the CPU and memory

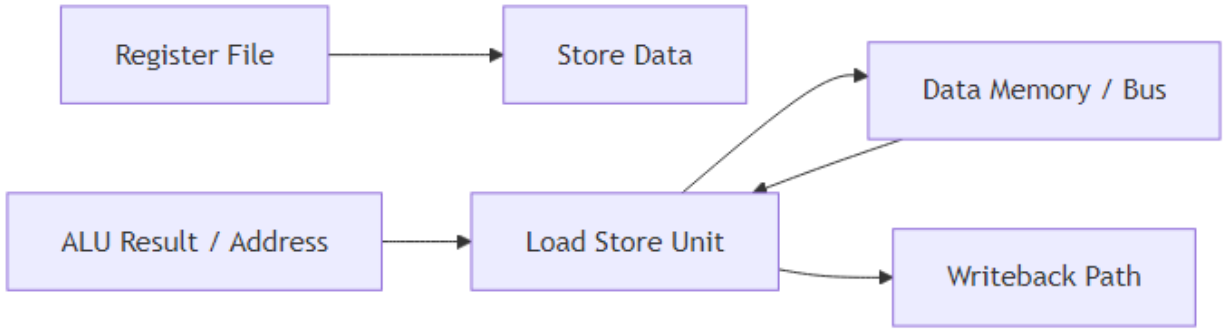
This module is essential because it enables interaction between the CPU and the external system state stored in memory.

9.2 Architectural Role Inside the CPU

The Load Store Unit belongs to the **data memory access stage** of the CPU.

It sits between the ALU datapath and the system bus or memory system.

Conceptually:



This makes the LSU the bridge between the CPU core and the memory system.

9.3 Relationship with RV32I Memory Instructions

The RV32I ISA defines two main categories of memory operations:

9.3.1 Load Instructions

Load instructions read data from memory and write the result to a register.

Examples:

```
lw rd, offset(rs1)
```

The address is calculated as:

```
address = rs1 + offset
```

The value read from memory is written into register `rd`.

9.3.2 Store Instructions

Store instructions write data from a register into memory.

Examples:

```
sw rs2, offset(rs1)
```

The address is calculated as:

```
address = rs1 + offset
```

The value in register `rs2` is written to memory.

9.4 Address Generation

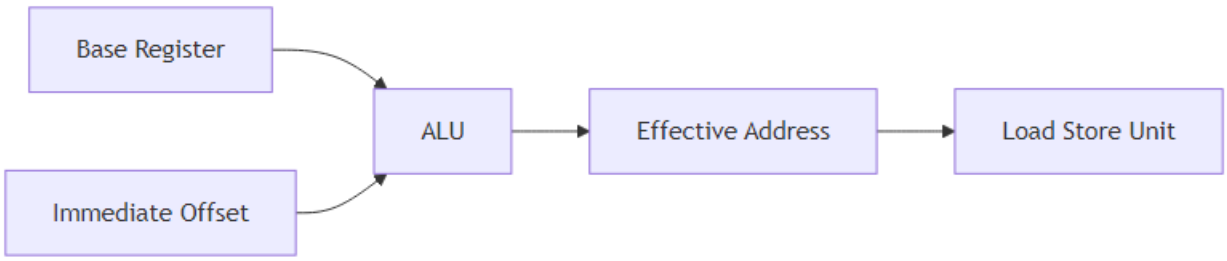
The effective memory address used by the LSU is typically generated by the ALU.

The ALU performs:

```
effective_address = base_register + immediate_offset
```

This address is then provided to the Load Store Unit.

Conceptually:



The LSU then performs the appropriate memory transaction.

9.5 Inputs and Outputs

Although the exact VHDL signal names depend on the implementation, the architectural interface of the LSU is straightforward.

9.5.1 Inputs

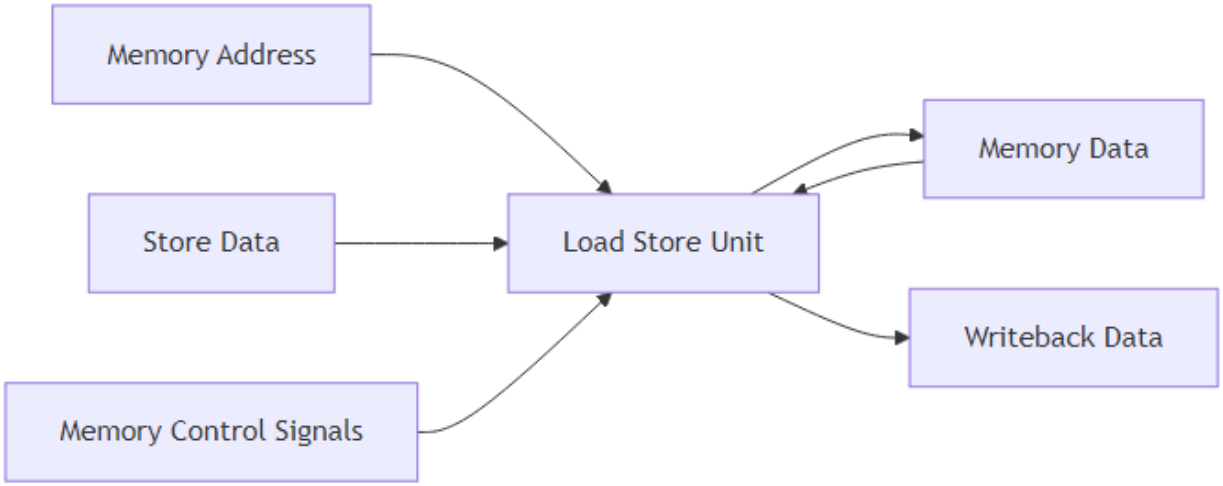
- effective memory address from the ALU
- store data from the register file
- memory operation type from the Control Unit

9.5.2 Outputs

- data read from memory
- write enable signals for memory

- data forwarded to the register writeback path

Conceptually:

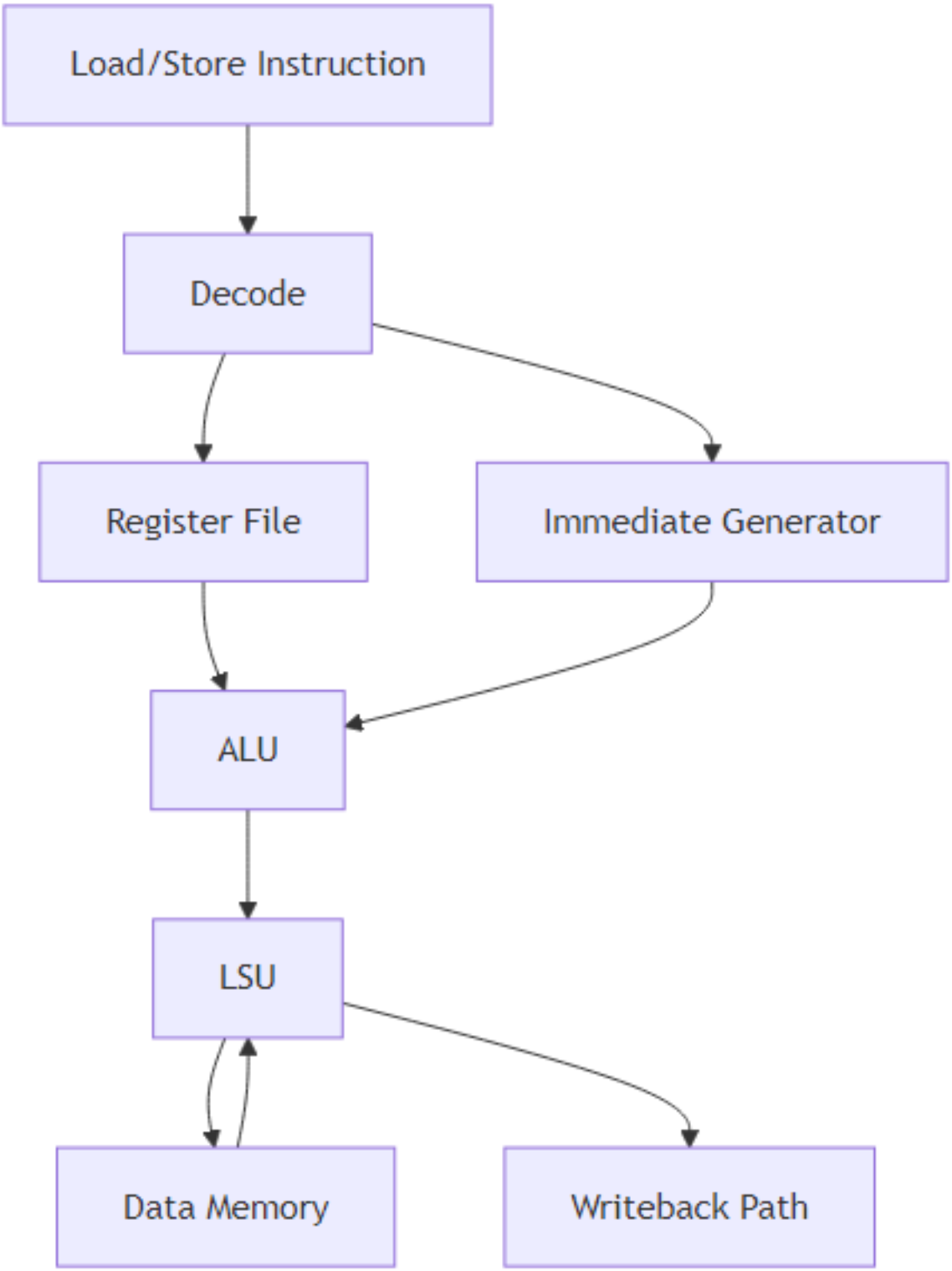


9.6 Memory Access Flow

The typical load/store execution flow is as follows:

1. instruction is decoded
2. register operands are read
3. immediate value is generated
4. ALU computes the effective address
5. LSU performs the memory transaction
6. data is forwarded to writeback (for loads)

Conceptually:



This sequence happens within a single cycle in the current CPU architecture.

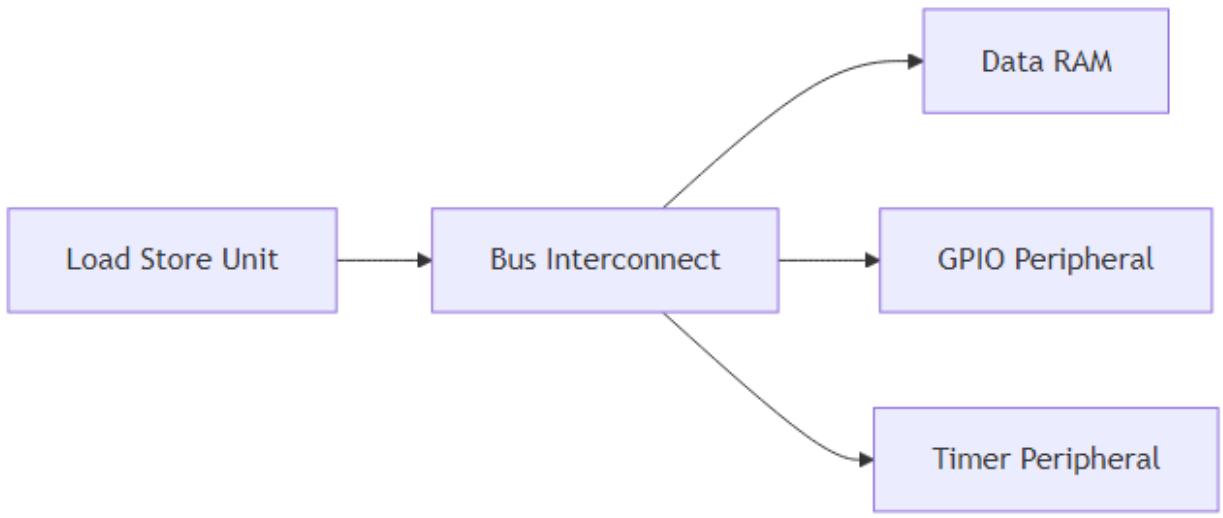
9.7 Interaction with the System Bus

The Load Store Unit does not interact only with RAM. It may also communicate with **memory-mapped peripherals** through the system bus.

Possible destinations include:

- RAM
- GPIO
- Timer
- other peripherals

Conceptually:



The bus interconnect determines which device receives the request.

9.8 Memory Read Operations

For load instructions, the LSU performs a memory read operation.

Steps:

1. address is provided to the memory system
2. memory returns the stored value
3. LSU forwards the value to the writeback path
4. the register file writes the value to `rd`

Conceptually:

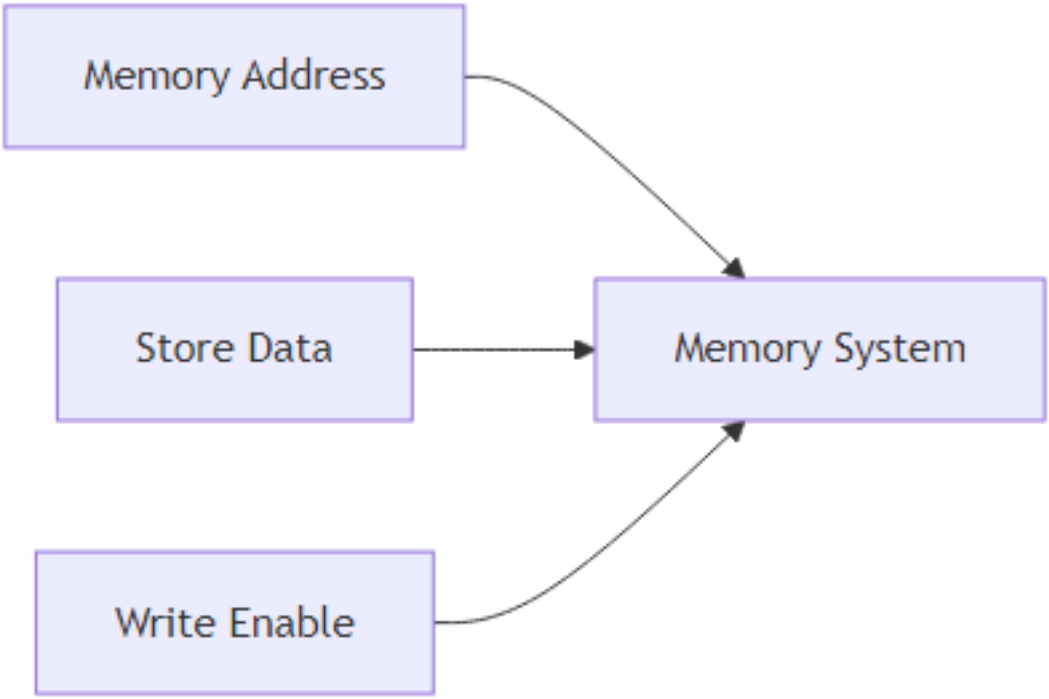
9.9 Memory Write Operations

For store instructions, the LSU performs a memory write operation.

Steps:

1. effective address is generated
2. store data is provided from the register file
3. LSU asserts write enable
4. memory stores the provided data

Conceptually:



In this case, no value is returned to the register file.

9.10 Alignment and Addressing Considerations

In RV32I, memory operations typically assume **word-aligned addresses** for 32-bit accesses.

Word alignment means:

```
address % 4 = 0
```

This simplifies the memory interface and avoids misaligned access complexities.

In minimal educational CPU designs, misaligned accesses may be:

- disallowed
- handled as traps
- ignored depending on implementation choices

9.11 Interaction with Other CPU Modules

The Load Store Unit interacts closely with several CPU components.

9.11.1 ALU

Provides the effective memory address.

9.11.2 Register File

Provides store data and receives load results.

9.11.3 Control Unit

Indicates whether the operation is:

- load
- store
- no memory access

9.11.4 Bus Interconnect

Routes memory requests to RAM or peripherals.

9.11.5 Writeback Logic

Receives data read from memory during load instructions.

9.12 Combinational vs Sequential Behavior

The LSU may contain both combinational and sequential logic depending on the memory interface.

Typical responsibilities include:

- combinational routing of addresses and data
- generation of control signals
- synchronization with memory timing

In a simple single-cycle design, the LSU is typically implemented with minimal internal state.

9.13 Role in the CPU Critical Path

For load instructions, the LSU participates in the CPU critical path.

A typical load instruction path may include:

1. instruction fetch
2. decode
3. register file read

4. address generation in ALU
5. memory read
6. register writeback

Because the memory access occurs within the same cycle, LSU timing directly affects the maximum clock frequency of the processor.

9.14 Verification Expectations

A good LSU testbench should verify:

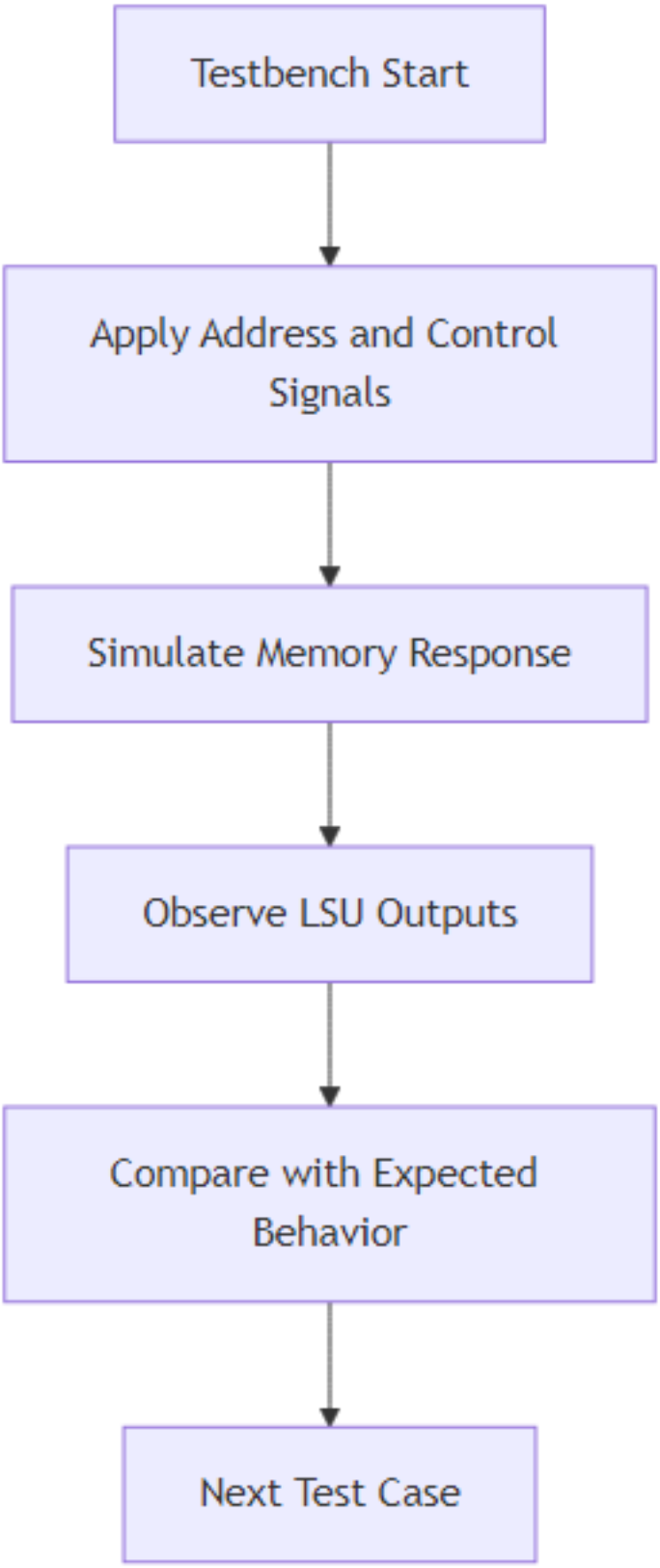
- correct address forwarding
- correct store operations
- correct load operations
- correct writeback behavior
- correct bus routing
- correct interaction with memory

Example scenarios include:

- load from RAM
- store to RAM
- peripheral access
- sequential memory operations
- edge address cases

9.15 Example Verification Flow

A basic LSU verification sequence may follow this pattern:


Testbench Start
Apply Address and Control Signals
Simulate Memory Response
Observe LSU Outputs
Compare with Expected Behavior
Next Test Case

Because the LSU interacts with memory, verification often includes memory models inside the testbench.

9.16 Design Strengths of the Module

The Load Store Unit contributes several architectural advantages.

9.16.1 Clear Memory Interface

All memory transactions are centralized in one module.

9.16.2 Clean Separation

The CPU datapath remains independent from memory implementation details.

9.16.3 Peripheral Integration

Memory-mapped peripherals can easily be integrated.

9.16.4 Scalable Design

The LSU can be extended to support additional memory operations.

9.17 Limitations and Tradeoffs

A minimal LSU design may have some limitations.

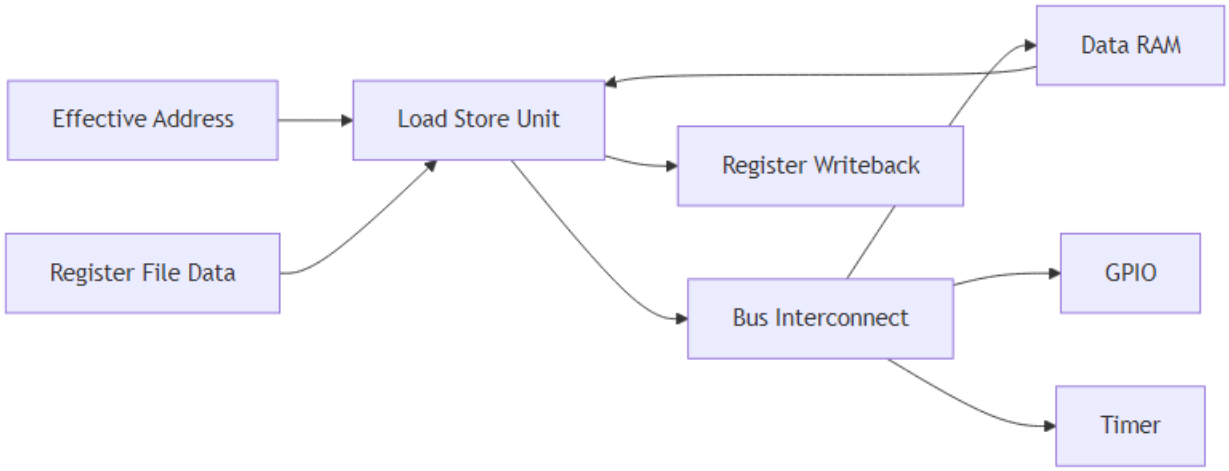
Possible tradeoffs include:

- no support for misaligned accesses
- limited access width support
- simplified bus protocol

However, these simplifications are appropriate for an educational single-cycle RISC-V processor.

9.18 Summary Diagram

The following diagram summarizes the role of the Load Store Unit.



This diagram highlights the LSU as the interface between the CPU datapath and the system memory architecture.

9.19 Final Summary

The **Load Store Unit** manages all memory interactions of the RV32I Minimal SoC CPU.

It receives effective addresses from the ALU, performs memory reads and writes, and forwards load results to the register writeback path.

The LSU connects the CPU core with the system memory and memory-mapped peripherals through the bus interconnect.

Although conceptually simple, the module plays a crucial role in enabling data storage, program state management, and interaction with external devices in the overall SoC architecture.

10 Program Counter Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

10.1 Purpose of the Module

The **Program Counter (PC)** is the register that holds the address of the instruction currently being executed by the CPU.

At every instruction cycle, the Program Counter provides the address used by the Instruction Memory to fetch the next instruction.

The Program Counter therefore controls the **flow of program execution**.

Its responsibilities include:

- storing the current instruction address

- providing the address to instruction memory
- updating the address for the next instruction
- redirecting execution during branches and jumps
- responding to trap or control-flow redirection events

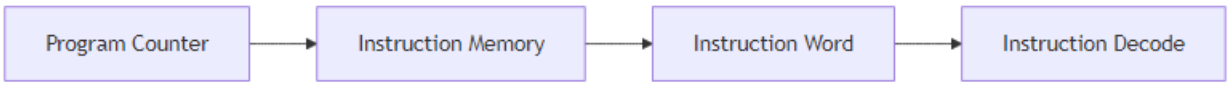
Because every instruction begins with a PC value, the Program Counter is one of the most fundamental components of the CPU.

10.2 Architectural Role Inside the CPU

The Program Counter belongs to the **instruction fetch stage** of the CPU.

It provides the address used to access instruction memory and determines the order in which instructions are executed.

Conceptually:



The PC is updated once per instruction cycle.

10.3 Sequential Instruction Execution

During normal program execution, instructions are executed sequentially.

Since RV32I instructions are **32 bits (4 bytes)** wide, the next instruction address is calculated as:

```
next_pc = current_pc + 4
```

This results in the following instruction sequence:

```
Address      Instruction
0x00000000   instruction 0
0x00000004   instruction 1
0x00000008   instruction 2
0x0000000C   instruction 3
```

Sequential execution continues unless a control-flow instruction modifies the PC.

10.4 Program Counter Update Sources

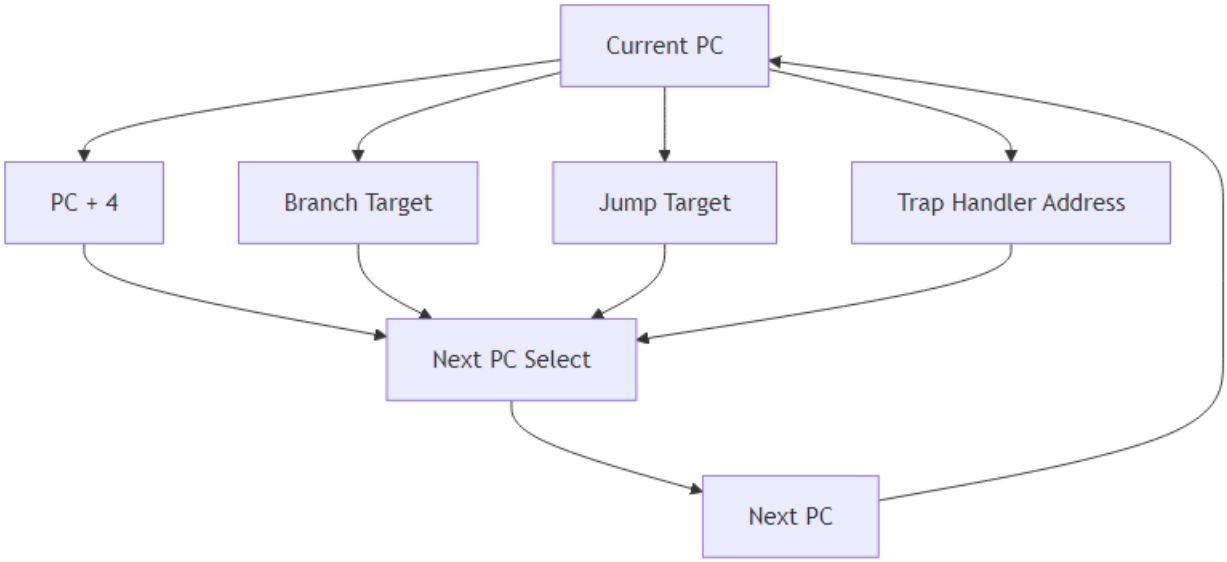
The Program Counter does not always increment by 4. Several instruction types can modify the next PC value.

Typical PC update sources include:

- sequential execution (`PC + 4`)
- branch target addresses

- jump target addresses
- trap or exception handlers

Conceptually:



A multiplexer selects which value becomes the next PC.

10.5 Inputs and Outputs

The exact signal names depend on the VHDL implementation, but the architectural interface of the Program Counter module is straightforward.

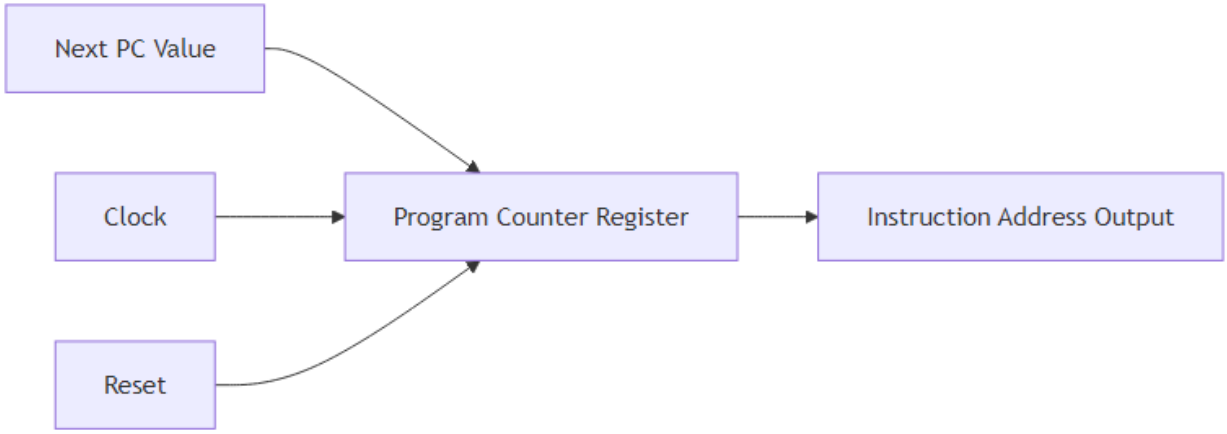
10.5.1 Inputs

- clock
- reset signal
- next PC value
- PC update enable

10.5.2 Outputs

- current instruction address

Conceptually:



The PC register updates on the active clock edge.

10.6 Reset Behavior

When the CPU is reset, the Program Counter must be initialized to a known starting address.

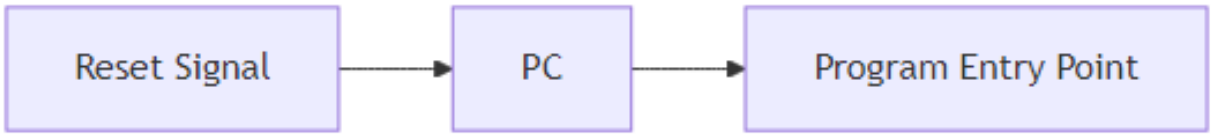
Typically this is the **reset vector**, which is the first instruction executed by the CPU.

Example:

```
PC = 0x00000000
```

This allows the CPU to begin executing the program stored at the start of instruction memory.

Conceptually:



After reset, normal PC update logic resumes.

10.7 Interaction with Branch Instructions

Branch instructions modify the PC when a condition is satisfied.

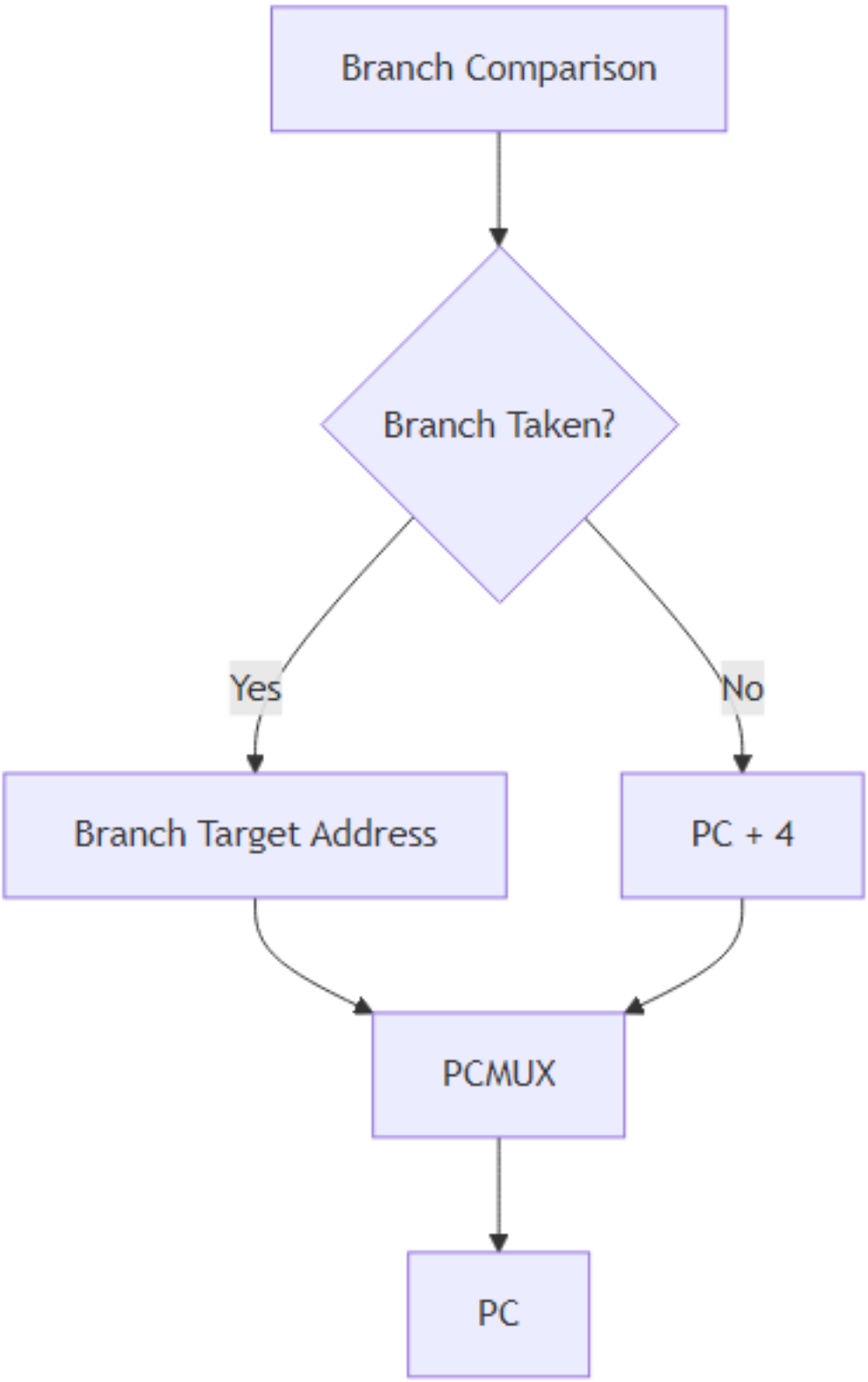
Example:

```
beq rs1, rs2, offset
```

Branch behavior:

1. operands are compared
2. branch condition is evaluated
3. if true, the PC is updated with the branch target
4. otherwise execution continues with `PC + 4`

Conceptually:



The Program Counter therefore implements conditional control flow.

10.8 Interaction with Jump Instructions

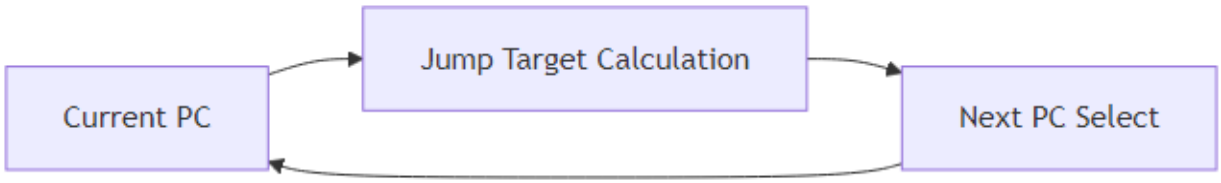
Jump instructions unconditionally modify the Program Counter.

Example instructions:

```
jal
jalr
```

These instructions compute a new target address that becomes the next PC.

Conceptually:



Jump instructions allow programs to implement:

- function calls
- returns
- unconditional control transfer

10.9 Interaction with Trap Handling

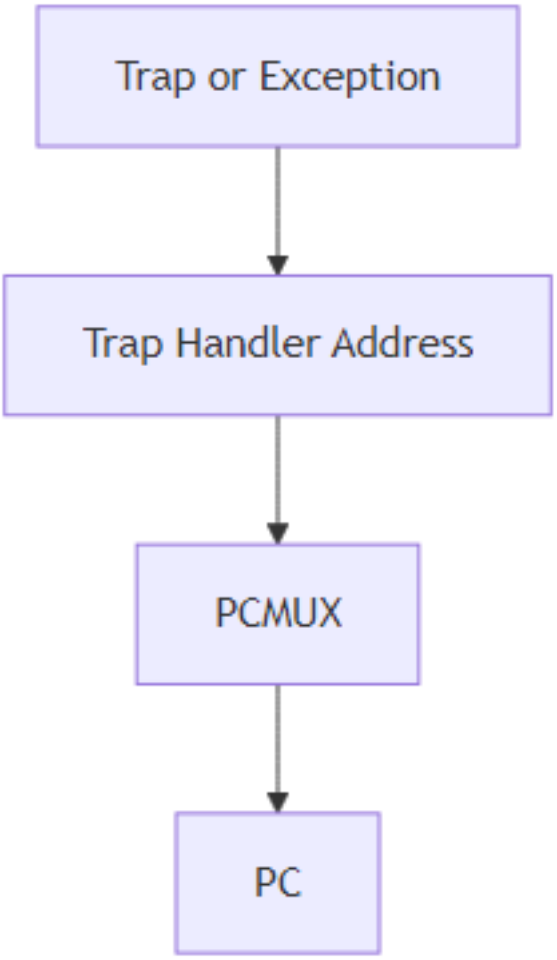
The Program Counter may also be redirected when exceptional events occur.

Examples include:

- illegal instruction detection
- system traps
- interrupt requests (future extensions)

In these cases, the PC is redirected to a **trap handler address**.

Conceptually:



This ensures that exceptional conditions can be handled safely.

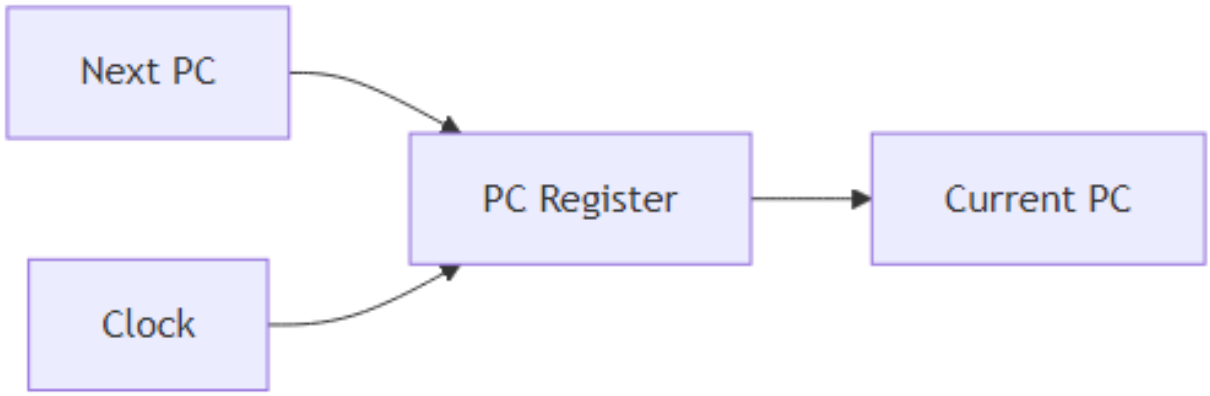
10.10 Program Counter as a Register

The Program Counter is implemented as a **state-holding register**.

Characteristics:

- updated on clock edge
- holds 32-bit address
- retains value between cycles

Conceptually:



This stored value becomes the address used by instruction memory.

10.11 Role in the CPU Critical Path

The Program Counter participates in the instruction fetch path.

A typical instruction cycle includes:

1. PC provides address
2. instruction memory returns instruction
3. decode logic interprets instruction
4. execution occurs
5. next PC is selected
6. PC updates on next clock edge

Because the PC feeds instruction memory, its timing affects the start of every instruction execution cycle.

10.12 Interaction with Other CPU Modules

The Program Counter interacts with multiple components of the CPU.

10.12.1 Instruction Memory

Receives the PC address to fetch instructions.

10.12.2 Branch Compare

Determines whether conditional branches redirect the PC.

10.12.3 Immediate Generator

Provides branch and jump offsets.

10.12.4 ALU

May participate in target address calculations.

10.12.5 Trap Unit

Can override normal PC update logic during exceptional conditions.

10.13 Verification Expectations

A Program Counter testbench should verify:

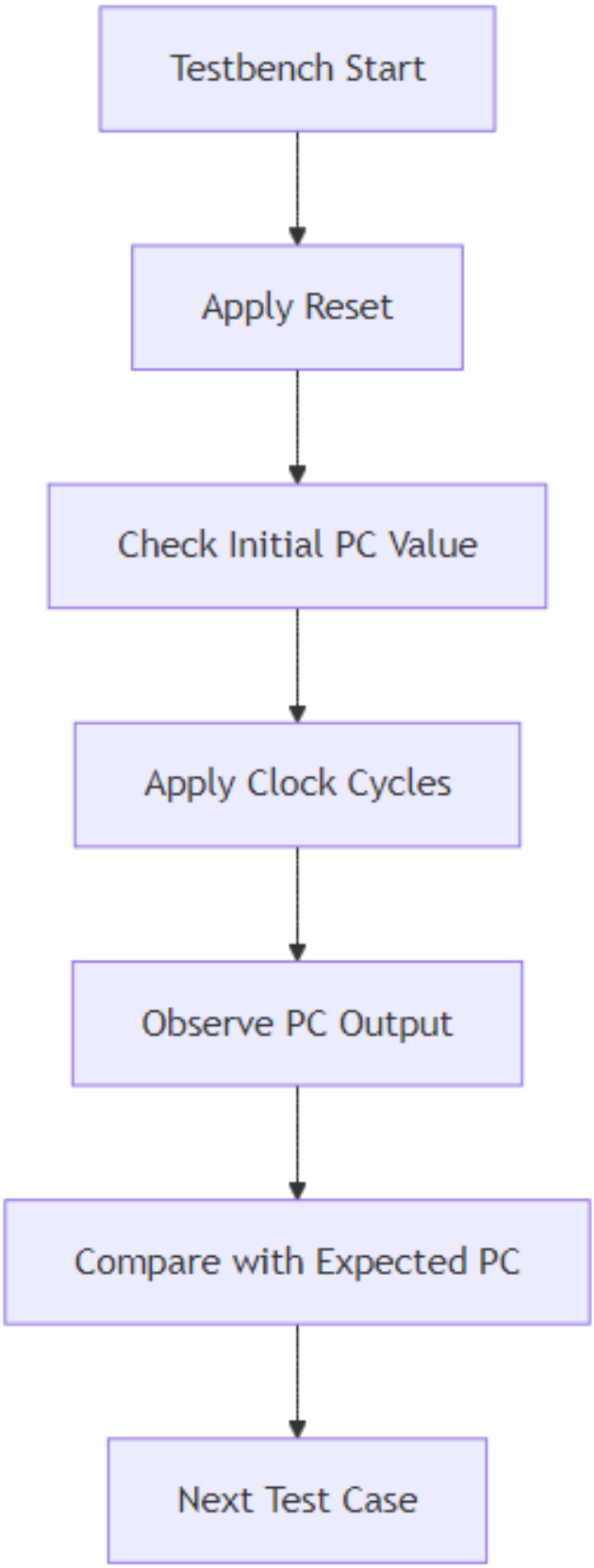
- correct reset initialization
- sequential PC increment
- correct branch redirection
- correct jump redirection
- correct trap redirection
- stable behavior across clock cycles

Example test scenarios:

- sequential instruction execution
- taken branch
- non-taken branch
- jump instruction
- reset behavior

10.14 Example Verification Flow

A typical verification procedure may follow this structure:


Testbench Start
Apply Reset
Check Initial PC Value
Apply Clock Cycles
Observe PC Output
Compare with Expected PC
Next Test Case

Because the Program Counter is a stateful module, verification focuses on correct updates across clock cycles.

10.15 Design Strengths of the Module

The Program Counter module offers several architectural benefits.

10.15.1 Simple Control Flow

Sequential execution is easy to implement using `PC + 4`.

10.15.2 Centralized Execution Control

All instruction execution begins with the PC.

10.15.3 Flexible Redirection

Branches, jumps, and traps can easily redirect execution.

10.15.4 Easy Verification

The module has a clear and predictable state transition behavior.

10.16 Limitations and Tradeoffs

A minimal Program Counter implementation may include some simplifications.

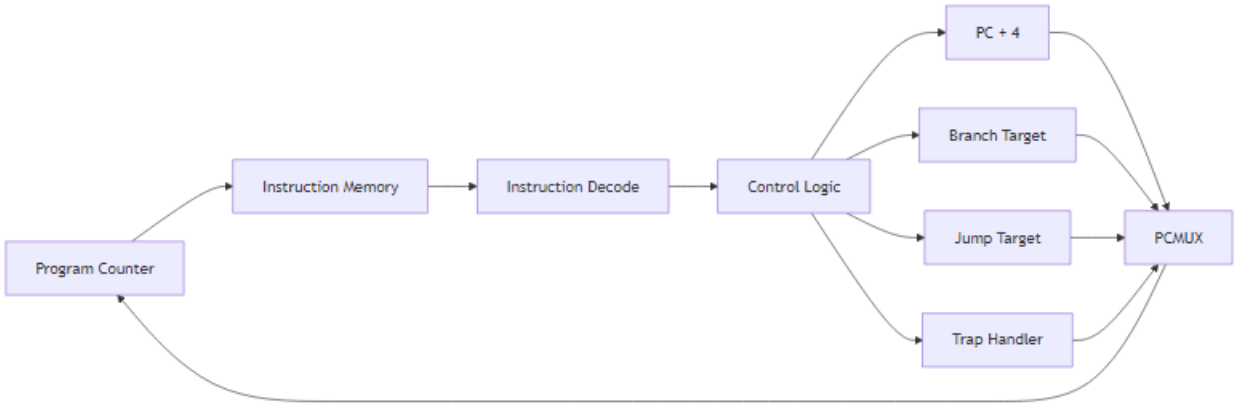
Possible limitations include:

- limited support for advanced control-flow features
- simple trap redirection mechanisms
- no speculative execution

These simplifications are appropriate for a single-cycle educational CPU.

10.17 Summary Diagram

The following diagram summarizes the role of the Program Counter.



This highlights the PC as the central controller of instruction flow.

10.18 Final Summary

The **Program Counter** is the register that determines which instruction the CPU executes next.

It provides instruction addresses to instruction memory and updates its value based on sequential execution, branches, jumps, and trap events.

As the starting point of every instruction cycle, the Program Counter plays a fundamental role in controlling program execution and maintaining correct instruction flow within the RV32I Minimal SoC CPU.

11 Register File Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

11.1 Purpose of the Module

The **Register File** is the storage unit that holds the general-purpose registers used by the CPU during program execution.

In the RV32I architecture, the register file contains **32 registers**, each 32 bits wide. These registers store operands, intermediate values, and results of instruction execution.

The Register File is responsible for:

- storing the CPU general-purpose registers
- providing operands to the execution units
- receiving results from the writeback stage
- enabling simultaneous reading of source operands

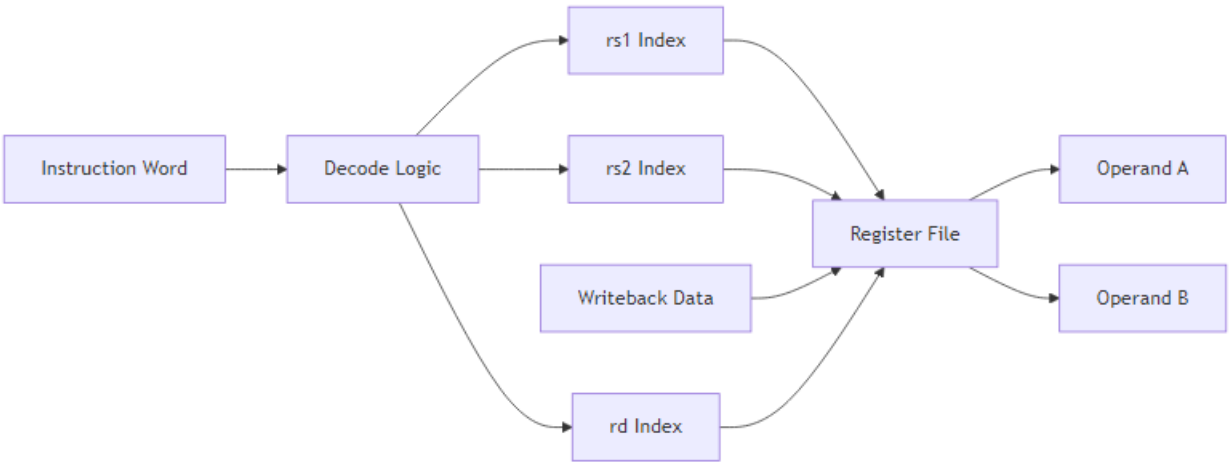
Because almost every instruction interacts with registers, the Register File is one of the most frequently accessed components of the CPU.

11.2 Architectural Role Inside the CPU

The Register File belongs to the **datapath execution stage** of the CPU.

It provides operand values used by the ALU, branch comparator, and other execution logic.

Conceptually:



This module acts as the main **operand storage and retrieval system** for the CPU.

11.3 Register Set in RV32I

The RV32I architecture defines **32 general-purpose registers**.

Each register is 32 bits wide.

Typical naming convention:

```
x0  - zero register
x1  - return address
x2  - stack pointer
x3  - global pointer
x4  - thread pointer
x5-x7
x8-x9
x10-x17
x18-x27
x28-x31
```

However, in hardware the registers are typically addressed by index values:

```
0 - 31
```

These indices are encoded in the instruction fields.

11.4 Special Behavior of Register x0

Register **x0** has special semantics in the RISC-V architecture.

Properties of register x0:

- always reads as **zero**
- writes to x0 are ignored

Example:

```
addi x0, x1, 5
```

Even though the instruction attempts to write to x0, the value is discarded and the register remains zero.

This behavior simplifies many instruction patterns and avoids the need for dedicated zero-generation hardware.

11.5 Inputs and Outputs

Although exact signal names depend on the VHDL implementation, the architectural interface is well defined.

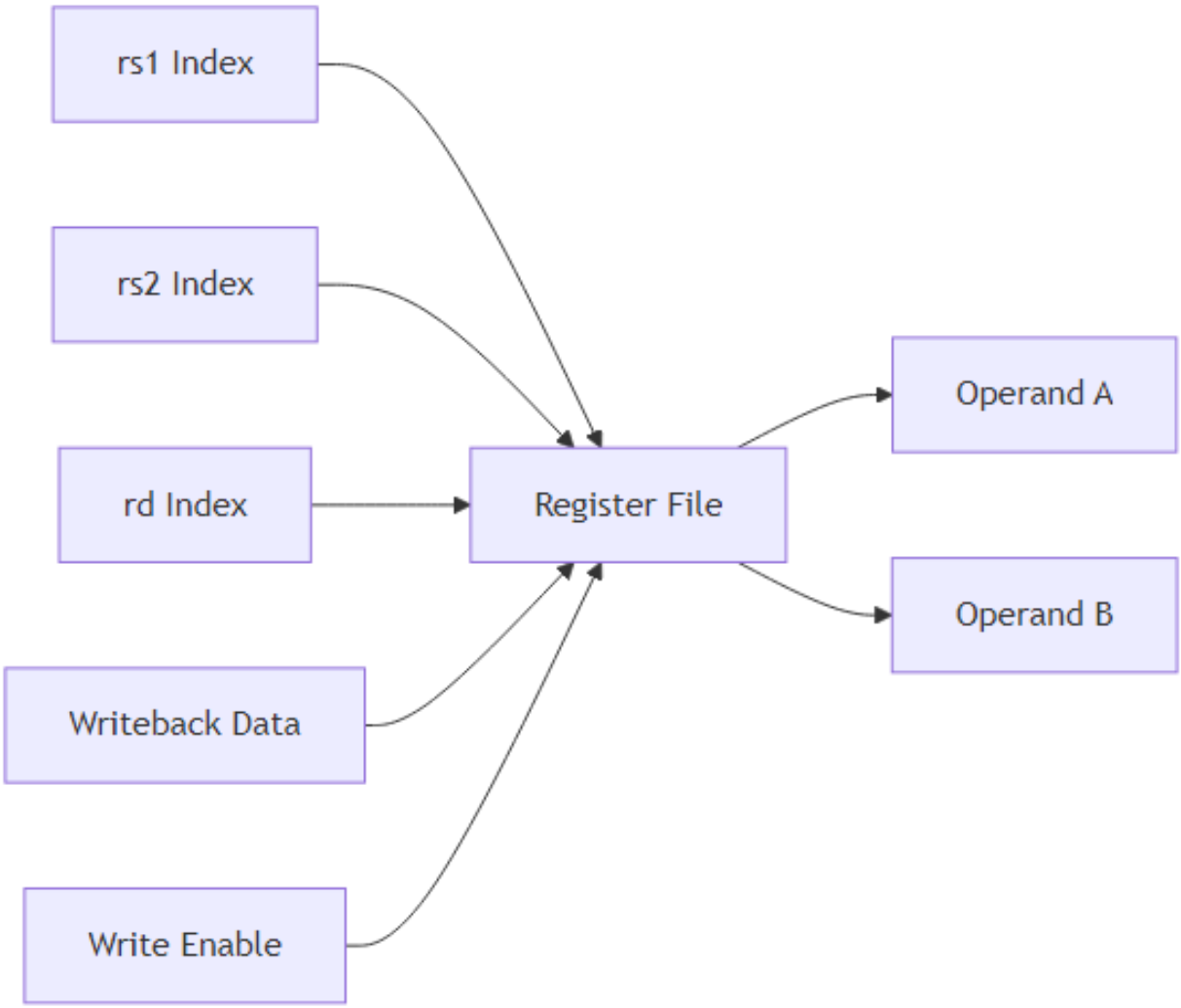
11.5.1 Inputs

- clock
- register write enable
- destination register index (`rd`)
- writeback data
- source register index (`rs1`)
- source register index (`rs2`)

11.5.2 Outputs

- operand value from `rs1`
- operand value from `rs2`

Conceptually:



The Register File therefore supports both **read and write operations**.

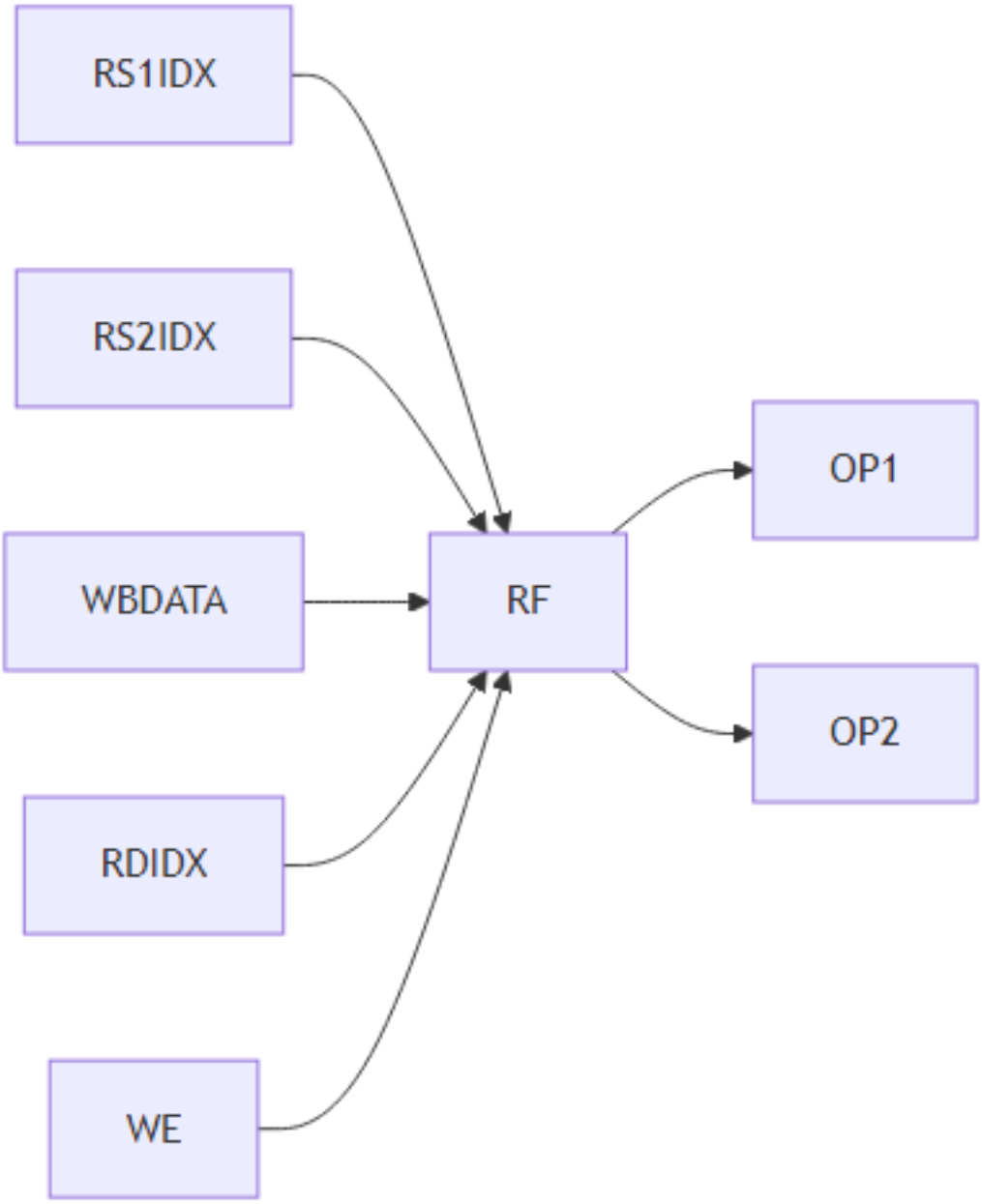
11.6 Dual Read Port Architecture

Most RISC-V CPUs use a **dual read port register file**.

This means:

- two registers can be read simultaneously
- one register can be written per cycle

Conceptually:



This allows instructions like:

`add x5, x1, x2`

to read `x1` and `x2` in the same cycle.

11.7 Register Read Behavior

Register reads are typically implemented as **combinational operations**.

This means:

- operands are available immediately after indices are applied
- no clock cycle is required to read register values

Conceptually:

This behavior is important for maintaining single-cycle CPU execution.

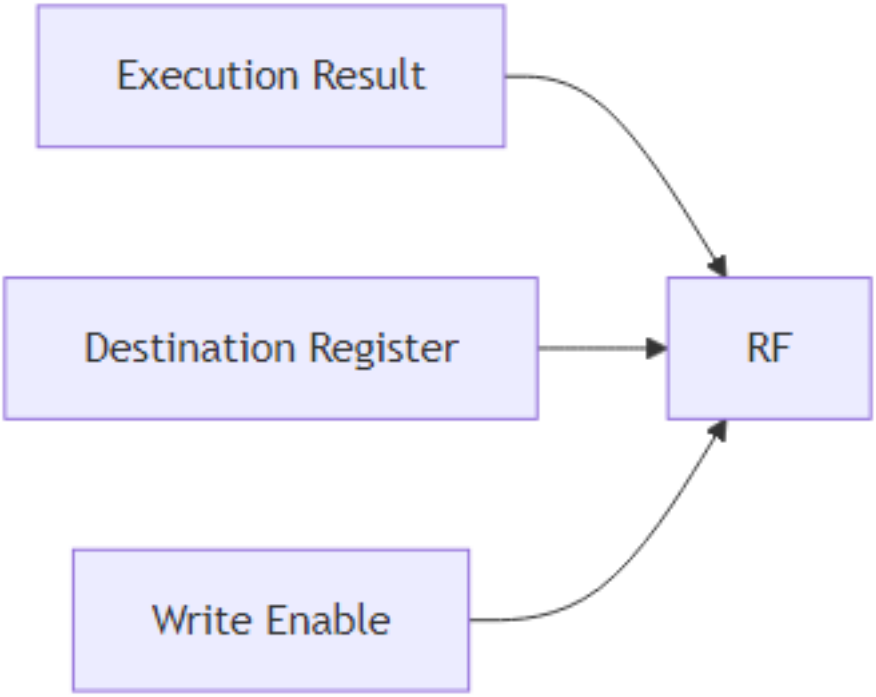
11.8 Register Write Behavior

Register writes occur during the **writeback stage** of instruction execution.

Typical process:

1. execution unit produces result
2. control logic determines destination register
3. write enable signal is asserted
4. result is written to the register file

Conceptually:



Writes usually occur on the **active clock edge**.

11.9 Instruction Interaction

Many instruction types interact with the Register File.

11.9.1 R-Type Instructions

Example:

```
add rd, rs1, rs2
```

Process:

1. read `rs1`
2. read `rs2`
3. ALU computes result
4. result written to `rd`

11.9.2 I-Type Instructions

Example:

```
addi rd, rs1, imm
```

Process:

1. read `rs1`
2. immediate generated
3. ALU computes result
4. result written to `rd`

11.9.3 Load Instructions

Example:

```
lw rd, offset(rs1)
```

Process:

1. read base register `rs1`
2. compute memory address
3. load data from memory
4. write loaded value to `rd`

11.9.4 Store Instructions

Example:

```
sw rs2, offset(rs1)
```

Process:

1. read `rs1`
2. read `rs2`
3. compute address
4. store `rs2` value to memory

No register write occurs.

11.10 Interaction with Other CPU Modules

The Register File interacts with many parts of the CPU datapath.

11.10.1 Control Unit

Determines when register writes occur.

11.10.2 ALU

Receives operand values from registers.

11.10.3 Immediate Generator

Provides immediate operands for certain instructions.

11.10.4 Branch Comparator

Uses register values to evaluate branch conditions.

11.10.5 Load Store Unit

Uses register values for memory addressing and store operations.

11.10.6 Writeback Logic

Provides the data written to destination registers.

11.11 Combinational and Sequential Components

The Register File typically combines both combinational and sequential logic.

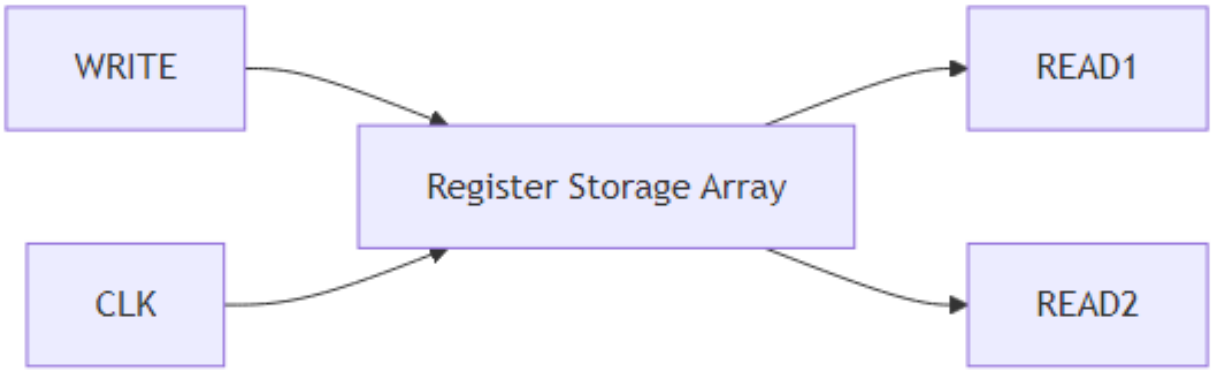
11.11.1 Combinational Logic

- register read paths

11.11.2 Sequential Logic

- register write storage

Conceptually:



11.12 Role in the CPU Critical Path

Register file access appears in the middle of the CPU datapath.

A typical instruction path may include:

1. instruction fetch
2. decode
3. register file read
4. ALU computation
5. memory access
6. writeback

Because operands must be available quickly, register read latency directly affects the CPU timing.

11.13 Verification Expectations

A Register File testbench should verify:

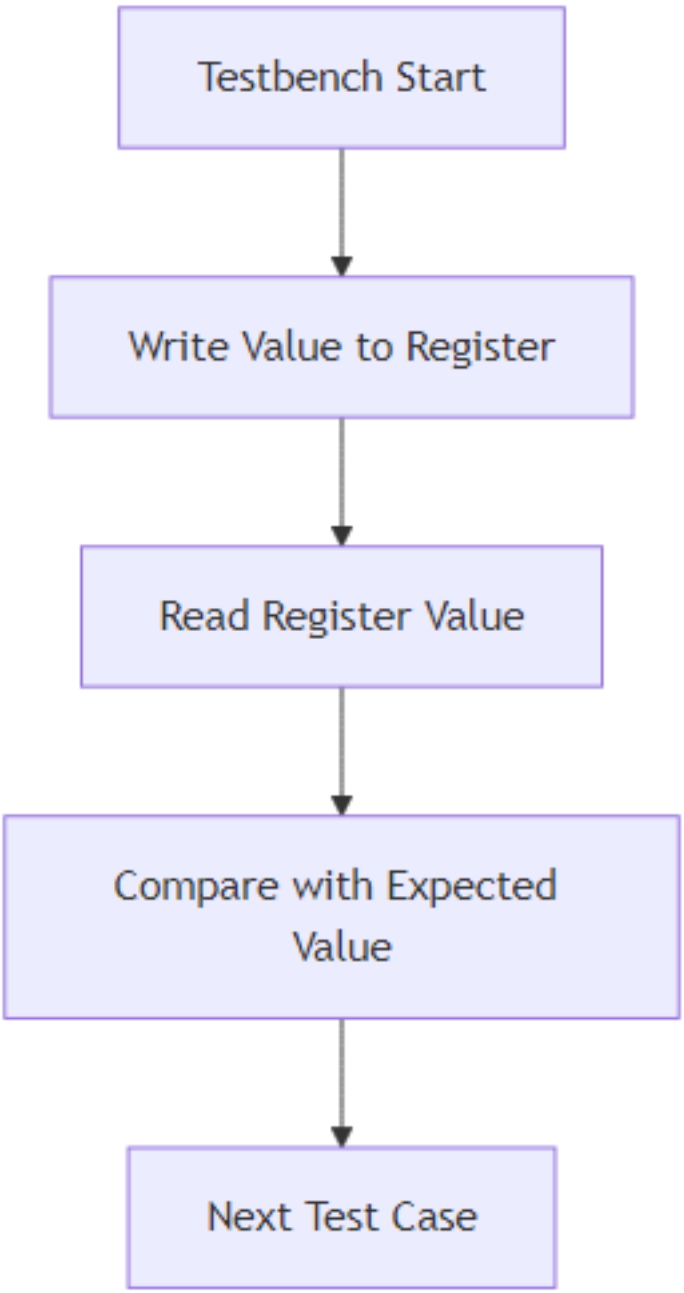
- correct register reads
- correct register writes
- write enable behavior
- x0 register immutability
- simultaneous dual reads
- correct indexing

Example tests:

- write and read same register
- write multiple registers
- attempt to write x0
- read different register combinations

11.14 Example Verification Flow

A typical testbench sequence may follow this pattern:



Because the module has simple behavior, verification can be systematic.

11.15 Design Strengths of the Module

The Register File offers several advantages in the CPU architecture.

11.15.1 Fast Operand Access

Operands are immediately available for execution.

11.15.2 Clean Datapath Integration

All register interactions are centralized.

11.15.3 Efficient Instruction Execution

Dual-read architecture supports most instruction patterns.

11.15.4 Predictable Behavior

Deterministic register storage simplifies verification.

11.16 Limitations and Tradeoffs

A minimal register file implementation may include simplifications.

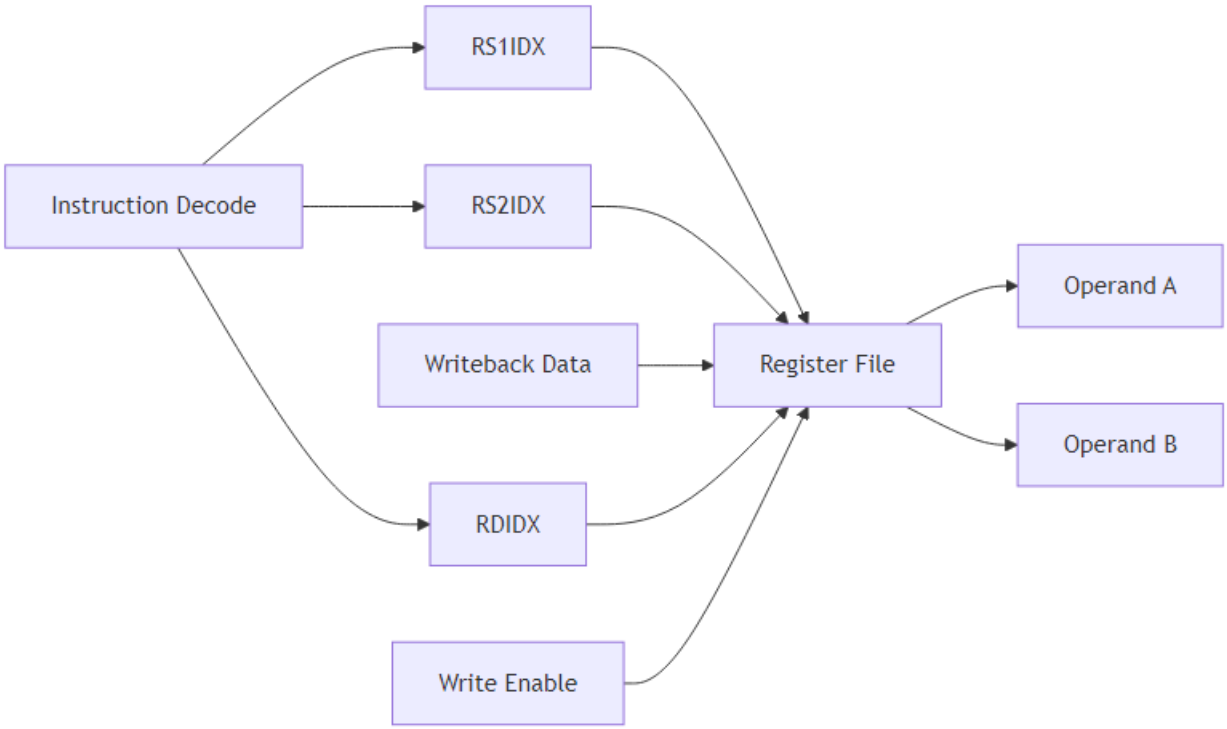
Possible tradeoffs include:

- limited number of write ports
- simple write conflict handling
- no advanced register renaming mechanisms

These simplifications are appropriate for a minimal single-cycle CPU design.

11.17 Summary Diagram

The following diagram summarizes the Register File's role in the CPU.



The Register File therefore acts as the central operand storage system for the CPU.

11.18 Final Summary

The **Register File** stores the 32 general-purpose registers defined by the RV32I architecture.

It provides operands for instruction execution and receives results during the writeback stage.

The module supports two simultaneous register reads and one register write, allowing efficient instruction execution in the single-cycle CPU design.

Through its integration with the ALU, Load Store Unit, Branch Comparator, and Control Unit, the Register File plays a central role in the operation of the RV32I Minimal SoC processor.

12 Trap Unit Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

12.1 Purpose of the Module

The **Trap Unit** is responsible for handling exceptional conditions that occur during CPU execution.

In a processor, not every instruction sequence can proceed normally. Certain situations require the CPU to interrupt the current execution flow and redirect control to a special handler routine. These situations are called **traps**.

The Trap Unit detects these exceptional events and redirects the CPU to the appropriate handler address.

Typical responsibilities include:

- detecting illegal instructions
- detecting execution errors
- redirecting the Program Counter to a trap handler
- maintaining safe CPU behavior during exceptional situations

Although traps may be minimal in a simple educational CPU, the Trap Unit provides the architectural foundation for more advanced features such as interrupts and exception handling.

12.2 Architectural Role Inside the CPU

The Trap Unit belongs to the **control path** of the CPU.

It monitors execution signals and can override the normal control flow when an exceptional condition occurs.

Conceptually:

When no trap occurs, normal instruction execution continues.

When a trap is detected, control flow is redirected.

12.3 What is a Trap

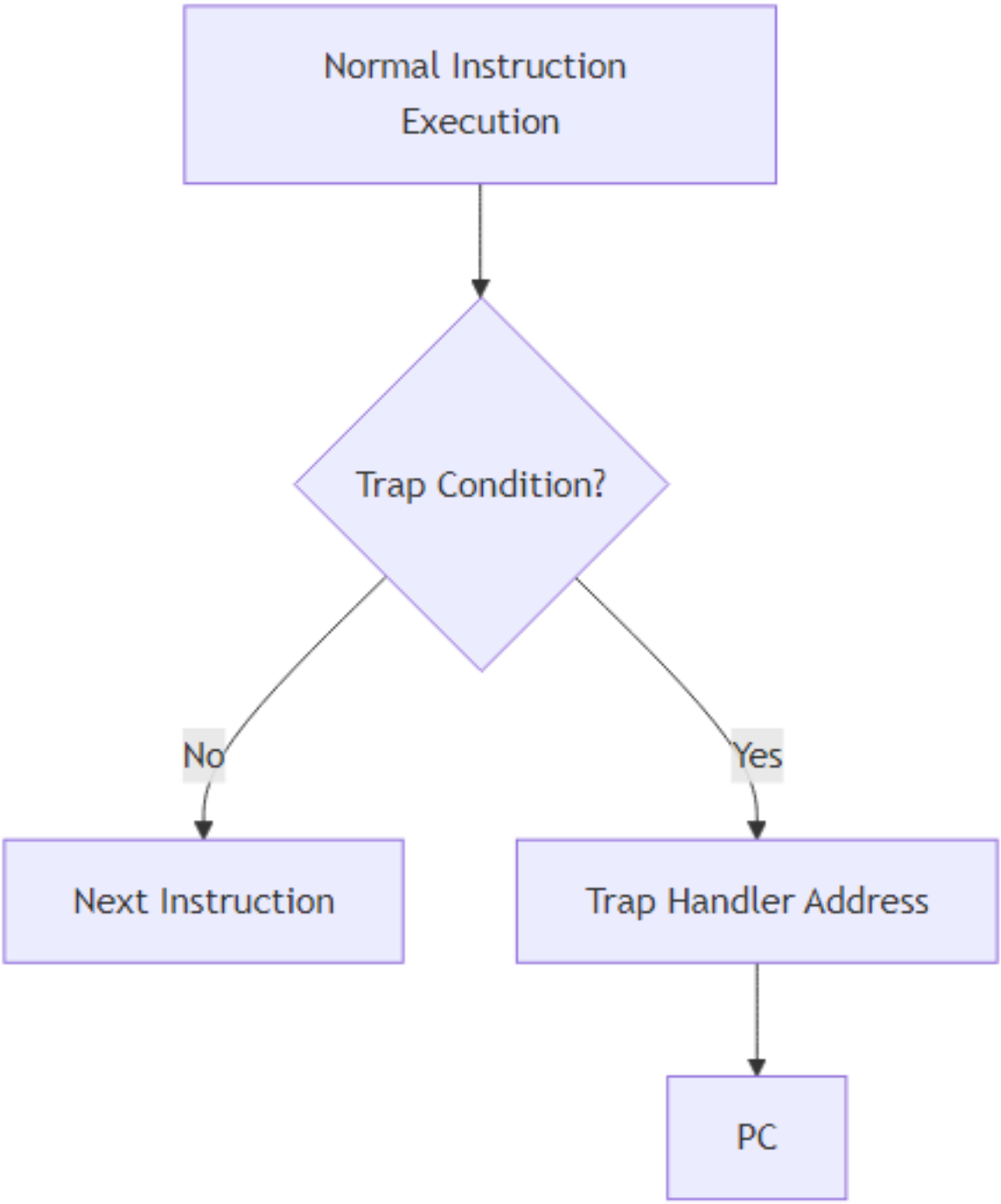
A **trap** is a controlled transfer of execution to a special handler routine that deals with abnormal or exceptional situations.

Examples of trap-causing events include:

- illegal instructions
- invalid operation encodings
- memory access violations
- system call instructions (future extensions)

Instead of allowing undefined behavior, the CPU redirects execution to a dedicated handler.

Conceptually:



This mechanism ensures safe CPU operation.

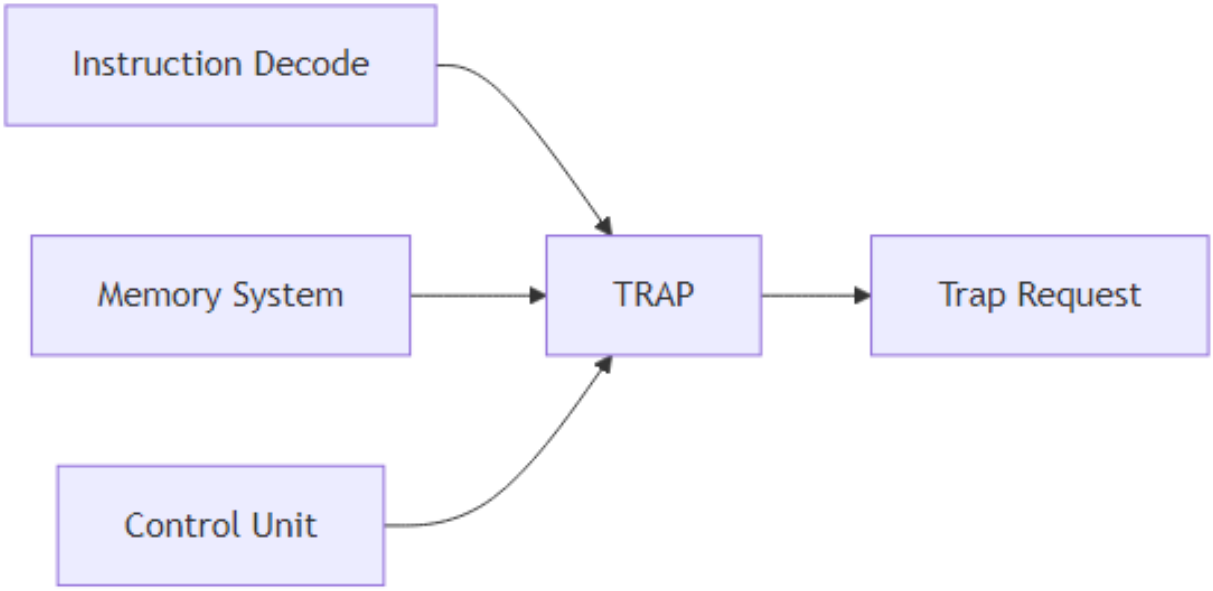
12.4 Trap Detection

The Trap Unit receives signals from multiple parts of the CPU indicating potential exceptional conditions.

Possible sources include:

- instruction decode
- control unit
- memory access logic
- system instructions

Conceptually:



When a trap condition is detected, the Trap Unit generates a trap request.

12.5 Illegal Instruction Detection

One common trap condition is the detection of **illegal instructions**.

An illegal instruction occurs when:

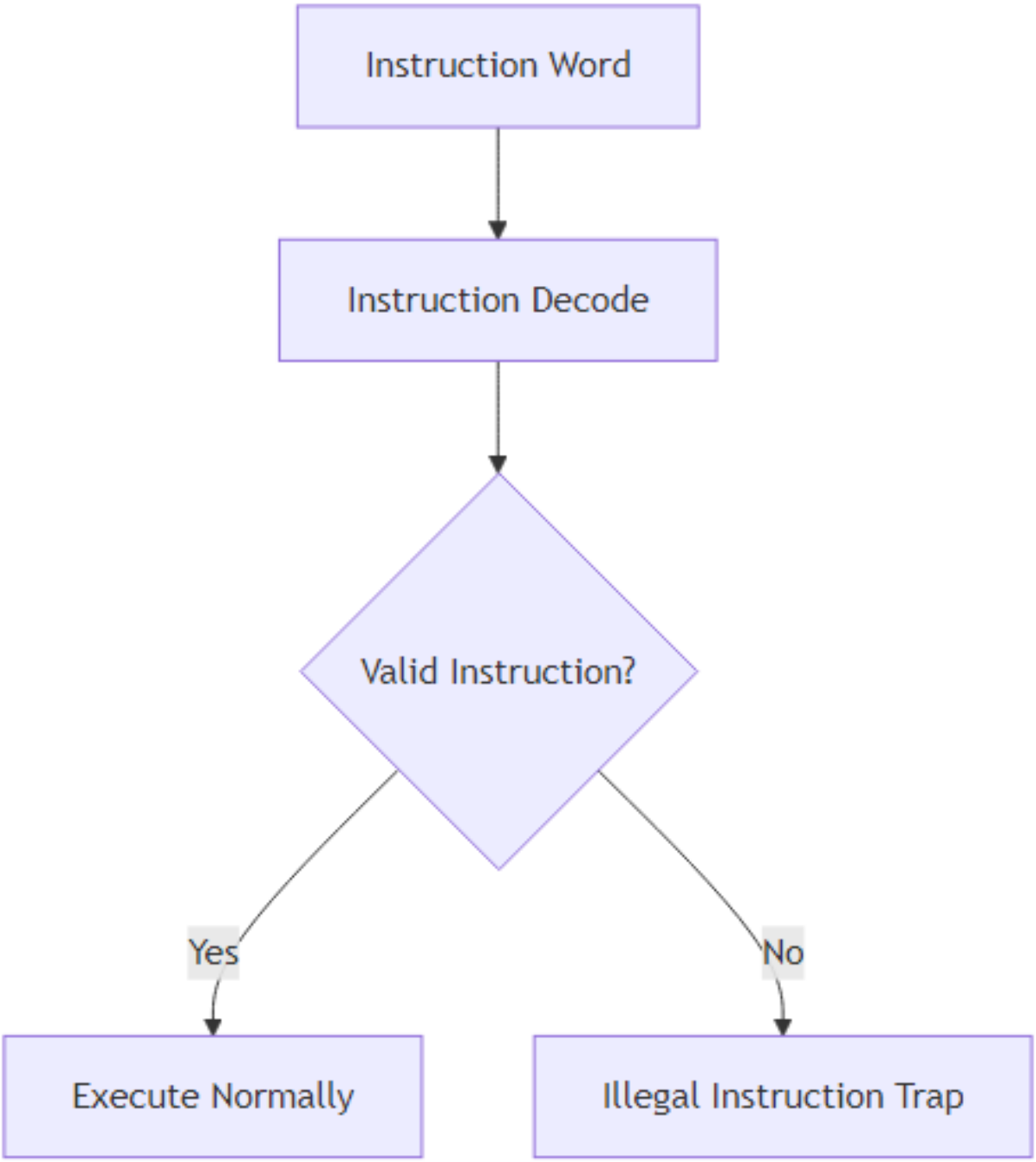
- the opcode is not recognized
- instruction fields contain invalid combinations
- the instruction is not supported by the implemented ISA subset

Example:

```
unknown_opcode
```

When this happens, the Trap Unit signals that execution must be redirected.

Conceptually:



12.6 Trap Handling Flow

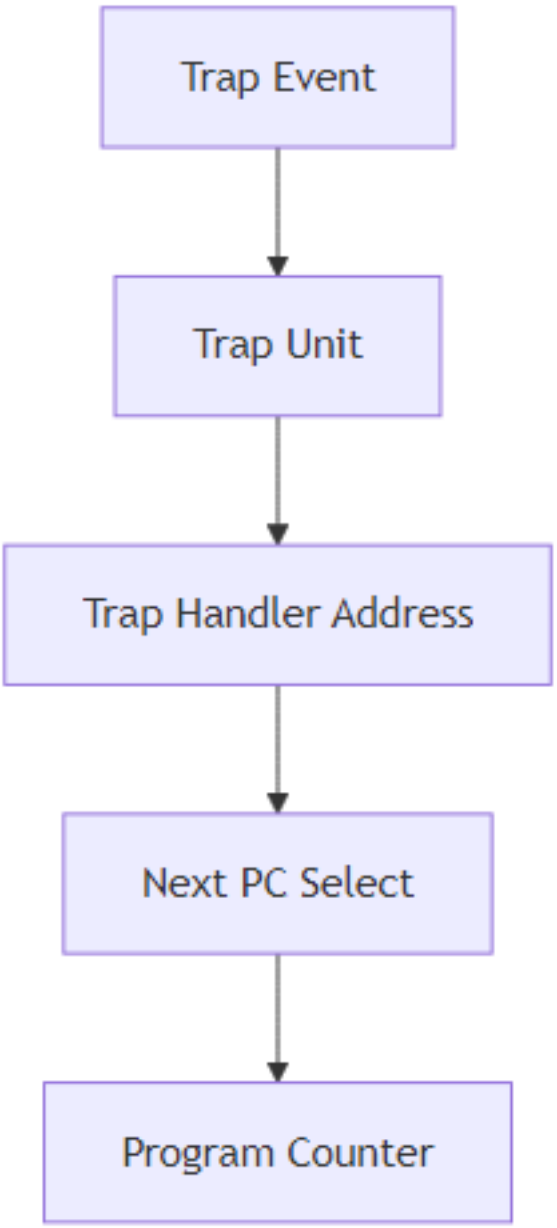
When a trap occurs, the CPU must redirect execution to a trap handler.

Typical flow:

1. trap condition detected
2. trap signal generated

3. Program Counter selects trap handler address
4. execution jumps to handler code

Conceptually:



This ensures that execution continues in a controlled environment.

12.7 Trap Handler Address

The trap handler address is a predefined location in instruction memory where the trap handling code resides.

Example:

`0x00000080`

When a trap occurs, the Program Counter is updated with this address.

Conceptually:

The handler then performs whatever recovery or reporting actions are required.

12.8 Inputs and Outputs

The Trap Unit receives signals related to execution status and produces control signals that affect the Program Counter.

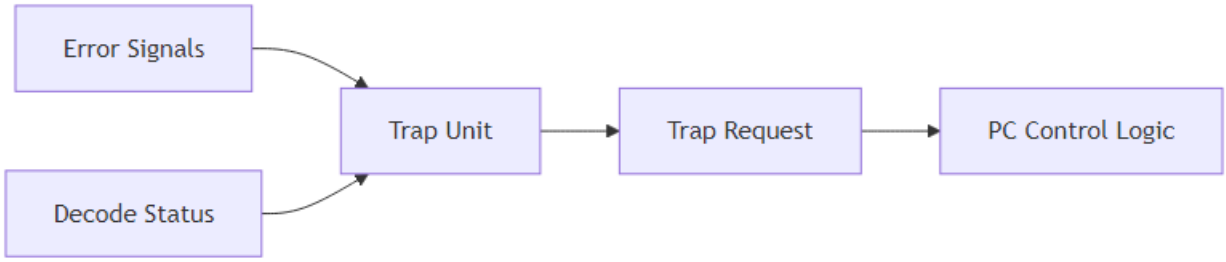
12.9 Inputs

- illegal instruction signal
- control error signals
- instruction decode information

12.9.1 Outputs

- trap request signal
- trap handler address
- control signal for PC redirection

Conceptually:



12.10 Interaction with the Program Counter

The Trap Unit works closely with the Program Counter logic.

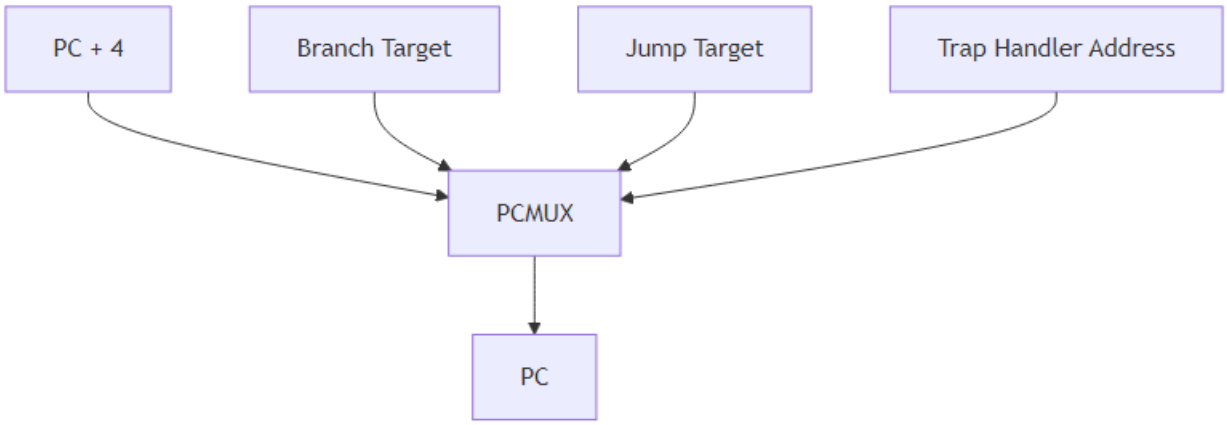
Normally the PC selects:

```
PC + 4
branch target
jump target
```

However, when a trap occurs, the PC must instead select:

```
trap_handler_address
```

Conceptually:



The Trap Unit provides the control signal selecting the trap handler.

12.11 Interaction with Other CPU Modules

The Trap Unit interacts with several CPU components.

12.11.1 Control Unit

Provides instruction validity and execution status signals.

12.11.2 Instruction Decode

Identifies illegal or unsupported instructions.

12.11.3 Program Counter

Receives trap redirection signals.

12.11.4 Execution Units

May generate runtime error signals.

These interactions allow the CPU to respond safely to exceptional situations.

12.12 Combinational vs Sequential Behavior

The Trap Unit is typically implemented using **combinational logic**.

This means:

- trap conditions are detected immediately
- the trap request signal is generated within the same cycle

Conceptually:

This allows the CPU to redirect execution without additional delay.

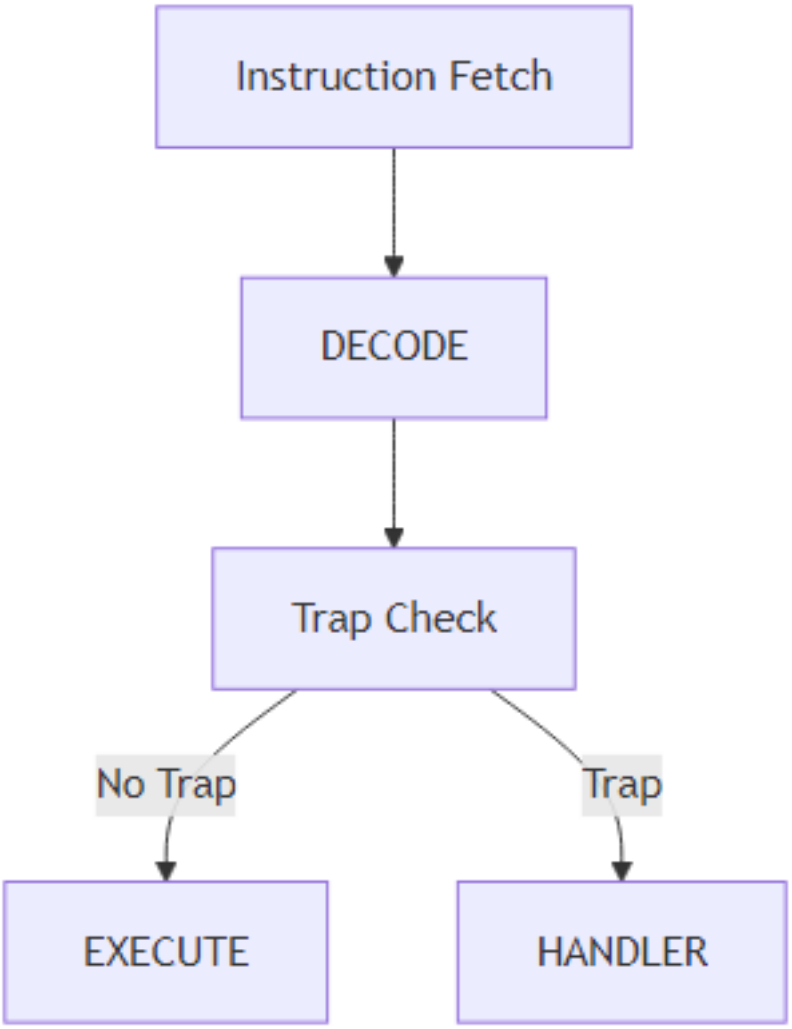
12.13 Role in the CPU Control Path

Although traps may occur rarely, the Trap Unit is an important part of the CPU control path.

Execution flow:

1. instruction fetched
2. instruction decoded
3. trap conditions checked
4. normal execution or trap redirection

Conceptually:



This ensures robust CPU behavior.

12.14 Verification Expectations

A Trap Unit testbench should verify:

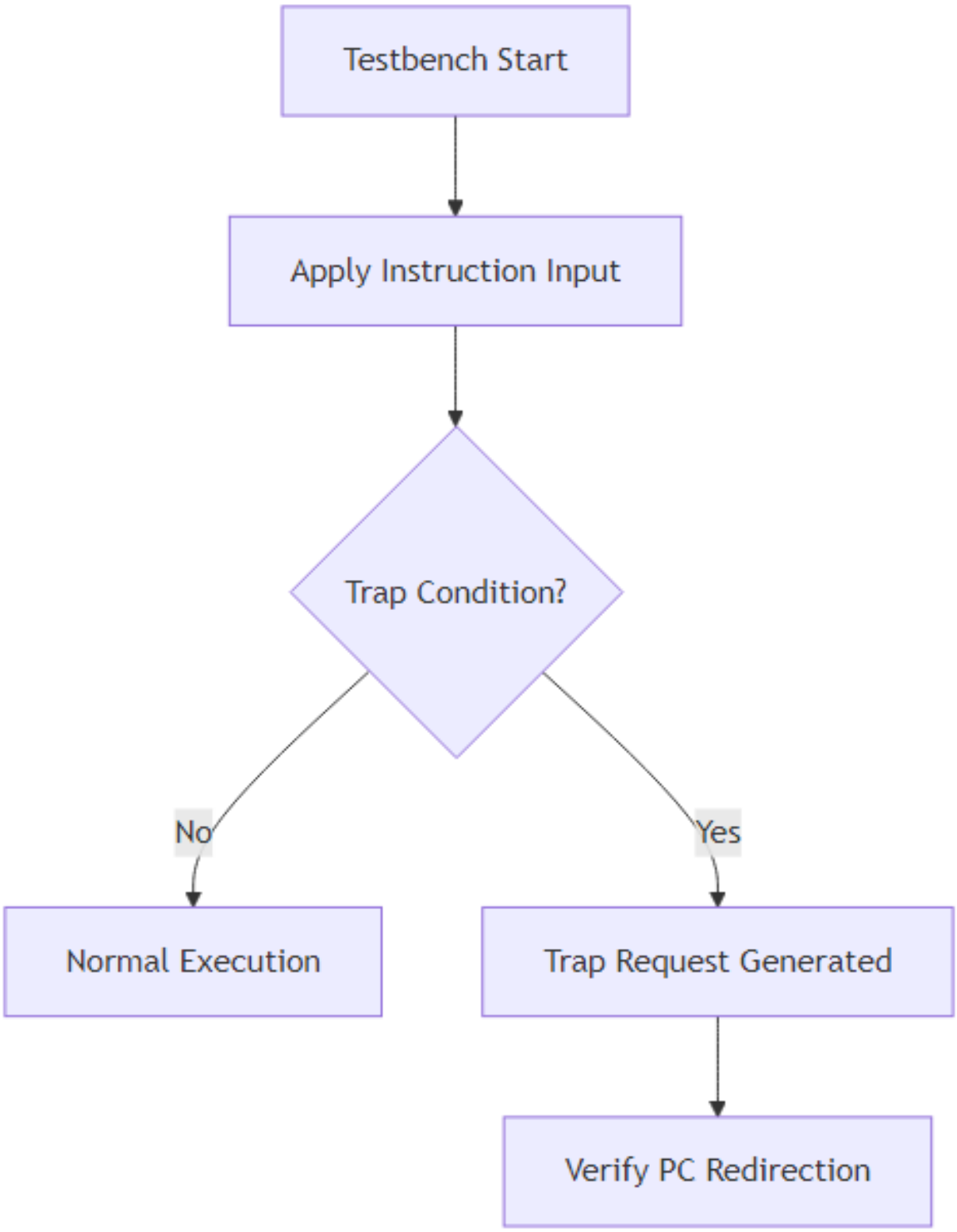
- detection of illegal instructions
- generation of trap request signals
- correct trap handler address selection
- correct interaction with PC control

Example scenarios:

- valid instruction execution
- illegal opcode detection
- forced trap condition
- normal execution recovery

12.15 Example Verification Flow

A typical verification procedure may follow this structure:



Because trap behavior is deterministic, verification is relatively straightforward.

12.16 Design Strengths of the Module

The Trap Unit improves the robustness of the CPU architecture.

12.16.1 Safe Execution

Prevents undefined behavior when unexpected instructions occur.

12.16.2 Clean Control Flow

Separates exceptional execution paths from normal instruction flow.

12.16.3 Extensibility

Provides a foundation for advanced features such as interrupts and system calls.

12.16.4 Modular Design

Trap detection logic is isolated from other execution modules.

12.17 Limitations and Tradeoffs

In a minimal educational CPU, trap handling may be simplified.

Possible limitations include:

- limited trap types
- simplified handler mechanism
- minimal error reporting

These simplifications are acceptable for a minimal RV32I implementation.

12.18 Summary Diagram

The following diagram summarizes the role of the Trap Unit.

This highlights the Trap Unit as the mechanism that ensures safe execution during exceptional conditions.

12.19 Final Summary

The **Trap Unit** monitors CPU execution for exceptional conditions such as illegal instructions or control errors.

When a trap condition occurs, the module generates a trap request and redirects the Program Counter to a predefined trap handler address.

Although relatively small in hardware complexity, the Trap Unit is essential for ensuring reliable and safe CPU behavior, and it provides the architectural foundation for future extensions such as interrupts, exceptions, and operating system support.

13 Bus Interconnect Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

13.1 Purpose of the Module

The **Bus Interconnect** module is responsible for routing memory access requests from the CPU to the correct destination inside the SoC.

In the RV32I Minimal SoC, the CPU does not connect directly to every memory or peripheral block. Instead, memory requests are sent through a shared interconnect structure that examines the requested address and forwards the transaction to the proper target.

Its main responsibilities are:

- receive memory access requests from the CPU side
- decode the requested address
- select the correct target device
- forward write data and control signals
- return read data from the selected target
- provide a unified access mechanism for RAM and peripherals

This module is what transforms the CPU from an isolated core into a real **memory-mapped SoC**.

13.2 Architectural Role Inside the SoC

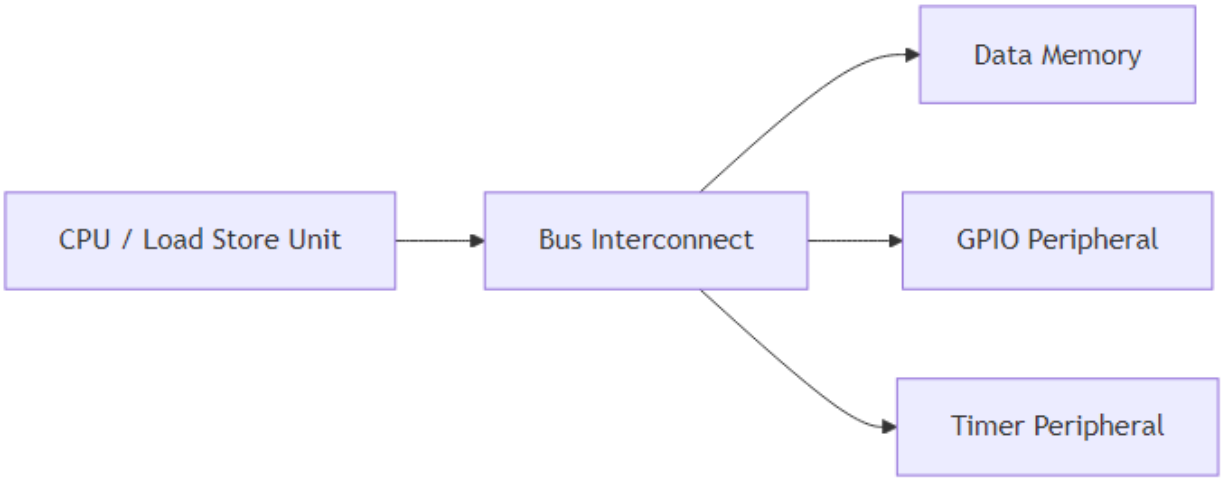
The Bus Interconnect sits between the CPU memory-access path and the SoC slaves.

On the master side, it connects to the CPU, usually through the **Load Store Unit (LSU)**.

On the slave side, it connects to the memory-mapped devices, such as:

- data memory
- GPIO
- timer

Conceptually:



This makes the Bus Interconnect the central routing point of the SoC data path.

13.3 Why the Bus Interconnect Exists

Without a bus interconnect, the CPU would need separate dedicated wiring for each memory or peripheral device.

That would make the design:

- harder to scale
- less modular
- harder to document
- harder to verify

By introducing a dedicated bus interconnect, the design gains:

- centralized address decoding
- clean separation between CPU and peripherals
- easier peripheral expansion
- consistent memory-mapped access model

This is a standard architectural approach in SoC design.

13.4 Memory-Mapped SoC Model

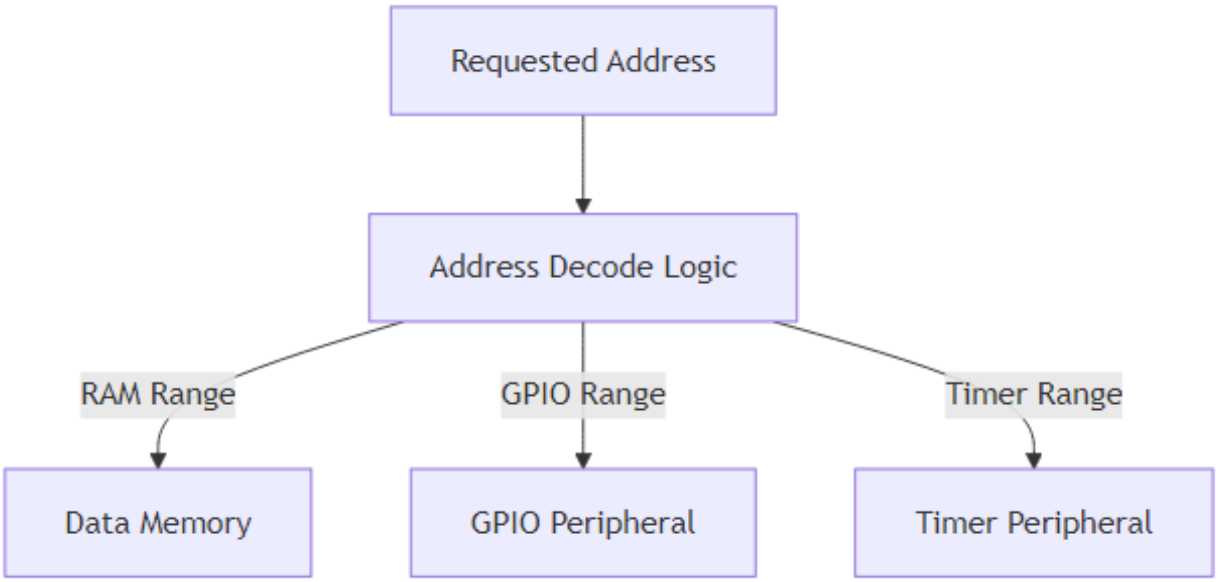
The RV32I Minimal SoC uses a **memory-mapped I/O model**.

In this approach:

- RAM occupies part of the address space
- peripherals also occupy parts of the address space
- the CPU uses ordinary load/store instructions to access both

That means the CPU does not need special instructions for GPIO or timer access. Instead, it performs standard memory transactions, and the bus interconnect routes them based on address.

Conceptually:



This is one of the key features that makes the design a true SoC rather than just a standalone CPU plus RAM.

13.5 Inputs and Outputs

Although the exact VHDL port names come from the RTL implementation, the architectural interface of the Bus Interconnect is usually very clear.

13.5.1 Inputs from CPU Side

Typical input categories include:

- address
- write data
- read enable
- write enable

13.5.2 Outputs to CPU Side

Typical output categories include:

- read data
- response signals or selected data path

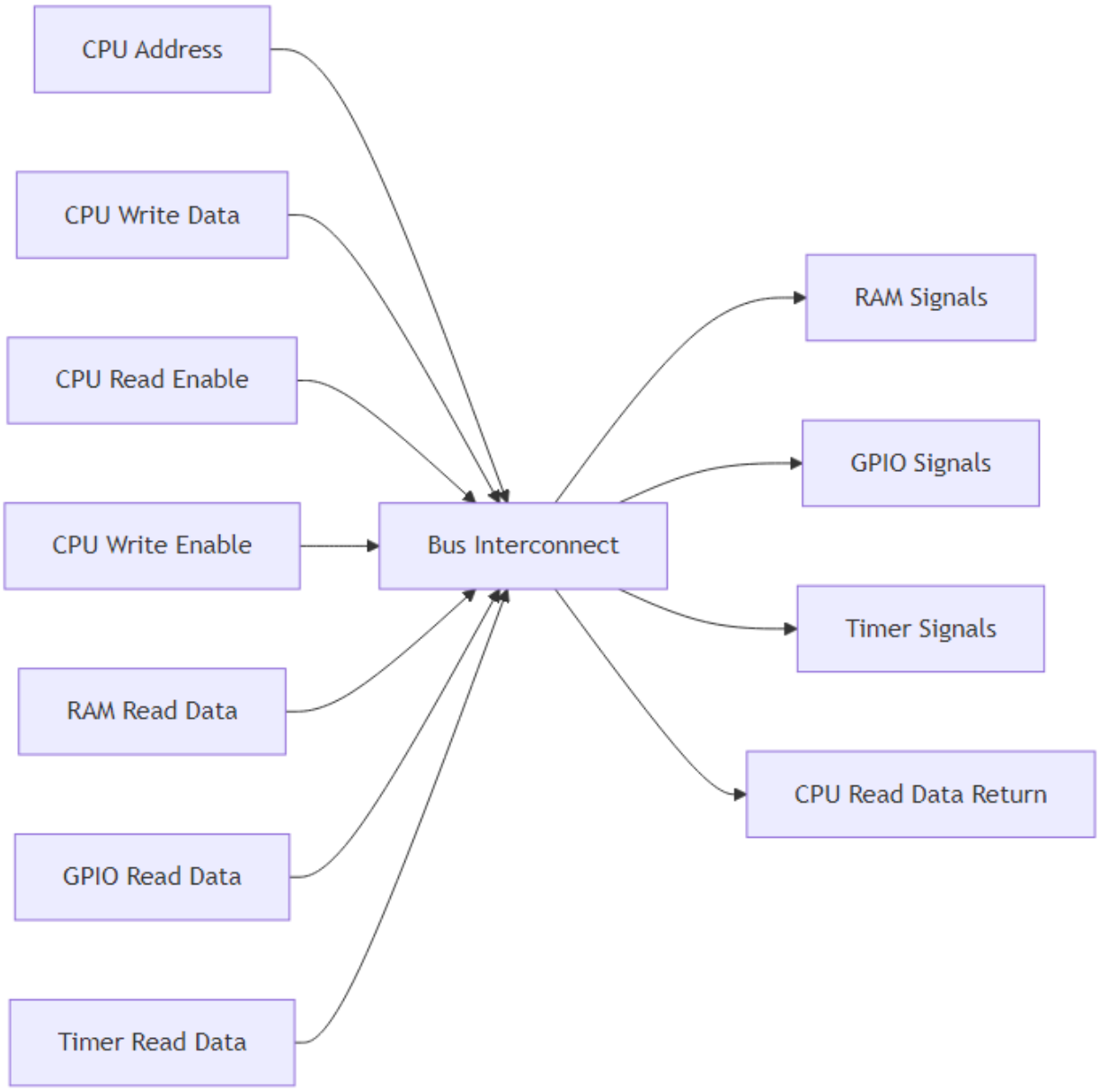
13.5.3 Outputs to Slave Devices

Typical outputs include:

- forwarded address
- forwarded write data

- per-device read/write enables

Conceptually:



This interface allows the interconnect to act as a protocol translator and routing block between one master and multiple slaves.

13.6 Address Decoding

The core function of the Bus Interconnect is **address decoding**.

Address decoding means:

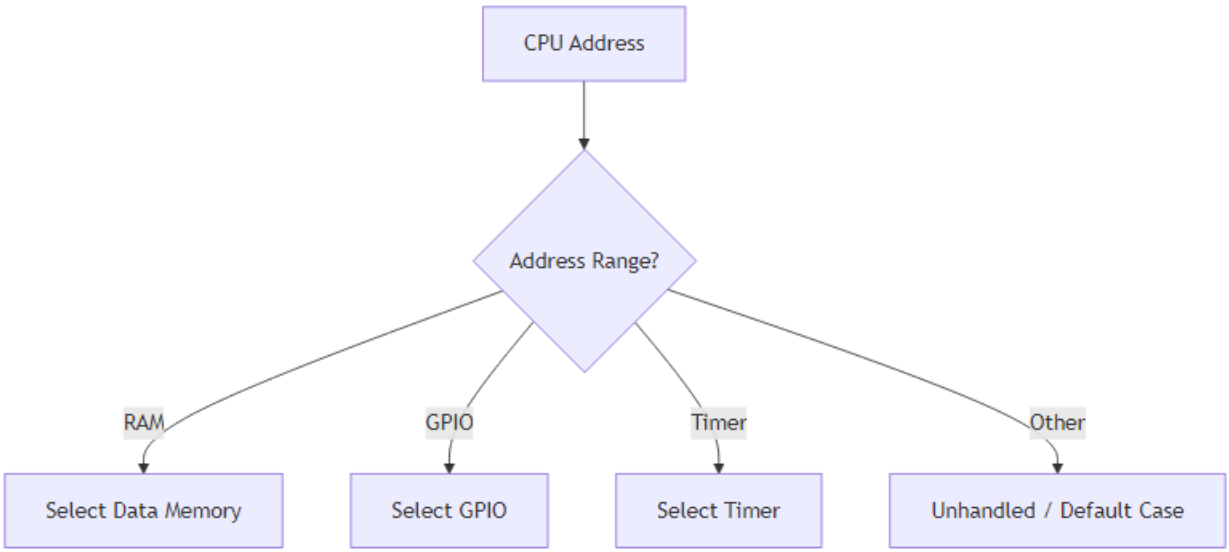
- inspect the address generated by the CPU
- determine which device owns that address range
- activate only the correct target

For example, conceptually:

Address Range	Target
RAM range	Data Memory
GPIO range	GPIO
Timer range	Timer

The exact address map depends on the RTL implementation and project-level memory map, but architecturally the mechanism is the same.

Conceptually:



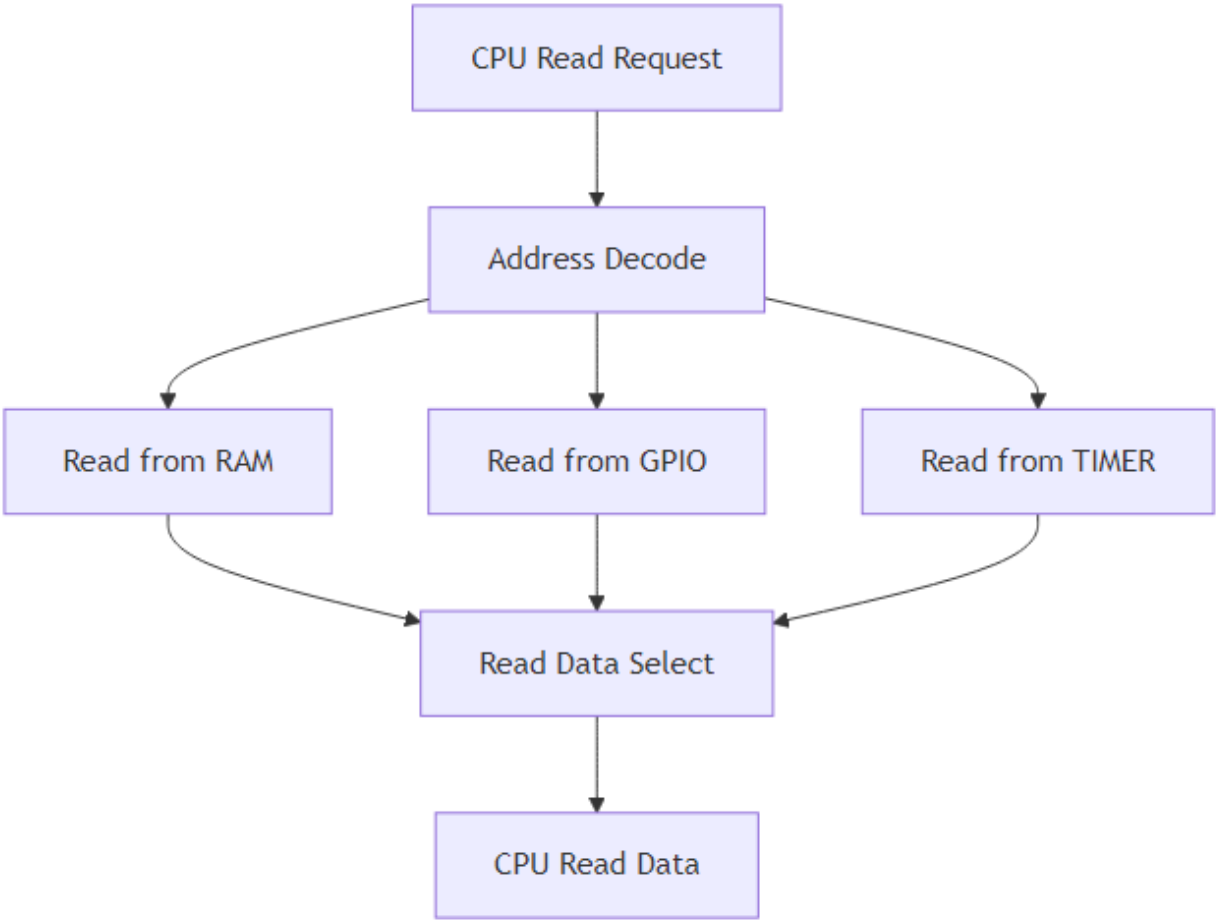
Only one target should be selected for a valid access.

13.7 Read Transaction Routing

For a read transaction, the Bus Interconnect must:

1. receive the CPU address
2. decode the target device
3. enable read access to that device
4. collect the returning data
5. forward the selected read data back to the CPU

Conceptually:



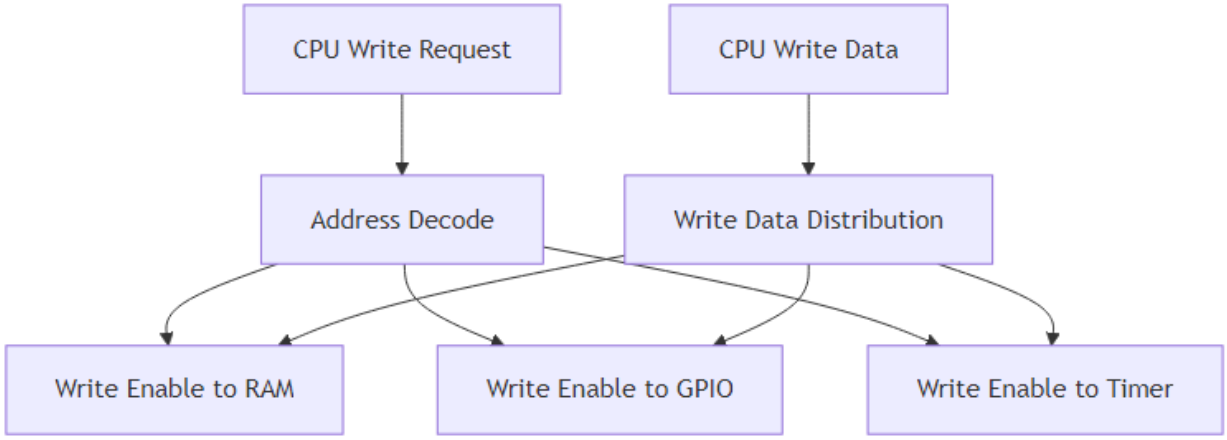
This means the interconnect not only selects devices, but also selects which device's return data is visible to the CPU.

13.8 Write Transaction Routing

For a write transaction, the Bus Interconnect must:

1. receive the CPU address
2. receive write data
3. decode the target device
4. forward the write enable only to the selected device
5. forward write data to the selected device

Conceptually:



For a correct write operation, only the addressed device should react to the transaction.

13.9 One-Master, Multiple-Slave Organization

The architecture of this interconnect is typically:

- one master: CPU / LSU
- multiple slaves: RAM, GPIO, timer

This is a simple and very common structure in educational SoCs.

Advantages include:

- easy address mapping
- low logic complexity
- straightforward verification
- clear documentation

Because there is only one master, the interconnect does not need complex arbitration logic between multiple competing initiators.

That keeps the design lightweight.

13.10 Relationship with the Load Store Unit

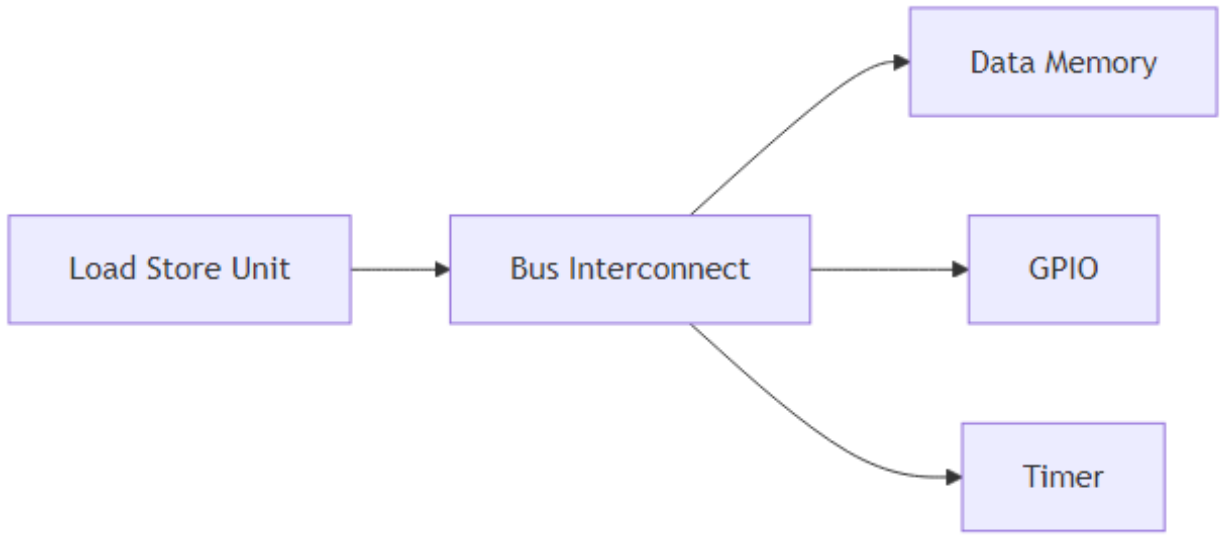
The Bus Interconnect receives requests from the CPU memory-access path, which is usually driven by the **Load Store Unit**.

The LSU provides:

- effective address
- store data
- read/write intent

The Bus Interconnect then translates that into per-device accesses.

Conceptually:



This means the LSU and Bus Interconnect together form the CPU's external data access path.

13.11 Relationship with Data Memory

One of the main bus targets is **data memory**.

When the CPU accesses an address within the RAM range, the Bus Interconnect:

- selects RAM
- forwards the address
- forwards write data when needed
- forwards read data back to the CPU

This allows ordinary load/store instructions to work with data memory through the same general interface used for peripherals.

13.12 Relationship with GPIO

The GPIO peripheral is also memory-mapped.

When the CPU accesses an address inside the GPIO address range, the Bus Interconnect forwards the transaction to the GPIO module.

This allows software to control or observe GPIO state using standard load/store instructions.

Example conceptual uses:

- writing output values
- reading input values
- accessing GPIO control registers if implemented

13.13 Relationship with Timer

The timer is another memory-mapped peripheral connected as a bus slave.

When the requested address falls into the timer range, the Bus Interconnect routes the transaction to the Timer module.

This allows software to:

- read timer values
- configure timer registers
- acknowledge timer-related status, depending on implementation

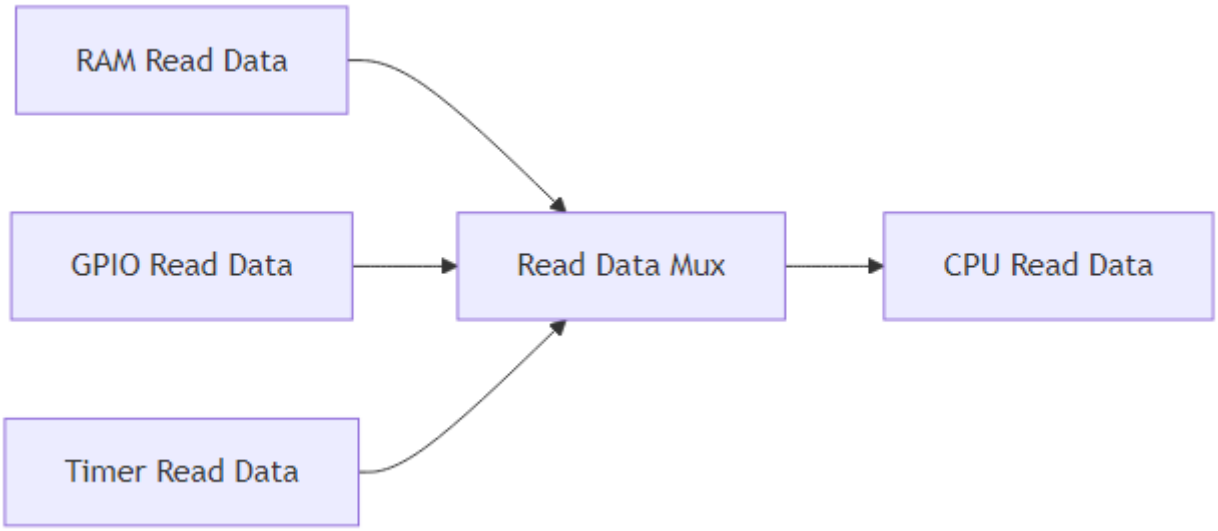
Again, the bus makes this possible using the same load/store mechanism used for RAM and GPIO.

13.14 Read Data Multiplexing

After address decoding, multiple slave devices may exist physically, but only one device's read data should be returned for a valid access.

The Bus Interconnect therefore includes a **read-data selection** function.

Conceptually:



The selection input for this mux is derived from address decoding.

This is one of the most important data-path functions of the interconnect.

13.15 Invalid or Unmapped Addresses

A well-structured interconnect should define behavior for invalid or unmapped addresses.

Possible architectural behaviors include:

- return zero

- return a default value
- suppress all device accesses
- signal an error or trap condition if supported

The exact behavior depends on the RTL implementation and system philosophy.

Even in a minimal design, it is important that unmapped addresses do not accidentally activate the wrong device.

13.16 Combinational vs Sequential Behavior

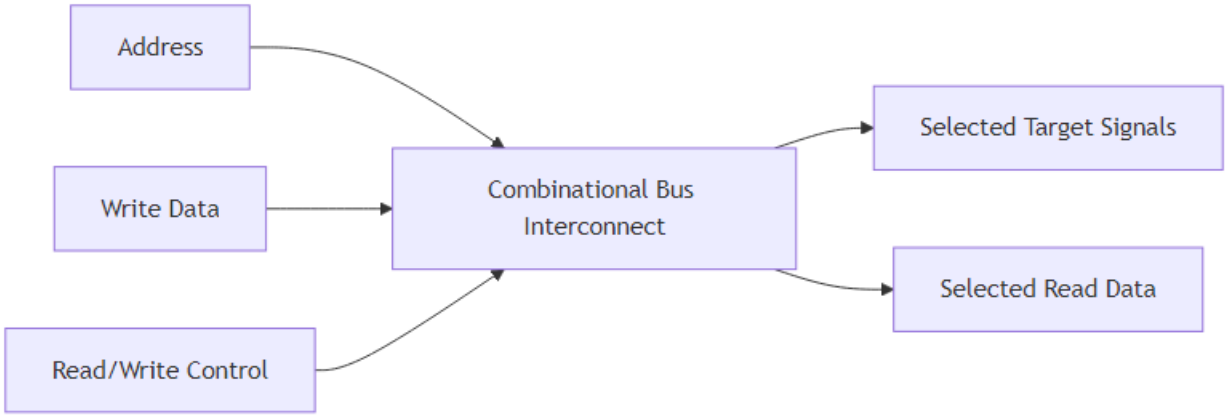
The Bus Interconnect is usually largely **combinational**.

Typical combinational functions include:

- address decoding
- device selection
- read-data multiplexing
- signal forwarding

The slave devices themselves may contain sequential logic, but the interconnect often acts as a routing layer rather than a storage element.

Conceptually:



This is a natural choice for a minimal single-cycle SoC.

13.17 Role in the SoC Critical Path

Because the interconnect sits between the CPU and the memory/peripheral subsystem, it can contribute to the timing path of load and store operations.

A typical load path may include:

1. instruction fetch
2. decode
3. register file read

4. ALU address calculation
5. bus address decode
6. slave device access
7. read data mux
8. writeback

This means the interconnect should remain simple and efficient to avoid unnecessarily extending the cycle time.

Its simplicity is one of the reasons a minimal custom interconnect is a good fit for this project.

13.18 Verification Expectations

A good Bus Interconnect testbench should verify:

- correct address decoding
- correct target selection
- correct read-data routing
- correct write routing
- proper handling of unmapped addresses
- isolation between slave devices

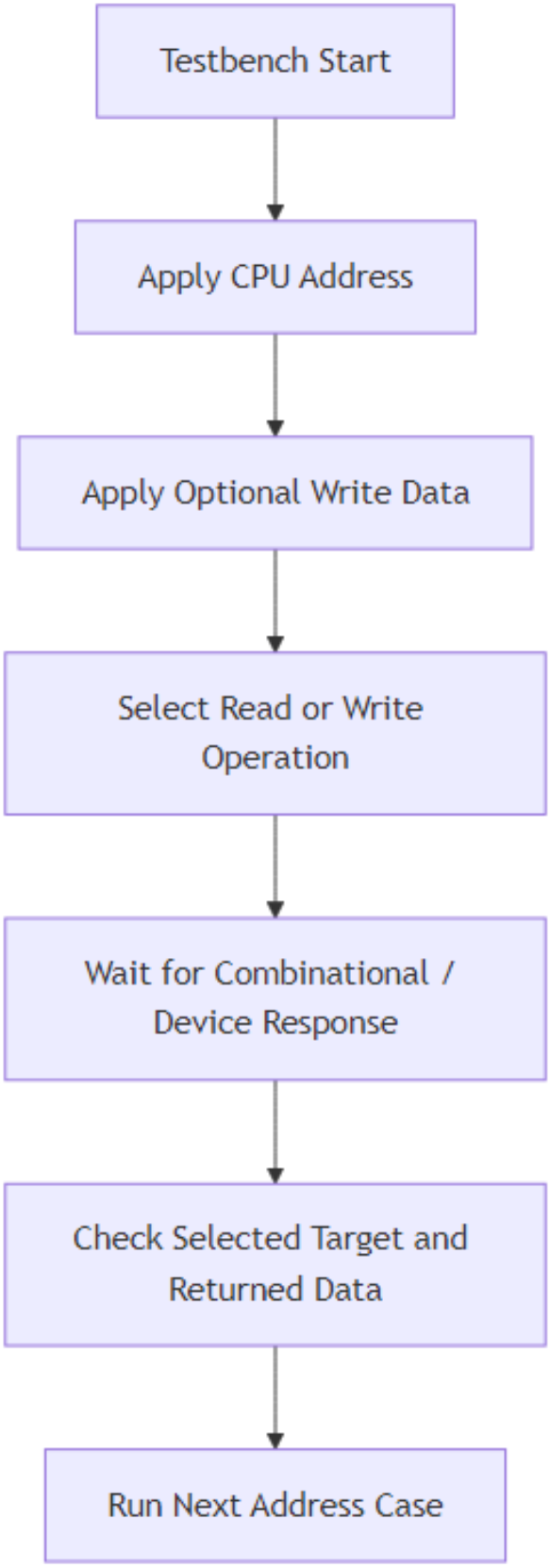
Representative tests include:

- read from RAM address and verify RAM data is returned
- write to RAM address and verify only RAM is updated
- read from GPIO address and verify GPIO data is returned
- write to GPIO address and verify only GPIO reacts
- read from timer address and verify timer data is returned
- verify that an unmapped address does not activate the wrong slave

Because the interconnect is central to SoC behavior, its correctness is very important.

13.19 Example Verification Flow

A conceptual testbench flow might look like this:


Testbench Start
Apply CPU Address
Apply Optional Write Data
Select Read or Write Operation
Wait for Combinational / Device Response
Check Selected Target and Returned Data
Run Next Address Case

This style of verification is effective because address decoding is systematic and can be covered with representative address ranges.

13.20 Design Strengths of the Module

The Bus Interconnect provides several architectural benefits.

13.20.1 Centralized SoC Routing

All CPU data transactions are routed in one place.

13.20.2 Clean Memory-Mapped Model

RAM and peripherals are accessed uniformly.

13.20.3 Modularity

New peripherals can be added by extending the address decode and slave routing.

13.20.4 Simplicity

A one-master, multi-slave design keeps hardware logic understandable.

13.20.5 Good Portfolio Value

Including a dedicated bus interconnect makes the project clearly identifiable as a full SoC design rather than only a CPU datapath exercise.

13.21 Limitations and Tradeoffs

A minimal custom interconnect also has natural limitations.

Examples include:

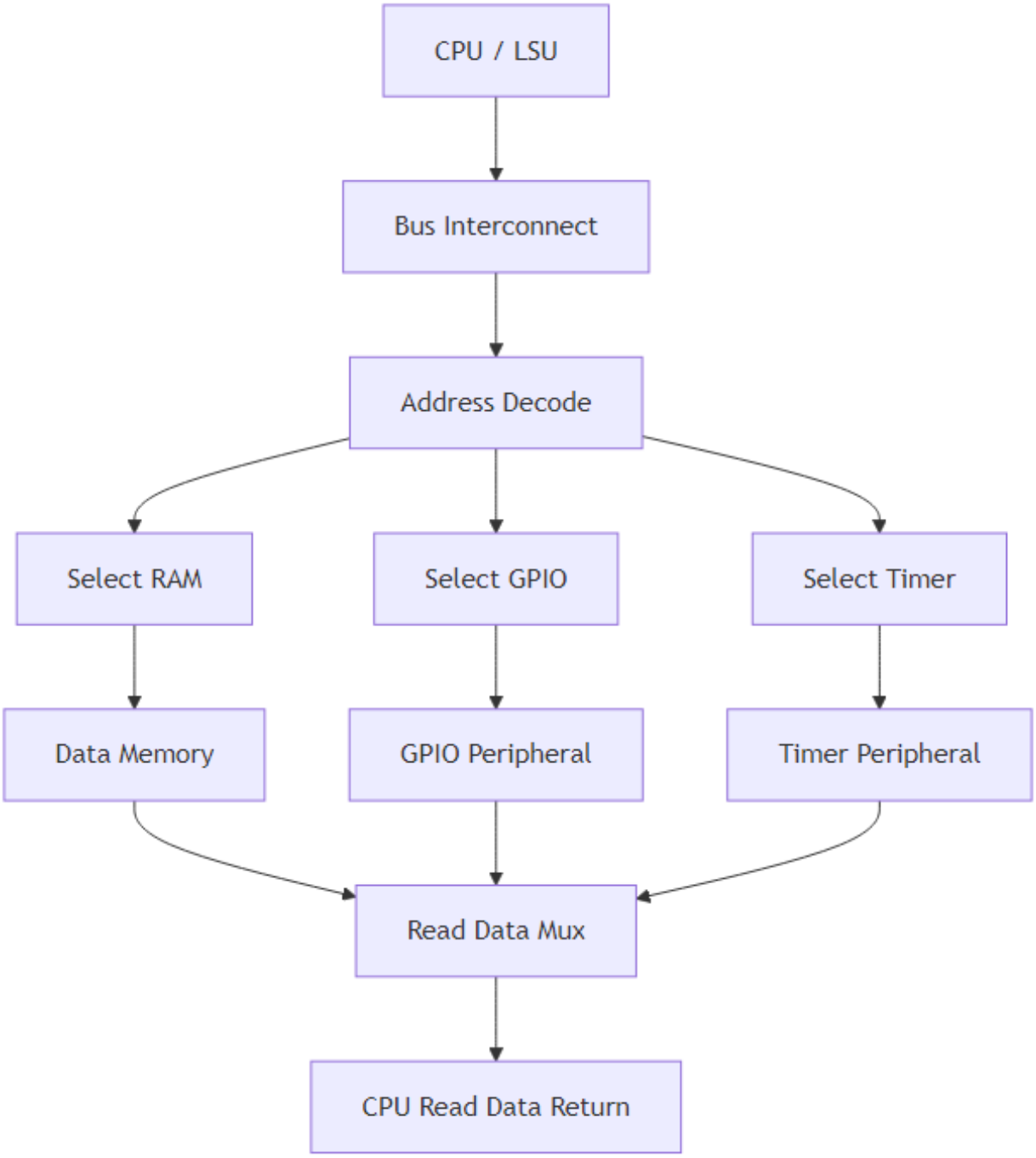
- no multi-master arbitration
- no standard external bus protocol such as AXI or Wishbone
- limited scalability compared to industrial bus fabrics
- simple response model

These tradeoffs are entirely reasonable for a small educational SoC.

In fact, the simplicity is a strength for learning and documentation.

13.22 Summary Diagram

The following diagram summarizes the Bus Interconnect's role in the SoC.



This diagram highlights the interconnect as both:

- an address decoder
- a transaction router
- a read-data selector

13.23 Final Summary

The **Bus Interconnect module** is the central routing structure that connects the CPU memory-access path to the memory-mapped resources of the **RV32I Minimal SoC**.

It receives address, control, and write-data information from the CPU side, decodes the requested address, selects the correct target device, and returns the appropriate read data back to the CPU.

By enabling uniform access to:

- data memory
- GPIO
- timer

the Bus Interconnect is what allows the system to function as a complete memory-mapped SoC rather than as an isolated processor core.

14 Data Memory Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

14.1 Purpose of the Module

The **Data Memory** module provides the main storage area used by the CPU for load and store operations.

While instruction memory holds the program code executed by the processor, data memory stores runtime information such as:

- program variables
- stack data
- intermediate computation results
- memory buffers

The Data Memory module therefore acts as the **primary read/write storage resource** available to software running on the CPU.

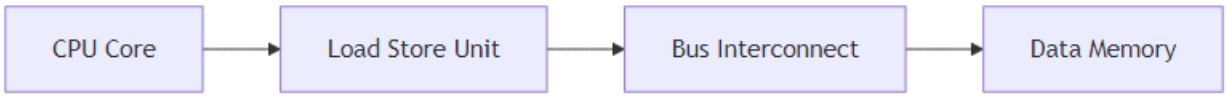
In the RV32I Minimal SoC architecture, this memory is accessed through the Load Store Unit and Bus Interconnect.

14.2 Architectural Role Inside the SoC

Data Memory is one of the **memory-mapped slave devices** connected to the Bus Interconnect.

This means that the CPU does not interact with it directly. Instead, memory requests travel through the SoC interconnect infrastructure.

Conceptually:



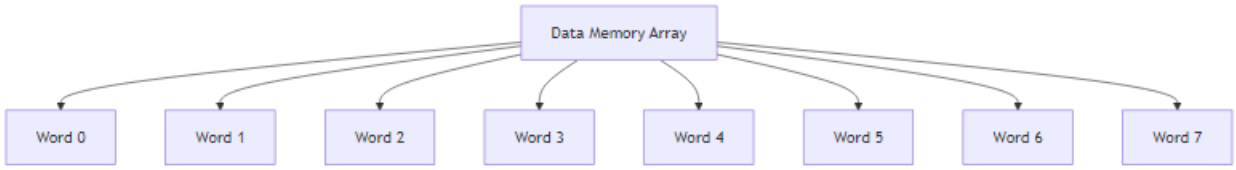
The Bus Interconnect selects the Data Memory when the CPU accesses an address that falls inside the RAM address range.

14.3 Memory Organization

In an RV32I system, data memory is typically organized as an array of 32-bit words.

Each memory entry contains one word that can be accessed through load and store instructions.

Conceptually:



The total size of the memory depends on the implementation parameters in the RTL design.

14.4 Addressing Model

The CPU generates byte addresses for memory operations.

However, because the RV32I architecture primarily operates on 32-bit words, data memory is often implemented internally as a **word-addressed array**.

Typical addressing flow:

1. CPU generates a byte address
2. address is forwarded through the bus
3. memory logic selects the corresponding word entry

Conceptually:

The lower address bits may be used for alignment purposes depending on the memory implementation.

14.5 Inputs and Outputs

Although the exact signal names depend on the VHDL implementation, the architectural interface of the Data Memory module usually contains the following signals.

14.5.1 Inputs

Typical input signals include:

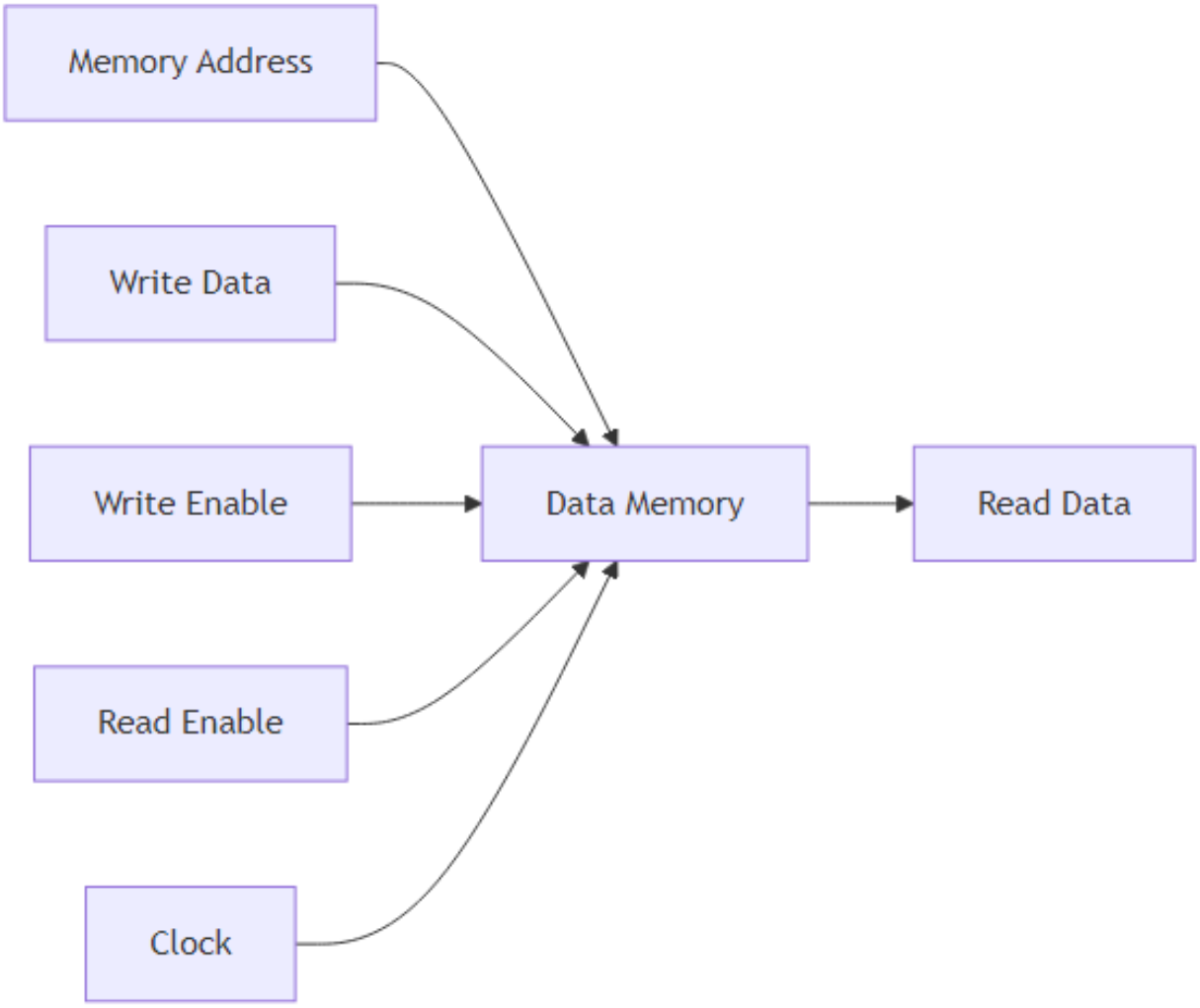
- memory address
- write data
- write enable
- read enable
- clock

14.5.2 Outputs

Typical output signals include:

- read data

Conceptually:



These signals allow the memory module to participate in read and write transactions initiated by the CPU.

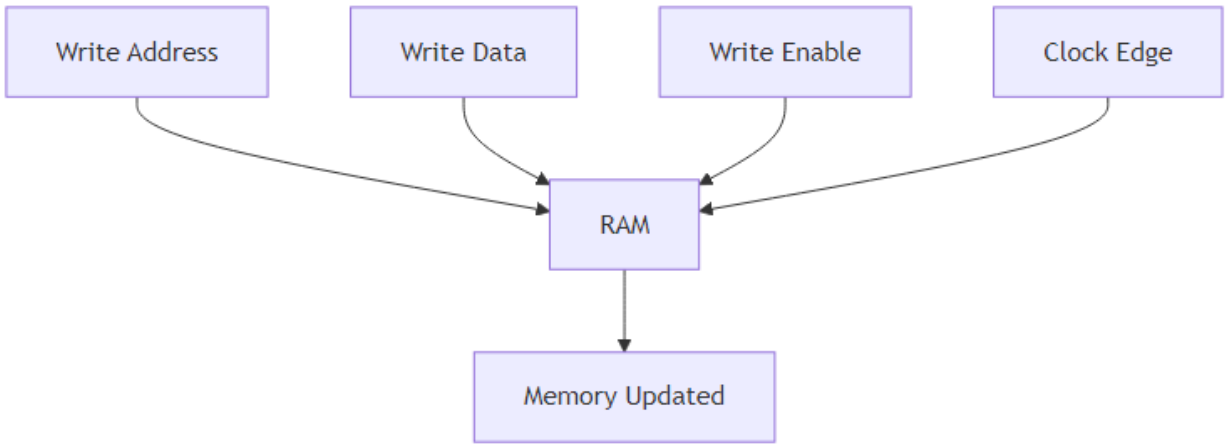
14.6 Write Operation

A write operation occurs when the CPU executes a store instruction.

The typical sequence is:

1. CPU computes effective address
2. address is forwarded through the bus
3. store data is forwarded from the register file
4. memory write enable is asserted
5. memory location is updated on the clock edge

Conceptually:



Only the addressed memory location is modified during the transaction.

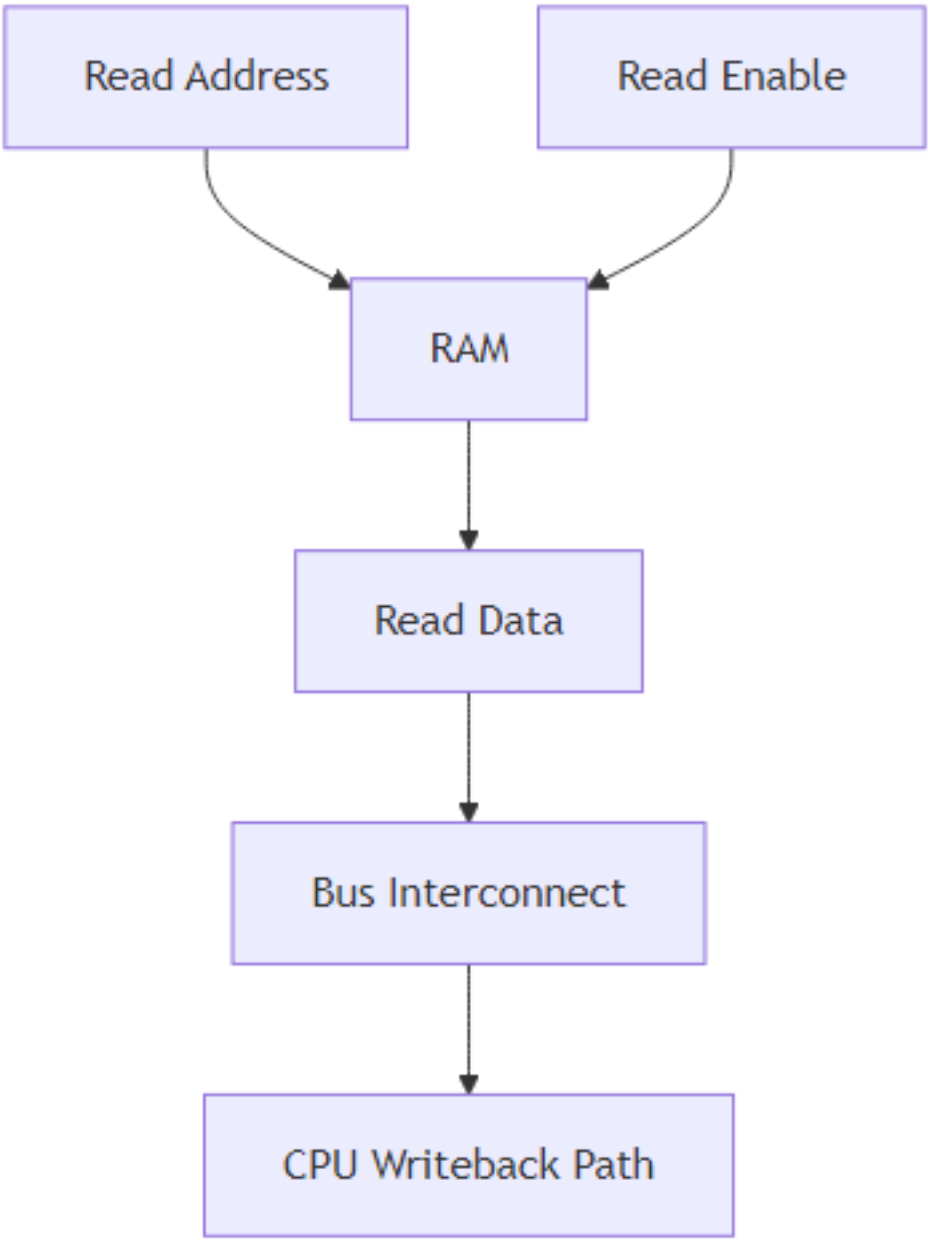
14.7 Read Operation

A read operation occurs when the CPU executes a load instruction.

Typical sequence:

1. CPU generates memory address
2. address is decoded by the bus
3. read enable is asserted
4. memory returns stored data
5. data is forwarded back to the CPU writeback path

Conceptually:



The returned data becomes the result of the load instruction.

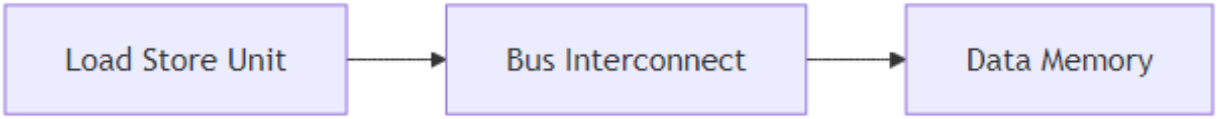
14.8 Interaction with the Load Store Unit

The Data Memory module receives its requests from the **Load Store Unit (LSU)**.

The LSU generates:

- effective memory addresses
- store data
- control signals indicating read or write operations

Conceptually:



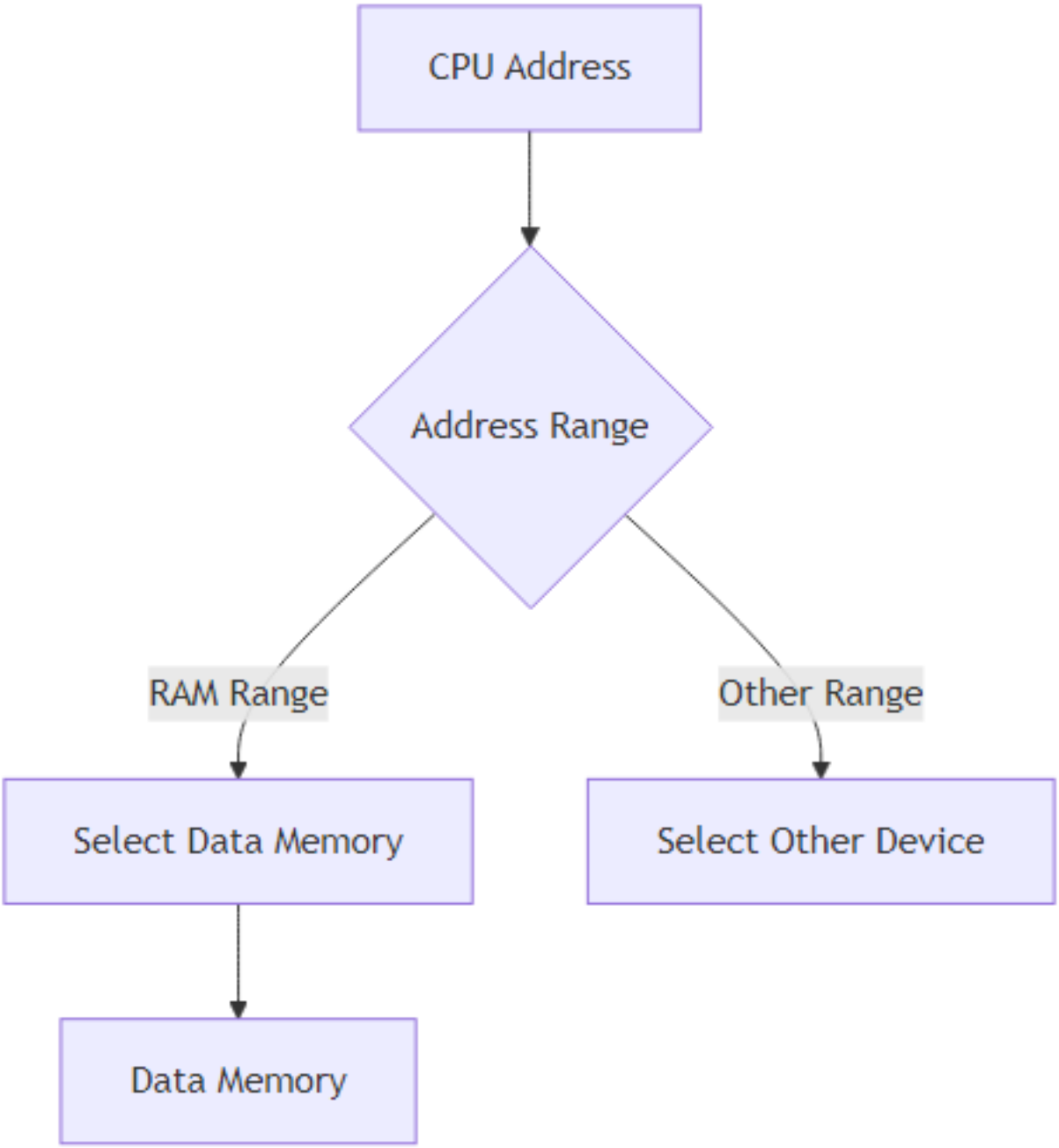
The Data Memory itself does not perform address calculations; it simply stores and retrieves values.

14.9 Interaction with the Bus Interconnect

The Bus Interconnect determines whether a memory request should be routed to Data Memory.

If the requested address falls inside the RAM address range, the interconnect activates the memory interface.

Conceptually:



This mechanism allows the CPU to access both RAM and peripherals using the same load/store instructions.

14.10 Word Alignment

Because the CPU uses 32-bit instructions and registers, data memory is typically accessed using **word-aligned addresses**.

This means:

- addresses should normally be multiples of 4
- the lowest two bits of the address select the byte position

In a simplified implementation, the memory may assume correct alignment.

Conceptually:

Alignment simplifies memory indexing and improves performance.

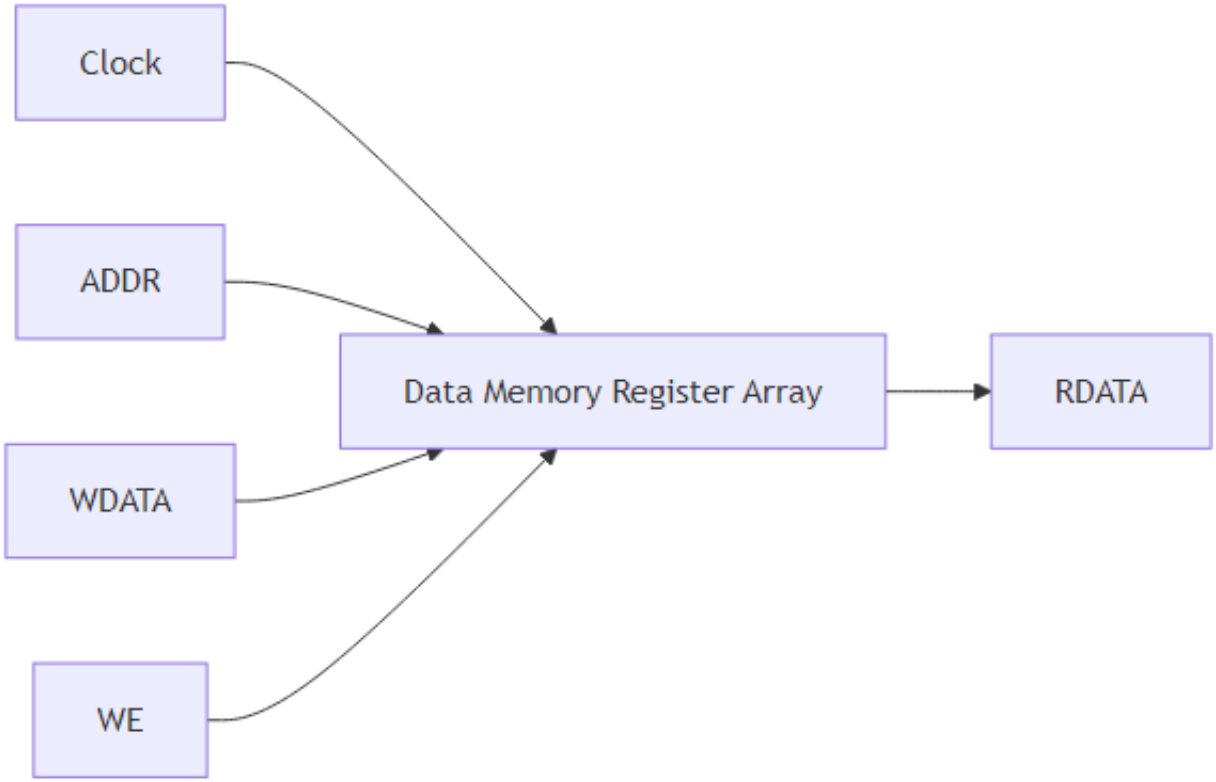
14.11 Sequential Behavior

The Data Memory module is usually **sequential**, meaning that write operations occur on clock edges.

Characteristics:

- writes update memory at the clock edge
- reads may be combinational or synchronous depending on implementation

Conceptually:



This behavior makes the memory stable and predictable.

14.12 Role in the CPU Execution Path

Data Memory participates in the execution of load and store instructions.

Typical load path:

1. register read
2. immediate generation
3. address calculation
4. bus routing
5. memory read
6. writeback to register file

Typical store path:

1. register read
2. address calculation
3. bus routing
4. memory write

Because load operations may require a memory read before writeback, memory latency can influence the overall CPU cycle time.

14.13 Verification Expectations

A Data Memory testbench should verify:

- correct memory writes
- correct memory reads
- correct address indexing
- proper read-after-write behavior
- stable data storage across cycles

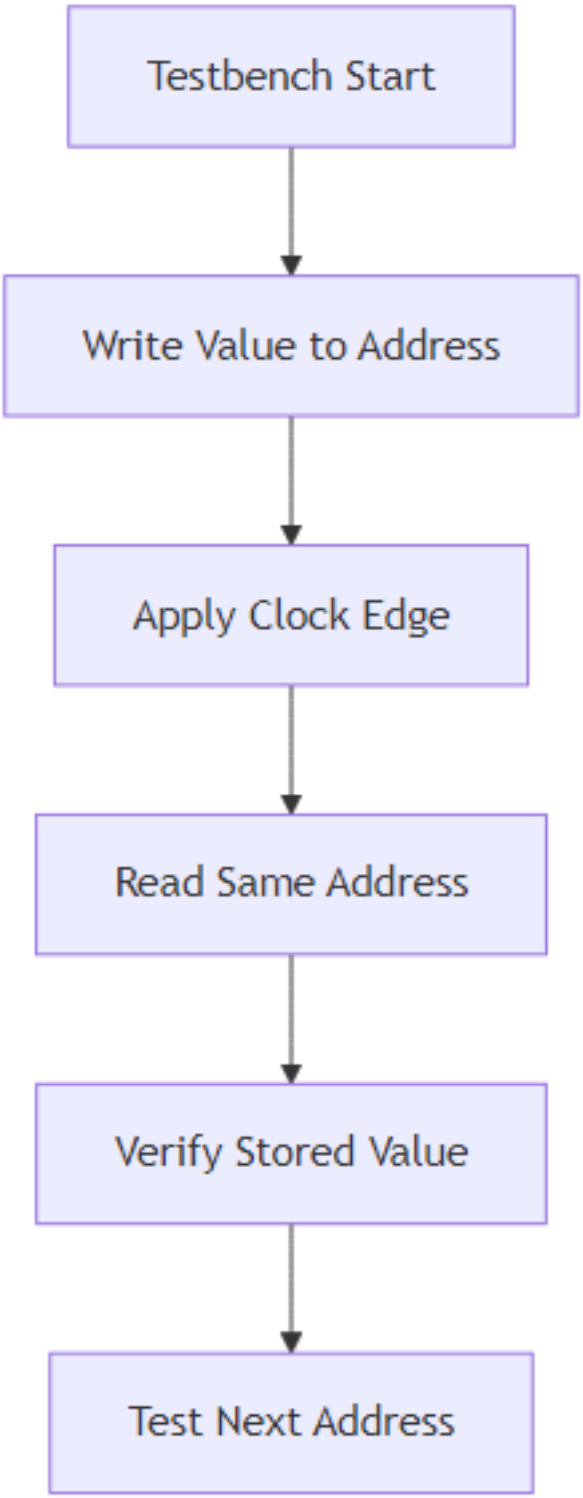
Typical tests include:

- writing to multiple memory locations
- reading stored values
- verifying data persistence
- verifying no unintended writes occur

Because memory is a fundamental storage element, correctness is critical.

14.14 Example Verification Flow

A conceptual verification sequence:



This verifies that data is correctly stored and retrieved.

14.15 Design Strengths of the Module

The Data Memory module contributes several advantages.

14.15.1 Simple Interface

Memory access is controlled using straightforward read/write signals.

14.15.2 Modular SoC Integration

The memory connects cleanly through the Bus Interconnect.

14.15.3 Scalable Storage

Memory capacity can be expanded without changing CPU logic.

14.15.4 Realistic System Behavior

Including data memory makes the processor capable of running meaningful programs.

14.16 Limitations and Tradeoffs

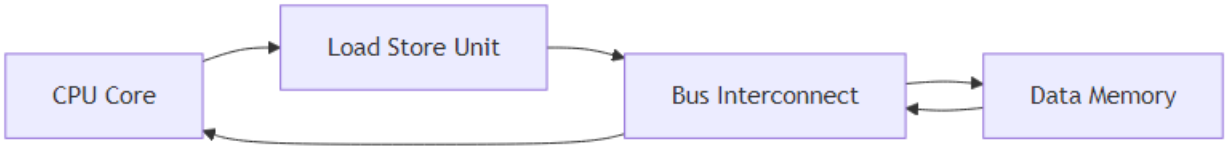
A minimal educational memory system may lack advanced features such as:

- caching
- burst transactions
- memory protection
- virtual memory support
- multi-port access

These features are not required for a minimal RV32I SoC and would significantly increase system complexity.

14.17 Summary Diagram

The following diagram summarizes the role of Data Memory in the SoC.



This diagram highlights Data Memory as the primary storage element used by software executing on the processor.

14.18 Final Summary

The **Data Memory module** provides the main runtime storage used by the RV32I Minimal SoC.

It stores program data, variables, and intermediate results, and it supports load and store operations issued by the CPU.

Through its integration with the Bus Interconnect and Load Store Unit, Data Memory enables the processor to read and write memory using the standard load/store instructions defined by the RV32I architecture.

15 GPIO Peripheral Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

15.1 Purpose of the Module

The **GPIO (General Purpose Input/Output)** module provides a simple interface between the CPU and external digital signals.

In a system-on-chip architecture, GPIO peripherals allow software running on the CPU to interact with external hardware components such as:

- LEDs
- buttons
- switches
- sensors
- external logic signals

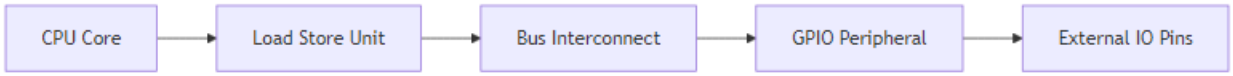
The GPIO module exposes a set of memory-mapped registers that can be accessed using standard load and store instructions. This allows the CPU to read input signals and drive output signals using the same memory access mechanisms used for normal RAM.

15.2 Architectural Role Inside the SoC

The GPIO module is implemented as a **memory-mapped peripheral** connected to the Bus Interconnect.

This means that the CPU interacts with GPIO through the same memory access path used for RAM and other peripherals.

Conceptually:



When the CPU accesses an address that belongs to the GPIO address range, the Bus Interconnect routes the transaction to the GPIO module.

15.3 General Purpose Input/Output Concept

GPIO stands for **General Purpose Input/Output**, meaning the peripheral provides programmable access to digital signals.

Two main operations are possible:

- **Input**: reading the state of external pins
- **Output**: driving values onto external pins

Software can therefore interact with the external environment using simple memory operations.

Example use cases:

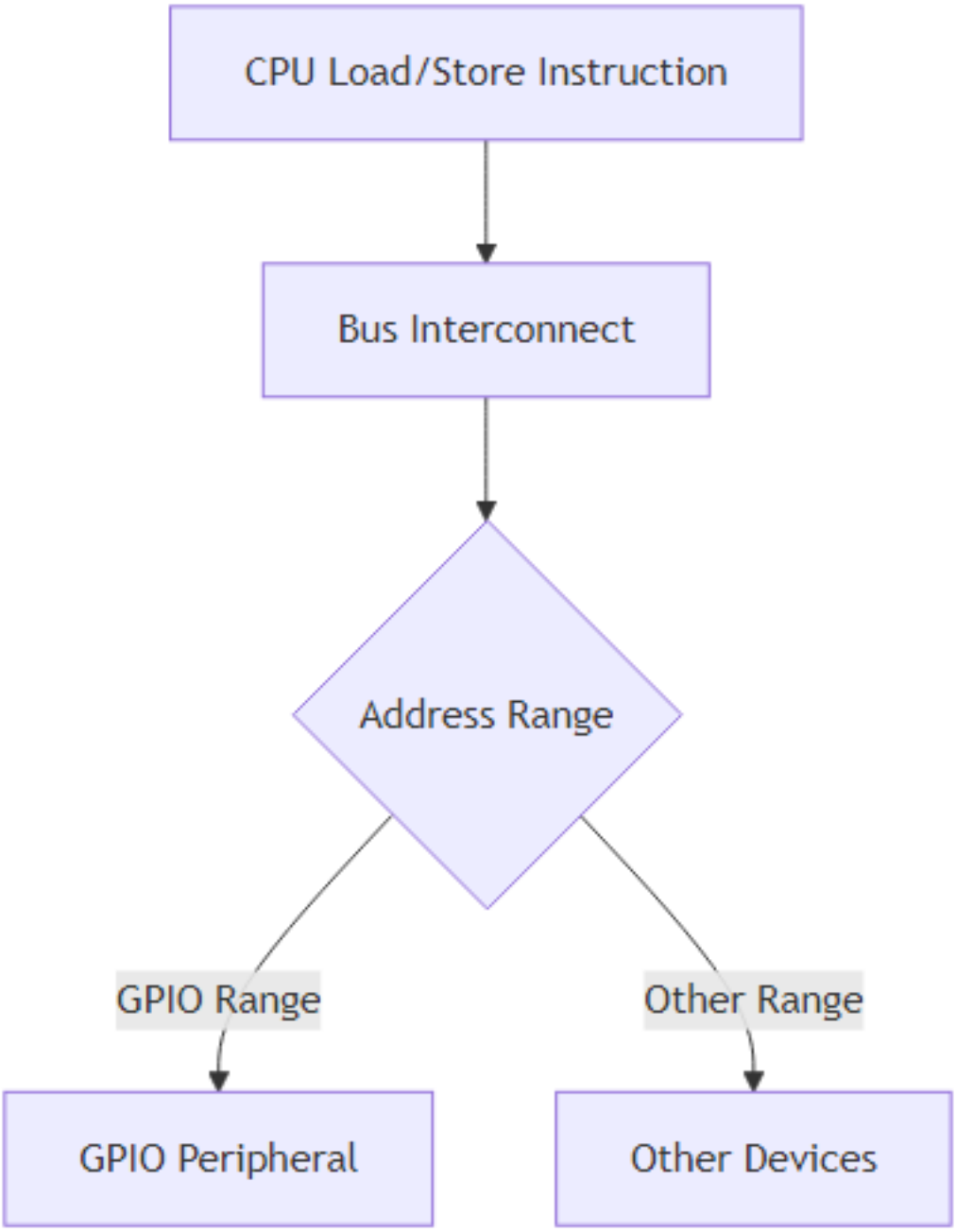
- turning LEDs on or off
- reading button states
- sending control signals to other hardware modules
- monitoring external digital inputs

15.4 Memory-Mapped Peripheral Model

The GPIO module is accessed through the **memory address space** of the SoC.

Instead of special instructions, the CPU uses normal load and store instructions.

Conceptually:



This design simplifies both hardware and software because the same memory interface is used for RAM and peripherals.

15.5 GPIO Register Interface

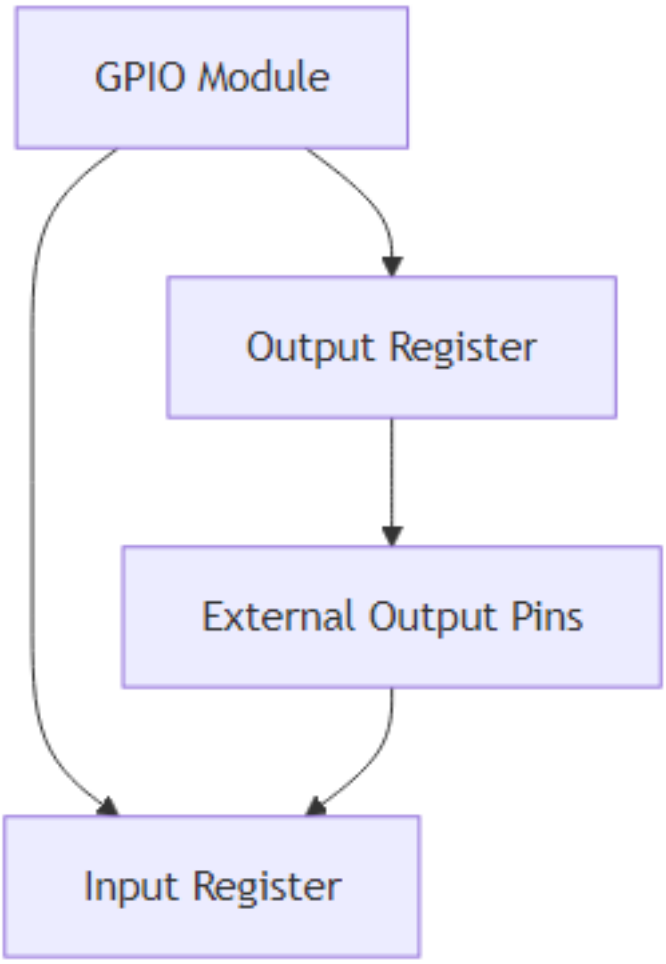
The GPIO peripheral typically exposes one or more **control registers** accessible through memory-mapped addresses.

Typical registers include:

Register	Purpose
Output Register	Drives values onto output pins
Input Register	Reports the current state of input pins

These registers are implemented as internal storage elements inside the GPIO module.

Conceptually:



The CPU writes to the output register and reads from the input register.

15.6 Inputs and Outputs

Although the exact signal names depend on the RTL implementation, the GPIO module usually has two groups of interfaces.

15.6.1 Bus Interface

Signals connected to the Bus Interconnect:

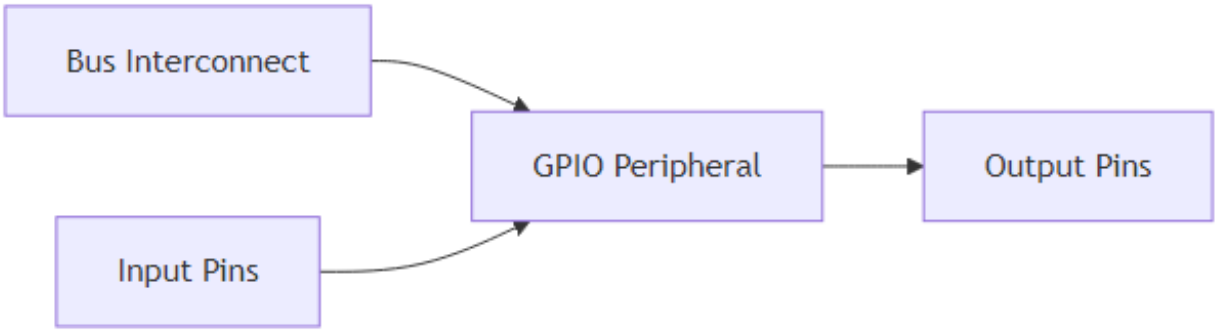
- address
- write data
- read data
- read enable
- write enable
- clock

15.6.2 External IO Interface

Signals connected to the outside world:

- GPIO input pins
- GPIO output pins

Conceptually:



This allows software to control hardware pins and observe external signals.

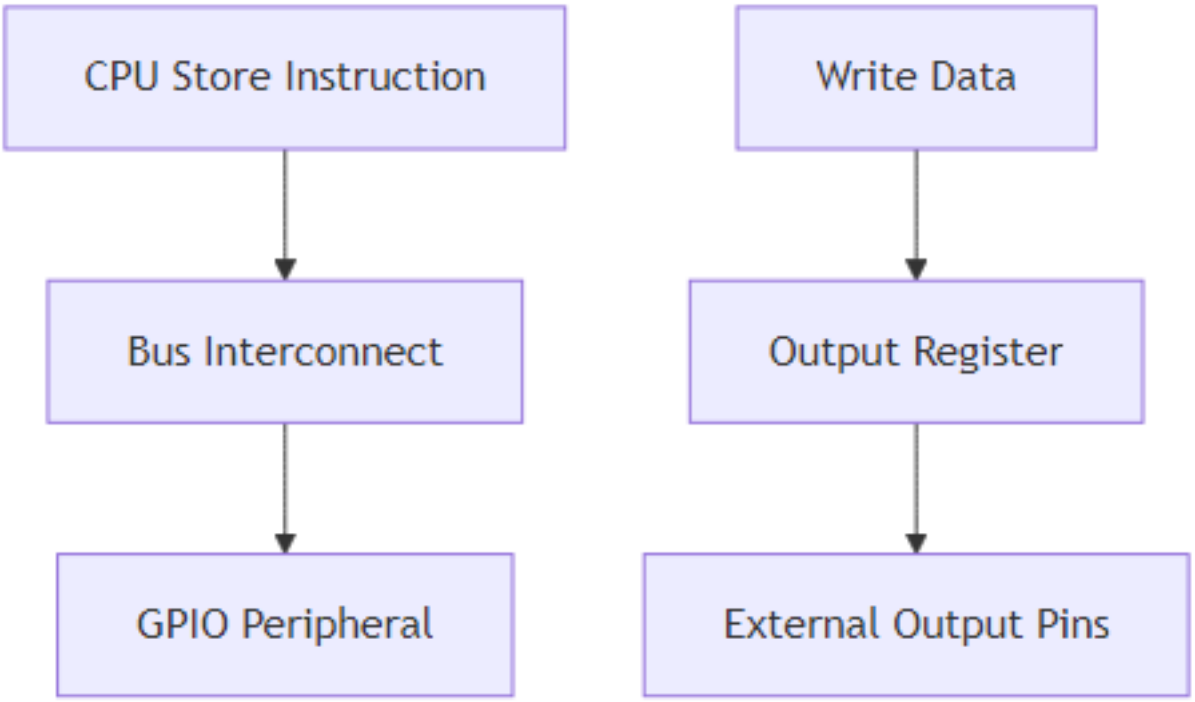
15.7 Output Operation

To drive values onto external pins, the CPU performs a **store instruction** to the GPIO output register.

Typical flow:

1. CPU writes value to GPIO address
2. Bus Interconnect routes the request
3. GPIO module updates output register
4. output register drives the external pins

Conceptually:



The output pins maintain the stored value until the register is updated again.

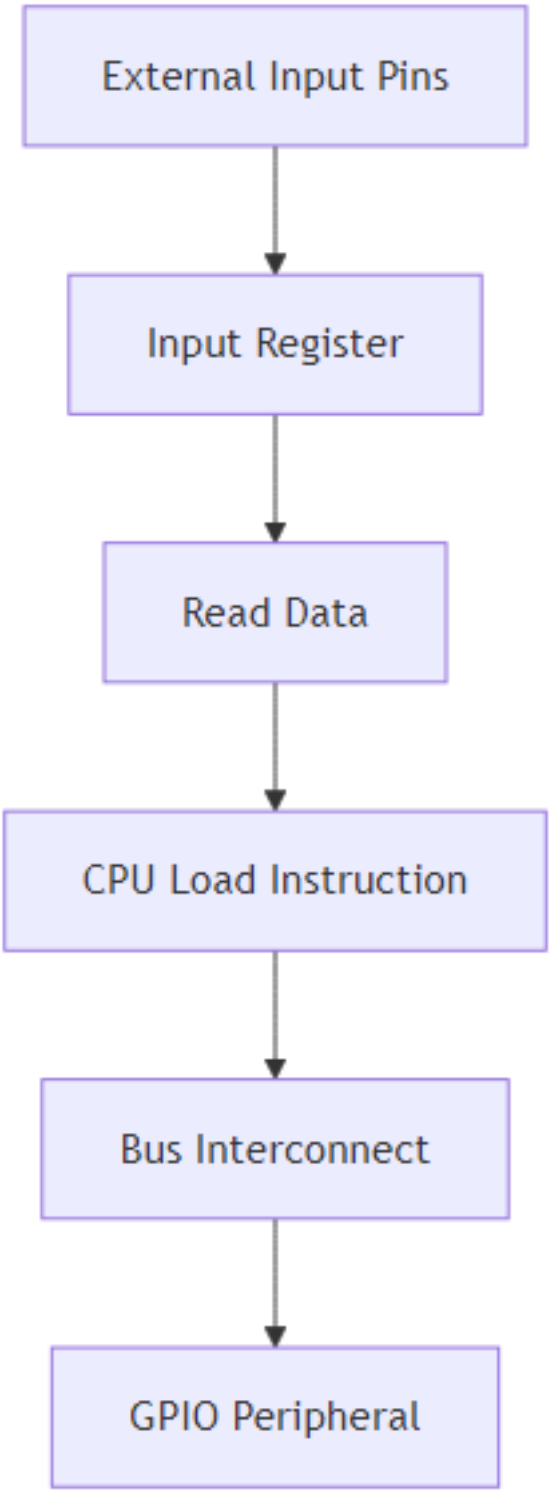
15.8 Input Operation

To read external signals, the CPU performs a **load instruction** from the GPIO input register.

Typical flow:

1. external hardware drives GPIO input pins
2. GPIO module captures input values
3. CPU reads the input register
4. the register value is returned to the CPU

Conceptually:

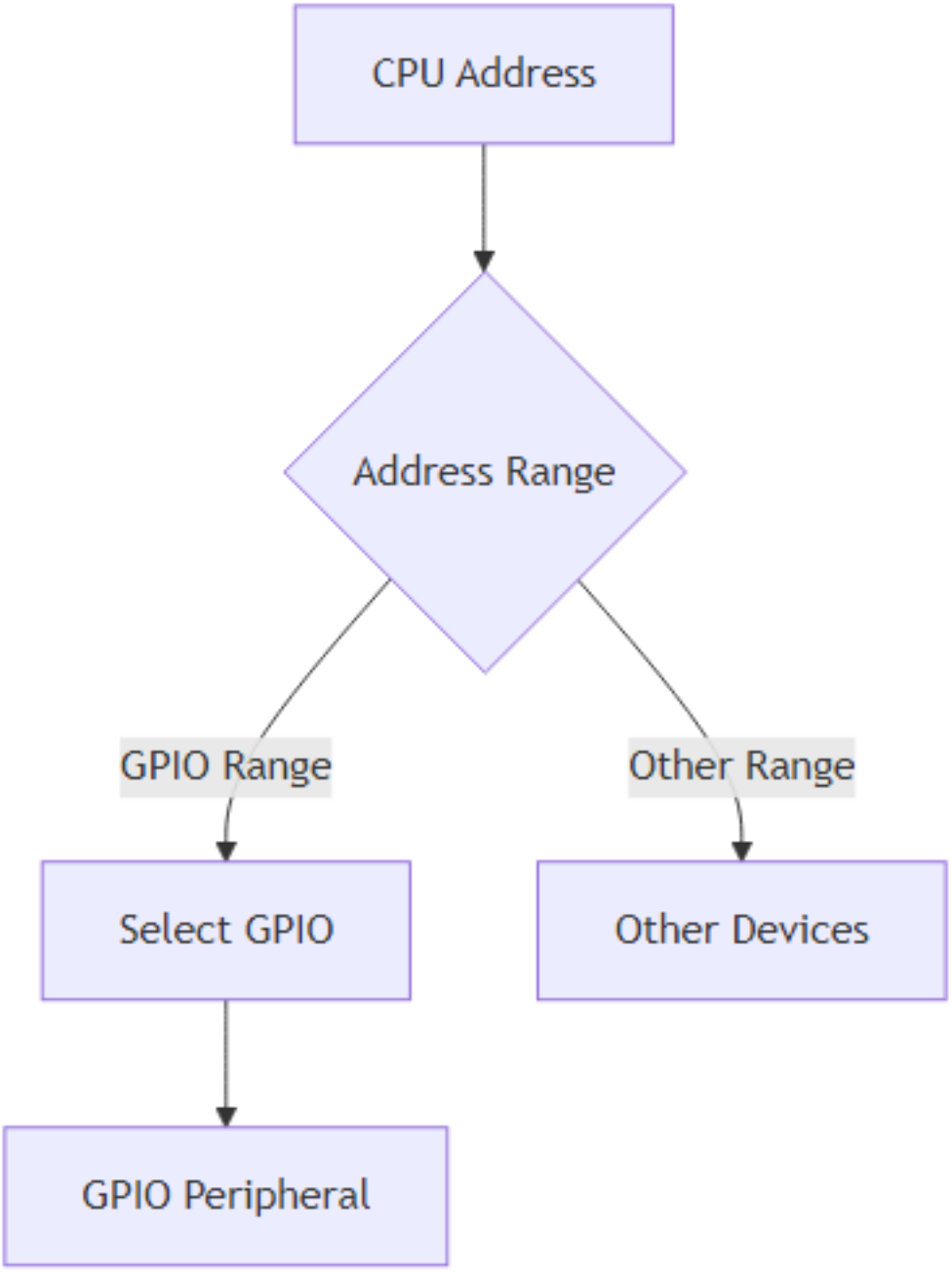


This allows software to monitor external signals.

15.9 Interaction with the Bus Interconnect

The Bus Interconnect determines when a transaction belongs to the GPIO module.

Conceptually:



When the GPIO address range is selected:

- write operations update GPIO registers
- read operations return GPIO register values

15.10 Interaction with the CPU

From the CPU's perspective, GPIO behaves like a memory location.

Example conceptual program flow:

```
store value to GPIO_output_address
```

This writes a value that drives external pins.

```
load value from GPIO_input_address
```

This reads the current state of input pins.

The CPU does not need to know that the device is a peripheral; it simply performs load/store operations.

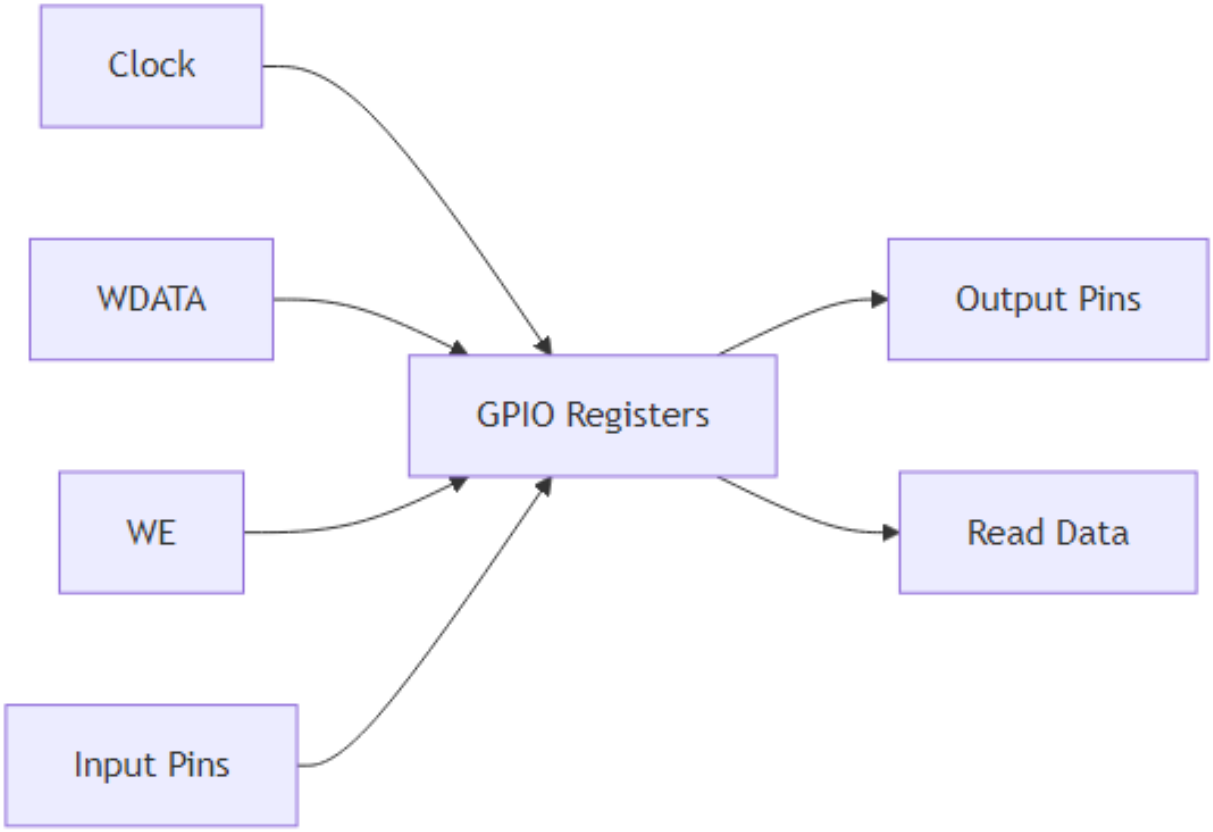
15.11 Sequential Behavior

GPIO registers are typically implemented as **clocked registers**.

This means:

- output registers update on clock edges
- input values may be captured synchronously

Conceptually:



This ensures stable and predictable behavior.

15.12 Example GPIO Use Cases

GPIO peripherals are widely used for simple system control tasks.

Examples include:

- controlling LEDs
- reading push buttons
- selecting configuration modes
- generating simple digital control signals

Example conceptual LED control:

```
store 1 to GPIO address
```

This might turn an LED on.

```
store 0 to GPIO address
```

This might turn the LED off.

15.13 Verification Expectations

A GPIO testbench should verify:

- correct write behavior for output registers
- correct read behavior for input registers
- proper propagation of output values to pins
- proper capture of input pin values

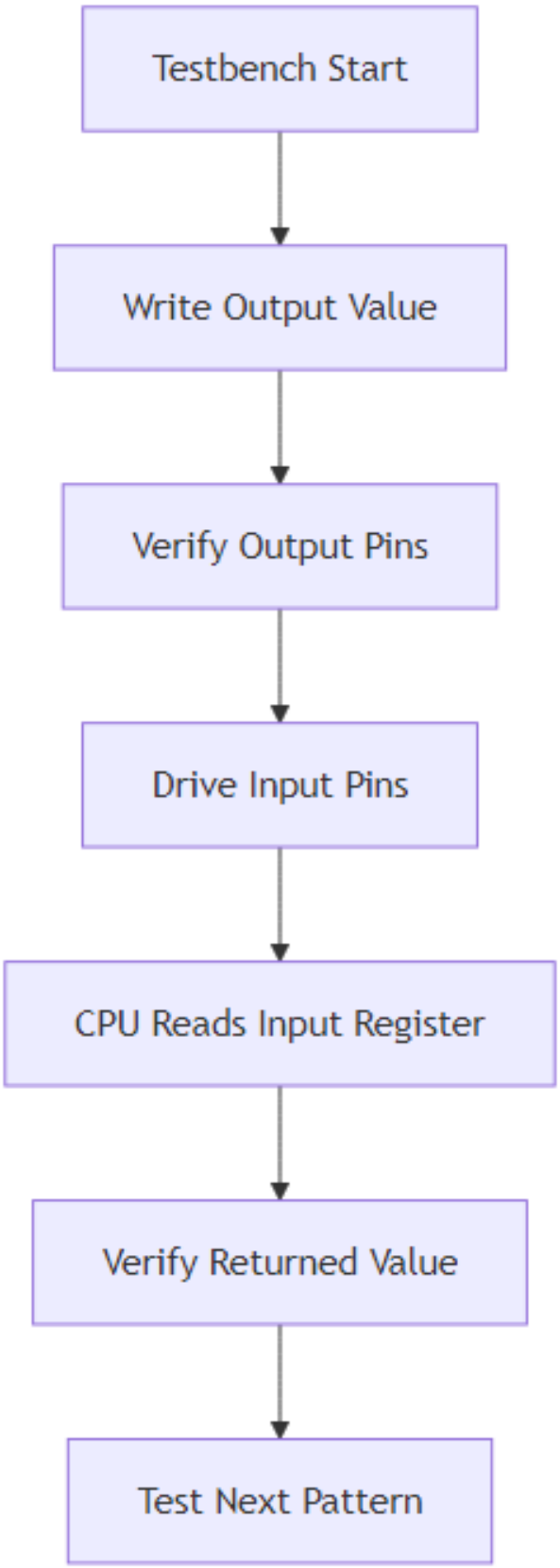
Typical tests include:

- writing different output patterns
- verifying output pin values
- driving input pins externally
- verifying CPU reads correct input values

Because GPIO connects to external hardware, verification often involves stimulating simulated external signals.

15.14 Example Verification Flow

A typical verification sequence:


Testbench Start
Write Output Value
Verify Output Pins
Drive Input Pins
CPU Reads Input Register
Verify Returned Value
Test Next Pattern

This confirms correct bidirectional behavior.

15.15 Design Strengths of the Module

The GPIO module provides several advantages.

15.15.1 Simple Hardware Interface

Only basic register logic is required.

15.15.2 Flexible External Interaction

Software can control external devices easily.

15.15.3 Clean SoC Integration

The module fits naturally into the memory-mapped architecture.

15.15.4 Educational Value

GPIO peripherals make the processor interact with the real world, which is valuable in FPGA-based demonstrations.

15.16 Limitations and Tradeoffs

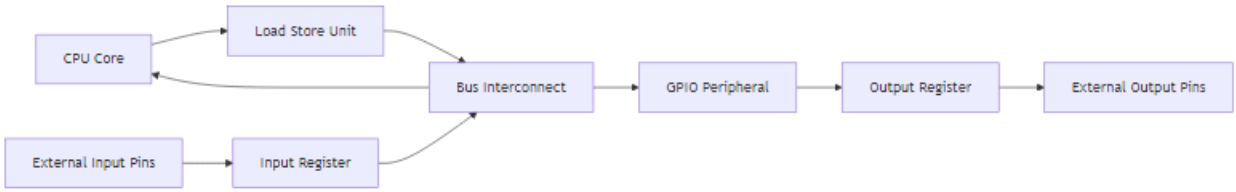
A minimal GPIO design may lack advanced features such as:

- programmable direction control
- interrupt generation
- debouncing logic
- configurable pin modes

These features can be added later without changing the CPU architecture.

15.17 Summary Diagram

The following diagram summarizes the role of the GPIO peripheral.



This diagram shows how the CPU communicates with external hardware through the GPIO peripheral.

15.18 Final Summary

The **GPIO module** provides a memory-mapped interface that allows the CPU to interact with external digital signals.

By exposing input and output registers accessible through load and store instructions, the GPIO peripheral enables software to control hardware devices and monitor external signals.

Through its connection to the Bus Interconnect, the GPIO module integrates seamlessly into the RV32I Minimal SoC architecture and provides a simple bridge between the processor and the external environment.

16 Timer Peripheral Module

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

16.1 Purpose of the Module

The **Timer** module provides a hardware time reference for the system.

It implements a counter that increments with each clock cycle and can be accessed by the CPU through the memory-mapped bus. Timers are widely used in embedded systems and SoC designs to measure time, generate delays, schedule tasks, and implement periodic events.

The timer allows software running on the CPU to:

- read the current system time
- measure elapsed time
- implement software delays
- synchronize operations with hardware timing

Even in a minimal SoC design, including a timer peripheral significantly improves system capabilities.

16.2 Architectural Role Inside the SoC

The Timer is implemented as a **memory-mapped peripheral** connected to the Bus Interconnect.

This means that the CPU interacts with the timer using normal load and store instructions.

Conceptually:

When the CPU accesses the timer's address range, the Bus Interconnect routes the transaction to the Timer module.

16.3 Timer Concept

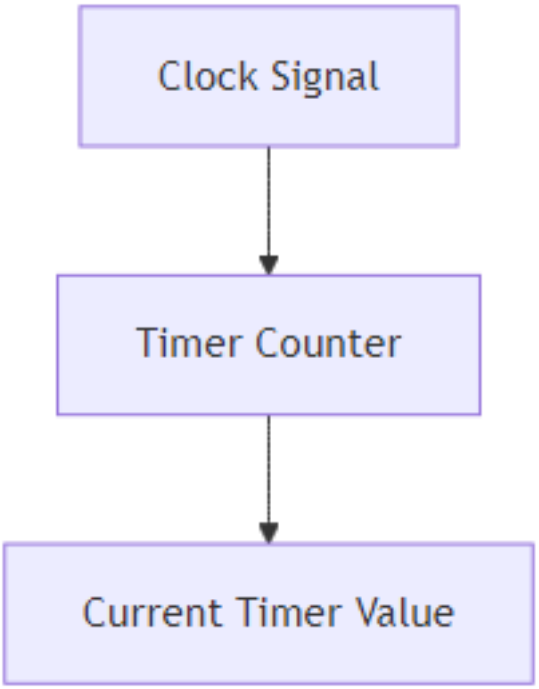
A hardware timer typically consists of a counter that increments with the system clock.

Every clock cycle:

```
counter = counter + 1
```

This counter therefore represents the passage of time in units of clock cycles.

Conceptually:



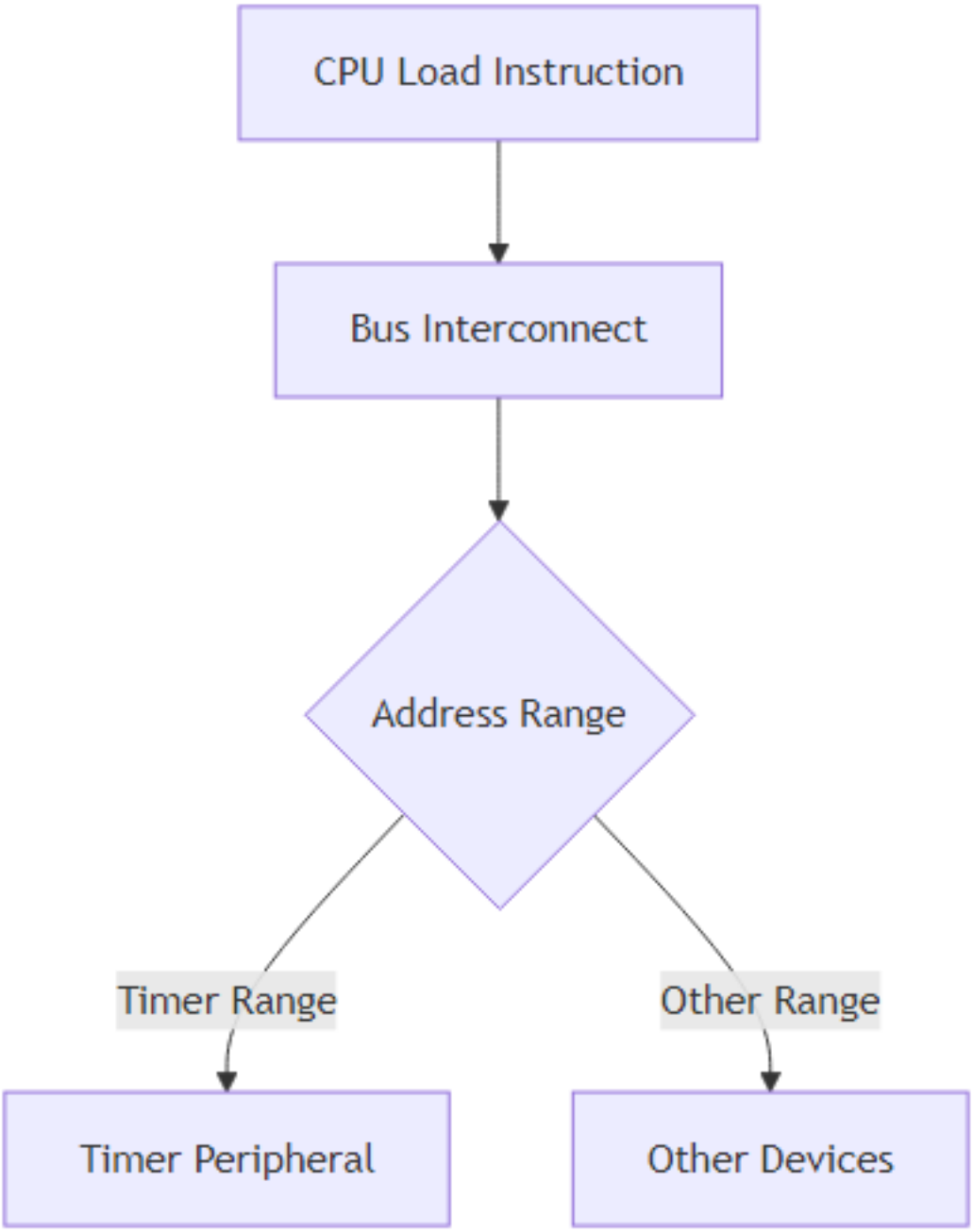
The CPU can read this value through memory-mapped registers.

16.4 Memory-Mapped Interface

The Timer module is accessed through the SoC address space.

This allows the CPU to read the timer value using a standard load instruction.

Conceptually:



The returned value corresponds to the current timer count.

16.5 Timer Register Interface

The Timer module typically exposes one or more registers.

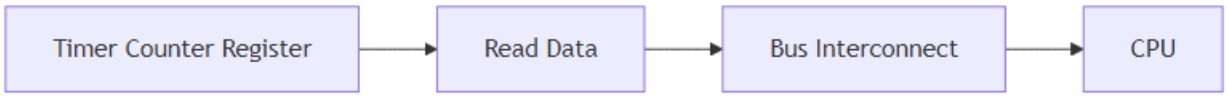
A minimal design may include a **single counter register**.

Example conceptual register map:

Register	Description
Timer Counter	Current value of the timer

The CPU reads this register to determine the current timer value.

Conceptually:



16.6 Inputs and Outputs

Although the exact signals depend on the RTL implementation, the Timer module typically includes the following interface signals.

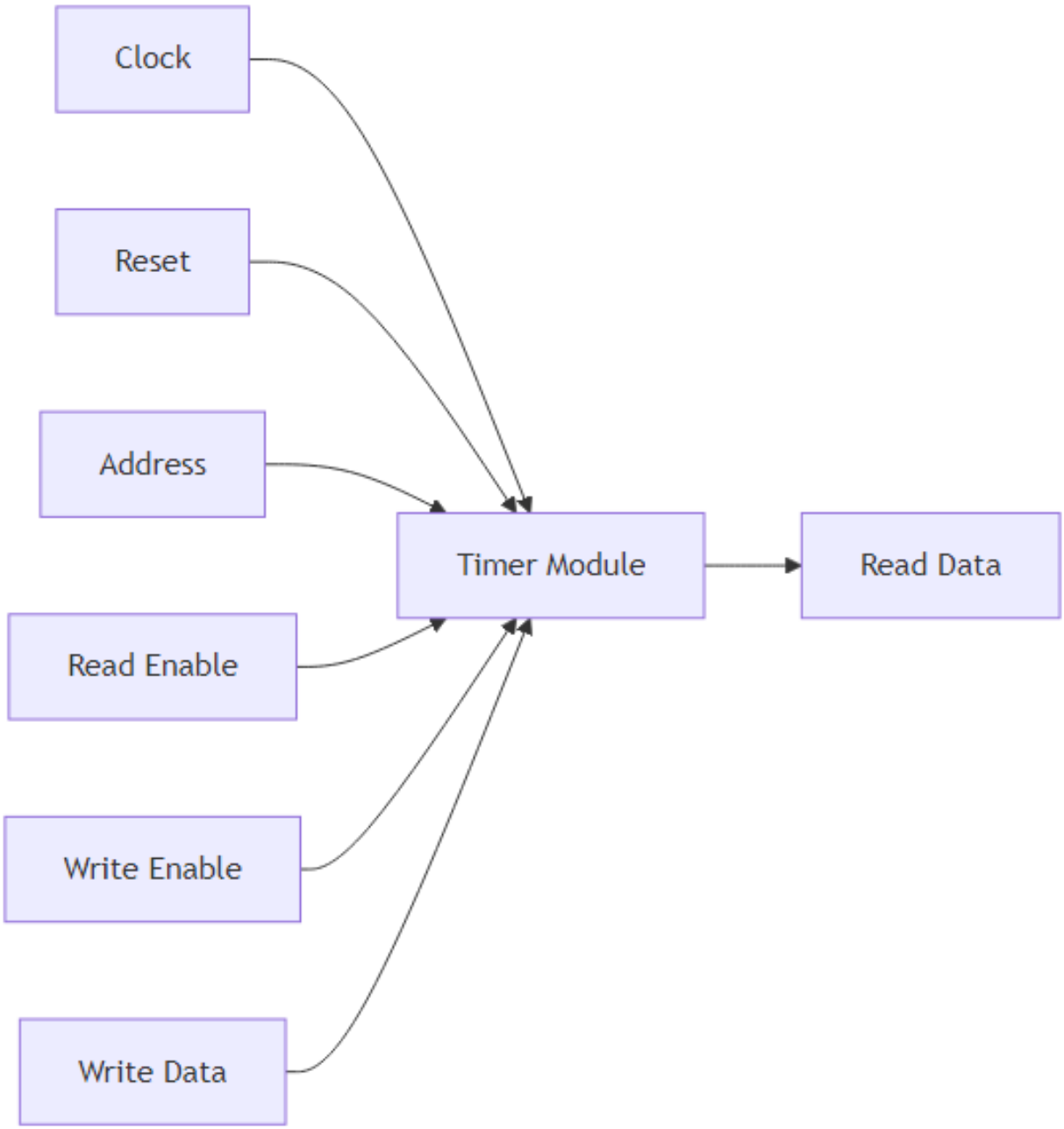
16.6.1 Inputs

- clock
- reset
- memory address
- read enable
- write enable (optional depending on design)
- write data (optional)

16.6.2 Outputs

- read data

Conceptually:



16.7 Counter Operation

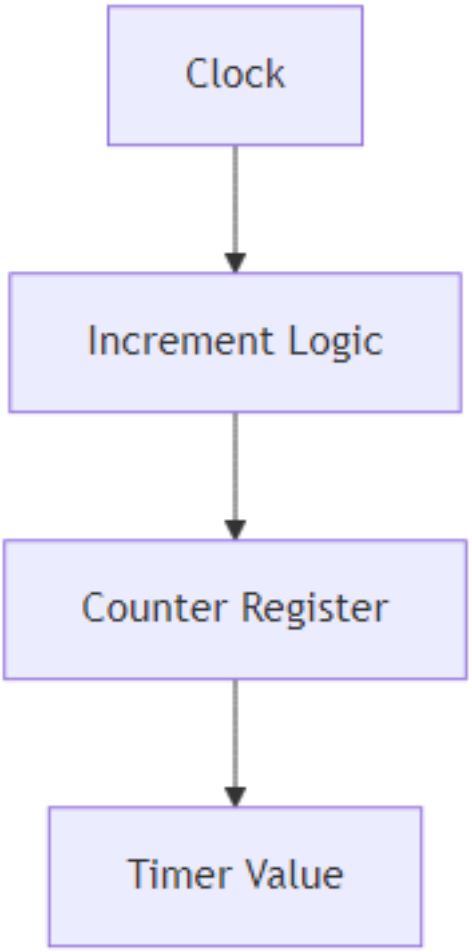
The core function of the Timer module is the counter.

Every clock cycle:

```
counter = counter + 1
```

This creates a continuously increasing time reference.

Conceptually:

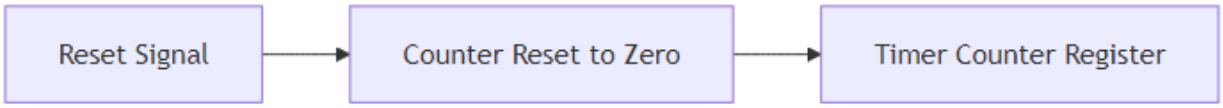


The current counter value can be returned to the CPU through the bus interface.

16.8 Reset Behavior

When the system reset is asserted, the timer counter typically returns to zero.

Conceptually:



After reset is released, the counter resumes incrementing with the system clock.

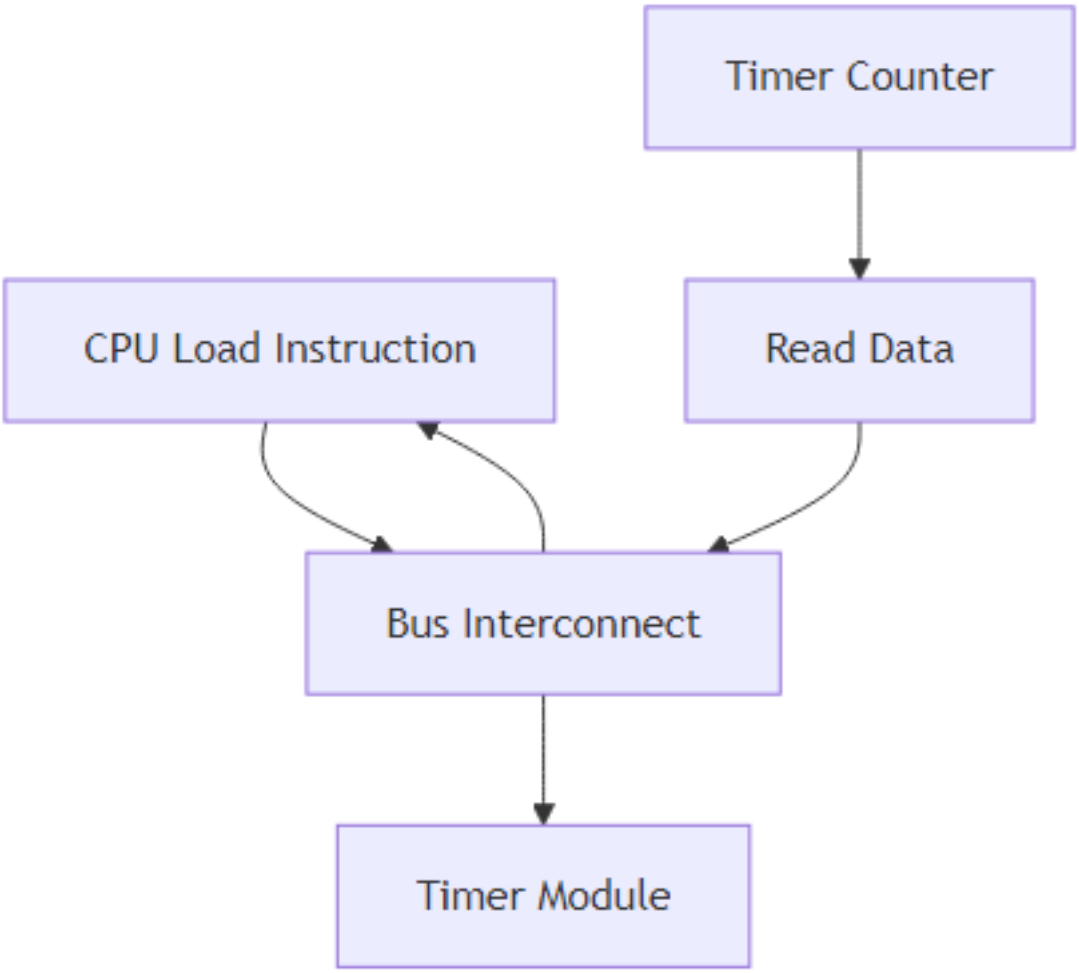
16.9 Timer Read Operation

When the CPU performs a load instruction targeting the timer address, the current counter value is returned.

Typical sequence:

1. CPU issues load instruction
2. address is routed through the bus
3. timer module is selected
4. current counter value is returned

Conceptually:



This allows software to measure time intervals.

16.10 Timer Write Operation (Optional)

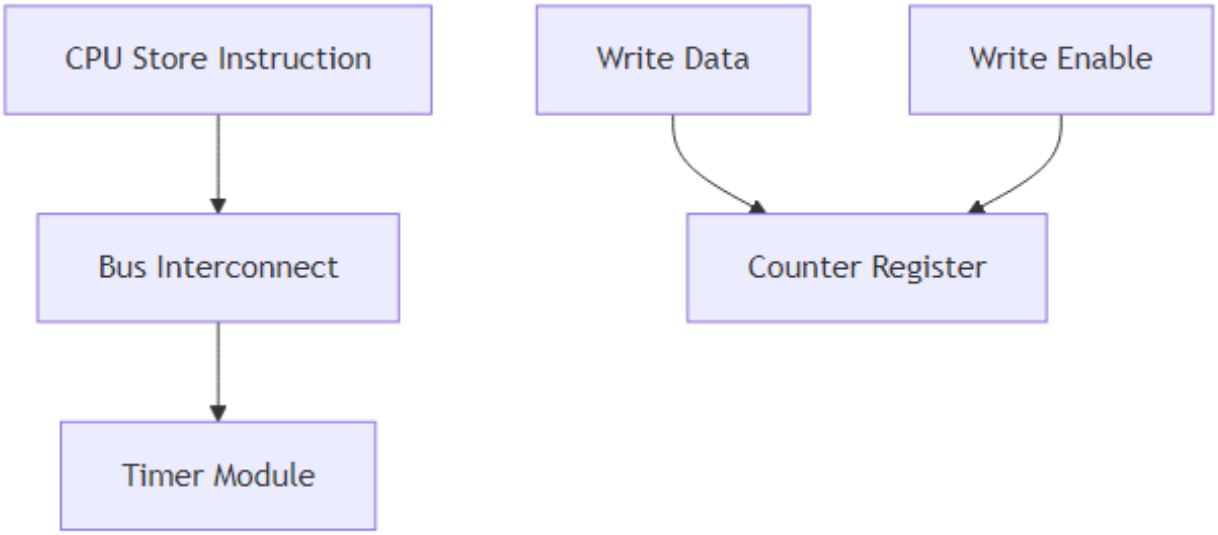
Some timer designs allow software to modify the timer value.

Possible operations include:

- resetting the counter
- writing a new counter value
- configuring timer behavior

In a minimal design, writes may be ignored or used to reset the counter.

Conceptually:

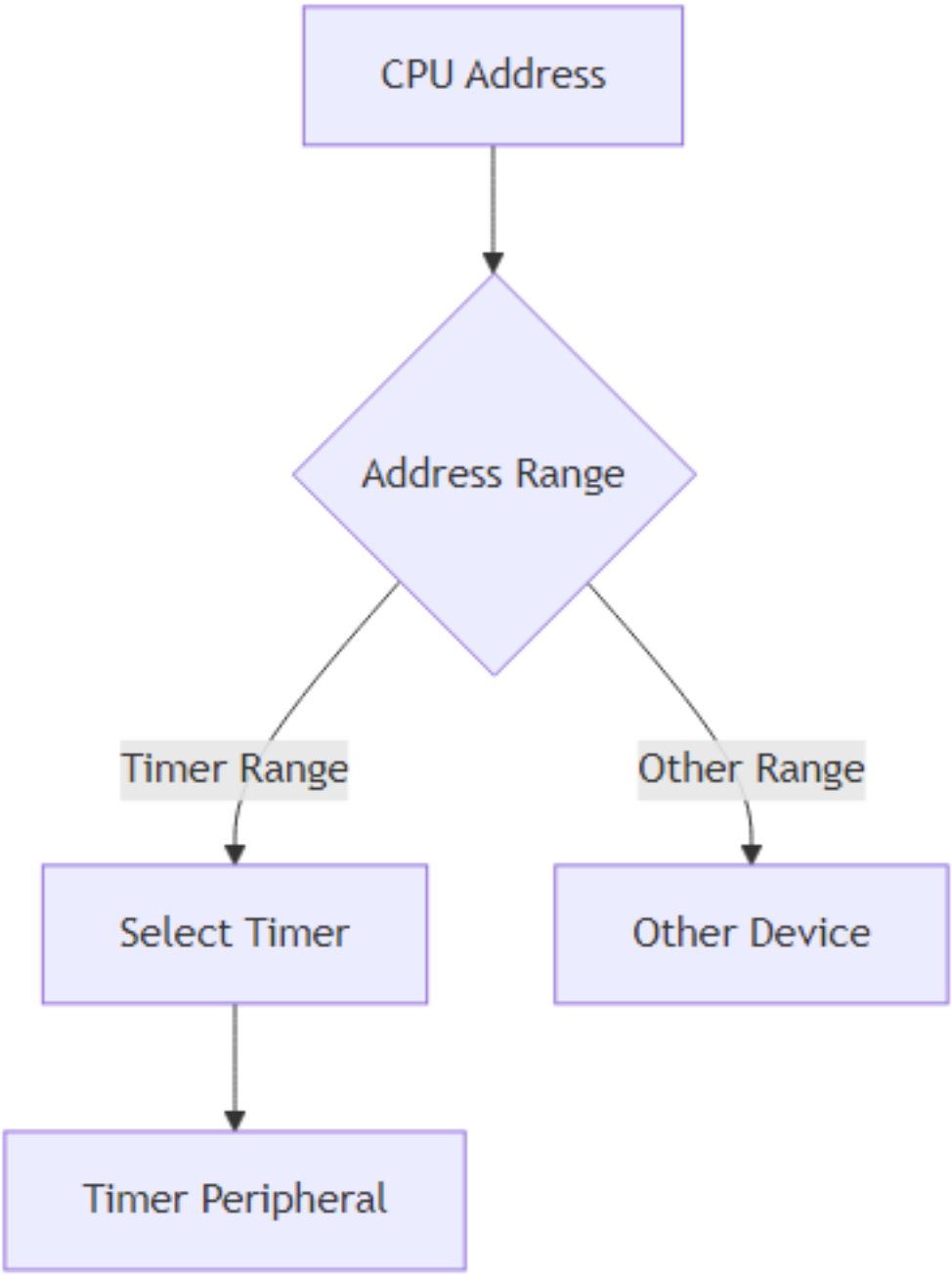


The exact behavior depends on the implementation in the RTL code.

16.11 Interaction with the Bus Interconnect

The Bus Interconnect routes timer accesses based on the address range.

Conceptually:



When the timer address range is selected, the timer module handles the transaction.

16.12 Interaction with the CPU

From the CPU's perspective, the timer behaves like a memory location.

Example conceptual program flow:

```
load x5, TIMER_ADDRESS
```

This loads the current timer value into register `x5`.

The CPU can then perform operations such as:

- measuring elapsed cycles
- implementing delays
- comparing timer values

16.13 Example Software Use Case

A timer can be used to measure elapsed time.

Example conceptual flow:

```
start_time = read_timer()
perform_operation()
end_time = read_timer()

elapsed_time = end_time - start_time
```

This allows software to measure the number of clock cycles consumed by an operation.

16.14 Sequential Behavior

The Timer module is inherently **sequential** because the counter changes with each clock cycle.

Conceptually:

This continuous update provides a real-time reference for the system.

16.15 Verification Expectations

A Timer testbench should verify:

- counter increment behavior
- reset functionality
- correct read operations
- correct interaction with the bus

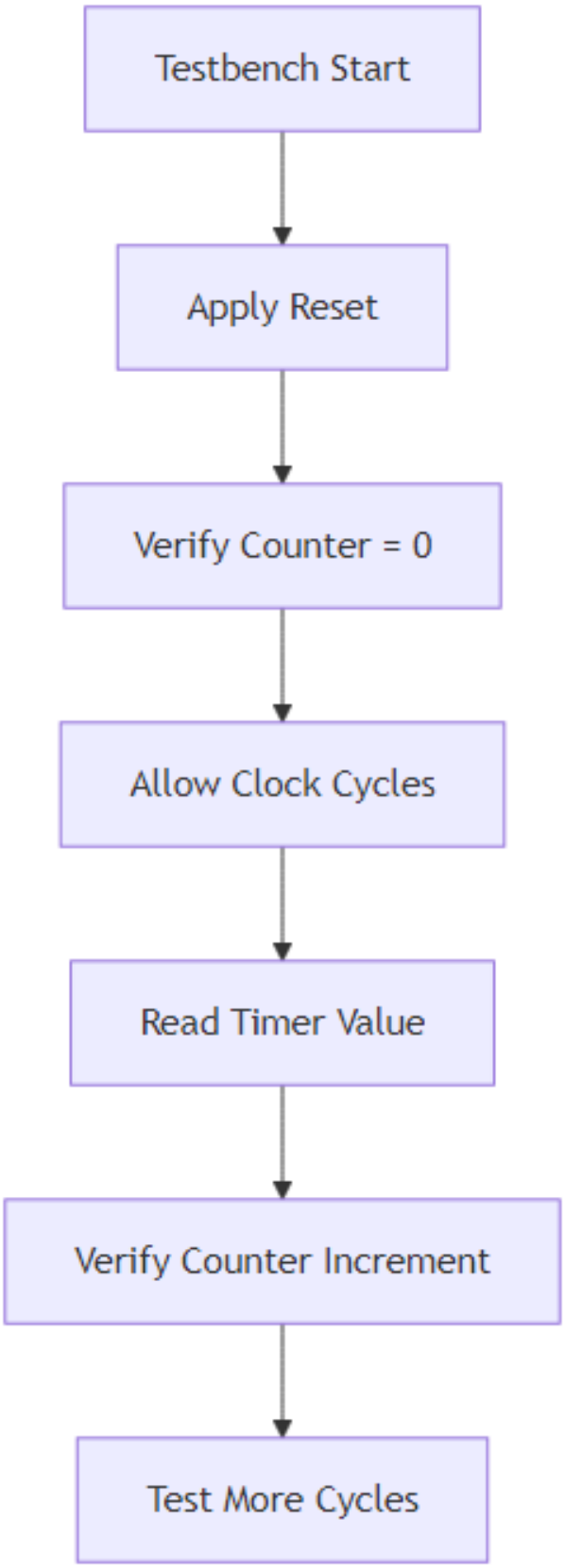
Typical tests include:

- verifying counter increments each cycle
- verifying reset clears the counter
- verifying correct read data
- verifying stable behavior across many cycles

Because the timer runs continuously, verification should observe multiple clock cycles.

16.16 Example Verification Flow

A conceptual verification sequence:


Testbench Start
Apply Reset
Verify Counter = 0
Allow Clock Cycles
Read Timer Value
Verify Counter Increment
Test More Cycles

This confirms that the timer behaves correctly over time.

16.17 Design Strengths of the Module

The Timer module provides several advantages.

16.17.1 System Time Reference

The CPU gains a reliable notion of elapsed time.

16.17.2 Software Scheduling

Programs can measure delays and schedule tasks.

16.17.3 Low Hardware Complexity

The timer is simple to implement but extremely useful.

16.17.4 Realistic SoC Feature

Most real embedded systems include timers, making this a valuable addition.

16.18 Limitations and Tradeoffs

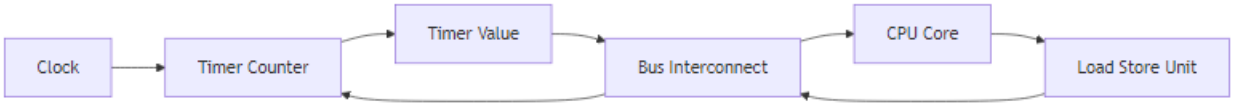
A minimal timer implementation may lack advanced features such as:

- programmable compare registers
- interrupt generation
- multiple timer channels
- configurable clock prescalers

These features can be added later without changing the CPU core.

16.19 Summary Diagram

The following diagram summarizes the Timer's role in the SoC.



This shows the timer acting as a continuously running hardware counter that the CPU can access through the memory-mapped bus.

16.20 Final Summary

The **Timer peripheral** provides a continuously incrementing hardware counter that allows the CPU to measure time and implement timing-based behavior.

By exposing the counter through a memory-mapped interface, the timer can be accessed using standard load and store instructions.

Through its integration with the Bus Interconnect, the Timer becomes a valuable system component that enables timing measurements, delays, and future extensions such as interrupt-driven scheduling within the RV32I Minimal SoC.

17 Verification Strategy

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

17.1 Purpose of the Verification Strategy

The purpose of the **verification strategy** is to ensure that the RV32I Minimal SoC behaves correctly according to its architectural specification.

Verification is the process of validating that the implemented hardware design performs the intended functions and produces correct results under all expected operating conditions.

A structured verification approach provides:

- confidence in functional correctness
- early detection of design bugs
- reproducible test procedures
- clear documentation of validation methods

In this project, verification is performed primarily through **VHDL testbenches** that simulate the behavior of individual modules and integrated system components.

17.2 Verification Goals

The verification process aims to confirm the following properties:

- each module behaves according to its functional specification
- the CPU correctly executes RV32I instructions
- memory and peripherals respond correctly to bus transactions
- control flow behaves correctly for branches, jumps, and traps
- the SoC architecture operates correctly as an integrated system

Verification is therefore divided into two major levels:

1. **Module-Level Verification**
2. **System-Level Verification**

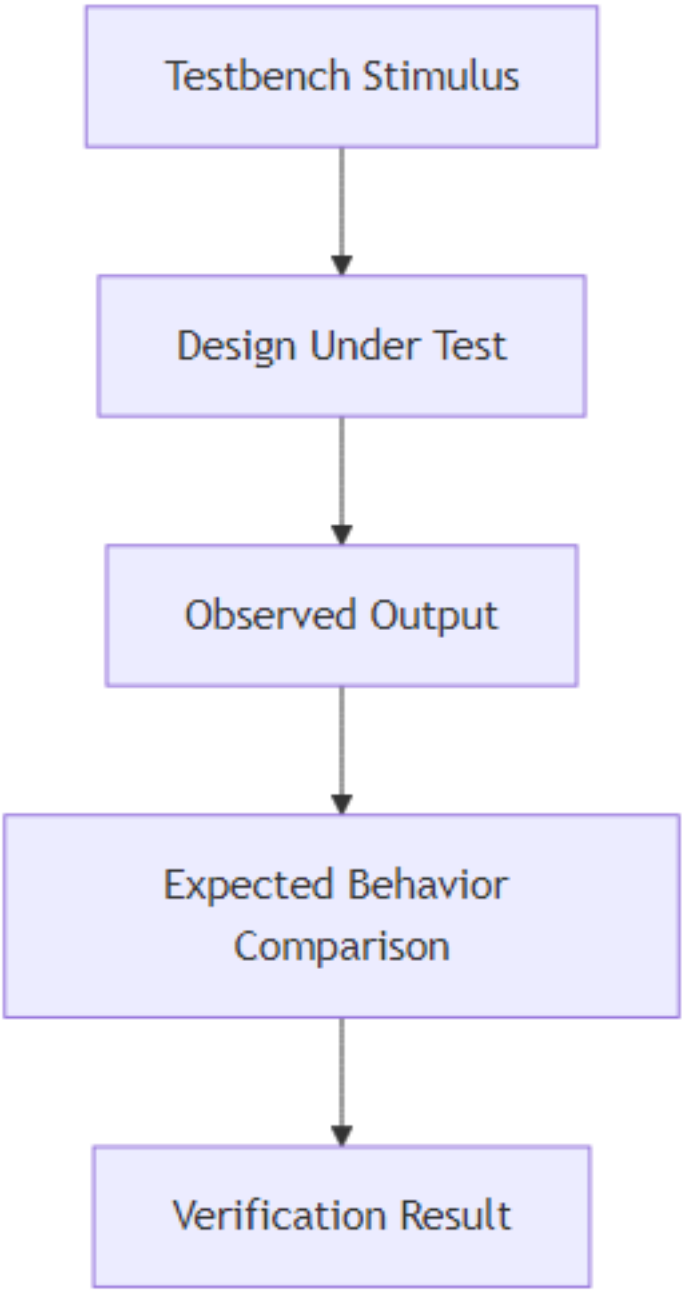
17.3 Verification Methodology

The project uses a **simulation-based verification methodology**.

In this approach:

1. hardware modules are instantiated in simulation testbenches
2. input stimuli are applied to the design
3. outputs are observed and validated
4. expected behavior is compared with actual results

Conceptually:



This approach allows the design to be validated before synthesis or FPGA deployment.

17.4 Design Under Test (DUT)

In each verification scenario, the hardware module being tested is referred to as the **Design Under Test (DUT)**.

Examples of DUTs in this project include:

- ALU
- Control Unit
- Register File
- Immediate Generator
- Program Counter
- Load Store Unit
- Bus Interconnect
- Timer
- GPIO
- CPU core

Each DUT is instantiated inside a dedicated testbench.

17.5 Module-Level Verification

Module-level verification focuses on testing individual hardware blocks in isolation.

Each module is verified using a dedicated testbench designed specifically for its functionality.

Examples include:

Module	Testbench
ALU	tb_alu
ALU Control	tb_alu_control
Branch Comparator	tb_branch_compare
Control Unit	tb_control_unit
Immediate Generator	tb_immediate_generator
Program Counter	tb_program_counter
Register File	tb_register_file
Load Store Unit	tb_load_store_unit
Timer	tb_timer
GPIO	tb_gpio
Data Memory	tb_data_memory
Bus Interconnect	tb_bus_interconnect
Trap Unit	tb_trap_unit

These testbenches verify the correctness of each component before system integration.

17.6 Unit Testing Approach

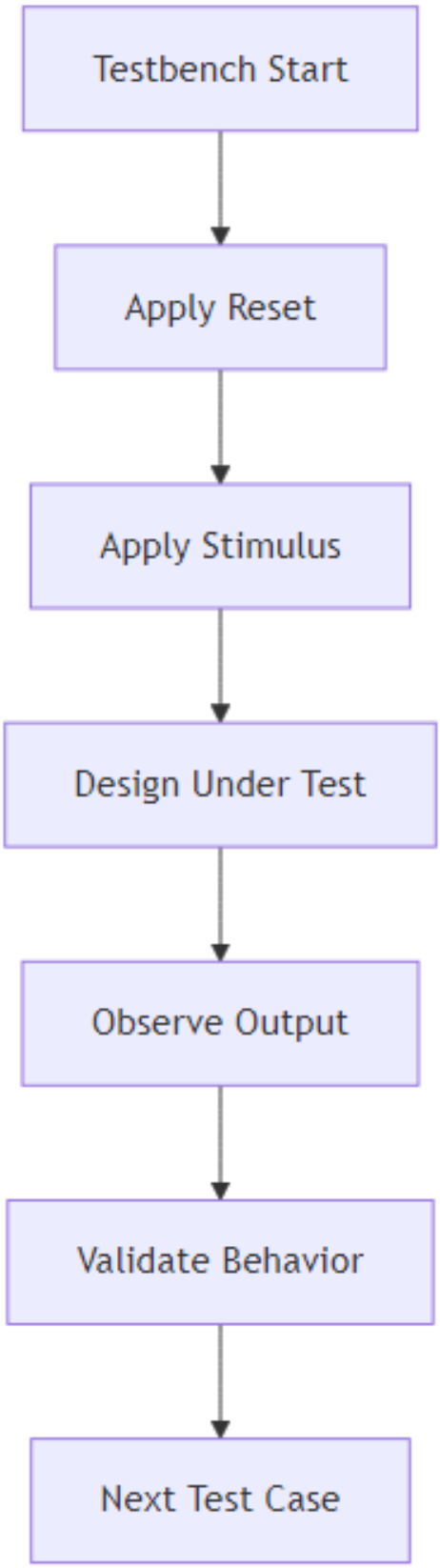
Each module testbench follows a structured unit-testing methodology.

Typical steps include:

1. instantiate the DUT
2. generate clock and reset signals
3. apply input stimuli

4. observe output signals
5. compare outputs with expected results

Conceptually:


Testbench Start
Apply Reset
Apply Stimulus
Design Under Test
Observe Output
Validate Behavior
Next Test Case

Unit testing isolates module functionality and simplifies debugging.

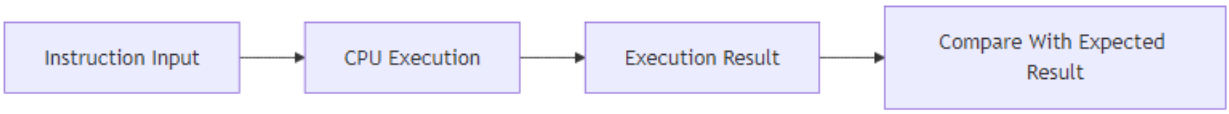
17.7 Instruction-Level Verification

The CPU core must correctly execute instructions defined in the **RV32I instruction set architecture**.

Instruction-level verification ensures that:

- arithmetic operations produce correct results
- logical operations behave correctly
- load and store instructions access memory properly
- branch instructions correctly control program flow
- immediate values are generated correctly

Conceptually:



Testbenches simulate representative instruction sequences to validate CPU functionality.

17.8 Control Flow Verification

Special attention is given to instructions that alter the program counter.

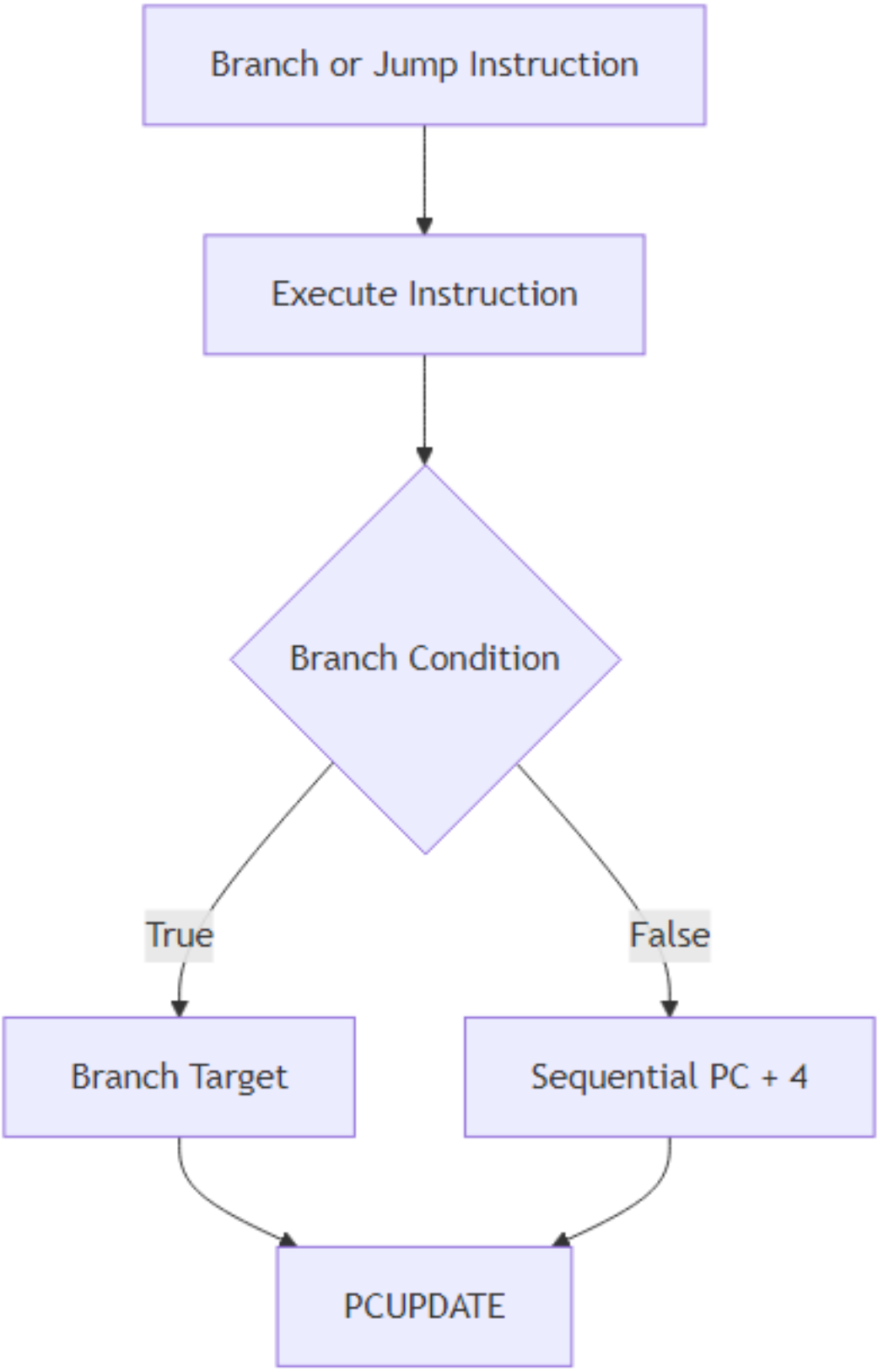
These include:

- branch instructions
- jump instructions
- trap handling

Verification confirms that:

- branches are taken when conditions are true
- branches are not taken when conditions are false
- jump targets are computed correctly
- trap redirection overrides normal execution flow

Conceptually:


Branch or Jump Instruction
Execute Instruction
Branch Condition
True
False
Branch Target
Sequential PC + 4
PCUPDATE

Correct control flow is essential for proper program execution.

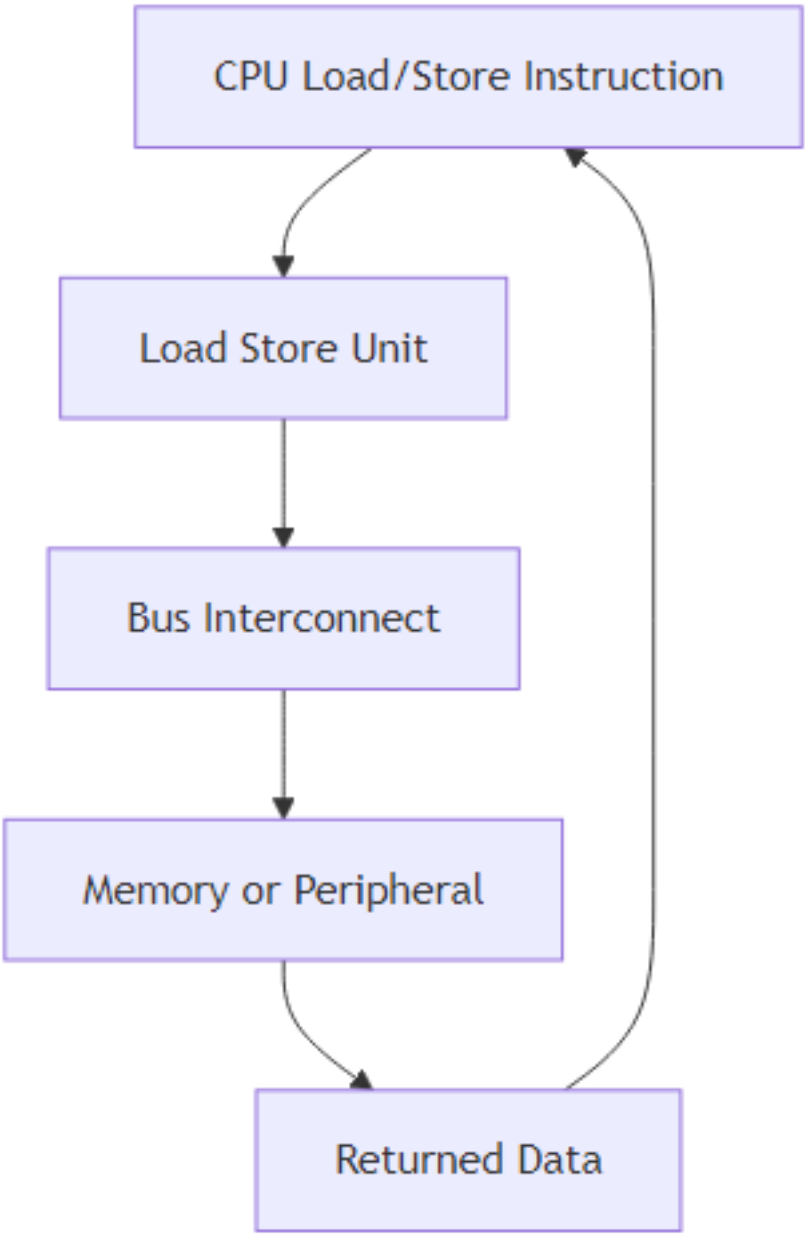
17.9 Memory Access Verification

Memory access behavior must also be verified.

Verification ensures:

- loads return correct values
- stores update memory correctly
- address calculations are correct
- bus routing selects the correct device

Conceptually:



Testbenches confirm that memory transactions behave correctly.

17.10 Peripheral Verification

Peripheral modules are verified independently.

Peripherals tested include:

- GPIO
- Timer

Verification ensures:

- correct register behavior
- proper bus interactions
- correct signal propagation

Example verification goals:

GPIO: - output register updates correctly - input pins are read correctly

Timer: - counter increments each clock cycle - reset behavior is correct - read operations return the current counter value

---

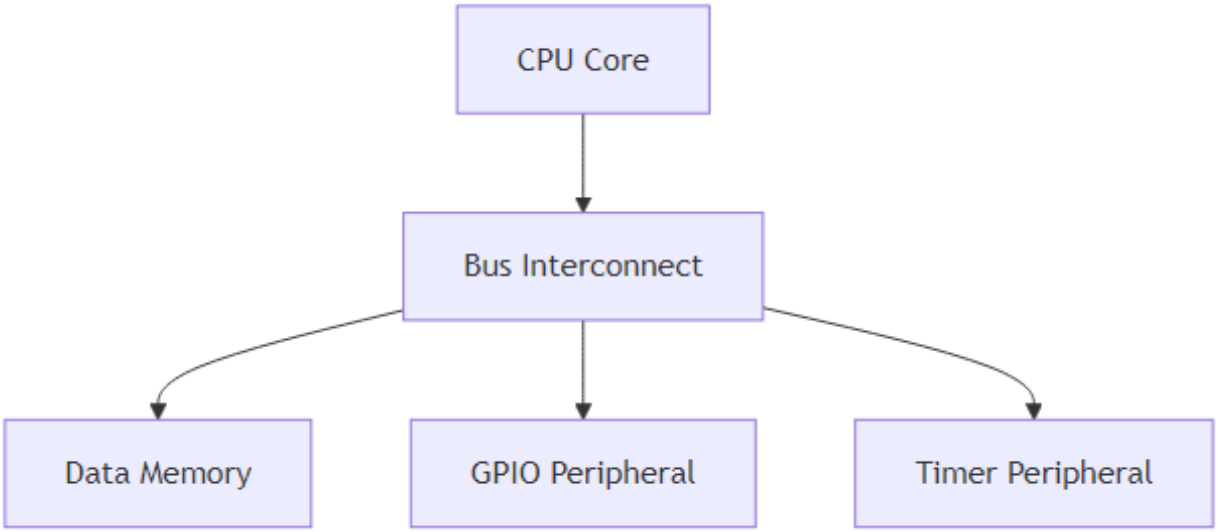
17.11 Integrated System Verification

After individual modules are verified, integrated system testing validates interaction between modules.

System-level verification confirms that:

- CPU communicates correctly with peripherals
- bus routing operates correctly
- memory transactions behave properly
- control flow functions correctly across the full system

Conceptually:



Integrated tests confirm correct cooperation between all system components.

17.12 Clock and Reset Handling

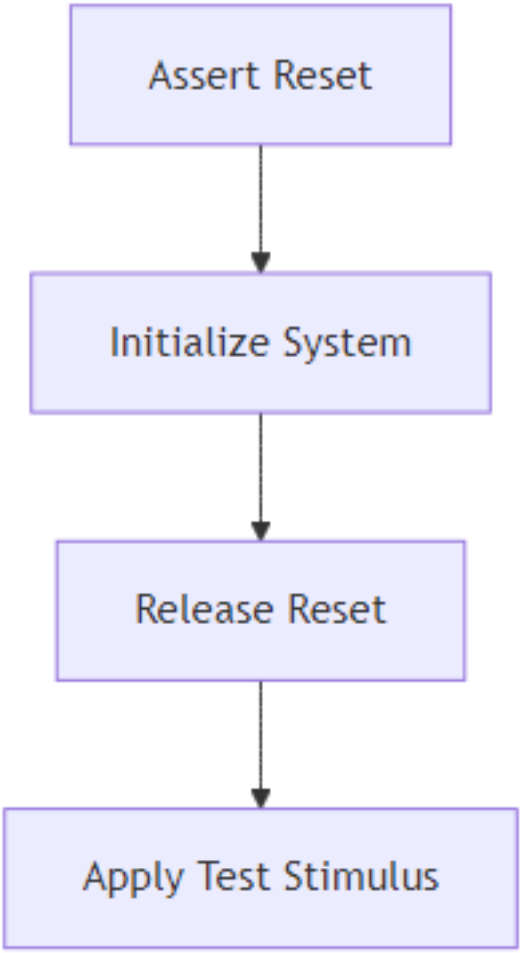
Testbenches generate clock and reset signals to simulate realistic hardware behavior.

Typical simulation pattern:

1. assert reset

2. initialize system state
3. release reset
4. begin stimulus generation

Conceptually:



Correct reset behavior is essential for predictable system startup.

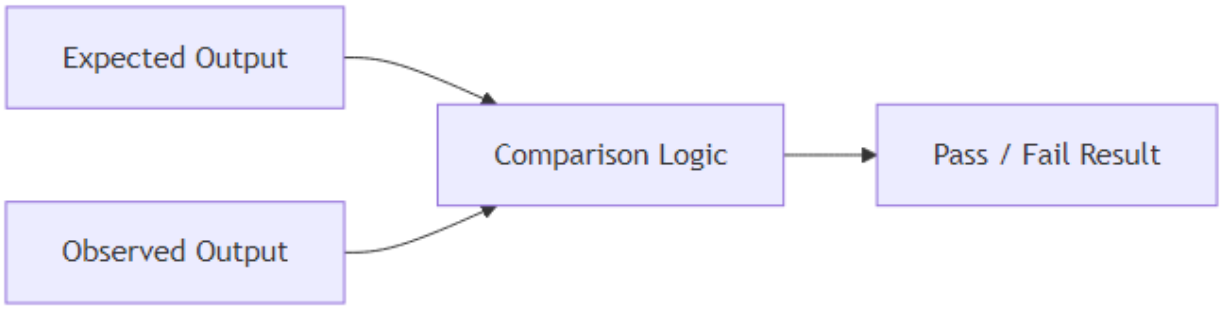
17.13 Expected vs Observed Behavior

Verification compares **expected behavior** with **observed outputs**.

Two approaches may be used:

- waveform inspection
- assertion-based checks
- explicit result comparisons

Conceptually:



This ensures that each test produces deterministic validation results.

17.14 Simulation Environment

All verification in this project is performed using a **VHDL simulation environment**.

The simulation environment provides:

- waveform visualization
- signal inspection
- execution tracing
- debugging capabilities

Simulation tools typically allow engineers to observe internal signals and verify the behavior of complex digital systems.

17.15 Debugging Strategy

When verification detects incorrect behavior, debugging typically follows a structured approach:

1. identify failing test case
2. inspect waveform traces
3. identify incorrect signal behavior
4. trace the signal back to the source logic
5. correct the RTL implementation
6. rerun verification

This iterative process ensures that design bugs are systematically removed.

17.16 Coverage Considerations

Although full coverage analysis is beyond the scope of a minimal project, the verification approach aims to cover the following categories:

- arithmetic instruction coverage
- branch condition coverage
- memory access coverage

- peripheral interaction coverage
- control signal coverage

Test cases are selected to exercise representative behaviors of each module.

17.17 Benefits of the Verification Strategy

This verification methodology provides several advantages.

17.17.1 Early Bug Detection

Errors can be identified before hardware synthesis.

17.17.2 Modular Debugging

Problems can be isolated to individual modules.

17.17.3 Repeatable Testing

Testbenches can be rerun automatically after design changes.

17.17.4 Clear Documentation

The verification process becomes part of the project documentation.

17.18 Limitations of the Verification Approach

Simulation-based verification also has limitations.

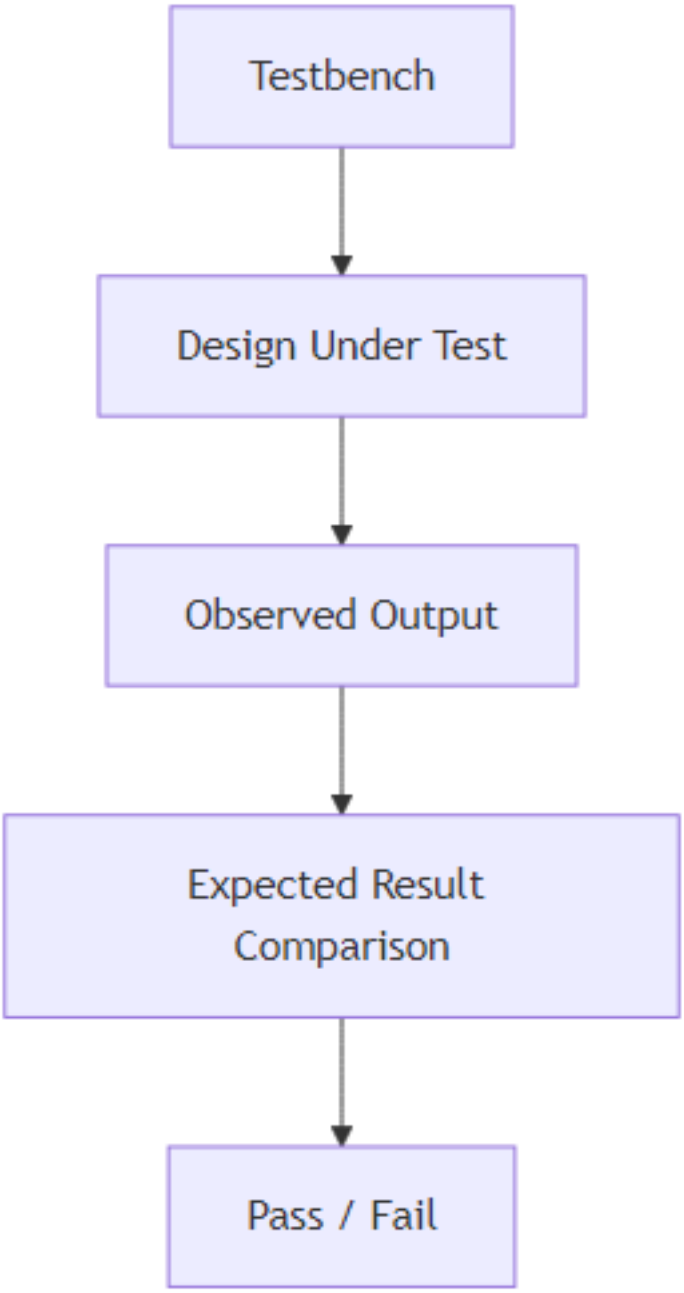
Examples include:

- incomplete coverage of all possible input combinations
- dependence on manually written test cases
- absence of formal verification methods
- limited performance compared to hardware execution

Despite these limitations, simulation remains the most common verification method used in early hardware design stages.

17.19 Summary Diagram

The following diagram summarizes the overall verification workflow.



Each module and subsystem of the RV32I Minimal SoC is validated using this approach.

17.20 Final Summary

The **verification strategy** for the RV32I Minimal SoC relies on a simulation-based methodology using dedicated VHDL testbenches.

Verification is performed at both the module level and the system level to ensure that individual components behave correctly and that the integrated SoC architecture operates as intended.

By systematically applying test stimuli, observing outputs, and comparing results with expected behavior, this strategy provides strong confidence in the correctness of the processor implementation.

18 Testbench Architecture

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

18.1 Purpose of the Testbench Architecture

The **testbench architecture** defines the structure used to simulate and verify the hardware modules of the RV32I Minimal SoC.

A testbench is a simulation environment that instantiates a hardware module, applies input stimuli, observes outputs, and verifies the correctness of the design behavior.

Unlike synthesizable hardware modules, testbenches exist only for simulation and are not implemented in the final hardware.

The goals of the testbench architecture are:

- provide controlled stimulus for the design under test
- simulate realistic system behavior
- validate functional correctness
- enable debugging through waveform observation
- support repeatable verification procedures

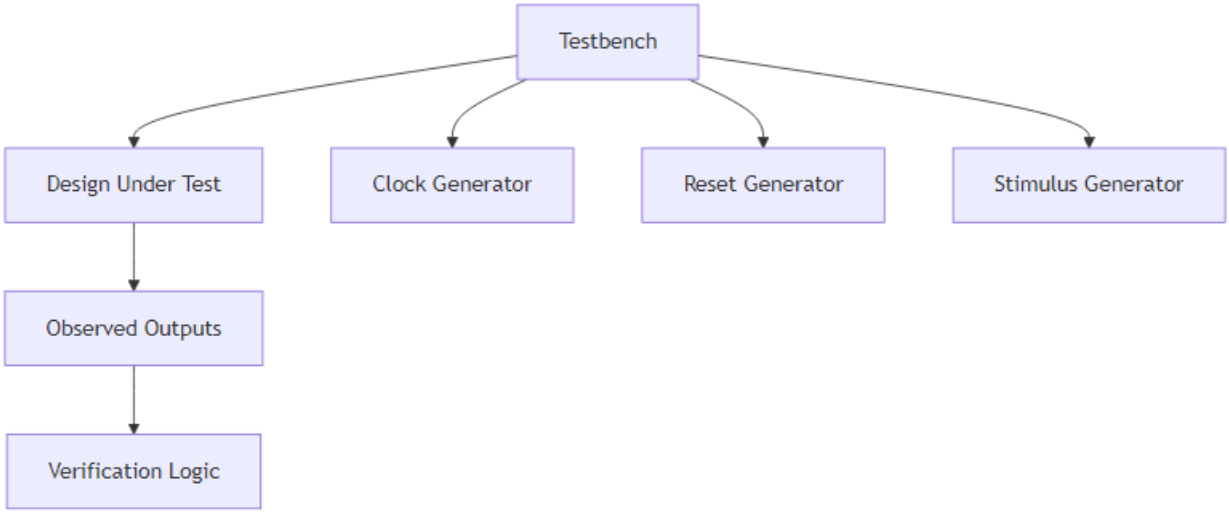
18.2 Testbench Structure

Each testbench follows a common structural organization.

The main components of a testbench include:

1. **Design Under Test (DUT)**
2. **Clock Generation Logic**
3. **Reset Logic**
4. **Stimulus Generation**
5. **Output Observation**
6. **Verification Checks**

Conceptually:



This structure ensures consistent testing across all modules.

18.3 Design Under Test (DUT)

The **Design Under Test (DUT)** is the hardware module being verified.

In each testbench, the DUT is instantiated as a component and connected to signals defined inside the testbench environment.

Example DUTs in this project include:

- ALU
- Control Unit
- Immediate Generator
- Program Counter
- Register File
- Load Store Unit
- Bus Interconnect
- GPIO
- Timer
- CPU Core

Each module is verified independently before integration.

18.4 Clock Generation

Most digital hardware designs require a clock signal.

The testbench generates a simulated clock to drive sequential logic inside the DUT.

Conceptually:

A common clock generation pattern is:

- initialize clock to zero
- toggle clock at fixed intervals
- repeat continuously during simulation

This simulates the periodic behavior of real hardware clocks.

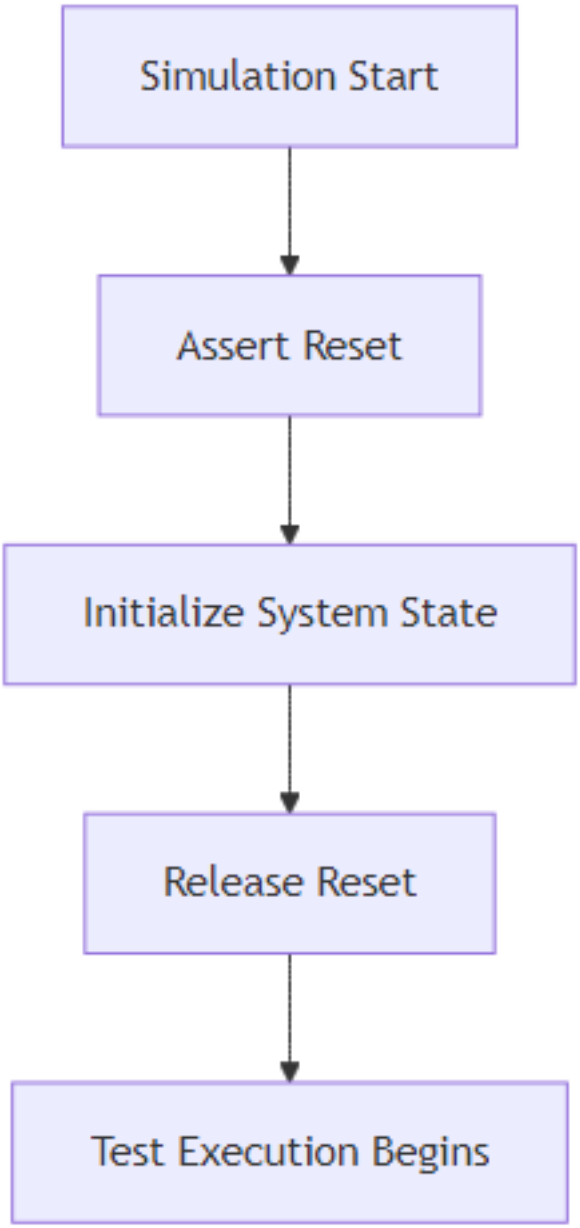
18.5 Reset Generation

A reset signal initializes the system into a known state before simulation begins.

Typical reset sequence:

1. assert reset at simulation start
2. allow internal registers to initialize
3. release reset
4. begin test stimulus

Conceptually:



Proper reset handling ensures predictable behavior across tests.

18.6 Stimulus Generation

Stimulus generation drives input signals to the DUT.

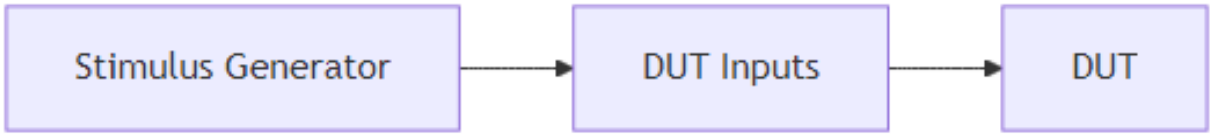
Stimulus may include:

- instruction values
- register indices
- data values

- control signals
- memory addresses

Stimulus sequences are designed to test specific functionality.

Conceptually:



Stimulus may be applied sequentially to simulate multiple test cases.

18.7 Output Observation

The testbench observes output signals generated by the DUT.

Observed outputs may include:

- arithmetic results
- memory values
- register outputs
- control signals
- peripheral responses

Conceptually:

These signals are used to determine whether the DUT behaves correctly.

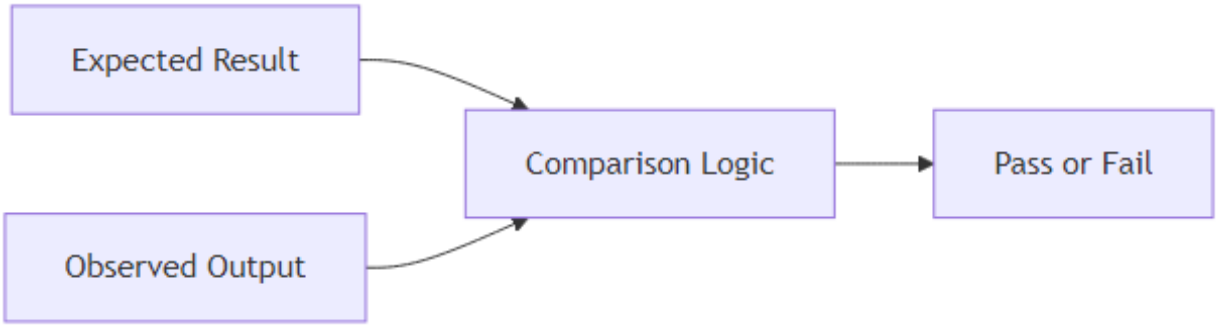
18.8 Verification Checks

Verification checks compare the observed outputs with expected results.

Typical verification methods include:

- manual waveform inspection
- conditional checks inside the testbench
- assertions
- printed simulation messages

Conceptually:



This comparison step is critical to determine whether the design is correct.

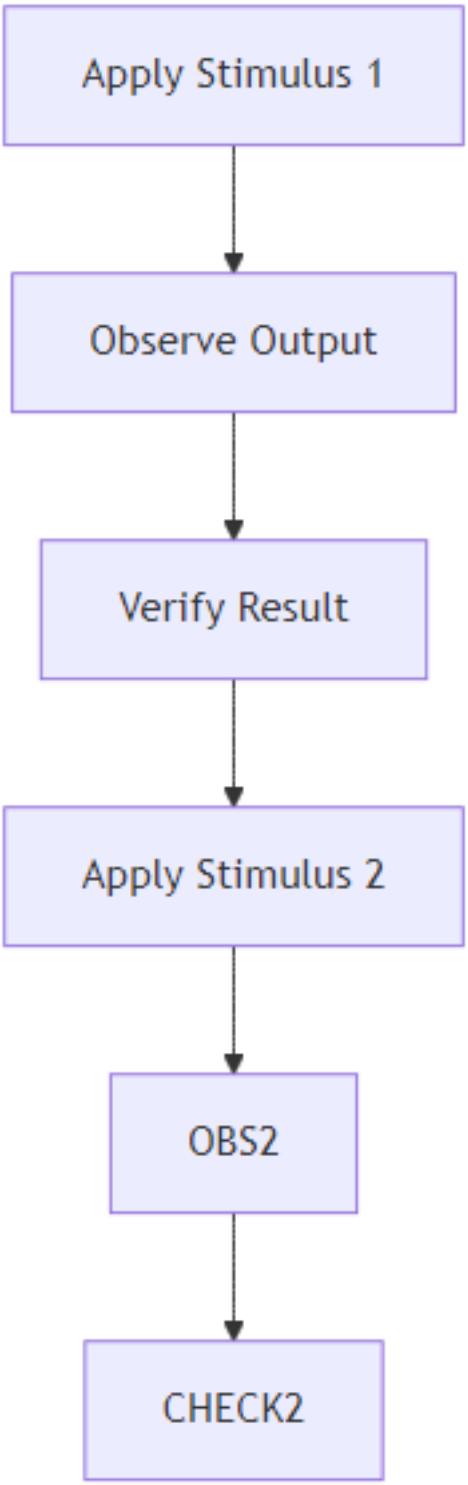
18.9 Sequential Test Execution

Testbenches often execute multiple test cases in sequence.

Typical pattern:

1. apply stimulus
2. wait for propagation or clock cycle
3. observe result
4. check correctness
5. proceed to next test case

Conceptually:



This allows many behaviors to be verified in a single simulation run.

18.10 Timing Control

Because hardware operates on clock cycles, testbenches must control timing carefully.

Common timing mechanisms include:

- waiting for clock edges
- waiting fixed simulation delays
- synchronizing stimulus with clock cycles

Conceptually:

Correct timing ensures realistic simulation behavior.

18.11 Module-Level Testbenches

Each module in the design has its own dedicated testbench.

Examples include:

Module	Testbench
ALU	tb_alu
ALU Control	tb_alu_control
Branch Compare	tb_branch_compare
Control Unit	tb_control_unit
Immediate Generator	tb_immediate_generator
Program Counter	tb_program_counter
Register File	tb_register_file
Load Store Unit	tb_load_store_unit
Bus Interconnect	tb_bus_interconnect
Data Memory	tb_data_memory
GPIO	tb_gpio
Timer	tb_timer
Trap Unit	tb_trap_unit

This modular approach simplifies debugging and verification.

18.12 CPU-Level Testbench

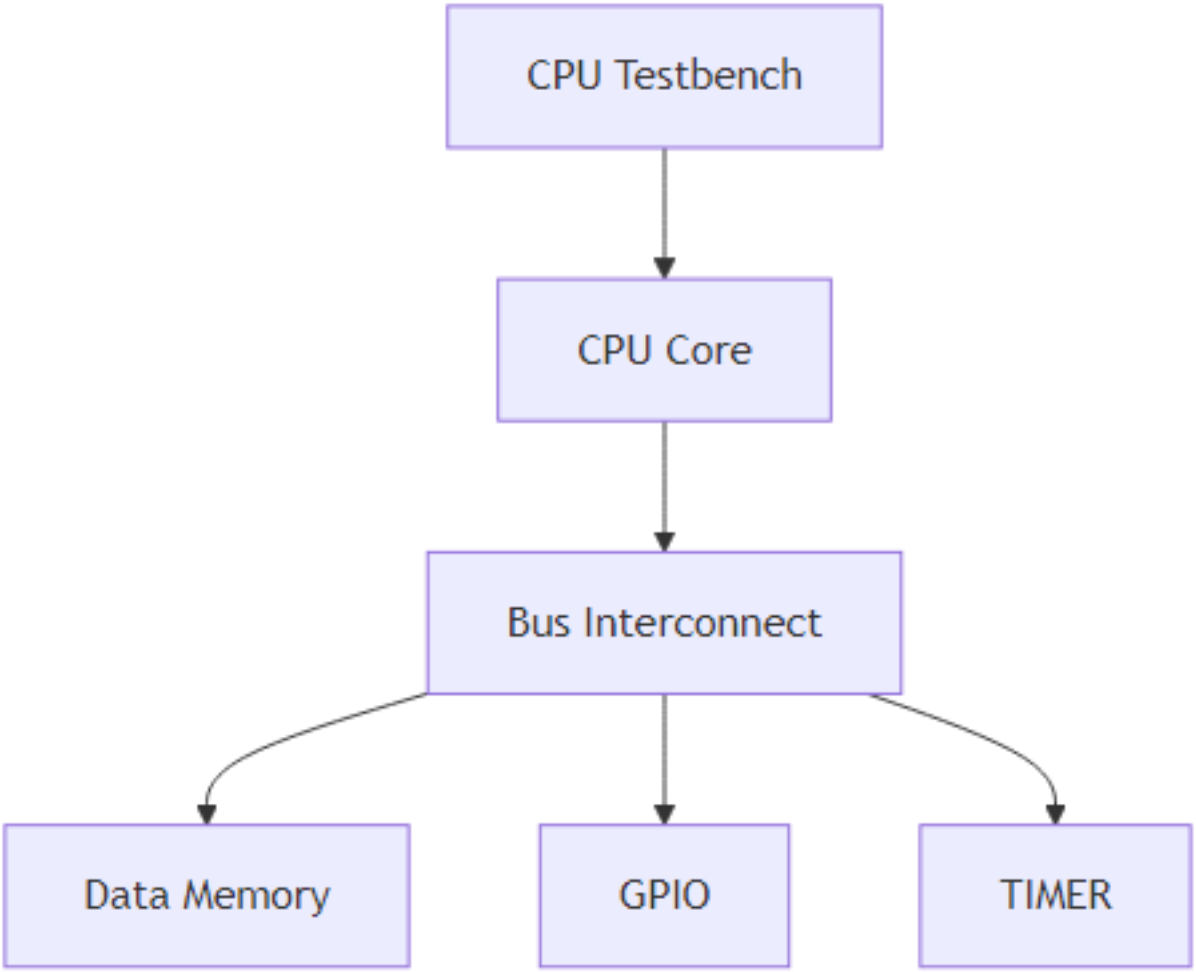
In addition to module-level tests, the full CPU can be verified using a dedicated testbench.

The CPU testbench may simulate:

- instruction execution sequences

- memory operations
- branch behavior
- interactions with peripherals

Conceptually:



This verifies that all modules work correctly together.

18.13 Waveform Analysis

During simulation, engineers often inspect signal waveforms to understand the behavior of the hardware.

Waveform analysis allows observation of:

- signal transitions
- clock cycles
- internal states
- control signal timing

This is one of the most powerful debugging tools available in hardware design.

18.14 Testbench Advantages

The testbench architecture provides several benefits.

18.14.1 Early Error Detection

Design errors can be discovered before synthesis or FPGA deployment.

18.14.2 Controlled Environment

All input signals are generated by the simulation environment.

18.14.3 Repeatable Testing

Testbenches can be executed repeatedly after each design modification.

18.14.4 Debugging Support

Waveform inspection helps identify incorrect signal behavior.

18.15 Limitations of Simulation-Based Testbenches

Although testbenches are powerful, they have limitations.

Examples include:

- incomplete coverage of all input combinations
- simulation speed limitations
- reliance on manually created test scenarios
- difficulty reproducing complex real-world conditions

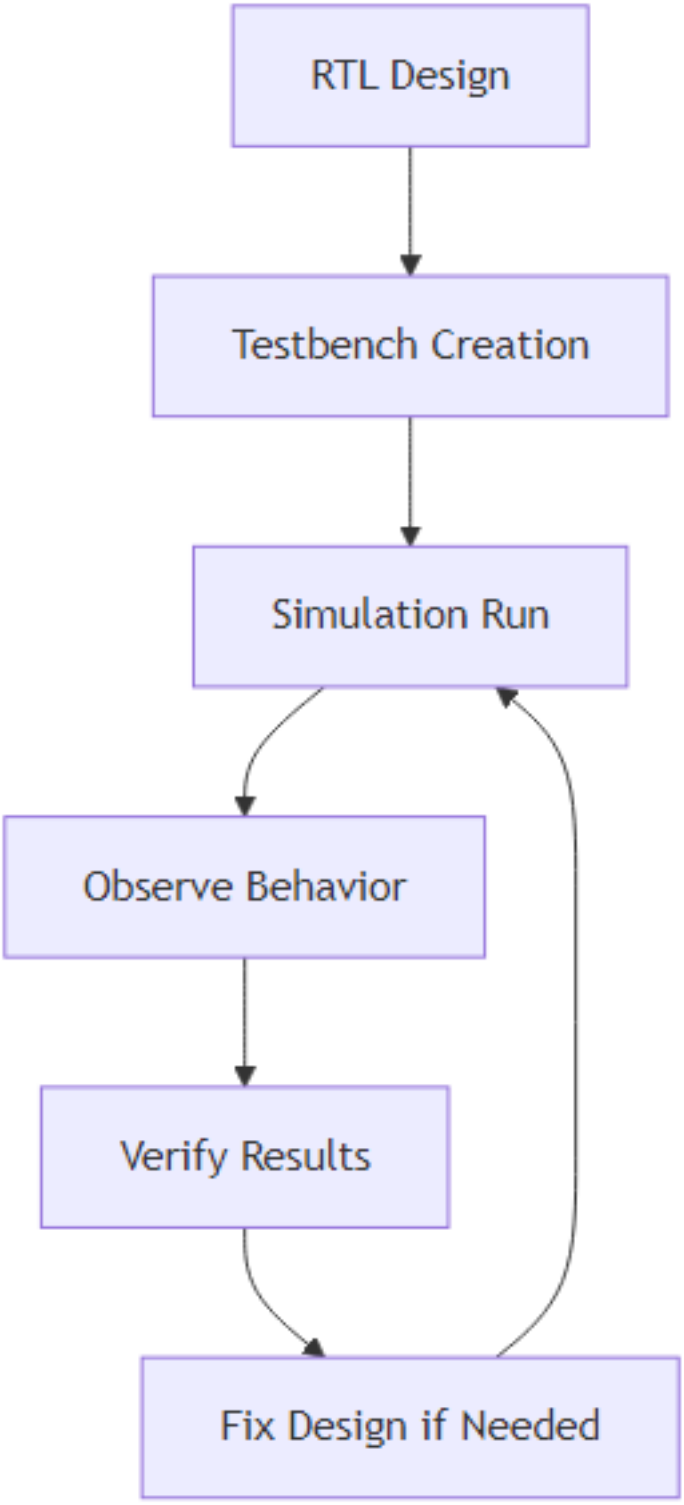
Despite these limitations, simulation remains the primary method used in early hardware verification.

18.16 Simulation Workflow

The typical simulation workflow for this project is:

1. write RTL design module
2. write corresponding testbench
3. compile design and testbench
4. run simulation
5. observe outputs and waveforms
6. verify expected behavior
7. debug and refine design

Conceptually:



This iterative process ensures continuous design improvement.

18.17 Good Testbench Design Practices

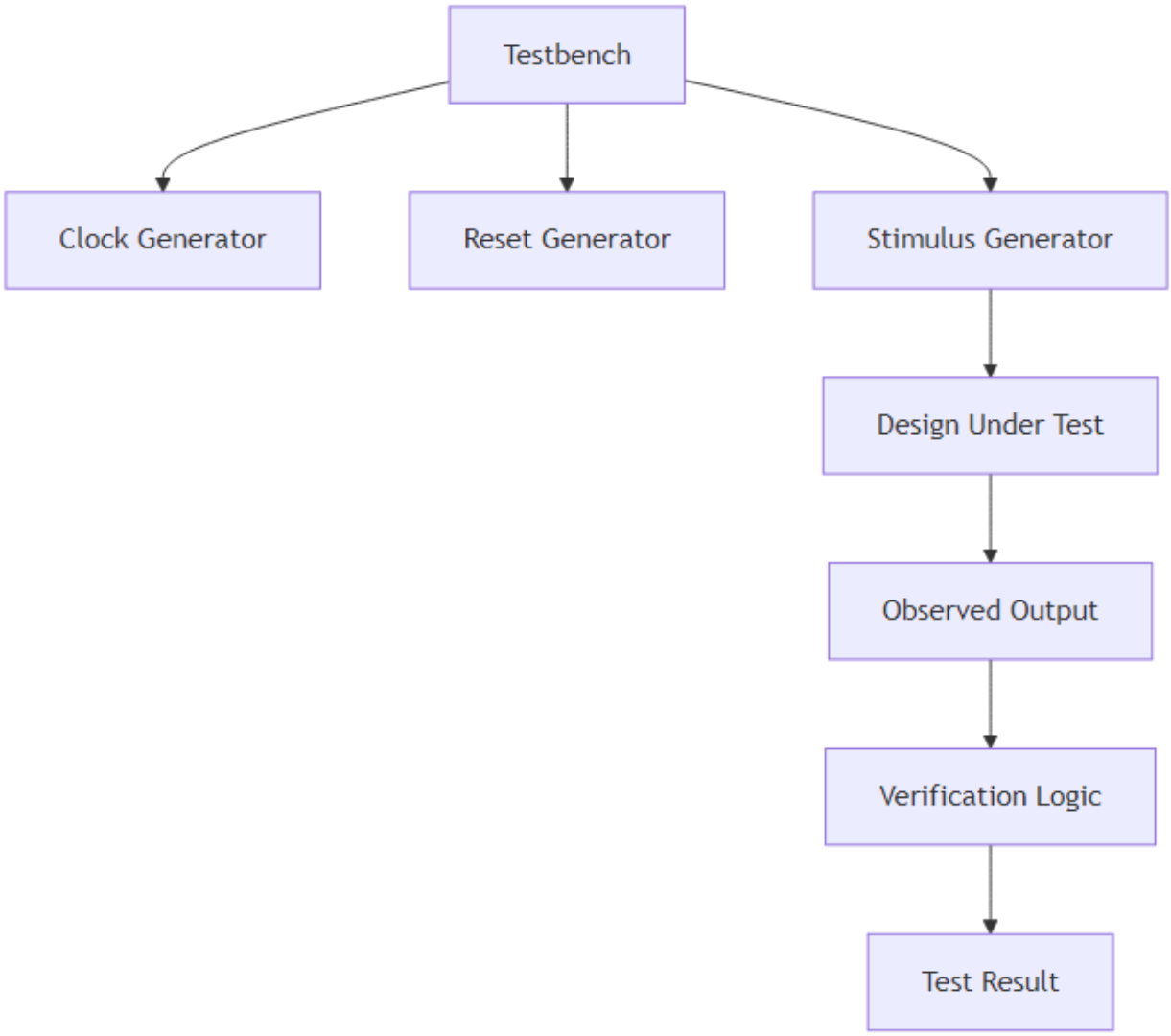
Effective testbenches follow several best practices:

- keep testbenches separate from synthesizable RTL
- generate clean and deterministic stimulus
- isolate modules during testing
- verify corner cases
- document expected behavior

Following these practices improves verification reliability.

18.18 Summary Diagram

The following diagram summarizes the overall testbench architecture.



This architecture is reused across all module-level testbenches in the project.

18.19 Final Summary

The **testbench architecture** defines the simulation environment used to verify each module and subsystem of the RV32I Minimal SoC.

By generating clock and reset signals, applying stimulus, observing outputs, and comparing results with expected behavior, the testbench framework provides a structured and repeatable verification methodology.

This approach ensures that both individual components and the integrated system behave correctly before hardware synthesis or deployment.

19 CPU Datapath Diagram

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

19.1 Purpose of the Datapath Diagram

The **CPU datapath diagram** illustrates how data flows through the processor during instruction execution.

While individual modules such as the ALU, register file, and control unit are described in their own documents, the datapath diagram shows how these components interact as part of a complete execution pipeline.

This visualization helps explain:

- how instructions are fetched
- how operands are retrieved
- how operations are executed
- how results are written back
- how control flow decisions are made

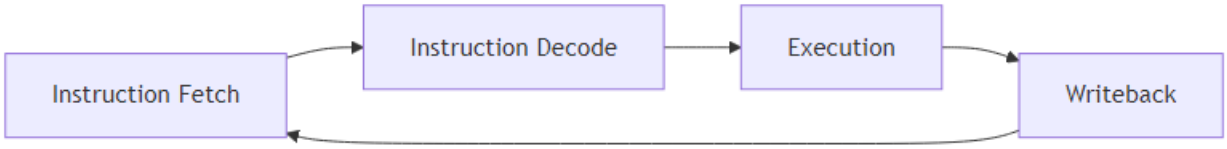
Understanding the datapath is essential for analyzing the architecture of the RV32I CPU.

19.2 High-Level CPU Datapath Overview

At the highest level, the CPU datapath consists of four major functional stages:

1. **Instruction Fetch**
2. **Instruction Decode**
3. **Execution**
4. **Writeback**

Conceptually:



Because this design uses a **single-cycle architecture**, all of these stages are effectively executed within one clock cycle.

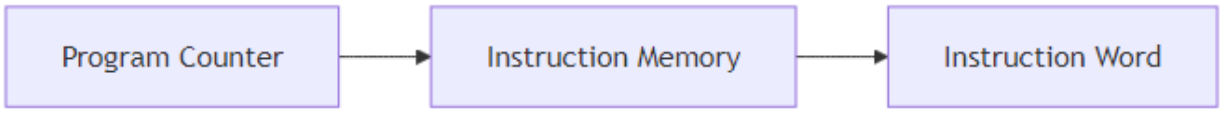
19.3 Instruction Fetch Stage

Instruction fetch retrieves the instruction from instruction memory using the current Program Counter (PC).

Components involved:

- Program Counter
- Instruction Memory

Conceptually:



The fetched instruction is forwarded to the decode stage.

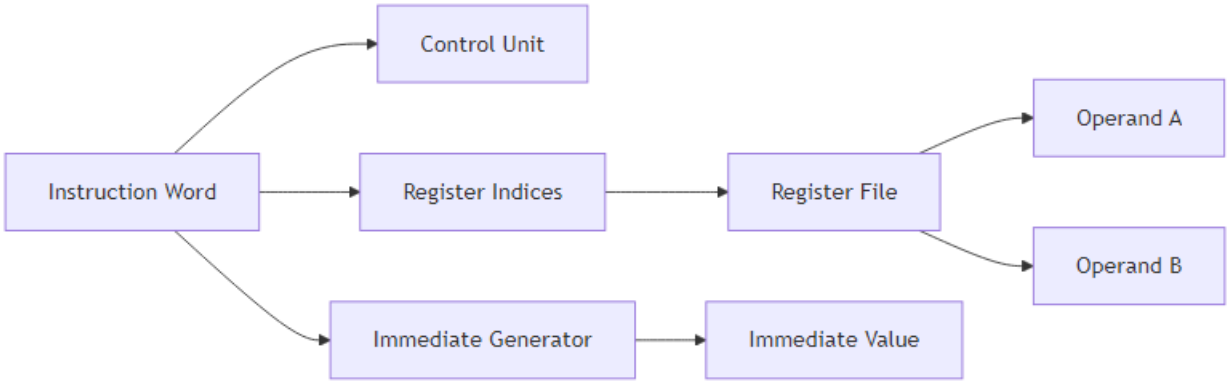
19.4 Instruction Decode Stage

During the decode stage, the CPU determines how the instruction should be executed.

Components involved:

- Control Unit
- Register File
- Immediate Generator

Conceptually:



The control unit produces signals that guide the rest of the datapath.

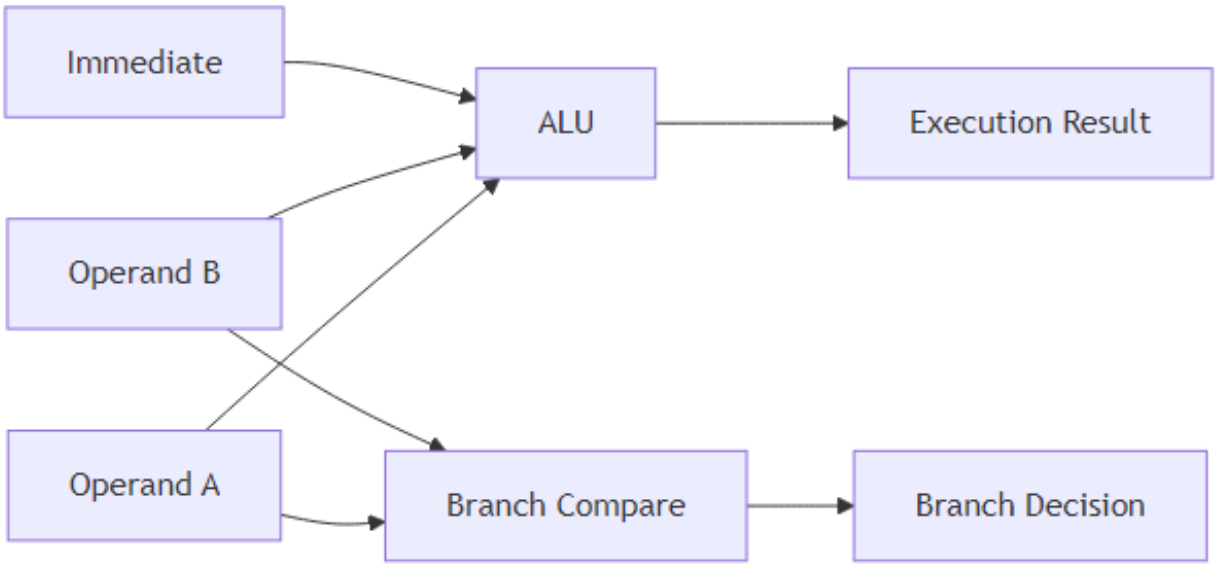
19.5 Execution Stage

The execution stage performs arithmetic operations, comparisons, and address calculations.

Components involved:

- ALU
- Branch Comparator
- Load Store Unit

Conceptually:

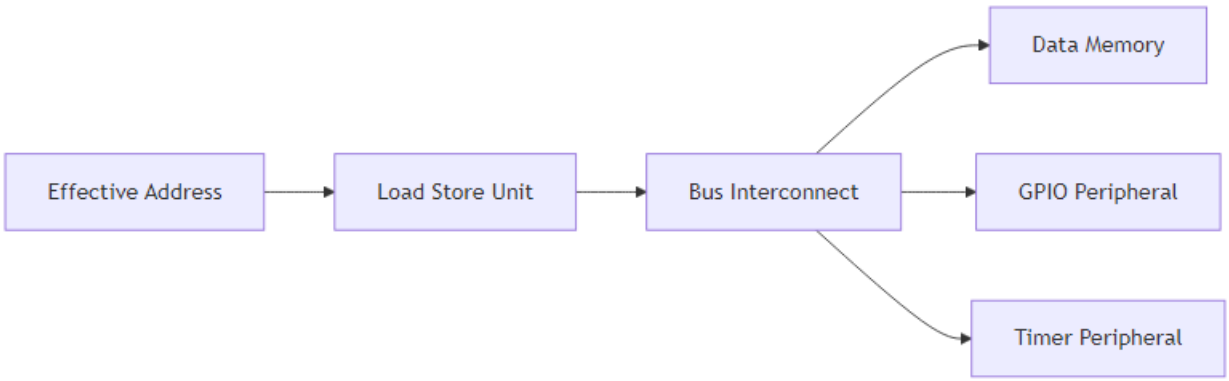


The ALU performs arithmetic operations, while the branch comparator evaluates conditional branches.

19.6 Memory Access Stage

If the instruction involves memory access, the Load Store Unit and Bus Interconnect perform the required transaction.

Conceptually:



Load operations return data that will be written back to the register file.

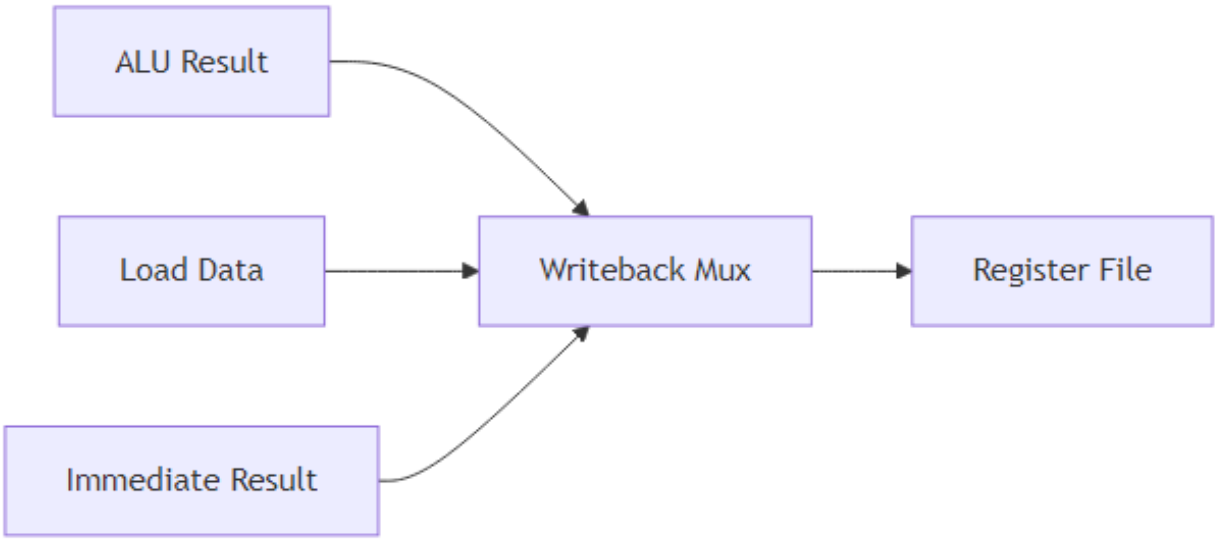
19.7 Writeback Stage

The writeback stage stores results in the register file.

Possible sources of writeback data include:

- ALU results
- load data from memory
- immediate values
- PC-related values

Conceptually:

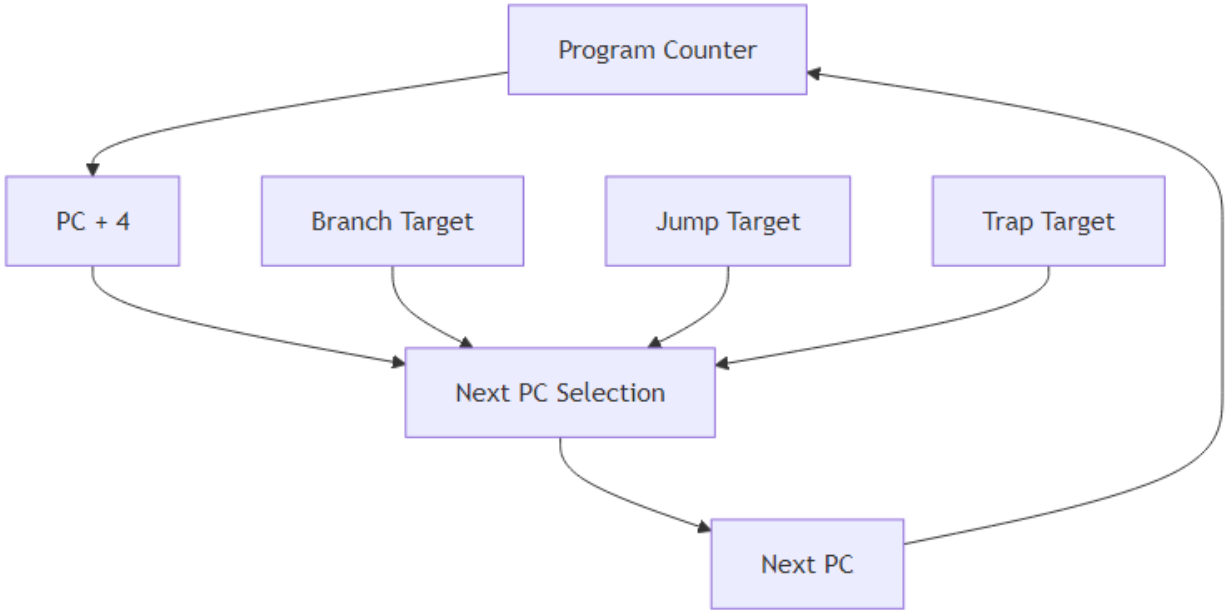


The control unit selects the correct writeback source.

19.8 Control Flow Path

Branch and jump instructions modify the normal sequential instruction flow.

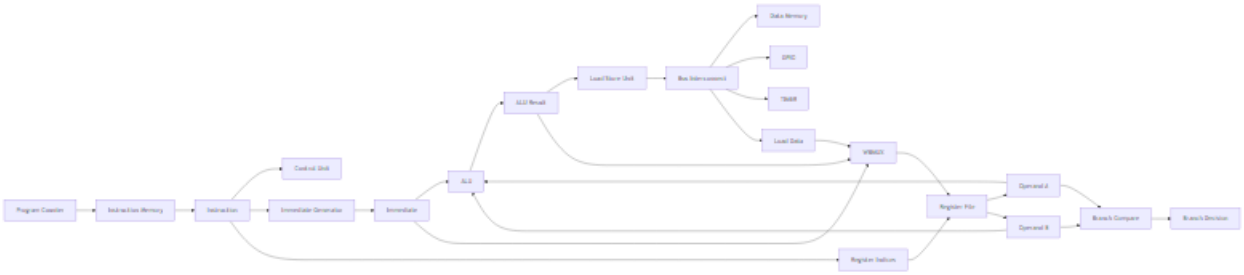
Conceptually:



This mechanism allows the CPU to redirect execution based on branch conditions or system events.

19.9 Complete CPU Datapath Diagram

The following diagram shows the complete datapath for the RV32I CPU.



This diagram highlights the full interaction between CPU modules and peripherals.

19.10 Single-Cycle Datapath Characteristics

The CPU uses a **single-cycle execution model**, meaning:

- each instruction completes within one clock cycle
- the datapath performs all required operations before the next clock edge

Advantages of this architecture include:

- simple control logic
- straightforward datapath design
- easy verification

However, it also means the clock period must be long enough to accommodate the longest instruction path.

19.11 Critical Path Considerations

The longest datapath delay typically occurs during **load instructions**.

Typical load path:

1. instruction fetch
2. register read
3. immediate generation
4. ALU address calculation
5. bus routing
6. memory read
7. writeback

This path determines the minimum clock period for the CPU.

Understanding this critical path is important for timing analysis.

19.12 Educational Value of the Datapath

The datapath architecture used in this project closely follows classic textbook CPU designs.

This makes the design useful for:

- teaching processor architecture
- demonstrating CPU design concepts
- understanding RISC execution models
- exploring hardware/software interaction

19.13 Summary

The CPU datapath illustrates how data flows through the processor during instruction execution.

By connecting modules such as the Program Counter, Register File, ALU, Control Unit, and memory interface, the datapath forms the core of the RV32I Minimal SoC processor architecture.

Understanding this datapath is essential for analyzing the design, debugging hardware behavior, and explaining the architecture during technical discussions or interviews.

20 SoC Architecture Diagram

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

20.1 Purpose of the SoC Architecture Diagram

The **SoC architecture diagram** illustrates the complete organization of the RV32I Minimal SoC system.

While the CPU datapath diagram focuses only on the processor core, the SoC architecture diagram shows how the CPU integrates with the rest of the system.

This includes:

- instruction memory
- data memory
- bus interconnect
- memory-mapped peripherals
- external interfaces

The diagram provides a system-level overview of how all components interact within the System-on-Chip architecture.

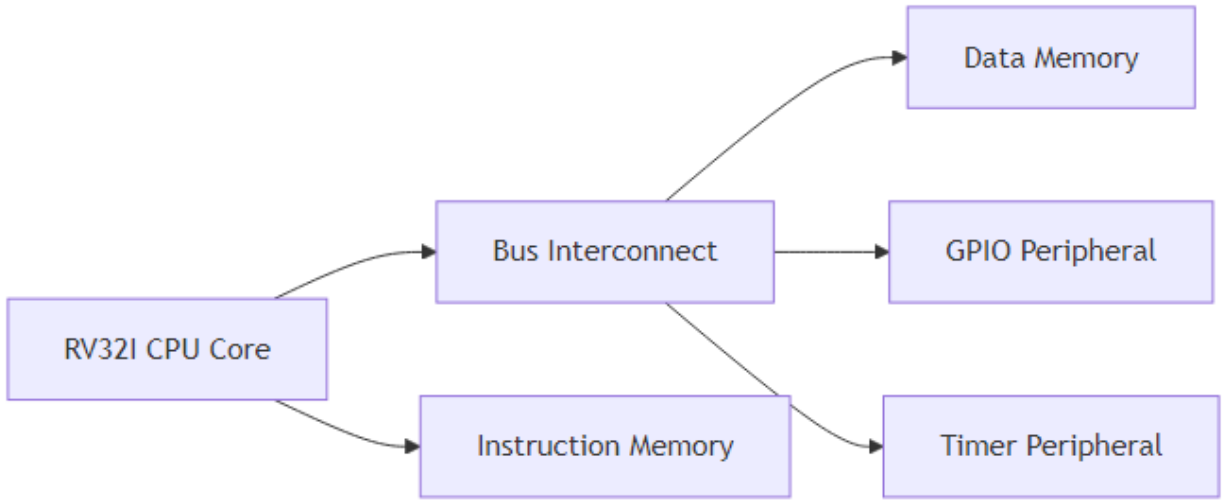
20.2 System Overview

The RV32I Minimal SoC consists of a processor core connected to a memory subsystem and several peripherals through a shared bus interconnect.

Major components include:

- RV32I CPU Core
- Instruction Memory
- Load Store Unit
- Bus Interconnect
- Data Memory
- GPIO Peripheral
- Timer Peripheral

Conceptually:



This architecture allows the CPU to interact with memory and peripherals using the same load/store instruction mechanism.

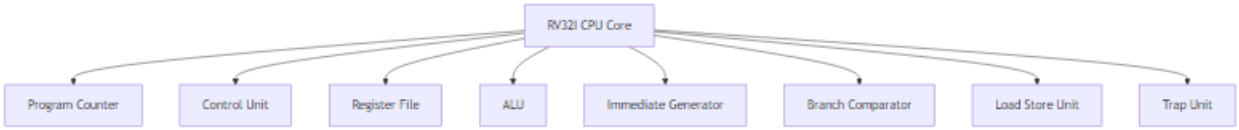
20.3 CPU Core

The CPU core implements the RV32I instruction set architecture.

Its internal modules include:

- Program Counter
- Instruction Memory Interface
- Control Unit
- Register File
- Immediate Generator
- ALU
- Branch Comparator
- Load Store Unit
- Trap Unit

Conceptually:



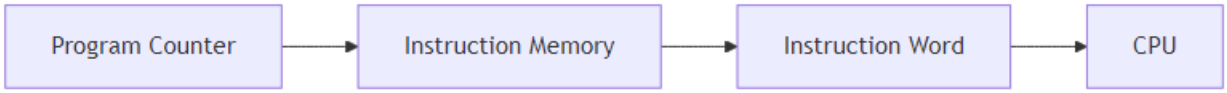
These modules form the internal execution architecture of the processor.

20.4 Instruction Memory

Instruction memory stores the program instructions executed by the CPU.

The Program Counter generates the address of the next instruction, which is then fetched from instruction memory.

Conceptually:



Instruction memory is typically read-only during program execution.

20.5 Data Access Path

Data access operations are handled by the Load Store Unit.

The LSU generates memory requests that are forwarded to the Bus Interconnect.

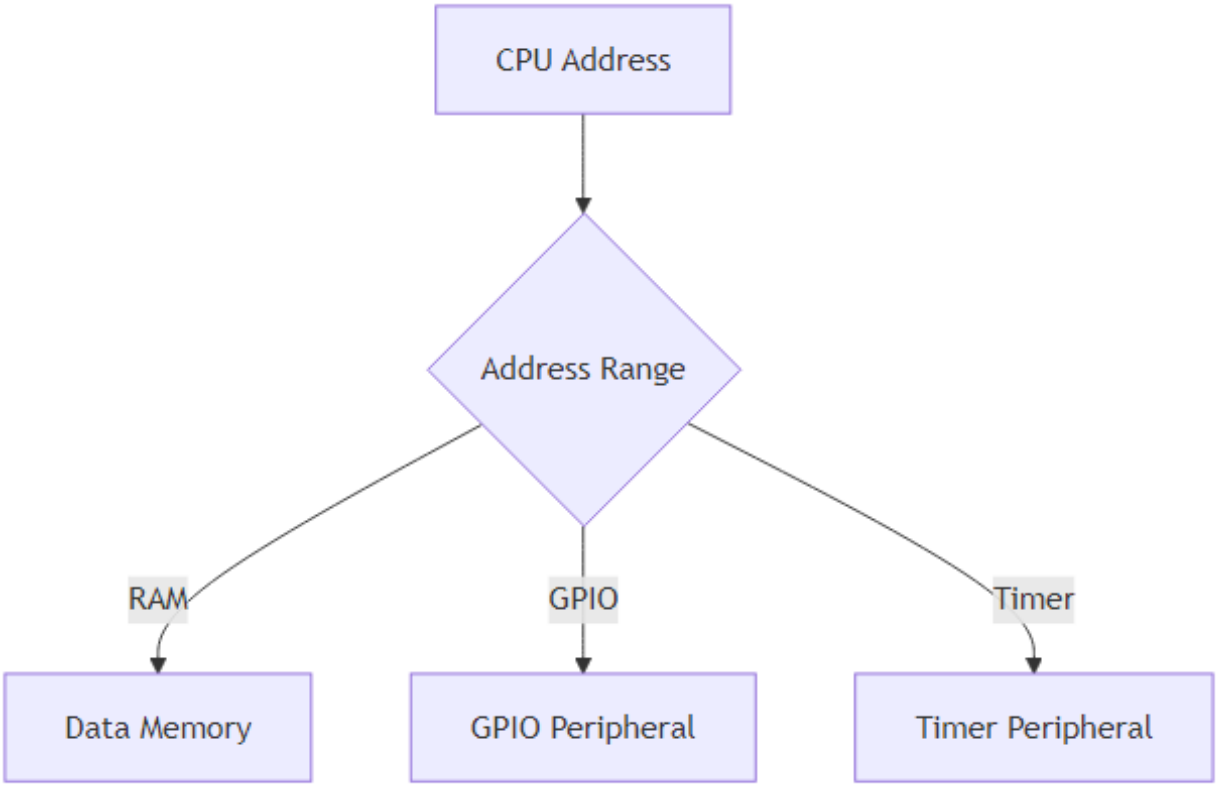
Conceptually:

This path handles load and store instructions issued by the CPU.

20.6 Bus Interconnect

The Bus Interconnect routes memory transactions to the appropriate destination based on the requested address.

Conceptually:



This module acts as the central communication fabric of the SoC.

20.7 Data Memory

Data memory stores runtime data used by programs.

Typical stored data includes:

- variables
- stack data
- buffers
- intermediate computation results

Conceptually:

Load instructions retrieve data from RAM, while store instructions update it.

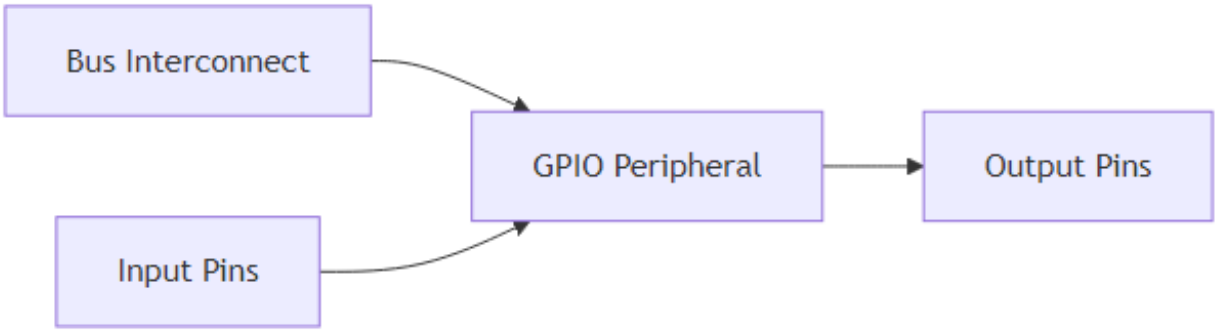
20.8 GPIO Peripheral

The GPIO module provides an interface between the CPU and external digital signals.

Typical uses include:

- controlling LEDs
- reading switches
- interacting with external digital hardware

Conceptually:



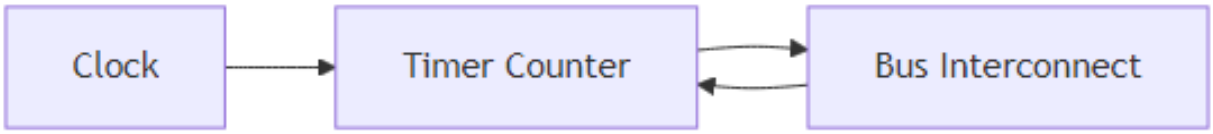
GPIO registers are accessed through memory-mapped addresses.

20.9 Timer Peripheral

The Timer provides a continuously incrementing counter driven by the system clock.

This counter allows the CPU to measure time intervals and implement delays.

Conceptually:



Software can read the timer value using standard load instructions.

20.10 Memory Map Concept

The SoC uses a **memory-mapped architecture**, where both memory and peripherals occupy regions of the address space.

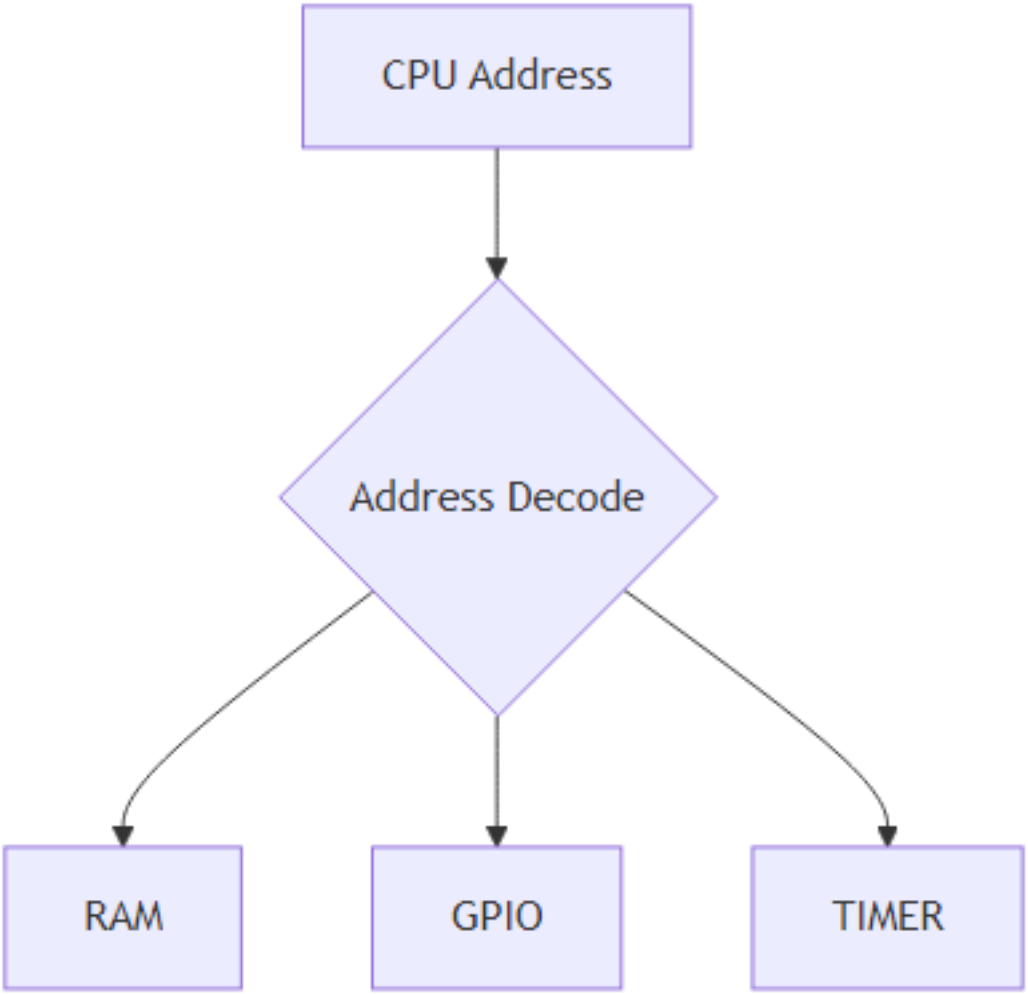
Conceptual address map:

Address Range	Device
RAM Range	Data Memory
GPIO Range	GPIO Peripheral

Address Range	Device
Timer Range	Timer Peripheral

The Bus Interconnect selects the correct device based on the address.

Conceptually:



This unified address space simplifies software and hardware design.

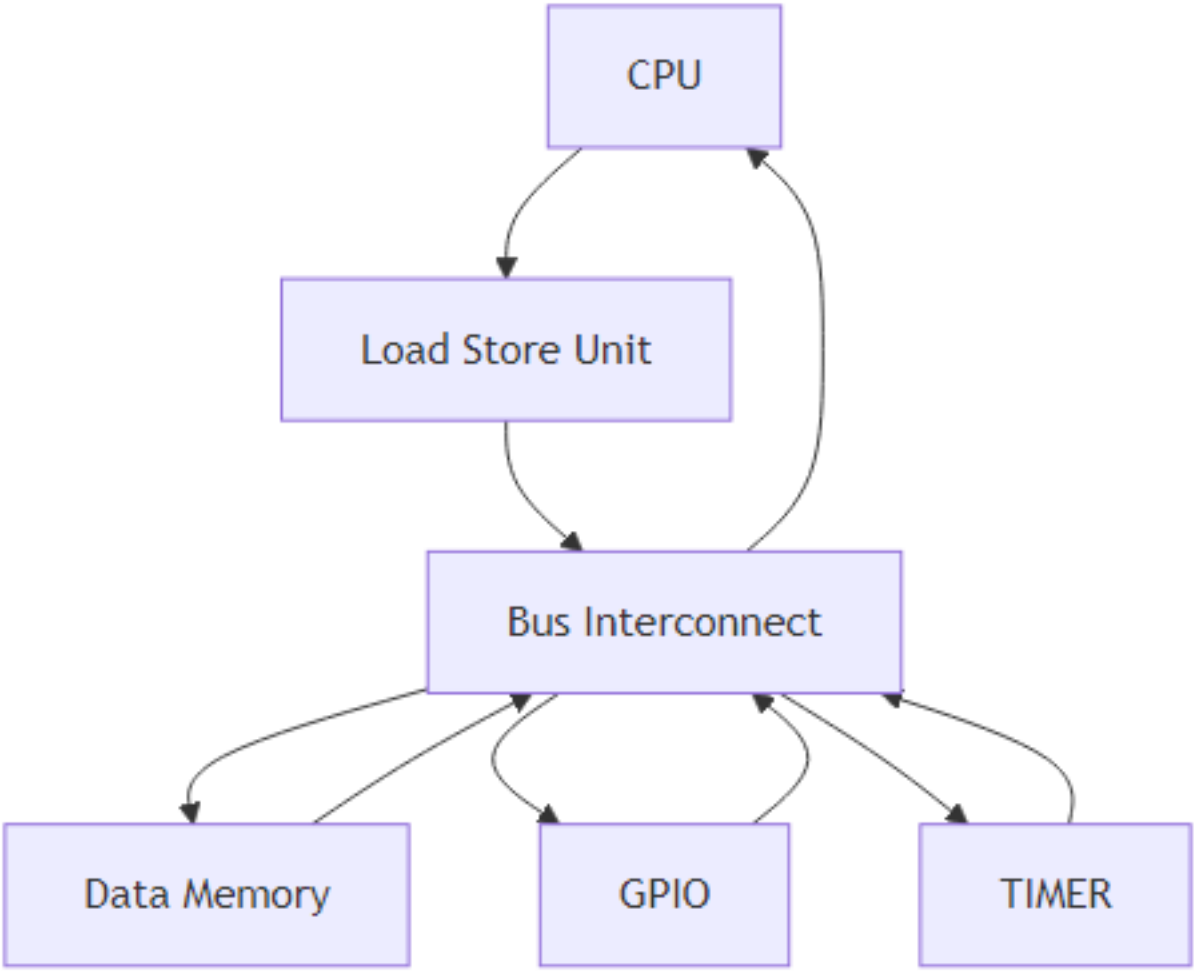
20.11 Data Flow Through the SoC

The typical data flow for a load instruction is:

1. CPU executes load instruction
2. LSU generates address
3. Bus Interconnect routes request
4. target device returns data

5. data is written back to the register file

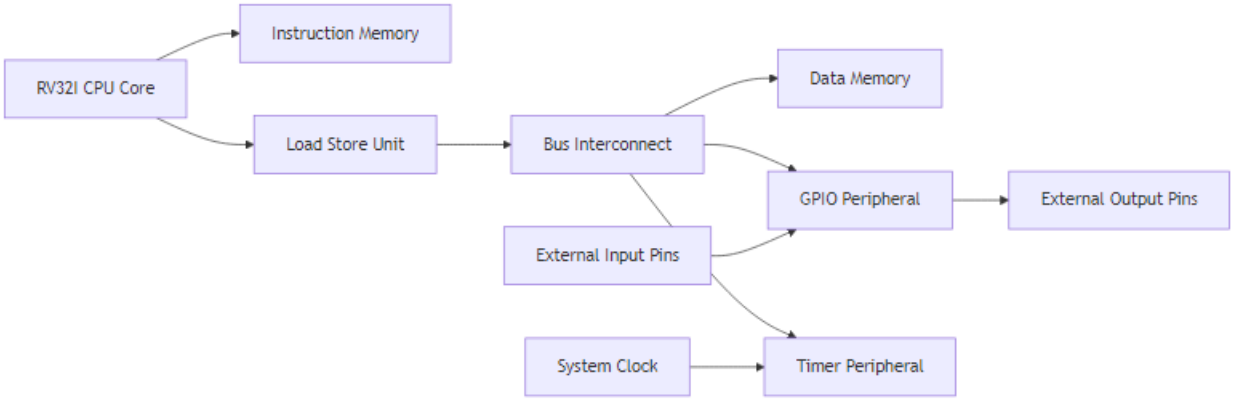
Conceptually:



This flow enables communication between the processor and system components.

20.12 Complete SoC Architecture Diagram

The following diagram shows the complete architecture of the RV32I Minimal SoC.



This diagram illustrates how the CPU core interacts with memory and peripherals through the system bus.

20.13 Advantages of the SoC Architecture

The architecture used in this project offers several advantages.

20.13.1 Modular Design

Each component can be designed and verified independently.

20.13.2 Memory-Mapped Peripherals

Peripherals can be accessed using normal load/store instructions.

20.13.3 Expandability

New peripherals can be added by extending the bus interconnect.

20.13.4 Clear System Organization

The system structure is easy to understand and document.

20.14 Educational Value

The RV32I Minimal SoC architecture provides a compact example of how a processor core can be integrated into a complete digital system.

It demonstrates key concepts used in modern hardware systems, including:

- processor datapaths
- bus-based communication
- memory-mapped peripherals
- hardware/software interaction

These concepts are fundamental to embedded systems and FPGA-based designs.

20.15 Summary

The SoC architecture diagram shows how the RV32I CPU core interacts with memory and peripherals through a shared bus interconnect.

By integrating instruction memory, data memory, GPIO, and a timer into a memory-mapped system, the design demonstrates the structure of a simple yet functional System-on-Chip architecture.

This system-level view complements the CPU datapath diagram and helps explain how the processor operates within a complete digital system.

21 Timer Architecture Diagram

Project: RV32I Minimal SoC – VHDL Implementation
Author: Mateus Telles Nébias

21.1 Purpose of the Timer Architecture Diagram

The **Timer architecture diagram** illustrates the internal structure and operation of the Timer peripheral used in the RV32I Minimal SoC.

While the timer module documentation explains the functionality of the peripheral, this diagram provides a structural representation of how the timer operates internally.

The diagram highlights:

- the counter register
- the increment logic
- the clock and reset inputs
- the memory-mapped bus interface
- the data returned to the CPU

Understanding this structure helps clarify how the timer integrates into the SoC architecture and how software interacts with it.

21.2 Overview of the Timer Module

The timer is implemented as a continuously incrementing counter driven by the system clock.

Every clock cycle, the counter increases by one. The CPU can read the current counter value through the memory-mapped bus.

Conceptually:

This mechanism provides a simple hardware time reference for the system.

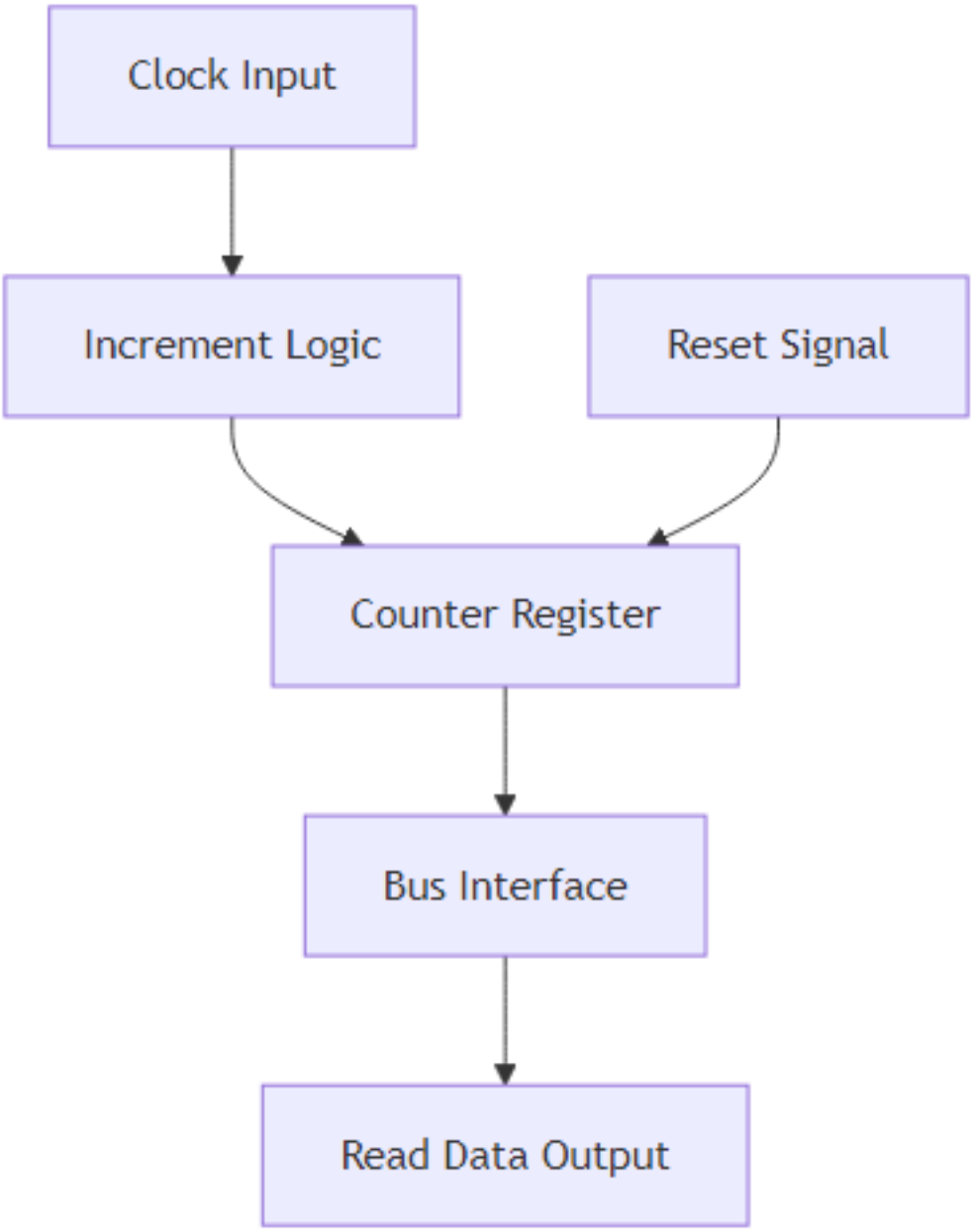
21.3 Core Timer Components

The Timer peripheral consists of several key components.

21.3.1 Main elements

- Counter Register
- Increment Logic
- Clock Input
- Reset Logic
- Bus Interface
- Read Data Output

Conceptually:



These components form the internal architecture of the timer.

21.3.1.1 Counter Register The counter register stores the current timer value.

Characteristics:

- increments once per clock cycle
- resets to zero when reset is asserted
- stores the current time reference

Conceptually:

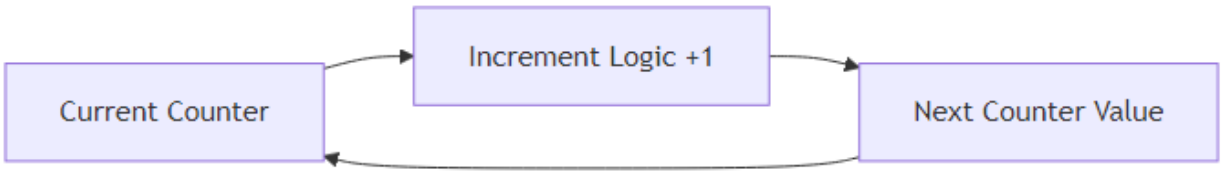
The counter continuously updates as long as the system clock is active.

21.3.1.2 Increment Logic The increment logic computes the next value of the counter.

Each clock cycle:

```
next_counter = counter + 1
```

Conceptually:



This simple arithmetic operation drives the timer progression.

21.3.1.3 Clock Input The timer is driven by the **system clock**.

Every rising clock edge triggers an update of the counter value.

Conceptually:

The frequency of the clock determines the resolution of the timer.

Example:

- 50 MHz clock → timer increments every 20 ns

21.3.1.4 Reset Logic The reset signal initializes the timer when the system starts.

When reset is asserted:

- the counter value is cleared
- the timer begins counting again from zero

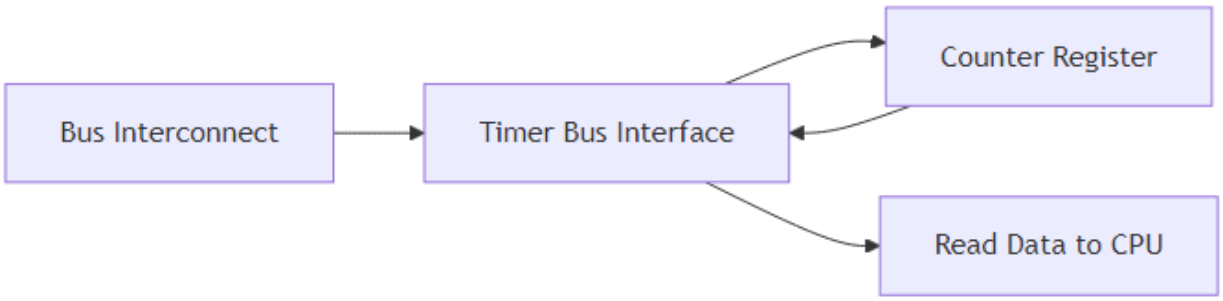
Conceptually:

This ensures predictable timer behavior after system initialization.

21.3.1.5 Bus Interface The timer connects to the Bus Interconnect through a memory-mapped interface.

The bus interface allows the CPU to access the timer value using load instructions.

Conceptually:



This interface allows the timer to behave like a memory-mapped register.

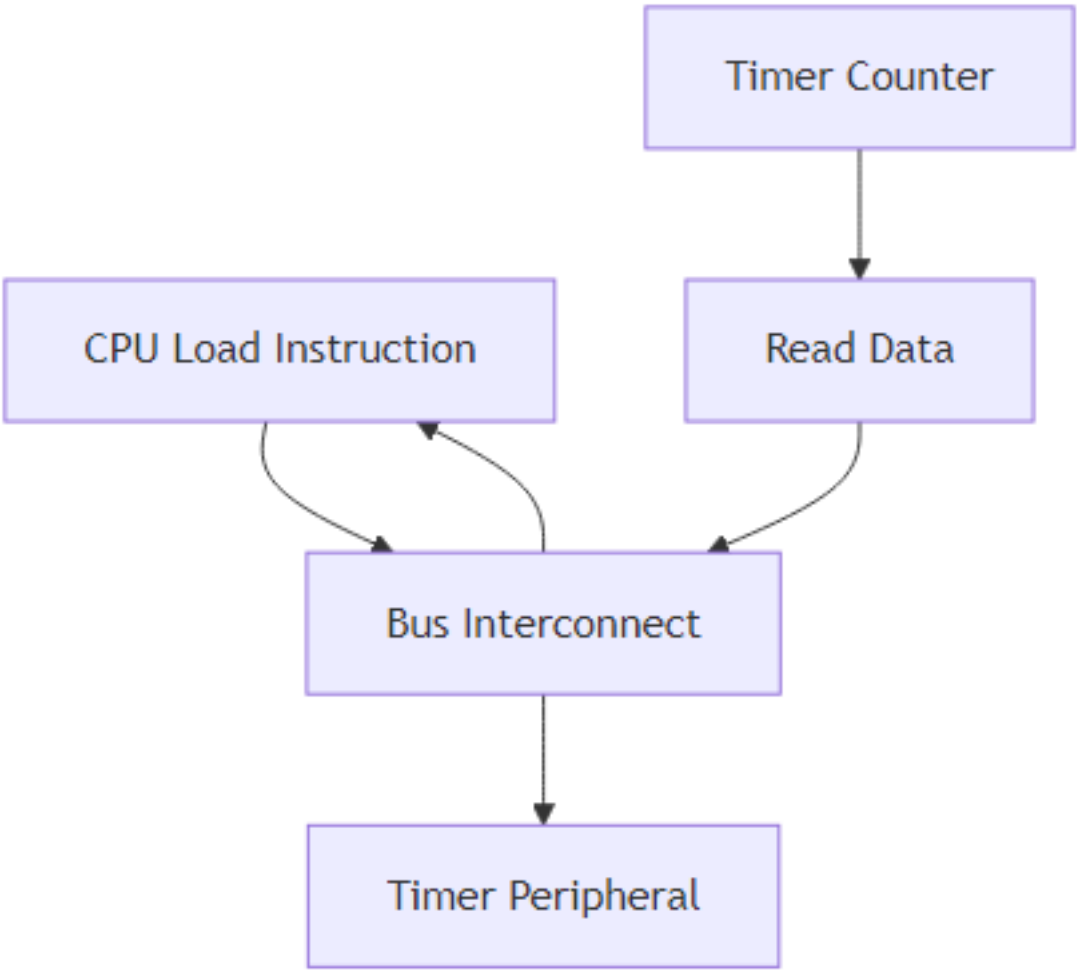
21.3.1.6 CPU Interaction The CPU reads the timer value by executing a load instruction targeting the timer address.

Example conceptual instruction:

```
lw x5, TIMER_ADDRESS
```

This operation loads the current timer value into register `x5`.

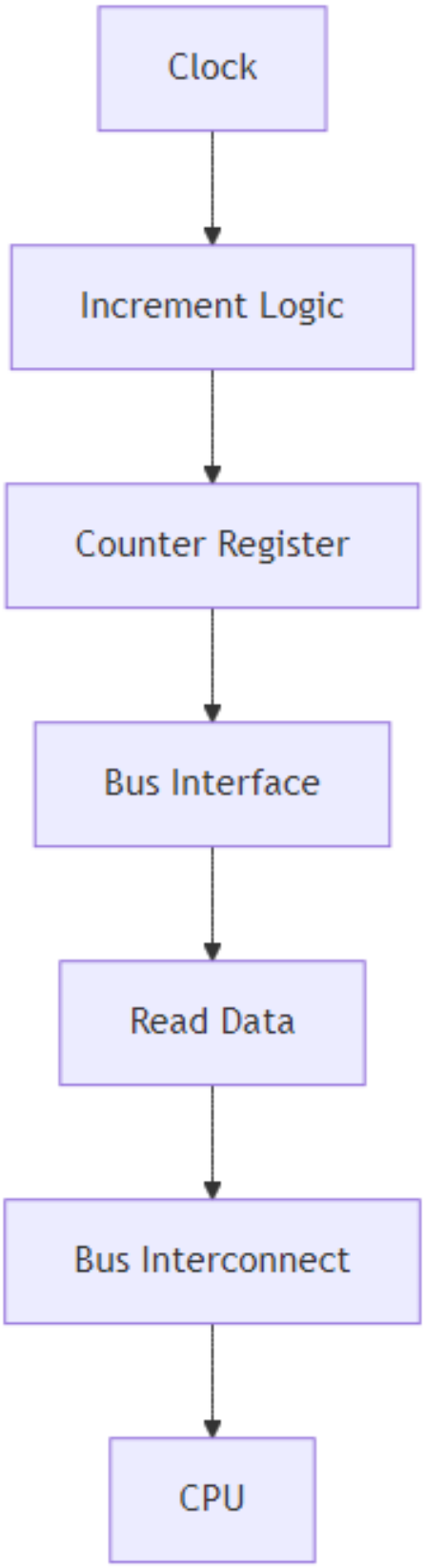
Conceptually:



Software can then use this value for timing calculations.

21.3.1.7 Timer Data Flow The internal data flow of the timer module can be summarized as follows.

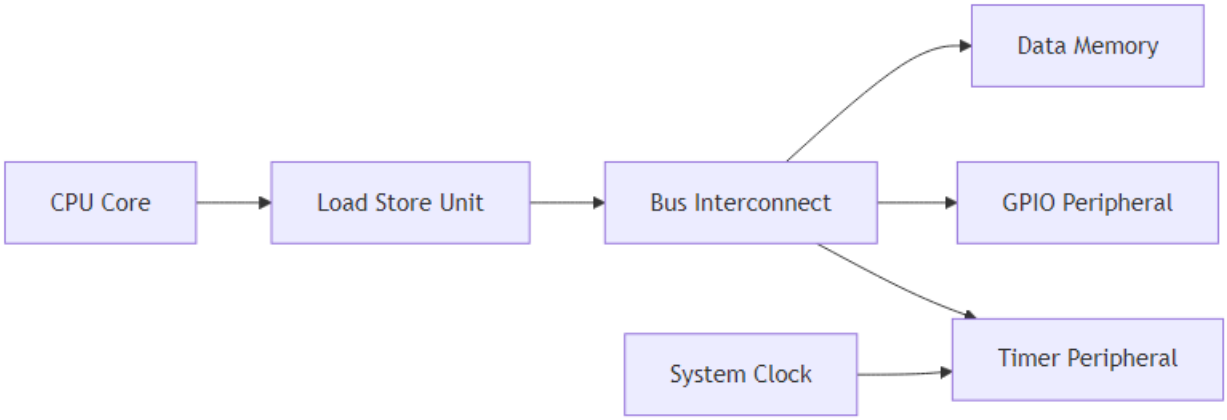
Conceptually:


Clock
Increment Logic
Counter Register
Bus Interface
Read Data
Bus Interconnect
CPU

This diagram highlights the simple and efficient architecture of the timer.

21.3.1.8 Integration with the SoC The timer integrates with the rest of the system through the Bus Interconnect.

Conceptually:



This integration allows the CPU to access the timer in the same way as other memory-mapped devices.

21.4 Example Timing Measurement

The timer can be used to measure the number of clock cycles consumed by an operation.

Example conceptual program flow:

```
start = read_timer()
execute_operation()
end = read_timer()

elapsed = end - start
```

The difference between the two readings corresponds to the number of clock cycles that occurred during the operation.

21.5 Advantages of the Timer Architecture

The timer architecture provides several advantages.

21.6 Simple Implementation

Only a counter and increment logic are required.

21.6.1 Reliable Time Reference

The counter provides a consistent measure of elapsed time.

21.6.2 Easy CPU Access

Memory-mapped interface allows the CPU to read the timer using load instructions.

21.6.3 Expandability

Future extensions may include:

- interrupt generation
- compare registers
- programmable prescalers
- multiple timers

21.7 Educational Value

The timer architecture demonstrates how simple hardware blocks can provide important system functionality.

It illustrates several important hardware design concepts:

- sequential logic
- clock-driven state updates
- memory-mapped peripherals
- hardware/software interaction

These concepts are common in real-world embedded systems.

21.8 Summary

The timer architecture consists of a clock-driven counter that increments every cycle and exposes its value through a memory-mapped interface.

This simple design allows the CPU to measure elapsed time and implement software delays while integrating seamlessly into the RV32I Minimal SoC architecture.